XRayLabTool

Release 0.5.0

Wei Chen

Oct 28, 2026

USER GUIDE

1 Key Features **3**

2 Quick Start **5**
- 2.1 Interactive Notebooks . . . 5
- 2.2 Installation . . . 5
- 2.3 Basic Usage . . . 5

3 Navigation **7**
- 3.1 Getting Started . . . 7
- 3.2 CLI Reference . . . 13
- 3.3 Shell Completion Guide . . . 17
- 3.4 Kramers-Kronig Guide . . . 25
- 3.5 Examples . . . 30
- 3.6 X-ray Optics Fundamentals . . . 33
- 3.7 Atomic Scattering Factor Data . . . 36
- 3.8 Calculation Methods . . . 40
- 3.9 Kramers-Kronig Transform . . . 47
- 3.10 API Reference . . . 51
- 3.11 Architecture . . . 205
- 3.12 Contributing Guide . . . 234
- 3.13 Performance and Optimization . . . 237
- 3.14 Testing Guide . . . 239
- 3.15 Changelog . . . 243

4 Performance Highlights **255**

5 Target Audience **257**

6 Citation **259**

7 Indices and Tables **261**

Python Module Index **263**

Index **265**

XRayLabTool is a Python package and command-line tool for calculating X-ray optical properties of materials. It provides both a Python API and CLI for synchrotron scientists, materials researchers, and X-ray optics developers.

CHAPTER

ONE

KEY FEATURES

Performance

- One JAX float64 implementation (CPU or NVIDIA GPU), no NumPy twin (ADR-006)
- Scattering tables resident on the device; one compiled program, one upload and one download per call
- Batches run as blocks of materials through the same program, with no worker pools
- Measured gates and the accepted single-call latency are in *ADR-006: The Core Calculators Are JAX-Only*

X-ray Physics

- Complex refractive index calculations
- Attenuation coefficients and penetration depths
- Critical angles for total external reflection
- Transmission and reflection coefficients

Architecture (v0.5.0)

- **JAX-only core**: `set_backend("jax")` is a no-op and `set_backend("numpy")` raises `ValueError`
- Modular design with focused sub-packages:
  - **calculators**: Core X-ray physics calculations
  - **data_handling**: Atomic data caching and batch processing
  - **interfaces**: CLI and completion systems
  - **io**: File operations and export functionality
  - **validation**: Input validation and error handling
  - **analysis**: Material comparison and absorption edge detection
  - **export**: CSV/JSON export for downstream analysis
  - **backend**: JAX runtime helpers, shape buckets and the in-tree PCHIP interpolator
  - **gui**: PySide6 desktop application with PyQtGraph interactive plots
- Type-safe with complete type hints
- 252 characterization tests with golden-value assertions
- Cross-platform compatibility (Linux, macOS, Windows)

CLI Interface

- 10+ commands for common tasks and completion management
- Multiple output formats (table, CSV, JSON)
- Virtual environment-centric shell completion (bash, zsh, fish, PowerShell)
- Batch processing from CSV files
- Environment isolation for completion

Scientific Data Handling

- Built on CXRO/NIST atomic scattering databases
- Support for energy ranges and arrays
- Chemical formula parsing and validation
- Export capabilities for downstream analysis

CHAPTER

TWO

QUICK START

2.1 Interactive Notebooks

Open the notebook gallery (Colab / nbviewer links) on the *Examples page*.

2.2 Installation

```
pip install xraylabtool                 # Core (NumPy + JAX CPU, CLI, GUI)
pip install "xraylabtool[gpu_cuda13]"   # NVIDIA GPU, CUDA 13 driver
pip install "xraylabtool[gpu_cuda12]"   # NVIDIA GPU, CUDA 12 driver

# Or with conda (CPU only; dependencies come from conda-forge)
conda install -c imewei -c conda-forge xraylabtool
```

2.3 Basic Usage

Python API:

```
import xraylabtool as xrt

# Calculate X-ray properties for silicon at 8 keV
result = xrt.calculate_single_material_properties("Si", 8.0, 2.33)  # formula, keV, g/cm³

print(f"Critical angle: {result.critical_angle_degrees[0]:.3f}°")
print(f"Attenuation length: {result.attenuation_length_cm[0] * 1e4:.1f} μm")
```

Command Line:

```
# Single material calculation
xraylabtool calc Si -e 8.0 -d 2.33

# Batch processing
xraylabtool batch materials.csv -o results.csv
```

CHAPTER

THREE

NAVIGATION

3.1 Getting Started

This guide shows XRayLabTool installation and basic usage.

3.1.1 Interactive Notebook

Open the Getting Started notebook directly (no embedded render in the docs):

- Open in Colab
- View on nbviewer
- Download (.ipynb)

3.1.2 Installation

System Requirements

XRayLabTool requires:

- **Python 3.12 or higher**
- **Operating System**: Windows, macOS, or Linux
- **Memory**: Minimum 512 MB RAM (recommended 2 GB for large calculations)
- **Storage**: 50 MB for basic installation

Install from PyPI

Install XRayLabTool using pip:

```
# Core package (JAX on CPU, CLI, GUI, matplotlib)
pip install xraylabtool

# NVIDIA GPU acceleration: pick the CUDA major matching your driver
pip install "xraylabtool[gpu_cuda13]"
pip install "xraylabtool[gpu_cuda12]"
```

`jax` and `jaxlib` are required at `>=0.12.0.1` with no upper bound (the Kramers-Kronig goldens are verified on that release; earlier ones differ in the last digits). The GPU extras add the matching `jax-cuda*-plugin` and `jax-cuda*-pjrt` packages, which must be the same version as `jaxlib`.

Install from conda

The package is also published on the `imewei` channel at https://anaconda.org/imewei/xraylabtool. Keep `conda-forge` in the command, because the dependencies are resolved from it:

```
conda install -c imewei -c conda-forge xraylabtool
```

The conda package is `noarch` and CPU only. It does not provide the `gpu_cuda12` / `gpu_cuda13` extras; use pip or uv for GPU support. `mamba` accepts the same arguments.

Development Installation

For development or to get the latest features (uses `uv`):

```
git clone https://github.com/imewei/pyXRayLabTool.git
cd pyXRayLabTool
uv sync
uv run pytest  # verify installation
```

Verify Installation

Verify installation:

```
# Test CLI (xtool and xlt are short aliases for xraylabtool)
xraylabtool --version
xtool --version
xraylabtool --help

# Test Python API
python -c "import xraylabtool; print('Installation successful!')"
```

Shell Completion (Virtual Environment-Centric)

Install completion in your current virtual environment:

```
# Install completion in current environment
xraylabtool completion install

# Verify installation
xraylabtool completion status

# List all environments with completion status
xraylabtool completion list
```

The new completion system provides:

- **Environment Isolation**: Completion only available when environment is active
- **Multi-Shell Support**: Native completion for bash, zsh, fish, PowerShell
- **No System Changes**: No sudo required, installs per environment
- **Auto-Activation**: Completion activates/deactivates with environment
- **Alias Support**: Completion works for both `xraylabtool` and its short aliases `xtool` and `xlt`

For legacy compatibility, the old commands still work:

```
xraylabtool install-completion    # Uses new system backend
```

3.1.3 First Steps

Simple Calculation

Let's calculate X-ray properties for silicon at 8 keV:

Using the CLI:

```
xraylabtool calc Si -e 8.0 -d 2.33
```

Using Python:

```python
import xraylabtool as xrt

# Positional order is (formula, energy_keV, density)
result = xrt.calculate_single_material_properties("Si", 8.0, 2.33)

print(f"Formula: {result.formula}")
print(f"Critical angle: {result.critical_angle_degrees[0]:.3f}°")
print(f"Attenuation length: {result.attenuation_length_cm[0] * 1e4:.1f} μm")
```

Expected output:

```
Formula: Si
Critical angle: 0.225°
Attenuation length: 69.7 μm
```

All per-energy fields on `XRayResult` are NumPy arrays, even for a single energy, so index `[0]` for a scalar.

Understanding the Results

The main properties calculated are:

- **Critical angle**: Angle for total external reflection
- **Attenuation length**: Distance for 1/e intensity reduction
- **Delta (δ)**: Real part of refractive index decrement
- **Beta (β)**: Imaginary part related to absorption

Multiple Energies

Calculate properties across an energy range:

CLI:

```
xraylabtool calc Si -e 5.0,8.0,10.0 -d 2.33
```

Python:

```python
result = xrt.calculate_single_material_properties("Si", [5.0, 8.0, 10.0], 2.33)

for e, theta_c in zip(result.energy_kev, result.critical_angle_degrees):
    print(f"{e:.1f} keV: θc = {theta_c:.3f}°")
```

Different Materials

Compare several materials in one call:

```python
formulas = ["Si", "SiO2", "Al", "Cu"]
densities = [2.33, 2.20, 2.70, 8.96]

results = xrt.calculate_xray_properties(formulas, 8.0, densities)

for formula, r in results.items():
    print(f"{formula:5}: θc = {r.critical_angle_degrees[0]:.3f}°, "
          f"attenuation length = {r.attenuation_length_cm[0] * 1e4:.1f} μm")
```

3.1.4 Batch Processing

For many materials, use the `batch` command. Create `materials.csv` with lowercase `formula,density,energy` columns (energy in keV; comma-separate several energies in one cell):

```
formula,density,energy
Si,2.33,8.0
SiO2,2.20,8.0
Al,2.70,"5.0,8.0,10.0"
Cu,8.96,8.0
```

Process the batch:

```
xraylabtool batch materials.csv -o results.csv
```

Or in Python, `calculate_xray_properties()` (above) handles material lists directly.

3.1.5 Graphical User Interface (GUI)

XRayLabTool includes a modern desktop application for interactive analysis.

Launch via:

```
python -m xraylabtool.gui
```

Key Features:

- **Interactive Analysis**: Single material, multiple material comparison and Kramers-Kronig tabs; the window opens maximized.
- **Modern Interface**: Clean design with Light and Dark data-optimized themes.
- **Theme Toggle**: Switch between Light and Dark modes via the status bar toggle.
- **Persisted Settings**: Your theme preference is saved automatically.

3.1.6 Common Use Cases

Mirror Design

For X-ray mirror applications:

```
# Compare substrate materials
substrates = ["Si", "SiO2", "SiC"]
densities = [2.33, 2.20, 3.21]
energy = 8.0  # keV

print("Mirror substrate comparison at 8 keV:")
print("Material | Critical Angle | Attenuation Length")
print("---------|----------------|-------------------")

for formula, density in zip(substrates, densities):
    result = xrt.calculate_single_material_properties(formula, energy, density)
    print(f"{formula:8} | {result.critical_angle_degrees[0]:13.3f}° | "
          f"{result.attenuation_length_cm[0] * 1e4:15.1f} μm")
```

Beamline Planning

For synchrotron beamline design:

```
# Energy scan for beamline components
energies = np.logspace(0, np.log10(30), 50)  # 1 keV to 30 keV
```

(continues on next page)

(continued from previous page)

```
result = xrt.calculate_single_material_properties("Si", energies, 2.33)
critical_angles = np.radians(result.critical_angle_degrees) * 1e3  # mrad
attenuation_lengths = result.attenuation_length_cm

# Plot or analyze the energy dependence
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.loglog(energies, critical_angles)
ax1.set_xlabel('Energy (keV)')
ax1.set_ylabel('Critical Angle (mrad)')
ax1.set_title('Critical Angle vs Energy')

ax2.loglog(energies, attenuation_lengths)
ax2.set_xlabel('Energy (keV)')
ax2.set_ylabel('Attenuation Length (cm)')
ax2.set_title('Attenuation Length vs Energy')

plt.tight_layout()
plt.show()
```

3.1.7 Understanding the Architecture

XRayLabTool uses a modular architecture:

backend/
: JAX runtime helpers: x64 setup, counted host/device transfers, shape buckets and the in-tree PCHIP interpolator. XRayLabTool runs on JAX only (ADR-006); `set_backend("jax")` is a no-op and `set_backend("numpy")` raises `ValueError`.

calculators/
: Core X-ray physics calculations and algorithms. Contains the main calculation functions, the `XRayResult` data structure, the resident scattering tables and the pure JAX device programs (`optics`).

data_handling/
: Atomic data caching and batch processing. Batches run as blocks of materials through one device program; there are no worker pools.

interfaces/
: User interfaces including the complete CLI and shell completion systems.

io/
: File operations and data export functionality. Handles CSV, JSON, and other format conversions.

validation/
: Input validation and error handling. Ensures data quality and provides detailed error messages.

gui/
: PySide6 desktop application with PyQtGraph interactive plots.

This modular design allows you to import only what you need:

```
# Import main calculation functions
from xraylabtool.calculators import calculate_single_material_properties

# Import specific utilities
from xraylabtool.utils import parse_formula, energy_to_wavelength
```

(continues on next page)

(continued from previous page)

```
# Pure device function (jit/vmap/grad-compatible) behind the calculators
from xraylabtool.calculators.optics import optical_properties

# Precompile the default programs once instead of on the first call
import xraylabtool
xraylabtool.warm_up(("SiO2",), energy_counts=(1, 500))  # also the energy buckets 64 and 512
```

3.1.8 Next Steps

Next steps:

1. **Explore the CLI**: Try all 9 commands with `xraylabtool --help`
2. **Read the Tutorials**: Learn techniques and workflows
3. **Study Examples**: See applications
4. **Check the API Reference**: View available functions
5. **Learn the Physics**: Understand the X-ray optics background

Key Documentation Sections:

- *CLI Reference* - Complete command-line interface documentation
- *Examples* - Real-world usage examples
- *API Reference* - Complete API reference
- *X-ray Physics* - X-ray physics background

3.1.9 Getting Help

If you encounter issues:

1. **Check the FAQ**: Common questions and solutions
2. **Read Error Messages**: XRayLabTool provides detailed error descriptions
3. **Use Help Commands**: `xraylabtool --help` and `xraylabtool <command> --help`
4. **Check Documentation**: This documentation covers most use cases
5. **Report Issues**: Use the GitHub issue tracker for bugs

Command-line help:

```
xraylabtool --help                    # General help
xraylabtool calc --help               # Help for calc command
xraylabtool list examples             # Show example materials
```

Python help:

```
import xraylabtool as xrt
help(xrt.calculate_single_material_properties)
```

```
# Or in IPython/Jupyter for interactive help
In [1]: xrt.calculate_single_material_properties?
```

3.1.10 Performance Tips

For better performance:

1. **Use preloaded elements**: Si, O, Al, Fe, C, etc. are cached for speed
2. **Batch processing**: Process multiple materials together when possible
3. **Energy arrays**: Use NumPy arrays for energy ranges

4. **Avoid repeated parsing**: Cache formula parsing results

```
# Good - one call, energy array, all materials
results = xrt.calculate_xray_properties(formulas, energies, densities)

# Less efficient - one call per (material, energy) pair
for formula, density in zip(formulas, densities):
    for energy in energies:
        result = xrt.calculate_single_material_properties(formula, energy, density)
```

3.2 CLI Reference

Command-line interface for X-ray property calculations and shell-completion management. Every option below is taken from `xraylabtool <command> --help`; run that for the authoritative list on your installed version.

Usage: `xraylabtool [GLOBAL OPTIONS] COMMAND [OPTIONS]`

Alias: `xtool` and `xlt` are short aliases for `xraylabtool` and accepts the same commands and options (e.g. `xtool calc Si -e 8.0 -d 2.33`). The desktop GUI is launched with `xraylabtool-gui` or its aliases `xtool-gui` and `xlt-gui`.

Commands: calc, batch, compare, convert, kk, formula, atomic, bragg, list, completion, install-completion, uninstall-completion

Global Options: `--version`, `--help`, `-v/--verbose`, `--debug`

> **Note**
>
> All energies are in **keV** (tabulated range 0.03–30 keV). `8000` triggers an out-of-range warning; use `8.0` for Cu Kα-like energies.

3.2.1 Energy specification

`-e/--energy` accepts:

- Single value: `10.0`
- Comma-separated: `5.0,10.0,15.0`
- Linear range with count: `5-15:11` (11 points from 5 to 15 keV)
- Log range with count: `1-30:100:log` (100 log-spaced points)

3.2.2 calc - Single Material Calculation

Usage: `xraylabtool calc FORMULA -e ENERGY -d DENSITY [OPTIONS]`

Options:

- `-e/--energy ENERGY` (required) — keV, see *Energy specification*
- `-d/--density DENSITY` (required) — g/cm^3
- `-o/--output FILE` — save to CSV or JSON (format inferred from extension)
- `--format {table,csv,json}` — stdout format (default: table)
- `--fields FIELDS` — comma-separated subset of result fields
- `--precision N` — decimal places
- `--debug`

```
xraylabtool calc Si -e 8.0 -d 2.33
xraylabtool calc SiO2 -e 5.0,8.0,10.0 -d 2.2 --format csv
xraylabtool calc Al -e 1-20:100 -d 2.70 -o al_sweep.csv
```

3.2.3 batch - Batch Processing

Usage: `xraylabtool batch INPUT_FILE -o OUTPUT [OPTIONS]`

CSV Format: `formula,density,energy` columns

Options:

- `-o/--output FILE` (required) — CSV or JSON by extension
- `--format {csv,json}` — override auto-detected format
- `--workers N` — accepted for compatibility and ignored (batches run as blocks on the JAX device, not in worker pools)
- `--fields FIELDS` — comma-separated subset of result fields
- `--progress` / `--no-progress` — progress bar control
- `--debug`

```
xraylabtool batch materials.csv -o results.csv
xraylabtool batch large_dataset.csv -o results.json --progress
```

3.2.4 compare - Multi-Material Comparison

Usage: `xraylabtool compare MATERIAL [MATERIAL ...] -e ENERGY [OPTIONS]`

Each `MATERIAL` is `formula,density` (e.g. `SiO2,2.2`).

Options:

- `-e/--energy ENERGY` (required)
- `--properties LIST` — comma-separated properties (default: all standard)
- `-o/--output FILE`, `--format {table,csv,json}`, `--precision N`
- `--report` — detailed comparison report

```
xraylabtool compare SiO2,2.2 Si3N4,3.2 -e 8.0,10.0,12.0 --properties dispersion_delta,absorption_beta
xraylabtool compare SiO2,2.2 Al2O3,3.95 -e 10.0 -o comparison.csv
```

3.2.5 convert - Unit Conversion

Usage: `xraylabtool convert {energy,wavelength} VALUES --to {energy,wavelength} [-o FILE]`

Energies in keV, wavelengths in Å. `VALUES` is a single number or a comma-separated list.

```
xraylabtool convert energy 8.0 --to wavelength
xraylabtool convert wavelength 1.55 --to energy
xraylabtool convert energy 5.0,8.0,10.0 --to wavelength
```

3.2.6 kk - Kramers-Kronig f1

Usage: `xraylabtool kk FORMULA [--nearedge FILE --input-type {nexafs,beta,asf}] [--energy-unit {eV, keV}] [--merge LOW,HIGH] [-d G_CM3] [--refine TOL] [--energies SPEC] [-o FILE] [--format {csv,json}]`

Calculates the real scattering factor f1 from the tabulated f2 by the Kramers-Kronig transform and prints `energy_keV,f1,f2` (CSV, or JSON with the same three keys). Without `--nearedge` this is the Kramers-Kronig-consistent f1 of the tabulated spectrum. With `--nearedge` the measured absorption data replace the tabulated f2 between the merge points after normalisation to it.

- `--input-type` is required with `--nearedge`: `nexafs` (photoabsorption cross-section values), `beta` (imaginary part of the refractive index) or `asf` (already f2).
- `--energy-unit` (default `eV`) is the unit of the near-edge file's first column and of `--merge`. `--energies` is always in keV.
- `--merge LOW,HIGH` must lie inside both the near-edge data and the tabulated range (10 eV to 500 keV); the default is their whole overlap.
- `--energies` takes the formats of `calc -e` (keV, 0.01 to 500). Without it the output grid is the spectrum's own knots within 0.03-30 keV.
- `--refine TOL` inserts midpoints until linear interpolation of f1 and f2 is within `TOL` electrons per formula unit; the output is then sorted and unique, and the number of points added is reported on stderr.
- `--density` is validated (finite, positive) and accepted for `beta` input; it cancels in the near-edge normalisation and does not change the result.
- Errors (bad file, missing `--input-type`, no overlap with the tabulated range, energies out of range) print to stderr and exit with code 1.

```
xraylabtool kk Si
xraylabtool kk SiO2 --energies 1.0,2.0,5.0
xraylabtool kk C --nearedge c_edge.txt --input-type nexafs
xraylabtool kk C --nearedge c_edge.txt --input-type nexafs --merge 285,315 --refine 0.05 -o␣
↪c_edge.csv
```

3.2.7 formula - Formula Analysis

Usage: `xraylabtool formula FORMULAS [-o FILE] [--debug]`

Prints element composition, atom counts, and molecular weight. `FORMULAS` may be comma-separated.

```
xraylabtool formula SiO2
xraylabtool formula "Ca5(PO4)3F"
xraylabtool formula SiO2,Al2O3 -o formulas.json
```

3.2.8 atomic - Atomic Data Lookup

Usage: `xraylabtool atomic ELEMENTS [-o FILE]`

```
xraylabtool atomic Si
xraylabtool atomic Si,O,Al -o elements.csv
```

3.2.9 bragg - Bragg Diffraction

Usage: `xraylabtool bragg -d DSPACING (-e ENERGY | -w WAVELENGTH) [--order N] [-o FILE]`

`DSPACING` in Å, single or comma-separated.

```
xraylabtool bragg -d 3.14 -e 8.0
xraylabtool bragg -d 3.14,1.92,1.64 -e 8.0
xraylabtool bragg -d 3.14 -w 1.55 --order 2
```

3.2.10 list - Reference Information

Usage: `xraylabtool list {constants,fields,examples}`

```
xraylabtool list constants   # physical constants used in calculations
xraylabtool list fields      # XRayResult field names for --fields
xraylabtool list examples    # example invocations
```

3.2.11 completion - Virtual Environment-Centric Shell Completion

Usage: `xraylabtool completion ACTION [OPTIONS]`

Installs per virtual environment and activates/deactivates with it.

```
xraylabtool completion install                  # current environment, auto-detected shell
xraylabtool completion install --shell zsh      # bash | zsh | fish | powershell
xraylabtool completion install --env myenv --force
xraylabtool completion list                     # environments with completion status
xraylabtool completion status                   # current environment
xraylabtool completion uninstall                # current environment
xraylabtool completion uninstall --all          # every environment
xraylabtool completion info                     # system information
```

Supported Environments: venv / virtualenv, conda / mamba, Poetry, Pipenv

Supported Shells: bash, zsh, fish, PowerShell

3.2.12 install-completion / uninstall-completion - Legacy Compatibility

Install: `xraylabtool install-completion [--user | --system] [--test]`

Uninstall: `xraylabtool uninstall-completion`

These legacy commands delegate to the `completion` backend:

```
xraylabtool install-completion           # current user (default)
xraylabtool install-completion --test    # verify completion works
xraylabtool uninstall-completion
```

3.2.13 Output Formats

`--format` selects the stdout format for `calc` and `compare`: `table` (default), `csv`, `json`. `-o/--output FILE` writes to disk with the format inferred from the extension.

```
xraylabtool calc Si -e 8.0 -d 2.33 --format csv
xraylabtool calc Si -e 8.0 -d 2.33 --format json
xraylabtool calc Si -e 8.0 -d 2.33 -o si.json
```

3.2.14 Error Handling

Invalid formulas and missing required arguments exit non-zero with a one-line message; energies outside 0.03–30 keV print a warning. Add `--debug` for a traceback.

3.2.15 Integration Examples

Shell Script:

```
for material in Si Al Cu; do
    xraylabtool calc $material -e 8.0 -d 2.33 --format csv >> results.csv
done
```

Python:

```
import json
import subprocess

result = subprocess.run(
    ["xraylabtool", "calc", "Si", "-e", "8.0", "-d", "2.33", "--format", "json"],
    capture_output=True,
    text=True,
```

(continues on next page)

(continued from previous page)

```
    check=True,
)
data = json.loads(result.stdout)
```

Performance Tips: Use `batch` for many materials; it runs them as blocks through one compiled program.

3.3 Shell Completion Guide

This comprehensive guide covers the new virtual environment-centric shell completion system for XRayLabTool.

3.3.1 Overview

XRayLabTool features a modern completion system that provides intelligent shell completion while respecting virtual environment boundaries. Unlike traditional system-wide completion, this system automatically activates and deactivates with your Python environments.

Key Features

Virtual Environment Isolation
: Completion is only available when the relevant virtual environment is active, preventing conflicts between projects.

Multi-Shell Support
: Native completion support for bash, zsh, fish, and PowerShell with shell-specific optimizations.

No System Modifications
: Installation requires no sudo privileges and makes no system-wide changes.

Automatic Management
: Completion automatically activates when you enter an environment and deactivates when you leave.

Environment Detection
: Supports venv, virtualenv, conda, mamba, Poetry, and Pipenv environments.

Command Aliases
: The same completion is registered for the `xtool` short alias as well as `xraylabtool`, so `xtool <TAB>` behaves identically. A single `xraylabtool completion install` covers both.

3.3.2 Quick Start

Basic Installation

```
# 1. Activate your virtual environment
conda activate myproject
# or: source venv/bin/activate
# or: poetry shell

# 2. Install completion in the environment
xraylabtool completion install

# 3. Verify installation
xraylabtool completion status

# 4. Test completion (start new shell if needed)
xraylabtool <TAB>
```

Environment Management

```
# List all environments with completion status
xraylabtool completion list

# Show detailed status for current environment
xraylabtool completion status

# Show system information
xraylabtool completion info

# Remove completion from current environment
xraylabtool completion uninstall

# Remove completion from all environments
xraylabtool completion uninstall --all
```

3.3.3 Shell-Specific Installation

The completion system auto-detects your shell, but you can specify explicitly:

Bash

```
# Install bash completion
xraylabtool completion install --shell bash

# Prerequisites (if not already installed)
# macOS (Homebrew)
brew install bash-completion@2

# Linux (Ubuntu/Debian)
sudo apt install bash-completion
```

Zsh

```
# Install native zsh completion
xraylabtool completion install --shell zsh

# Prerequisites (if not already installed)
# macOS (Homebrew)
brew install zsh-completions

# Add to ~/.zshrc if not present:
if type brew &>/dev/null; then
  FPATH="$(brew --prefix)/share/zsh-completions:${FPATH}"
  autoload -U compinit
  compinit
fi
```

Fish

```
# Install fish completion (no prerequisites required)
xraylabtool completion install --shell fish
```

PowerShell

```
# Install PowerShell completion (Windows, macOS, Linux)
xraylabtool completion install --shell powershell
```

3.3.4 Environment-Specific Workflows

Conda/Mamba Environments

```
# Create and activate environment
conda create -n xraywork python=3.12
conda activate xraywork

# Install XRayLabTool and completion
conda install -c imewei -c conda-forge xraylabtool
xraylabtool completion install

# Completion is now active
xraylabtool c<TAB>  # Completes to 'calc', 'completion', 'convert'

# Deactivate - completion unavailable
conda deactivate
xraylabtool c<TAB>  # No completion

# Reactivate - completion returns
conda activate xraywork
xraylabtool c<TAB>  # Completion works again
```

Venv/Virtualenv

```
# Create and activate venv
python -m venv myproject
source myproject/bin/activate

# Install and set up completion
pip install xraylabtool
xraylabtool completion install

# Test completion
xraylabtool <TAB>
```

Poetry Projects

```
# Create Poetry project
poetry new xray-analysis
cd xray-analysis

# Add XRayLabTool dependency
poetry add xraylabtool

# Activate Poetry shell
poetry shell

# Install completion in Poetry environment
xraylabtool completion install

# Completion works within Poetry shell
xraylabtool atomic <TAB>
```

Pipenv Projects

```
# Create Pipenv environment
mkdir xray-project && cd xray-project
pipenv install xraylabtool

# Activate Pipenv shell
pipenv shell

# Install completion
xraylabtool completion install

# Use completion
xraylabtool batch <TAB>
```

3.3.5 Advanced Configuration

Multiple Shells

You can install completion for multiple shells in the same environment:

```
# Install for multiple shells
xraylabtool completion install --shell bash
xraylabtool completion install --shell zsh
xraylabtool completion install --shell fish

# Check what's installed
xraylabtool completion status
```

Force Reinstallation

```
# Force reinstall (useful for updates)
xraylabtool completion install --force

# Or uninstall and reinstall
xraylabtool completion uninstall
xraylabtool completion install
```

3.3.6 Completion Features

The completion system provides intelligent suggestions for:

Command Completion

```
xraylabtool <TAB>
# Suggests: calc, batch, convert, formula, atomic, bragg, list, completion
```

Option Completion

```
xraylabtool calc --<TAB>
# Suggests: --material, --energy, --density, --output, --format, --help
```

File Path Completion

```
xraylabtool batch <TAB>
# Completes file paths in current directory
```

(continues on next page)

(continued from previous page)

```
xraylabtool calc Si --output <TAB>
# Completes file paths for output files
```

Value Completion

```
xraylabtool convert energy 8.0 --to <TAB>
# Suggests common energies: 8.048, 10.0, 12.4

xraylabtool calc <TAB>
# Suggests common formulas: Si, SiO2, Al2O3, etc.
```

3.3.7 Migration from Legacy System

If you previously used the old system-wide completion:

Removal of Old System

```
# Remove old system-wide completion (if installed)
# Check common locations:
ls /usr/local/share/bash-completion/completions/xraylabtool
ls ~/.bash_completion.d/xraylabtool

# Remove if found
sudo rm /usr/local/share/bash-completion/completions/xraylabtool
rm ~/.bash_completion.d/xraylabtool
```

Migration Steps

```
# 1. Update XRayLabTool to latest version
pip install --upgrade xraylabtool

# 2. For each environment where you want completion:
conda activate myenv
xraylabtool completion install

# 3. Legacy commands still work (using new backend)
xraylabtool install-completion    # Same as 'completion install'
```

Compatibility

The new system maintains backward compatibility:

```
# These legacy commands still work:
xraylabtool install-completion      # Uses new system
xraylabtool uninstall-completion    # Uses new system
xraylabtool --install-completion    # Uses new system

# But these new commands are recommended:
xraylabtool completion install      # More features
xraylabtool completion uninstall    # Better management
```

3.3.8 Troubleshooting

Completion Not Working

Check environment status:

```
xraylabtool completion status
```

Common issues:

1. **Environment not active**: Ensure your virtual environment is activated
2. **Need new shell**: Start a new shell session after installation
3. **Shell prerequisites**: Install bash-completion, zsh-completions, etc.

Debug steps:

```
# 1. Verify installation
xraylabtool completion status

# 2. Check if completion files exist
ls $VIRTUAL_ENV/share/xraylabtool/completion/

# 3. Force reinstall
xraylabtool completion install --force
```

Shell Prerequisites Missing

Bash:

```
# Test if bash-completion is available
type complete >/dev/null 2>&1 && echo "OK" || echo "Install bash-completion"

# Install if missing
# macOS: brew install bash-completion@2
# Linux: sudo apt install bash-completion
```

Zsh:

```
# Check if compinit is available
which compinit >/dev/null && echo "OK" || echo "Configure zsh completions"
```

Environment Detection Issues

```
# Check environment detection
xraylabtool completion list

# If your environment isn't detected, check:
echo $VIRTUAL_ENV      # For venv/virtualenv
echo $CONDA_PREFIX     # For conda/mamba
echo $POETRY_ACTIVE    # For Poetry
```

Permission Errors

The new system should never require sudo. If you see permission errors:

```
# Check that you're not trying to install system-wide
xraylabtool completion install  # Should work without sudo

# If you get permission errors, check environment activation
which python  # Should point to environment, not system
```

3.3.9 Performance and Caching

The completion system includes performance optimizations:

Caching System

- **Command caching**: Available commands are cached for faster access
- **Environment caching**: Detected environments are cached with timeout
- **Completion script caching**: Generated scripts are cached per shell

Cache Management

```
# Cache is automatically managed, but you can clear it if needed
# Cache location: ~/.xraylabtool/cache/

# Force cache refresh (completion will detect and update)
xraylabtool completion install --force
```

3.3.10 Best Practices

Development Workflow

```
# 1. Set up project environment
git clone myproject
cd myproject
python -m venv venv
source venv/bin/activate

# 2. Install dependencies including XRayLabTool
pip install -r requirements.txt
# (assuming xraylabtool is in requirements.txt)

# 3. Install completion for development
xraylabtool completion install

# 4. Now completion works during development
xraylabtool calc Si --energy <TAB>
```

Multiple Projects

```
# Each project can have its own completion environment
cd project1
conda activate project1-env
xraylabtool completion install

cd ../project2
conda activate project2-env
xraylabtool completion install

# Completion automatically switches between projects
```

Team Development

```
# Add completion setup to project documentation:
# In README.md or CONTRIBUTING.md:

# Setup Instructions:
```

(continues on next page)

(continued from previous page)

```
# 1. Install dependencies: pip install -r requirements.txt
# 2. Install completion: xraylabtool completion install
# 3. Start new shell or source shell config
```

CI/CD Considerations

```
# In CI environments, you typically don't need completion
# But if testing CLI features:

pip install xraylabtool
# Skip: xraylabtool completion install (not needed in CI)
xraylabtool calc Si --energy 10.0  # Test CLI functionality
```

3.3.11 API Reference

The completion system provides a programmatic API for advanced users:

Python API

```
from xraylabtool.interfaces.completion_v2 import (
    EnvironmentDetector,
    CompletionInstaller
)

# Detect environments
detector = EnvironmentDetector()
current = detector.get_current_environment()
all_envs = detector.discover_all_environments()

# Manage completion
installer = CompletionInstaller()
installer.install(shell='bash')
installer.list_environments()
```

Environment Information

```
# Get detailed environment information
from xraylabtool.interfaces.completion_v2 import EnvironmentDetector

detector = EnvironmentDetector()
env = detector.get_current_environment()

if env:
    print(f"Type: {env.env_type}")
    print(f"Path: {env.path}")
    print(f"Name: {env.name}")
    print(f"Has completion: {env.has_completion}")
    print(f"Python version: {env.python_version}")
```

3.3.12 See Also

- *CLI Reference* - Complete CLI command reference
- *Getting Started* - Basic installation and setup
- *XRayLabTool Documentation* - Main documentation index

For more help, use:

```
xraylabtool completion --help
xraylabtool completion install --help
xraylabtool completion list --help
```

3.4 Kramers-Kronig Guide

The `kk` tools compute the real part of the atomic scattering factor, `f1`, as the Kramers-Kronig transform of the imaginary part, `f2`. The input is either the tabulated `f2` alone or the tabulated `f2` with a measured near-edge spectrum spliced in. This is how you get `f1` (and hence δ) that is consistent with a measured NEXAFS spectrum near an absorption edge, where the tabulated values are too coarse. The method is described in *Kramers-Kronig Transform* and every function in *Kramers-Kronig (kk)*.

All three interfaces (Python, CLI, GUI) call the same function, *`kk_calculate_real()`*.

3.4.1 Python API

Tabulated spectrum only

Energies are in keV, as everywhere in the package:

```
>>> import numpy as np
>>> from xraylabtool.kk import kk_calculate_real
>>> res = kk_calculate_real("SiO2", energies_kev=[1.0, 8.0])
>>> np.round(res.f1, 4)
array([29.427 , 30.3636])
>>> np.round(res.f2, 4)
array([4.5794, 0.397 ])
>>> round(res.relativistic_correction, 4)
29.9771
```

Without `energies_kev` the result is on the spectrum's own knots, clipped to 0.03-30 keV (an INFO log line reports how many knots were dropped):

```
>>> grid = kk_calculate_real("SiO2")
>>> grid.energy_kev.size
1116
```

The result is a *`KKResult`* with the fields `formula`, `energy_kev`, `f1`, `f2`, `relativistic_correction`, `merge_points_kev`, `n_refined_points`, `backend`, `nearedge_scaled_kev_f2` and `refine_exhausted`. `to_xray_result(density)` turns it into an ordinary `XRayResult` (δ, β, critical angle, attenuation length, ...) computed from the KK `f1`. It accepts only energies in 0.03-30 keV:

```
>>> xr = res.to_xray_result(2.2)
>>> xr.critical_angle_degrees.shape
(2,)
```

With a near-edge spectrum

Load the measured file and say what kind of values it holds. The tabulated `f2` between the two merge points is replaced by the measured data, scaled and offset so that it meets the tabulated values at both merge points. For the polystyrene NEXAFS example distributed with upstream kkcalc (energies in eV):

```
from xraylabtool.kk import kk_calculate_real, load_nearedge

spec = load_nearedge("PS_004_-dc.txt", energy_unit="eV")
res = kk_calculate_real(
    "C8H8",
```

(continues on next page)

(continued from previous page)

```
    nearedge=spec,
    input_type="nexafs",
    merge_points_kev=(0.280, 0.320),  # keV in the Python API
    refine_tolerance=0.05,
)
energy_kev, f2_scaled = res.nearedge_scaled_kev_f2  # the data on the f2 scale
```

The same with a spectrum built in memory (`input_type="asf"` means the values are already on the `f2` scale):

```
>>> from xraylabtool.kk import NearEdgeSpectrum
>>> e = np.linspace(520.0, 560.0, 81)  # eV
>>> v = 1.0 + np.tanh((e - 535.0) / 2.0) + 2.0 * np.exp(-((e - 537.0) / 1.5) ** 2)
>>> spec = NearEdgeSpectrum(energy_ev=e, values=v)
>>> res = kk_calculate_real(
...     "SiO2",
...     nearedge=spec,
...     input_type="asf",
...     merge_points_kev=(0.525, 0.555),
...     energies_kev=[0.530, 0.537, 0.545],
... )
>>> np.round(res.f1, 3)
array([16.442, 14.934, 20.393])
>>> res.merge_points_kev
(0.525, 0.555)
```

`input_type` is required whenever `nearedge` is given (`TypeError` otherwise). If `merge_points_kev` is omitted, the whole overlap of the data and the tabulated range is used.

Adaptive refinement

`refine_tolerance` (electrons per formula unit, absolute) inserts midpoints wherever linear interpolation between neighbouring points misses `f1` or `f2` by more than the tolerance. It never removes points. The output is then sorted and unique:

```
>>> r = kk_calculate_real("SiO2", energies_kev=[0.5, 1.0, 2.0], refine_tolerance=0.05)
>>> r.n_refined_points, r.energy_kev.size
(119, 122)
```

`refine_max_passes` (default 50) and `refine_max_points` (default 100 000) bound the work. Hitting either limit is logged at WARNING.

Low-level functions

The steps can also be run one at a time, in eV: *`compound_spectrum()`* (coefficient table), *`relativistic_correction()`*, *`kk_real_spectrum()`* (`f1`), *`evaluate_f2()`*, *`to_asf_scale()`*, *`merge_spectra()`* and *`refine_spectrum()`*.

3.4.2 Command line

`xraylabtool kk` (also `xlt kk`) prints `energy_keV,f1,f2` as CSV, or JSON with the same keys. Every flag:

```
# Tabulated spectrum on its own grid within 0.03-30 keV
xraylabtool kk Si

# --energies: output energies, always keV (same formats as calc -e)
xraylabtool kk SiO2 --energies 1.0,8.0
xraylabtool kk SiO2 --energies 0.1-30:200:log
```

(continues on next page)

(continued from previous page)

```
# --nearedge + --input-type (required together); file energies in eV by default
xraylabtool kk C8H8 --nearedge PS_004_-dc.txt --input-type nexafs

# --merge LOW,HIGH in the file's unit (--energy-unit)
xraylabtool kk C8H8 --nearedge PS_004_-dc.txt --input-type nexafs --merge 280,320

# --energy-unit keV: the file's first column AND --merge are keV
xraylabtool kk C8H8 --nearedge ps_kev.txt --input-type nexafs --energy-unit keV --merge 0.28,0.32

# --input-type beta with -d/--density (accepted; it cancels, see below)
xraylabtool kk C8H8 --nearedge ps_beta.txt --input-type beta -d 1.05

# --refine TOL: insert midpoints; the count goes to stderr
xraylabtool kk C8H8 --nearedge PS_004_-dc.txt --input-type nexafs --refine 0.05

# -o/--output and --format (default: from the extension, else csv)
xraylabtool kk SiO2 --energies 1-10:10 -o sio2_kk.csv
xraylabtool kk SiO2 --energies 1-10:10 --format json
```

Example output:

```
$ xraylabtool kk SiO2 --energies 1.0,8.0
energy_keV,f1,f2
1.0,29.42698660162399,4.579444483959662
8.0,30.363551083333043,0.3970044362774753
```

Every usage or data error (missing `--input-type`, unreadable file, no overlap with the tabulated range, energies out of range) prints `Error: ...` to stderr and exits with code 1. *CLI Reference* lists the flags in the same format as the other commands.

Units

- `--energies` and every output are always **keV**.
- `--energy-unit` (default `eV`) applies only to the near-edge file's first column and to `--merge`.
- A keV file read as eV (energies near 0.28) does not overlap the 10 eV-500 keV table. The error message states the file's parsed range and the required range.
- Merge points that land within 1e-12 relative of an end of the data or of the table, which happens after a keV↔eV conversion, are snapped to that end instead of being rejected.

3.4.3 GUI

The **Kramers-Kronig** tab of `xraylabtool-gui` sits next to "Single Material" and "Multiple Materials".

1. Enter a **Formula** (default `SiO2`).
2. Optionally pick a **Near-edge file** with *Browse...*, then choose its **Input type** (`nexafs`, `beta`, `asf`) and **File energy unit** (`eV` or `keV`). Both are ignored without a file.
3. **Merge points**: `low, high` in the file's unit. Leave empty for the whole overlap. Merge points without a file are an error.
4. **Refine tolerance**: `Off` (0) disables refinement.
5. **Start energy**, **End energy** (0.01-500 keV, default 1-30 keV) and **Points** (default 500) define a linear output grid. The CXRO `f1` comparison curve is drawn only for energies inside 0.03-30 keV. When merge points are given, the merge window is also sampled with **Points** energies, even if it lies outside the start/end range, so a narrow near-edge region is never stepped over.

6. Click **Compute**. The calculation runs off the UI thread. An error is shown in the status bar, as a toast and in a message box.

The plot shows the measured `f2` (near-edge data on the `f2` scale), the spliced `f2`, the KK `f1` and, for comparison, the tabulated CXRO `f1`. The table lists energy, `f1` and `f2`. **Export CSV** writes `energy_keV,f1,f2` of the last result.

3.4.4 Near-edge file format

- Plain text with whitespace-separated columns. The **first** column is the photon energy and the **last** column is the value, so extra columns in between are ignored.
- Header lines before the first numeric row are skipped, as are comment lines starting with `#`, `%` or `;`. A non-numeric line after the data have started is an error that reports its line number. Upstream kkcalc silently drops such lines.
- At least two rows. Energies must be positive and strictly increasing, and all values finite. Each violation raises `NearEdgeDataError`.
- Input types: `nexafs` (also `photoabsorption`, `xanes` in Python) for absorption values proportional to the cross-section; `beta` for the imaginary part of the refractive index; `asf` for values already on the `f2` scale. The input type fixes the energy-dependent conversion (for `nexafs` the values are multiplied by E). Constant prefactors cancel in the affine match at the merge points, so for `beta` the density and formula mass do not change the result.

3.4.5 Behaviour decisions (D1-D7)

D1, transform
The fixed five-term kernel of upstream `KK_PP` (powers 1 to -3) is ported. The general-order variant is not ported, and there is no `orders` argument.

D2, data
Upstream's tabulated `f2` is used unchanged. For Sc, Mo, Ru, Be, Ti, Si, La, Gd and W it differs from the `.nff` tables used by `calc`, so `kk` and `calc` disagree on `f2` for those nine elements.

D3, energy range
The core accepts 10 eV-500 keV (the data range) and never extrapolates. The default output grid is clipped to 0.03-30 keV. Only `to_xray_result` enforces 0.03-30 keV.

D4, merge units
`--merge` uses the same unit as the file (`--energy-unit`). The Python API takes `merge_points_kev` in keV.

D5, knots and end points
At a tabulated knot where `f2` is continuous the result is the true principal value. At a jump in `f2` (only He, Ne, Ar, Kr in the data) and at the first and last knot, the integral diverges. There the result is upstream's finite value, a convention.

D6, negative f2
If the normalised near-edge data go negative, the values are kept and a WARNING reports how many there are and the minimum. They are never clipped. Non-finite values are rejected.

D7, refinement grid
With `refine_tolerance` the output is the sorted, unique union of the requested energies and the added midpoints. Without it, the output keeps the caller's order.

3.4.6 Device pipeline, warm-up and the compile cache

`xraylabtool.kk` is JAX-only: one `jax.numpy` implementation in float64 on the default JAX device (CPU, or the GPU of a CUDA install). `set_backend` has no effect on it. *`kk_calculate_real()`* uploads all inputs once, assembles and merges the spectrum on the device, fetches one small status record (the host raises the usual errors from it), evaluates f1 and f2, runs the refinement one device pass at a time and downloads the result once. Array shapes are rounded up to buckets, so a second call with other values and the same buckets compiles nothing.

The first call in a process compiles. Measured on a CPU, `SiO2` with `refine_tolerance=0.01`: about 1.3-1.5 s cold and 0.15-0.16 s warm (medians; the fastest warm calls were 0.10-0.12 s) (see *ADR-005: The Kramers-Kronig Package Is JAX-Only* for the table and its caveats). Two things reduce the cold cost:

- **Persistent compile cache.** The `xraylabtool kk` command and the GUI launcher enable JAX's on-disk compile cache in `xraylabtool/jax` under the OS user-cache directory (`$XDG_CACHE_HOME` or `~/.cache` on Linux, `~/Library/Caches` on macOS, `%LOCALAPPDATA%` on Windows). With a filled cache the cold refined run takes about 0.6 s. When it turns the cache on it also sets `jax_persistent_cache_min_compile_time_secs` to 0, because JAX's default of 1 s would skip nearly every kk program. It does nothing if you already set `jax_compilation_cache_dir` or if `XRAYLABTOOL_NO_JAX_CACHE=1`; a threshold you set yourself is kept. The library itself never enables the cache; to use it from Python set `jax_compilation_cache_dir` yourself.
- **Warm-up.** *`warm_up()`* compiles the programs a default calculation of the given formulas and a refined run touch. The GUI warms the programs of its own Compute click on an idle worker after the window is shown; the command line never warms up, so a one-shot call does not pay for formulas it was not asked for.

```
>>> from xraylabtool.kk import warm_up
>>> warm_up(("Si",), refine_tolerance=None)
```

The kk functions need 64-bit mode and switch `jax_enable_x64` on when they are called. A surrounding thread-local `jax.enable_x64(False)` context is not overridden: the call raises `TypeError` naming `jax_enable_x64`. Known deviations from the performance gates (CPU refined-warm, small-m GPU overhead, bare-library GPU cold start) are listed in *ADR-005: The Kramers-Kronig Package Is JAX-Only*.

Gradients

`Spectrum` (a padded piecewise-polynomial f2 table, a JAX pytree) and *`kk_f1()`* are pure functions compatible with `jax.jit`, `jax.vmap` and `jax.grad`. The gradient with respect to the energies agrees with a central difference:

```
>>> import jax
>>> import jax.numpy as jnp
>>> from xraylabtool.kk import Spectrum, kk_f1
>>> spec = Spectrum.from_numpy([2.0, 5.0], [[0.0, 1.0, 0.0, 0.0, 0.0]])
>>> def f1_at(e):
...     return kk_f1(e, spec, 0.0)
>>> e, h = jnp.array(10.0), 1e-4
>>> g = jax.grad(f1_at)(e)
>>> fd = (f1_at(e + h) - f1_at(e - h)) / (2 * h)
>>> bool(jnp.isfinite(g)) and abs(float(g - fd)) < 1e-9
True
```

and so does the gradient with respect to the coefficients (here the sensitivity of f1(10 eV) to the constant term `a0`, which equals f1 itself because f1 is linear in the coefficients; padded rows get gradient 0):

```
>>> def f1_of_coeffs(c):
...     return kk_f1(e, Spectrum(spec.seg_ev, c, spec.n_segments), 0.0)
>>> gc = jax.grad(f1_of_coeffs)(spec.coeffs)
>>> round(float(gc[0, 1]), 6), float(jnp.abs(gc[1:]).max())
(0.078578, 0.0)
```

3.4.7 Limits

- No `f2` contribution below 10 eV or above 500 keV, and no analytic high-energy tail. `f2` is zero outside the data (see the physics page for why 500 keV is the end).
- Upstream's "add background" and "fix distortions" options are not implemented.

- Heavy elements below about 20 eV: `f1` is good to only about 1e-4 absolute in float64, because the interval terms cancel. Upstream has the same limit.
- Refinement next to the 10 eV and 500 keV ends, where `f1` has a logarithmic singularity: a piece narrower than 2 % of its energy is left unrefined. The count is `RefineOutcome.n_pole_skipped` and is logged. *`KKResult`* does not carry the count. For Au the refined points equal those of an emulation of upstream's bisection. For Pb and U this rule stops a chain of bisections towards 500 keV that upstream runs to its recursion limit (Pb: 461 points added here, 505 in the emulation).
- Near-edge data whose values at the two merge points differ by less than 1e-6 of their size cannot be normalised and are rejected.
- Many negative `f2` points after a merge (the D6 WARNING) usually mean a wrong `--energy-unit`. An eV file declared `keV` still overlaps the table and is merged at 275-390 keV, outside the default output grid, so the output looks normal. The polystyrene example read that way gives 70 of 270 negative points.
- Within a few eV of 10 eV (and of 500 keV), `f1` is dominated by the truncation singularity of the integral, not by the data. For Au, `f1(10 eV) = -0.561` (the D5 end-point convention), `f1(10.0001 eV) = -4.645` and `f1(10.16 eV) = -0.70`.
- `-d/--density` is accepted for `beta` input but has no effect on the result.
- Without `--energies` (`energies_kev`), the output grid is the spectrum's knots inside 0.03-30 keV. If a merge leaves no knot there (for example, near-edge data spanning the whole 10 eV-500 keV range with only two points), this is an `EnergyError` (CLI exit code 1) asking for explicit energies; it is never an empty result.

3.4.8 Attribution

The `kk` module is a re-implementation of parts of the Kramers-Kronig Calculator (kkcalc), https://github.com/benajamin/kkcalc, at commit f331ca31a654af946f7d550a22cf4e9c11fa8afc. Copyright (c) 2013 Benjamin Watts, Daniel J. Lauk. It is distributed under the zlib/libpng (Zlib) licence. The files derived from kkcalc are marked "altered from the original". The vendored `xraylabtool/data/kkcalc/asf.npz` holds upstream's `ASF.json` values unmodified. The licence text ships as `xraylabtool/data/kkcalc/LICENSE-kkcalc.txt`, and `NOTICE` lists every derived file. The package as a whole is MIT-licensed and the conda recipe declares `MIT AND Zlib`.

Please cite the method as: B. Watts, *Opt. Express* **22**(19), 23628–23639 (2014), doi:10.1364/OE.22.023628.

3.4.9 References

- B. Watts, "Calculation of the Kramers-Kronig transform of X-ray spectra by a piecewise Laurent polynomial method", *Opt. Express* **22**(19), 23628–23639 (2014), doi:10.1364/OE.22.023628.
- kkcalc (commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), zlib/libpng licence, https://github.com/benajamin/kkcalc.

3.5 Examples

Real-world examples and use cases for XRayLabTool.

> **Note**
>
> Interactive examples will be added in future releases.

3.5.1 Interactive Examples

These examples include downloadable Jupyter notebooks and Python scripts demonstrating applications of XRayLabTool.

Quick Start Examples: - Interactive tutorial with examples - Simple calculations and common materials - Single energy calculations - Energy range analysis - Formula parsing and validation

Application Examples: - Beamline design and optimization - X-ray optics and substrate selection - Component characterization - High-throughput property screening

3.5.2 Example Categories

By Complexity:

Level	Topics Covered	Prerequisites
Beginner	Single materials, basic CLI usage	Python basics
Intermediate	Batch processing, energy analysis	NumPy, Matplotlib
Advanced	Optimization, integration workflows	Scientific computing

By Application Domain:

Domain	Example Applications
Synchrotron Science	Beamline design, monochromator selection, filter optimization
X-ray Optics	Mirror substrates, multilayer coatings, focusing elements
Materials Research	Property screening, comparative analysis, database studies
Education	Physics demonstrations, computational exercises

3.5.3 Notebook Links

These notebooks are provided as downloadable files (not embedded in the docs). Open them directly in Colab or nbviewer:

Notebook	Run in Colab	View (nbviewer)	Download
Getting Started	Open in Colab	nbviewer	Download (.ipynb)
Basic Examples	Open in Colab	nbviewer	Download (.ipynb)

3.5.4 Running the Examples

Download Notebooks: All examples are available as Jupyter notebooks:

```
# Clone the repository to get examples
git clone https://github.com/b80985/pyXRayLabTool.git
cd pyXRayLabTool/examples
```

Required Dependencies: Install additional packages for running examples:

```
pip install jupyter matplotlib pandas seaborn
```

Interactive Execution:

```
jupyter notebook examples/
```

Standalone Scripts: Many examples also work as standalone Python scripts:

```
python examples/basic_calculations.py
```

3.5.5 Example Data

Included Datasets: - `materials_database.csv` - Common materials with densities - `synchrotron_energies.csv` - Typical beamline energy ranges - `mirror_substrates.csv` - X-ray mirror material properties - `test_compounds.csv` - Chemical formulas for validation

Custom Data: Examples show how to: - Import your own material lists - Create custom energy ranges - Export results in various formats - Integrate with other analysis tools

3.5.6 Performance Demonstrations

Speed Comparisons: - Single vs batch processing efficiency - Energy array vs individual calculations - Memory usage optimization techniques - Cache performance benefits

Scaling Analysis: - Performance vs dataset size - Memory requirements for large calculations - Optimization strategies for different use cases

3.5.7 Best Practices

Examples demonstrate:

1. **Efficient Workflows:** - When to use batch processing - Energy range optimization - Memory management for large datasets
2. **Error Handling:** - Input validation strategies - Graceful error recovery - Results verification methods
3. **Data Management:** - Organizing material databases - Version control for datasets - Documentation and metadata
4. **Integration:** - Using XRayLabTool with other tools - Automation and scripting - Web application development

3.5.8 Contributing Examples

Community-contributed examples:

Submission Guidelines: 1. **Format**: Jupyter notebook with clear documentation 2. **Scope**: Focused on specific applications or techniques 3. **Testing**: Include data validation and expected results 4. **Documentation**: Explain the scientific context and methods

Example Template:

```
"""
Example: Silicon Properties Analysis

Description: Calculate X-ray properties for silicon substrate
Application: X-ray mirror design and optics
Level: Beginner
Dependencies: None (basic XRayLabTool functionality)
"""

import xraylabtool as xlt

# Calculate properties for silicon at 8 keV
result = xlt.calculate_single_material_properties("Si", 8.0, 2.33)
print(f"Critical angle: {result.critical_angle_degrees[0]:.3f}°")
```

Submission Process: 1. Fork the repository on GitHub 2. Add your example to the `examples/` directory 3. Include documentation and test data 4. Submit a pull request with description

3.5.9 Getting Help

For Example Issues: - Check the notebook outputs for expected results - Verify your XRayLabTool installation and version - Review the prerequisites and dependencies - Use GitHub Issues for bug reports

For Application Questions: - Join the discussion on GitHub Discussions - Reference the scientific literature cited in examples - Contact domain experts in the community

For Custom Applications: - Start with the closest existing example - Refer to the API documentation for detailed function information - Consider contributing your solution back to the community

3.6 X-ray Optics Fundamentals

This section provides the scientific background for X-ray optical property calculations performed by XRayLabTool.

3.6.1 Complex Refractive Index

For X-rays, materials have a complex refractive index:

$$n = 1 - \delta - i\beta$$

Where:

- **n**: Complex refractive index
- **δ (delta)**: Real part of refractive index decrement
- **β (beta)**: Imaginary part related to absorption
- **i**: Imaginary unit

Physical Interpretation

Real Part (δ): - Controls phase velocity: $v_{phase} = c/n_{real} = c/(1-\delta)$ - For X-rays, δ > 0, so phase velocity > c (but energy velocity < c) - Determines critical angle for total external reflection - Typically 10^{-6} to 10^{-4} for hard X-rays

Imaginary Part (β): - Controls absorption and attenuation - Related to absorption coefficient: $\mu = 4\pi\beta/\lambda$ - Determines penetration depth - Generally $\beta \ll \delta$ except near absorption edges

Calculation from Atomic Data

For a compound with multiple elements:

$$\delta = \frac{r_e \lambda^2}{2\pi} \sum_i n_i f_1^i$$

$$\beta = \frac{r_e \lambda^2}{2\pi} \sum_i n_i f_2^i$$

Where:

- $r_e = 2.818 \times 10^{-15}$ m (classical electron radius)
- λ: X-ray wavelength
- n_i: Number density of element i
- f_1^i, f_2^i: Atomic scattering factors for element i

3.6.2 Critical Angle for Total External Reflection

At grazing incidence, total external reflection occurs when:

$$\sin(\theta_c) = \sqrt{2\delta}$$

For small angles (θ_c in radians):

$$\theta_c \approx \sqrt{2\delta}$$

Converting to practical units:

$$\theta_c \text{ (degrees)} = \sqrt{2\delta} \times \frac{180}{\pi}$$
$$\theta_c \text{ (mrad)} = 1000 \times \sqrt{2\delta}$$

Physical Significance

- **Total reflection**: For angles θ < θc, X-rays are totally reflected
- **Mirror design**: Critical angle determines useful angular range
- **Energy dependence**: θc ∝ λ², so higher energies have smaller critical angles
- **Material choice**: Higher electron density → larger critical angle

3.6.3 Attenuation and Absorption

Linear Absorption Coefficient

The linear absorption coefficient relates to the imaginary part of the refractive index:

$$\mu = \frac{4\pi\beta}{\lambda}$$

With units of cm^{-1} (or m^{-1}).

Mass Absorption Coefficient

Often more convenient for comparing materials:

$$\mu/\rho = \frac{\mu}{\rho}$$

Where ρ is the material density, giving units of cm^2/g.

Attenuation Length

The 1/e attenuation length (distance for intensity to drop by factor e):

$$l_{att} = \frac{1}{\mu} = \frac{\lambda}{4\pi\beta}$$

Beer-Lambert Law

Intensity decreases exponentially with thickness:

$$I(t) = I_0 e^{-\mu t}$$

Where: - I_0: Initial intensity - t: Material thickness - μ: Linear absorption coefficient

3.6.4 Transmission and Reflection

Fresnel Equations for X-rays

For a smooth interface at grazing angle θ:

Reflectivity:

$$R = \left| \frac{n\cos\theta - \sqrt{1 - n^2\sin^2\theta}}{n\cos\theta + \sqrt{1 - n^2\sin^2\theta}} \right|^2$$

Transmission:

$$T = 1 - R \quad \text{(for non-absorbing case)}$$

For absorbing materials, both reflection and transmission are reduced, with energy lost to absorption.

3.6.5 Applications in Synchrotron Optics

Mirror Design

Substrate Selection: - Higher δ → larger critical angle → increased reflectivity at higher angles - Lower β → less absorption → higher throughput - Thermal properties important for high-power applications

Coating Optimization: - Multilayer coatings can enhance reflectivity - Periodic structures create artificial Bragg reflections - Material combinations: W/B_4C, Ni/C, Mo/Si

Beamline Components

Windows and Filters: - Balance between transmission and contamination protection - Optimize thickness: thin enough for transmission, thick enough for strength - Common materials: Be, diamond, SiN membranes

Monochromator Crystals: - Silicon most common due to crystal structure - Darwin width determines energy resolution - Thermal management crucial for stability

3.6.6 Energy Dependence

Absorption Edges

Near absorption edges, scattering factors show sharp changes:

- **Pre-edge**: Smooth energy dependence
- **Edge jump**: Sharp increase in f_2 (absorption)
- **Post-edge**: EXAFS oscillations in both f_1 and f_2

This creates opportunities and challenges: - Enhanced contrast near edges - Monochromator design must account for edge effects - Material choice depends on X-ray energy range

Scaling Laws

For energies well away from edges:

$$f_2 \propto Z^4/E^3$$
$$\delta \propto \lambda^2 \propto E^{-2}$$
$$\beta \propto \lambda^2 \propto E^{-2}$$

Therefore: - Critical angle decreases as E^{-1} - Attenuation length increases as E^3 - Higher energies are more penetrating

3.6.7 Practical Considerations

Surface Roughness

Real surfaces have roughness that reduces reflectivity:

$$R_{rough} = R_{smooth} \times e^{-(4\pi\sigma \sin\theta/\lambda)^2}$$

Where σ is the RMS surface roughness.

Contamination

Surface contamination (carbon, oxides) affects optical properties: - Reduces reflectivity - Changes effective critical angle - Time-dependent degradation in some environments

Temperature Effects

Thermal expansion changes: - Lattice spacing (important for crystals) - Surface figure (thermal distortion) - Bulk density (usually small effect)

3.6.8 Measurement and Characterization

Experimental Techniques

Reflectometry: - Measure reflectivity vs angle at fixed energy - Determine δ and β from curve fitting - Requires high-quality optical surfaces

Transmission Measurements: - Measure attenuation through known thickness - Direct determination of absorption coefficient - Easier for high-Z materials

Energy Scans: - Vary energy at fixed geometry - Map out absorption edge structure - Useful for identifying elemental composition

Data Sources

XRayLabTool uses atomic scattering factor data from:

1. **Henke Tables**: Widely used standard (10 eV - 30 keV)
2. **CXRO Database**: Extended energy range with updates
3. **NIST XCOM**: Photoabsorption cross-sections
4. **Theoretical calculations**: For very light elements or high energies

The data combines experimental measurements with theoretical calculations, with interpolation between tabulated values for smooth energy dependence.

3.6.9 Further Reading

Textbooks: - Als-Nielsen & McMorrow: "Elements of Modern X-ray Physics" - Attwood: "Soft X-rays and Extreme Ultraviolet Radiation" - Willmott: "An Introduction to Synchrotron Radiation"

Online Resources: - CXRO X-ray database: https://henke.lbl.gov/optical_constants/ - NIST XCOM database: https://www.nist.gov/pml/xcom-photon-cross-sections-database

3.7 Atomic Scattering Factor Data

Understanding the atomic data sources and interpolation methods used in XRayLabTool.

3.7.1 Atomic Scattering Factors

Definition

Atomic scattering factors describe how X-rays scatter from atoms:

$$f = f_0 + f' + if''$$

Where:

- **f_0**: Thomson scattering (classical, forward scattering)
- **f'**: Dispersion correction (real part)
- **f''**: Absorption (imaginary part)

For X-ray optics calculations, we use:

- **$f_1 = f_0 + f'$**: Total real part
- **$f_2 = f''$**: Imaginary part (absorption)

Physical Origin

Thomson Scattering (f_0): - Classical electron scattering - Energy-independent - Equals atomic number Z for forward scattering

Dispersion Correction (f'): - Quantum mechanical correction - Energy-dependent, especially near absorption edges - Can be positive or negative

Absorption (f''): - Photoabsorption cross-section - Always positive - Shows sharp edges at absorption thresholds

3.7.2 Energy Dependence

Away from Absorption Edges

For energies well away from absorption edges:

$$f' \approx -\frac{r_e mc^2}{2\pi} \sum_j \frac{\lambda^2 f_{j0}}{(\lambda^2 - \lambda_j^2)}$$

$$f'' \approx \frac{Z^4 \text{const}}{E^3}$$

Where: - λ_j: Absorption edge wavelengths - f_{j0}: Oscillator strengths - The f'' $\propto E^{-3}$ scaling is approximate

Near Absorption Edges

Near absorption edges, both f' and f'' show complex structure:

1. **Pre-edge region**: Smooth interpolation
2. **Edge jump**: Sharp discontinuity in f''
3. **Post-edge oscillations**: XANES and EXAFS structure

3.7.3 Data Sources

Henke Tables

Coverage: - Elements: H (Z=1) to U (Z=92) - Energy range: 10 eV to 30 keV - Energy spacing: Variable, denser near edges

Method: - Combines experimental photoabsorption data - Theoretical calculations for f' - Kramers-Kronig transformation ensures consistency

File Format: Standard .nff format with columns: - Energy (eV) - f_1 (real part) - f_2 (imaginary part)

CXRO Database

Extended Henke Tables: - Updated experimental data - Extended energy ranges for some elements - Web interface and downloadable files - Source: https://henke.lbl.gov/optical_constants/

Advantages: - Regular updates with new measurements - Quality control and validation - Widely accepted standard

NIST XCOM

Photoabsorption Data: - Primary source for absorption coefficients - Energy range: 1 keV to 100 GeV - Includes pair production and Compton scattering - Used to validate and extend other databases

3.7.4 Interpolation Methods

Linear Interpolation

XRayLabTool uses linear interpolation between tabulated values:

$$f(E) = f_1 + \frac{E - E_1}{E_2 - E_1}(f_2 - f_1)$$

This works well because: - Data points are closely spaced - Smooth variation between points - Computationally efficient

Logarithmic Interpolation

For some quantities, logarithmic interpolation may be more accurate:

$$\ln f(E) = \ln f_1 + \frac{\ln E - \ln E_1}{\ln E_2 - \ln E_1}(\ln f_2 - \ln f_1)$$

Used when: - Data spans many orders of magnitude - Exponential-like behavior expected - Higher accuracy needed

Spline Interpolation

For critical applications, spline interpolation provides: - Smooth derivatives - Better behavior near edges - Higher computational cost

3.7.5 Edge Handling

Absorption Edge Structure

Absorption edges create discontinuities in f'':

K-edge (1s electron): - Largest jump in f'' - Corresponding feature in f' - Most prominent for light elements

L-edges (2s, 2p electrons): - Multiple edges (L_1, L_2, L_3) - Fine structure from chemical environment - Important for medium-Z elements

M-edges and higher: - Many closely spaced edges - Complex fine structure - Important for heavy elements

Pre-edge Features

Near absorption edges: - **White lines**: Sharp peaks just above edge - **XANES**: X-ray Absorption Near Edge Structure - **Pre-edge peaks**: Forbidden transitions

These features contain chemical information but complicate optical calculations.

Kramers-Kronig Relations

The real and imaginary parts are related by:

$$f'(E) = \frac{2}{\pi} P \int_0^\infty \frac{\omega f''(\omega)}{\omega^2 - E^2} d\omega$$

Where P denotes the principal value. This ensures physical consistency.

3.7.6 Quality and Accuracy

Experimental Uncertainties

Photoabsorption Measurements: - Systematic errors: 2-5% typical - Statistical errors: 1-2% for good measurements - Sample contamination affects results - Temperature and pressure effects

Transmission Measurements: - Sample thickness uncertainty - Multiple scattering corrections - Surface oxidation effects - Grain size and texture effects

Theoretical Limitations

Isolated Atom Approximation: - Ignores chemical bonding effects - Assumes spherical atoms - No crystal field effects - Limited accuracy for light elements

Relativistic Effects: - Important for inner shells of heavy elements - Affects edge positions and intensities - Modern calculations include these

Validation Methods

Cross-checks between databases: - NIST XCOM vs Henke tables - Independent measurements - Sum rule tests

Experimental validation: - Reflectometry measurements - Transmission measurements - Interferometry techniques

3.7.7 Data Processing in XRayLabTool

Caching Strategy

XRayLabTool uses a multi-level caching system:

1. **Preloaded cache**: 92 common elements loaded at startup
2. **LRU cache**: Recently used interpolations cached
3. **Disk cache**: Computed values saved for reuse
4. **Memory management**: Automatic cleanup of old entries

Performance Optimization

Vectorized operations: - NumPy arrays for energy ranges - Batch interpolation for efficiency - SIMD operations where available

Smart interpolation: - Adaptive mesh refinement near edges - Coarse grids away from features - Error estimation and mesh adaptation

Error Estimation

XRayLabTool provides error estimates based on:

1. **Interpolation error**: From data spacing
2. **Experimental uncertainty**: From literature values
3. **Model limitations**: Isolated atom approximation
4. **Numerical precision**: Machine epsilon effects

3.7.8 Usage Guidelines

Energy Range Selection

Recommended ranges: - 100 eV - 30 keV: Henke data most reliable - 30-100 keV: Extrapolation, larger uncertainties - Below 100 eV: Strong chemical bonding effects

Avoiding problematic regions: - Very close to absorption edges (±10 eV) - Regions with sparse data coverage - Energies requiring large extrapolations

Material Considerations

Light elements (Z < 10): - Large relative bonding effects - Limited experimental data - Consider molecular form factors

Heavy elements (Z > 80): - Complex edge structure - Relativistic effects important - Multiple absorption edges

Compounds vs Elements: - Additivity assumption generally good - Chemical shifts usually small - Exceptions: strongly bonded materials

3.7.9 Future Developments

Database Updates

- New experimental measurements
- Improved theoretical calculations
- Extended energy ranges
- Better uncertainty estimates

Computational Improvements

- Machine learning interpolation
- Quantum mechanical calculations
- Many-body effects
- Temperature-dependent data

Integration Features

- Real-time database updates
- Quality metrics and validation
- User-contributed data
- Community feedback mechanisms

3.7.10 References

Primary Sources: - Henke, B.L., et al. “X-ray interactions: photoabsorption, scattering, transmission, and reflection at E=50-30000 eV, Z=1-92”, Atomic Data and Nuclear Data Tables 54, 181-342 (1993) - NIST XCOM: Photon Cross Sections Database - CXRO X-ray interactions database

Theoretical Background: - Bethe, H.A. & Salpeter, E.E. “Quantum Mechanics of One- and Two-Electron Atoms” - Brown, G.S. et al. “X-ray absorption spectroscopy and its applications”

3.8 Calculation Methods

Detailed explanation of the numerical methods and algorithms used in XRayLabTool.

3.8.1 Compound Analysis

Chemical Formula Parsing

XRayLabTool uses a parser for chemical formulas:

Grammar Rules: 1. Element symbols: Capital letter + optional lowercase letter 2. Stoichiometric coefficients: Integer numbers following elements 3. Parentheses: For grouping with multipliers 4. Hydration: Dot notation (·) for water molecules

Parser Algorithm:

```
FORMULA := TERM (TERM)*
TERM := ELEMENT COUNT? | '(' FORMULA ')' COUNT?
ELEMENT := [A-Z][a-z]?
COUNT := [0-9]+
```

Examples: - `SiO2` → {Si: 1, O: 2} - `Ca5(PO4)3F` → {Ca: 5, P: 3, O: 12, F: 1} - `CuSO4·5H2O` → {Cu: 1, S: 1, O: 9, H: 10}

Error Handling

The parser handles common errors:

Unknown Elements:

```
try:
    composition = parse_formula("XYZ")
except FormulaError as e:
    print(f"Error: {e}")
    # Error: Unknown element 'XYZ'
    # Suggestion: Check element symbols (case-sensitive)
```

Syntax Errors:

```
try:
    composition = parse_formula("Si-O2")
except FormulaError as e:
    print(f"Error: {e}")
    # Error: Invalid character '-' in formula
    # Suggestion: Use format like SiO2 or Al2O3
```

Number Density Calculation

For a compound with formula units per unit volume:

$$n_{total} = \frac{\rho N_A}{M}$$

Where: - ρ: Material density (g/cm³) - N_A: Avogadro's number - M: Molecular weight (g/mol)

Element-specific number densities:

$$n_i = n_{total} \times s_i$$

Where s_i is the stoichiometric coefficient of element i.

3.8.2 Atomic Scattering Factor Interpolation

Data Structure

Atomic data is stored as:

```
@dataclass
class AtomicData:
    element: str
    energies: np.ndarray      # Energy grid (eV)
    f1_values: np.ndarray     # Real scattering factors
    f2_values: np.ndarray     # Imaginary scattering factors
```

Linear Interpolation

For energy E between tabulated points E_i and E_{i+1}:

$$f_1(E) = f_{1,i} + \frac{E - E_i}{E_{i+1} - E_i}(f_{1,i+1} - f_{1,i})$$
$$f_2(E) = f_{2,i} + \frac{E - E_i}{E_{i+1} - E_i}(f_{2,i+1} - f_{2,i})$$

Implementation:

```
def interpolate_scattering_factors(element, energy):
    data = load_atomic_data(element)
    f1 = np.interp(energy, data.energies, data.f1_values)
    f2 = np.interp(energy, data.energies, data.f2_values)
    return f1, f2
```

Edge Handling

Special care near absorption edges:

Pre-edge extrapolation:

```
if energy < data.energies[0]:
    # Linear extrapolation from first two points
    slope = (data.f2_values[1] - data.f2_values[0]) / \
            (data.energies[1] - data.energies[0])
    f2 = data.f2_values[0] + slope * (energy - data.energies[0])
```

Post-edge extrapolation:

```
if energy > data.energies[-1]:
    # Power law extrapolation: f2 ∝ E^(-3)
    ratio = (energy / data.energies[-1]) ** (-3)
    f2 = data.f2_values[-1] * ratio
```

3.8.3 Complex Refractive Index Calculation

Individual Element Contributions

For each element i in the compound:

$$\delta_i = \frac{r_e \lambda^2}{2\pi} n_i f_{1,i}$$
$$\beta_i = \frac{r_e \lambda^2}{2\pi} n_i f_{2,i}$$

Where: - r_e = 2.818 × 10^{-15} m (classical electron radius) - λ: X-ray wavelength (m) - n_i: Number density of element i (m^{-3})

Total Compound Properties

Sum over all elements:

$$\delta_{total} = \sum_i \delta_i$$

$$\beta_{total} = \sum_i \beta_i$$

Implementation:

```
def calculate_compound_properties(composition, density, wavelength):
    delta_total = 0.0
    beta_total = 0.0

    molecular_weight = sum(ATOMIC_WEIGHTS[elem] * count
                           for elem, count in composition.items())
    number_density = (density * AVOGADRO) / molecular_weight  # molecules/cm³

    for element, count in composition.items():
        f1, f2 = interpolate_scattering_factors(element, energy)
        element_density = number_density * count * 1e6  # Convert to m⁻³

        delta_i = (CLASSICAL_ELECTRON_RADIUS * wavelength**2 *
                   element_density * f1) / (2 * np.pi)
        beta_i = (CLASSICAL_ELECTRON_RADIUS * wavelength**2 *
                  element_density * f2) / (2 * np.pi)

        delta_total += delta_i
        beta_total += beta_i

    return delta_total, beta_total
```

3.8.4 Derived Quantity Calculations

Critical Angle

From the refractive index decrement:

$$\theta_c = \sqrt{2\delta}$$

With unit conversions:

```
def calculate_critical_angle(delta):
    theta_rad = np.sqrt(2 * delta)
    theta_deg = theta_rad * 180 / np.pi
    theta_mrad = theta_rad * 1000
    return theta_rad, theta_deg, theta_mrad
```

Attenuation Coefficients

Linear absorption coefficient:

$$\mu = \frac{4\pi\beta}{\lambda}$$

Mass absorption coefficient:

$$\mu/\rho = \frac{\mu}{\rho}$$

Attenuation length:

$$l_{att} = \frac{1}{\mu}$$

Implementation:

```
def calculate_attenuation(beta, wavelength, density):
    mu_linear = 4 * np.pi * beta / wavelength  # m⁻¹
    mu_linear_cm = mu_linear / 100  # cm⁻¹
    mu_mass = mu_linear_cm / density  # cm²/g
    attenuation_length = 1 / mu_linear_cm  # cm
    return mu_linear_cm, mu_mass, attenuation_length
```

3.8.5 Numerical Considerations

Precision and Accuracy

Floating Point Precision: - Use 64-bit floats for intermediate calculations - Guard against underflow for small β values - Check for overflow in exponential calculations

Significant Figures: - Atomic data typically 3-4 significant figures - Final results should reflect input precision - Avoid false precision in output

Error Propagation:

```
def propagate_uncertainty(f1, f2, df1, df2):
    # δ and β uncertainties from f1, f2 uncertainties
    ddelta = df1 * (r_e * wavelength**2 * number_density) / (2 * np.pi)
    dbeta = df2 * (r_e * wavelength**2 * number_density) / (2 * np.pi)

    # Critical angle uncertainty
    dtheta = ddelta / np.sqrt(2 * delta)
    return ddelta, dbeta, dtheta
```

Vectorization

For efficiency with energy arrays:

```
def vectorized_calculation(energies, formula, density):
    """Calculate properties for array of energies."""
    energies = np.asarray(energies)
    results = []

    # Vectorize over energies for each element
    for element, count in composition.items():
        f1_array, f2_array = interpolate_scattering_factors(element, energies)
        # Process entire arrays at once

    return np.array(results)
```

Boundary Conditions

Energy limits:

```
def validate_energy(energy):
    if np.any(energy <= 0):
        raise EnergyError("Energy must be positive")
    if np.any(energy < MIN_ENERGY):
        warnings.warn(f"Energy below {MIN_ENERGY} eV may be unreliable")
    if np.any(energy > MAX_ENERGY):
        warnings.warn(f"Energy above {MAX_ENERGY} eV requires extrapolation")
```

Density validation:

```
def validate_density(density):
    if density <= 0:
```

(continues on next page)

(continued from previous page)

```
        raise ValidationError("Density must be positive")
    if density > MAX_REASONABLE_DENSITY:
        warnings.warn("Very high density - check units (g/cm³)")
```

3.8.6 Performance Optimizations

Caching Strategies

LRU Cache for Interpolation:

```
from functools import lru_cache

@lru_cache(maxsize=10000)
def cached_interpolation(element, energy_tuple):
    # Convert tuple back to array for interpolation
    energies = np.array(energy_tuple)
    return interpolate_scattering_factors(element, energies)
```

Precomputed Grids:

```
class PrecomputedGrid:
    def __init__(self, energy_min, energy_max, n_points):
        self.energy_grid = np.logspace(
            np.log10(energy_min), np.log10(energy_max), n_points
        )
        self.f1_grid = {}
        self.f2_grid = {}
        self._precompute_common_elements()
```

Memory Management

Chunked Processing:

```
def process_large_batch(materials, energies, chunk_size=1000):
    """Process large datasets in chunks to manage memory."""
    n_materials = len(materials)
    results = []

    for i in range(0, n_materials, chunk_size):
        chunk = materials[i:i+chunk_size]
        chunk_results = calculate_batch(chunk, energies)
        results.extend(chunk_results)

        # Optional: garbage collection
        if i % (chunk_size * 10) == 0:
            gc.collect()

    return results
```

Sparse Storage:

```
def store_sparse_results(results, threshold=1e-12):
    """Store only non-negligible values to save memory."""
    sparse_results = []
    for result in results:
        if result.beta > threshold:
            sparse_results.append(result)
    return sparse_results
```

Algorithm Complexity

Time Complexity: - Single calculation: O(N) where N is number of elements - Batch processing: O(M×N×E) where M=materials, E=energies - Interpolation: O(log K) where K is data points per element

Space Complexity: - Atomic data storage: O(K×Z) where Z=number of elements - Result storage: O(M×E) for batch calculations - Cache storage: O(C) where C is cache size

3.8.7 Validation and Testing

Unit Tests

Atomic Data Consistency:

```
def test_kramers_kronig_consistency():
    """Test that f' and f'' satisfy Kramers-Kronig relations."""
    # Implementation of discrete KK transform test
    pass

def test_sum_rules():
    """Test Thomas-Reiche-Kuhn sum rule."""
    # ∫ f''(E) dE should equal number of electrons
    pass
```

Physical Limits:

```
def test_physical_bounds():
    """Test that results are physically reasonable."""
    result = calculate_single_material_properties("Si", 8.0, 2.33)
    assert 0 < result.dispersion_delta[0] < 1e-3  # Reasonable range for δ
    assert 0 < result.absorption_beta[0] < result.dispersion_delta[0]  # Usually β << δ
    assert 0 < result.critical_angle_degrees[0] < 1  # Typical range
```

Integration Tests

Cross-validation with literature:

```
def test_literature_values():
    """Compare with published reference values."""
    # Silicon at 8 keV
    result = calculate_single_material_properties("Si", 8.0, 2.33)
    assert abs(result.critical_angle_degrees[0] - 0.2248) < 0.001
```

Consistency across energy ranges:

```
def test_energy_continuity():
    """Test smooth behavior across energy ranges."""
    energies = np.linspace(7.9, 8.1, 201)  # keV
    result = calculate_single_material_properties("Si", energies, 2.33)
    # Check for smooth derivatives, no discontinuities
```

3.8.8 Error Handling

Graceful Degradation

```
def robust_calculation(formula, energy_kev, density):
    try:
        return calculate_single_material_properties(formula, energy_kev, density)
    except AtomicDataError:
        # Fall back to approximate methods
```

(continues on next page)

(continued from previous page)

```
        return approximate_calculation(formula, energy_kev, density)
    except Exception as e:
        logger.error(f"Calculation failed: {e}")
        return None
```

User Feedback

```
def calculate_with_warnings(formula, energy_kev, density):
    warnings = []

    if energy_kev < 0.1:
        warnings.append("Low energy: results may be unreliable")
    if density > 20:
        warnings.append("High density: check units")

    result = calculate_single_material_properties(formula, energy_kev, density)
    result.warnings = warnings
    return result
```

3.9 Kramers-Kronig Transform

`xraylabtool.kk` computes the real part of the atomic scattering factor, f_1, from the imaginary part, f_2, with the piecewise Laurent polynomial method of Watts (2014). This page states the equations as the code implements them, where the implementation follows the paper, and where it departs from it. Equation numbers refer to Watts 2014 (see References). The API is described in *Kramers-Kronig (kk)* and usage in *Kramers-Kronig Guide*.

3.9.1 Scattering factors and the dispersion relation

In the Henke convention the refractive index is $n = 1 - \delta - i\beta$, with both parts proportional to sums of atomic scattering factors over the elements q of the material (see *X-ray Optics Fundamentals*):

$$\delta = \frac{r_e \lambda^2}{2\pi} \sum_q N_q f_{1,q}, \qquad \beta = \frac{r_e \lambda^2}{2\pi} \sum_q N_q f_{2,q}$$

f_1 and f_2 of one formula unit are related by the Kramers-Kronig transform (Watts 2014, Eq. (1)). Written with the sign used in the code and in its tests:

$$f_1(E) = Z^* + \frac{2}{\pi} \mathrm{P} \int_0^\infty \frac{x\, f_2(x)}{E^2 - x^2}\, \mathrm{d}x$$

where P is the Cauchy principal value and Z^* the relativistically corrected electron count (Eq. (9), below). With this sign a constant $f_2 = c$ on $[a, b]$ gives $f_1 - Z^* = (c/\pi) \ln|(E^2 - a^2)/(E^2 - b^2)|$, and a single Lorentz oscillator has $f_1 - Z^* < 0$ below resonance and > 0 above it; both closed forms are checked in `tests/unit/kk/test_transform.py`.

3.9.2 Piecewise Laurent representation

The tabulated f_2 of every element is stored as a polynomial in x and $1/x$ on each interval $x_j \le x < x_{j+1}$ (Watts 2014, Eq. (8)):

$$f_2(x) = a_{j,1}\, x + a_{j,0} + a_{j,-1}\, x^{-1} + a_{j,-2}\, x^{-2} + a_{j,-3}\, x^{-3}$$

The coefficient arrays have five columns in this order, `A_1, A_0, A_-1, A_-2, A_-3` (`coeffs[:, 0]` to `coeffs[:, 4]`), one row per interval, and one more knot than rows. Knots and coefficients are in **eV**: $a_{j,n}$ has units of electrons·eV^{-n}, so expressing x in keV would rescale each column by 1000^n. For this reason the transform runs in eV internally and the public functions convert keV inputs once, at the boundary.

A compound is the stoichiometric sum of its elements: the coefficient tables are re-cut on the union of all elemental knots and added with the formula counts (*`compound_spectrum()`*).

3.9.3 Closed form per interval

Because each term of Eq. (8) is a power of x, the integral over one interval has a closed form. For an energy E that is not a knot, the sum over the i intervals is (Watts 2014, Eq. (10), with the $a_{j,-3}$ term corrected as explained in the erratum below):

$$f_1(E) = Z^* + \frac{1}{\pi E}\sum_{j=1}^{i}\left\{\left(E^2 a_{j,1} - E a_{j,0} + a_{j,-1} - \frac{a_{j,-2}}{E} + \frac{a_{j,-3}}{E^2}\right)\ln\left|\frac{x_{j+1}+E}{x_j+E}\right| - \left(E^2 a_{j,1} + E a_{j,0} + a_{j,-1} + \frac{a_{j,-2}}{E} + \frac{a_{j,-3}}{E^2}\right)\ln\left|\frac{x_{j+1}-E}{x_j-E}\right| - 2E a_{j,1}\left(x_{j+1}-x_j\right) + \frac{2a_{j,-2}}{E}\ln\left|\frac{x_{j+1}}{x_j}\right| + \frac{2a_{j,-3}}{E}\left(x_j^{-1}-x_{j+1}^{-1}\right)\right\}$$

The same sum, arranged by coefficient (the layout of the paper's Table 1; every entry is multiplied by $1/(\pi E)$, a blank means the term does not occur and 0 means the contributions cancel):

Table 1: Coefficient of each term of the corrected Eq. (10)

	$x_{j+1}^2 - x_j^2$	$x_{j+1} - x_j$	$\ln\lvert\frac{x_{j+1}+E}{x_j+E}\rvert$	$\ln\lvert\frac{x_{j+1}-E}{x_j-E}\rvert$	$\ln\lvert\frac{x_{j+1}}{x_j}\rvert$	$x_{j+1}^{-1} - x_j^{-1}$
$a_{j,1}$	0	$-2E$	E^2	$-E^2$		
$a_{j,0}$		0	$-E$	$-E$		
$a_{j,-1}$			1	-1		
$a_{j,-2}$			$-1/E$	$-1/E$	$2/E$	
$a_{j,-3}$			$1/E^2$	$-1/E^2$	0	$-2/E$

`kk_piecewise_poly()` evaluates this sum. It keeps the arithmetic of upstream kkcalc's `KK_PP` term by term (decision D1) so that the two agree to rounding; an independent transcription of the corrected Eq. (10) agrees with it to 1.8e-8 (Si), 4.9e-8 (SiO_2), 9.6e-9 (C_8H_8 with near-edge data) and 3.6e-5 (Au) in absolute f_1 (`test_matches_paper_eq10_as_corrected`); the Au difference is rounding in the cancelling terms (see Accuracy).

Erratum: sign of the $a_{j,-3}$ term in the typeset Eq. (10)

The a_{-3} term of the published Eq. (10) has the wrong sign: the equation ends in $+\frac{2a_{j,-3}}{E}\left(x_{j+1}^{-1} - x_j^{-1}\right)$. Integrating the $a_{j,-3}$ term directly, with $x \cdot x^{-3}/(E^2 - x^2) = E^{-2}\left(x^{-2} + (E^2 - x^2)^{-1}\right)$, gives $+\frac{2a_{j,-3}}{E}\left(x_j^{-1} - x_{j+1}^{-1}\right)$ inside the bracket, as written above. The paper's own Table 1 has this sign ($-2/E$ in the $x_{j+1}^{-1} - x_j^{-1}$ column), and so does upstream `KK_PP`.

`test_printed_eq10_a3_sign_is_wrong` in `tests/unit/kk/test_transform.py` checks this against Cauchy quadrature of a pure $a_{j,-3}$ interval. The corrected term agrees to 1e-12 relative. The printed sign is wrong by 0.73, 0.067 and 0.015 at the three test energies. On the Au data the printed sign changes f_1 by up to 9.8e7 (at 10.3 eV). That figure was computed for this page with the test module's Eq. (10) transcription; the test does not assert it. The code implements the corrected term. In low-resolution renderings of the page the + in the subscript $j+1$ can look like a minus sign. The printed subscripts are $j+1$ and j, so the error is in the sign, not in the indices.

3.9.4 Singularities at knots and end points

The logarithms $\ln|x_j - E|$ diverge when E equals a knot. The paper handles an interior knot by treating the two intervals that meet there together (Watts 2014, Eqs. (5) and (6)). If $x f_2(x)$ is continuous at x_j, the divergent parts from the two sides are equal and opposite and cancel, leaving the finite principal value. That argument needs an interval on both sides and a continuous f_2. It therefore says nothing about the first or last knot of the data, or about a knot where f_2 jumps, and at those points the principal value itself diverges logarithmically.

The code applies these conventions (decision D5). The first one is a limit; the other two are conventions, not limits:

- **Interior knot, continuous** f_2: the two divergent logarithms are dropped and replaced by $P_j(x_j)\ln|(x_{j+1}-x_j)/(x_{j-1}-x_j)|$, with $P_j(E)=Ef_2(E)$ evaluated on the interval that starts at x_j. This is the principal value, continuous in E.
- **Interior knot where** f_2 **jumps**: the same expression, which gives upstream's finite value. Golden points pin it, e.g. Ar at 15.7 eV ($f_1=-12.653$) and Ne at 21.5 eV (-3.826).
- **First or last knot**: the boundary interval's divergent logarithm is dropped with no replacement, again upstream's finite value.

In the vendored data, absorption edges are steep but narrow intervals, so f_2 is continuous at them (relative gap below 1e-9). Real jumps occur only for He (24.5 eV), Ne (21.5 eV), Ar (15.7 eV) and Kr (14.0 eV), with negligible ones for Xe (12 eV, 3.7e-6 relative) and H. The default output grid therefore rarely lands on a jump. Energies passed in keV are multiplied by 1000 once. For about 1.6 % of tabulated knots that product is one ulp away from the knot, so the result is evaluated just beside the knot: this does not matter where f_2 is continuous, but at one of the jump knots above it returns a one-sided value instead of the D5 convention.

3.9.5 Integration limits

Eq. (1) integrates from 0 to infinity. Applied to the first and last intervals, the closed form stays finite only if the polynomials vanish appropriately (Watts 2014, discussion after Eq. (7)). The interval that starts at 0 must have $a_n = 0$ for every $n < 0$. The reason is not convergence of the integral, which holds for $n = -1$; it is that the polynomial describing f_2 in that interval ($F(x)$ in Watts 2014) must be finite at $x = 0$, as the Kramers-Kronig relations require (Watts 2014, discussion after Eq. (7)). The interval that extends to infinity must have $a_n = 0$ for every $n \geq -1$, so that the integrand decays fast enough.

The tables combine Henke data from 10 eV to 30 keV (point values joined by straight lines, i.e. only $a_{j,1}$ and $a_{j,0}$ nonzero) with Biggs-Lighthill photoabsorption fits up to 500 keV (terms of degree 0 to -3). Above 500 keV the Biggs-Lighthill polynomials have nonzero degree 0 and -1 terms, which breaks the condition for the last interval, so that range cannot be used. The vendored data and the transform therefore cover **10 eV to 500 keV**, with $f_2 = 0$ outside. *kk_calculate_real()* rejects requested energies outside that range with `EnergyError`, and nothing is extrapolated. Only *to_xray_result()* restricts energies further, to the package's 0.03-30 keV range (D3).

3.9.6 Relativistic correction and workflow

The integration constant is the high-energy limit of f_1 (Watts 2014, Eq. (9)), summed over the elements with counts n_q:

$$Z^* = \sum_q n_q \left(Z_q - \left(\frac{Z_q}{82.5} \right)^{2.37} \right)$$

It is added exactly once per formula unit, not once per element table.

The paper lists six steps for computing the scattering factors of a material. They map onto the package as follows:

Step	Paper	Implementation
1	Look up each element's f_2 and add them with the stoichiometry	*load_asf()*, *compound_spectrum()* (formula parsed by *xraylabtool.utils.parse_formula()*)
2	Convert a measured spectrum to scattering-factor form and merge it in	*load_nearedge()*, *to_asf_scale()*, *merge_spectra()*
3	Relativistic correction, Eq. (9)	*relativistic_correction()*
4	f_1 at the requested energies, Eq. (10)	*kk_piecewise_poly()*, called in validated chunks by *kk_real_spectrum()*
5	f_2 at the requested energies, Eq. (8)	*evaluate_f2()*
6	Return energies, f_1 and f_2 together	*kk_calculate_real()* returning *KKResult*

refine_spectrum() (adaptive insertion of midpoints) is upstream's `improve_accuracy` and is not part of the paper.

3.9.7 Accuracy

What the paper shows. Watts (2014) tests the method on the analytic model of Ohta and Ishida, an antisymmetric Lorentzian pair, approximated by polynomials with terms of 4th to 0th order. The rms deviation from the analytic transform over 1000-3000 cm^{-1} is 2.4e-7. That test needs positive powers up to x^4. The kernel here has the fixed five terms of Eq. (8) (D1), so it cannot reproduce that test, and no such claim is made.

What this implementation shows (measured values from the test suite):

- *Upstream parity.* Against golden outputs of kkcalc at the pinned commit (Si, Au, SiO_2, Ar, Ne, a synthetic spectrum, and polystyrene with its upstream near-edge example), the JAX kernel (the only implementation) agrees with f2 to 1e-12 relative (measured 4.3e-16) and with f1 to a relative tolerance of 1e-8 plus 32 times the rounding scale of the closed-form sum (the largest difference seen on the transform goldens is 2.1 times that scale, Au at 13.1 eV). The fused XLA expressions round differently from upstream's NumPy, which is why the tolerance has a rounding-scale term and why the JAX floor is `>=0.12.0.1` (the release the tolerances were measured on). The worst case is 6.0e-5 absolute (Au at 13.1 eV, $f_1 = -0.0204$).
- *Independent quadrature.* Each of the five basis terms on its own matches Cauchy principal-value quadrature to 8.4e-16 relative. A four-interval spectrum with all twenty coefficients nonzero matches to 7.8e-15. A constant band matches its closed form to 3.4e-15.
- *Lorentz oscillator.* A single oscillator sampled on 2000, 4000 and 8000 points reproduces the analytic f_1 to 6.0e-4, 1.55e-4 and 4.8e-5. The error falls as h^2, as expected for the discretisation error of the piecewise-linear input; it is not a property of the transform.
- *Tabulated CXRO f_1.* For the 83 elements outside the nine below, the transform of the tabulated f_2 was compared with the f_1 tables at their own energies (30 eV-30 keV, metric $|\Delta f_1| / \max(|f_1|, 1)$). The pooled median is 1.9e-4 and the 95th percentile 8.9e-3. The worst per-element median is 2.8e-3 (Lu). Points that sample an absorption edge deviate much more, by tens of percent at the worst edge points, because the tables sample the edge at discrete energies while the Henke data resolve it in a narrow interval. The test therefore bounds the distribution, not the maximum.
- *Conditioning.* For heavy elements at low energy the interval terms are far larger than their sum: for Au at 13 eV, terms near 7.9e12 add up to about 3e3. Below about 20 eV, f_1 of such elements is good to only about 1e-4 absolute in float64. Upstream has the same limit. The default output grid starts at 30 eV.

3.9.8 Truncating the integral at 30 keV

The 30-500 keV tables matter even when only energies below 30 keV are wanted. The figure shows the change in f_1 when f_2 is integrated only up to 30 keV instead of 500 keV. It is negligible for carbon, visible for iron and large for lead, the same trend the paper reports for these elements:

Table 2: $f_1^{\text{full}} - f_1^{\text{trunc}}$ (electrons per atom), pinned by `test_30kev_truncation_matches_paper_trend`

Element	0.1 keV	1 keV	10 keV	29 keV
C	-1.21e-4	-1.21e-4	-1.29e-4	-3.79e-4
Fe	-0.0968	-0.0969	-0.1027	-0.2831
Pb	-2.497	-2.497	-2.589	-5.229

The deficit is always negative and grows near the cut. The `.nff` tables used by `xraylabtool calc` stop at 30 keV, so they cannot supply this contribution. The `kk` module therefore carries its own data.

3.9.9 Data source and the nine differing elements

The transform uses upstream kkcalc's `ASF.json` unchanged, repacked to `xraylabtool/data/kkcalc/asf.npz` (decision D2). It is not rebuilt from the package's `.nff` files: those lack the 30-500 keV Biggs-Lighthill data, and a rebuild would break the comparison with upstream. For 83 elements

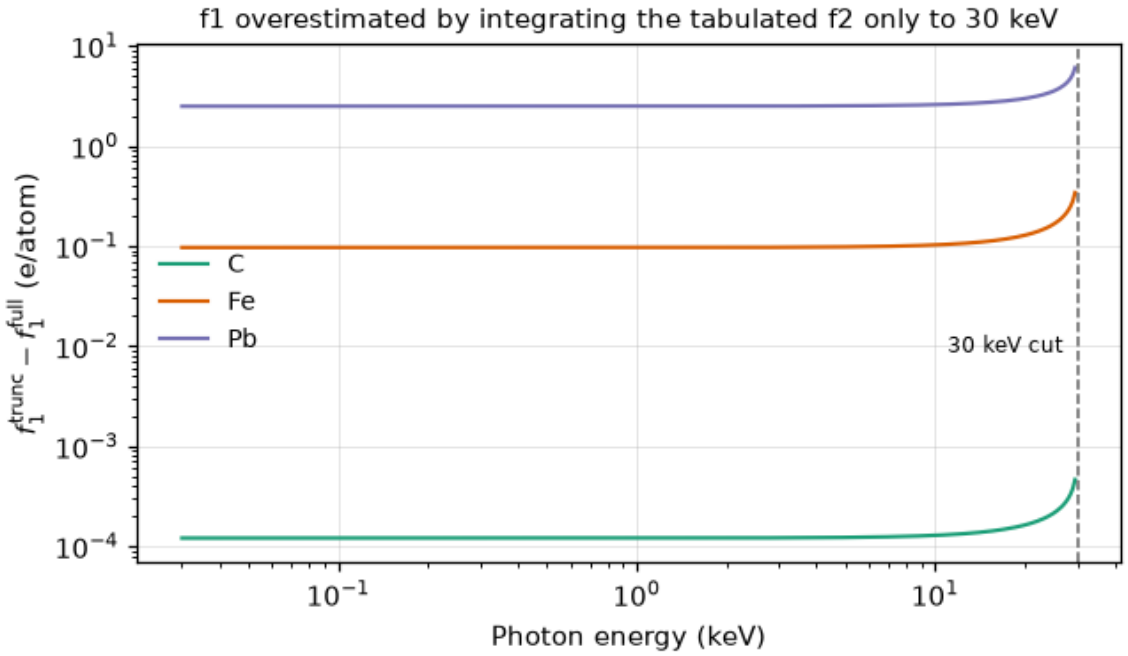


Fig. 1: $f_1^{\mathrm{trunc}} - f_1^{\mathrm{full}}$ against photon energy (log-log). Generated by `docs/_static/kk/make_truncation_figure.py` from this package's own code.

the two sources give the same f_2 at the `.nff` energies, with relative differences up to 3.8e-11 (Y). For nine elements they differ: **Sc, Mo, Ru, Be, Ti, Si, La, Gd and W**, with maximum relative deviations from 0.48 up to 7.4 (Sc). For these elements `xraylabtool kk` and `xraylabtool calc` give different f_2, and their KK f_1 is not compared with the tabulated f_1. The set is pinned by `test_nff_divergent_elements`.

3.9.10 References

- B. Watts, "Calculation of the Kramers-Kronig transform of X-ray spectra by a piecewise Laurent polynomial method", *Opt. Express* **22**(19), 23628–23639 (2014), doi:10.1364/OE.22.023628. Equations are re-typeset; the text and figures here are this project's own.
- Kramers-Kronig Calculator (kkcalc), https://github.com/benajamin/kkcalc, commit f331ca31a654af946f7d550a22cf4e9c11fa8afc, zlib/libpng licence (see *Kramers-Kronig Guide* and `NOTICE`).
- B. L. Henke, E. M. Gullikson and J. C. Davis, "X-ray interactions: photoabsorption, scattering, transmission, and reflection at E = 50-30,000 eV, Z = 1-92", *At. Data Nucl. Data Tables* **54**, 181-342 (1993).

3.10 API Reference

This section contains the API reference for XRayLabTool's modular architecture.

XRayLabTool is organized into focused sub-packages:

- **calculators**: Core X-ray physics calculations
- **data_handling**: Atomic data caching and batch processing
- **interfaces**: CLI and completion systems
- **validation**: Input validation and error handling
- **io_operations**: File operations and export functionality
- **analysis**: Material comparison and absorption edge detection

- **gui**: PySide6 desktop application
- **backend**: JAX runtime helpers, shape buckets and the in-tree PCHIP interpolator
- **kk**: Kramers-Kronig transform of f2 to f1 (kkcalc port, Watts 2014)
- **utils**: Utility functions and constants

3.10.1 Calculators Module

The calculators module contains the core X-ray physics calculations and the main result data structure.

Core Calculations

Core functionality for XRayLabTool.

This module is the orchestration layer for X-ray analysis. It re-exports domain types and cache utilities from focused submodules and contains the public API functions that wire everything together.

XRayResult fields (all per-energy fields are NumPy arrays aligned with `energy_kev`):

Field Name	Unit	Description
`formula`	•	Chemical formula of the material
`molecular_weight_g_mol`	g/mol	Molecular weight
`total_electrons`	•	Electrons per formula unit
`density_g_cm3`	g/cm³	Material density
`electron_density_per_ang3`	Å⁻³	Electron number density
`energy_kev`	keV	X-ray photon energies
`wavelength_angstrom`	Å	X-ray wavelengths
`dispersion_delta`	•	Real part of refractive index decrement (δ)
`absorption_beta`	•	Imaginary part of refractive index decrement (β)
`scattering_factor_f1`	electrons	Real atomic scattering factor summed over formula
`scattering_factor_f2`	electrons	Imaginary atomic scattering factor summed over formula
`critical_angle_degrees`	degrees	Critical angle for total external reflection
`attenuation_length_cm`	cm	1/e attenuation length
`real_sld_per_ang2`	Å⁻²	Real scattering length density
`imaginary_sld_per_ang2`	Å⁻²	Imaginary scattering length density

Legacy CamelCase accessors (`Formula`, `MW`, `Energy`, `Dispersion`, …) still resolve with a `DeprecationWarning`.

unique_result_key(*results*, *formula*, *density*)

Return a key for `formula` that never overwrites an earlier result.

The first occurrence keeps the plain formula; a repeat with another density becomes `formula@density` (with a `#n` suffix if even that is taken).

Return type

`str`

Parameters

- **results** (*Mapping[str, Any]*)
- **formula** (*str*)
- **density** (*float*)

calculate_multiple_xray_properties(
formula_list,
energy_kev,

mass_density_list,
)

Calculate X-ray optical properties for multiple chemical formulas.

Parameters

- **formula_list** (list[str]) – List of chemical formulas
- **energy_kev** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any]) – X-ray energies in keV (scalar, list, or array)
- **mass_density_list** (list[float]) – Mass densities in g/cm³

Return type
dict[str, dict[str, str | float | ndarray]]

Returns
Dictionary mapping formula strings to result dictionaries

Raises
ValueError – If input lists have different lengths or invalid values

calculate_single_material_properties(
formula,
energy_keV=None,
density=None,
* *(Keyword-only parameters separator (PEP 3102))*,
energy=None,
)

Calculate X-ray optical properties for a single material composition.

This is a pure function that calculates comprehensive X-ray optical properties for a single chemical formula at given energies and density. It returns an XRayResult dataclass containing all computed properties.

Parameters

- **formula** (str) – Chemical formula string (e.g., “SiO2”, “Al2O3”, “CaCO3”)
- **energy_keV** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any] | None) – X-ray energies in keV. Valid range: 0.03-30.0 keV
- **density** (float | floating[Any] | NDArray[] | None) – Material mass density in g/cm³ (must be positive)
- **energy** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any] | None) – Optional backward-compatible alias (eV). If provided, interpreted as electron-volts and converted to keV.

Returns
Dataclass containing all calculated X-ray properties.

Return type
XRayResult

Raises

- **FormulaError** – If chemical formula cannot be parsed or contains invalid elements
- **EnergyError** – If energy values are outside valid range (0.03-30.0 keV)
- **ValidationError** – If density is not positive or other validation failures

Examples

```
>>> import xraylabtool as xlt
>>> result = xlt.calculate_single_material_properties("SiO2", 8.0, 2.2)
>>> print(f"Formula: {result.formula}")
Formula: SiO2
```

See also

calculate_xray_properties : Calculate properties for multiple materials XRayResult : Complete documentation of returned dataclass

warm_up(*formulas=('SiO2',), energy_counts=(1,)*)

Precompile the device programs for `formulas` and energy counts.

Loads the resident scattering tables, then runs the single-material program and the supplied-factor program once per distinct element bucket of `formulas` and energy bucket of `energy_counts` at one material, so a later `calculate_single_material_properties` or `calculate_properties_from_factors` call whose energy count falls in a warmed bucket (up to 64 energies by default) compiles nothing. A new energy bucket costs a compile of 100-300 ms.

Parameters

- **formulas** (Sequence[str]) – Chemical formulas whose element counts select the programs to build.
- **energy_counts** (Sequence[int]) – Numbers of energies you expect to pass; each selects its bucket (*bucket_energy()*: powers of two up to 4096).

Raises

- **FormulaError** – If a formula cannot be parsed.
- **UnknownElementError** – If a formula names an element without tabulated data.
- **TypeError** – If an energy count is not an integer.
- **ValueError** – If `energy_counts` is empty or holds a count that is not positive.

Return type

None

References

Package addition; see `docs/superpowers/specs/2026-10-26-package-jax-first-design.md`.

Examples

```
>>> import xraylabtool as xlt
>>> xlt.warm_up(("SiO2",), energy_counts=(1, 500))
```

calculate_properties_from_factors(
formula,
energy_kev,
density,
f1_total,
f2_total,
)

Build an `XRayResult` from supplied scattering factors.

Computes δ, β and the derived quantities from per-formula-unit `f1` and `f2` that the caller provides instead of reading the tabulated `.nff` files. It runs the same code as `calculate_single_material_properties()` after its table lookup. This function is a package addition, not part of the cited databases or papers.

Parameters

- **formula** (str) – Chemical formula of one formula unit, e.g. `"SiO2"`.
- **energy_kev** (NDArray[]) – Photon energies in keV. Finite; the tabulated 0.03-30 keV range is not enforced here.
- **density** (float) – Mass density in g/cm^3; finite and positive.

- **f1_total** (NDArray[]) – Real scattering factor, electrons per formula unit. Negative values (anomalous dispersion near an edge) are kept: δ becomes negative and the critical angle is 0 there.
- **f2_total** (NDArray[]) – Imaginary scattering factor, electrons per formula unit. Negative values are kept; the attenuation length clamps β at `1e-30`.

Returns

All fields are arrays of shape `(m,)` except the scalar material properties; the order of `energy_kev` is preserved.

Return type

XRayResult

Raises

- **ValidationError** – If the three arrays are not non-empty 1-D arrays of equal length, any value is non-finite, or `density` is not finite and positive.
- **FormulaError** – If `formula` cannot be parsed or names an unknown element.

References

Package addition. The optical constants follow the CXRO/NIST convention used by `calculate_single_material_properties()`.

Examples

```
>>> import numpy as np
>>> from xraylabtool.calculators.core import calculate_properties_from_factors
>>> res = calculate_properties_from_factors(
...     "Si", np.array([8.0]), 2.33, np.array([14.2]), np.array([0.5])
... )
>>> res.scattering_factor_f1.shape
(1,)
>>> bool(res.dispersion_delta[0] > 0)
True
```

calculate_xray_properties(*formulas*, *energies*, *densities*)

Calculate X-ray optical properties for multiple material compositions.

Parameters

- **formulas** (list[str]) – List of chemical formula strings
- **energies** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any]) – X-ray energies in keV applied to all materials
- **densities** (list[float]) – List of material mass densities in g/cm³

Returns

Dictionary mapping formula strings to XRayResult objects.

Return type

dict[str, XRayResult]

Raises

- **ValueError** – If formulas or densities are empty or not lists, their lengths differ, a formula is not a non-empty string, or a density is not a positive number.
- **RuntimeError** – If no formulas were processed successfully

See also

calculate_single_material_properties : Single material calculations XRayResult : Documentation of returned data structure

class FastXRayCalculationEngine

Bases: object

High-performance X-ray calculation engine implementing CalculationEngine protocol.

__init__()

Initialize the calculation engine.

Return type
None

calculate_optical_constants(
formula,
energies,
density,
)

Calculate dispersion and absorption coefficients.

Return type
tuple[Any, Any]

Parameters

- **formula** (*str*)
- **energies** (*EnergyArray*)
- **density** (*FloatLike*)

calculate_derived_quantities(
dispersion,
absorption,
energies,
)

Calculate derived quantities from optical constants.

Return type
dict[str, ndarray]

Parameters

- **dispersion** (*Any*)
- **absorption** (*Any*)
- **energies** (*EnergyArray*)

warm_up_cache(*common_elements=None*)

Pre-warm caches for improved performance.

Return type
None

Parameters
common_elements (*list[str] | None*)

get_performance_info()

Get performance information about the calculation engine.

Return type
dict[str, Any]

get_calculation_engine()

Get the global calculation engine instance.

Return type
FastXRayCalculationEngine

Derived Quantities

Derived quantities calculator for X-ray optical properties.

This module contains functions for calculating derived X-ray properties such as critical angles, attenuation lengths, and other useful quantities. Each takes host arrays and returns NumPy arrays; the arithmetic runs in one JAX device program call (one upload, one fetch).

calculate_critical_angle(*dispersion_delta*)

Calculate critical angle for total external reflection.

Parameters
dispersion_delta (ndarray) – Dispersion coefficient δ

Return type
ndarray

Returns
Critical angle in degrees, `sqrt(2δ)·180/π`; NaN where δ < 0

calculate_attenuation_length(*wavelength_angstrom*, *absorption_beta*)

Calculate X-ray attenuation length (1/e length).

Parameters

- **wavelength_angstrom** (ndarray) – X-ray wavelength in Angstroms
- **absorption_beta** (ndarray) – Absorption coefficient β

Return type
ndarray

Returns
Attenuation length in centimeters; `inf` where β = 0

calculate_transmission(*thickness_cm*, *attenuation_length_cm*)

Calculate transmission through a material thickness.

Parameters

- **thickness_cm** (float) – Material thickness in centimeters
- **attenuation_length_cm** (ndarray) – Attenuation length in centimeters

Return type
ndarray

Returns
Transmission coefficient (0-1)

calculate_scattering_length_density(
dispersion_delta,
absorption_beta,
wavelength_angstrom,
)

Calculate real and imaginary scattering length densities.

Parameters

- **dispersion_delta** (ndarray) – Dispersion coefficient δ
- **absorption_beta** (ndarray) – Absorption coefficient β
- **wavelength_angstrom** (ndarray) – X-ray wavelength in Angstroms

Return type
tuple[ndarray, ndarray]

Returns
Tuple of (real_sld, imaginary_sld) in units of Å⁻²

calculate_scattering_factors(
energy_ev,
wavelength,
mass_density,
molecular_weight,
element_data,
)

Calculate X-ray scattering factors and properties of one material.

This function performs the core calculation of dispersion, absorption, and total scattering factors for a material based on its elemental composition. The PCHIP lookup and the arithmetic run in one device program call.

Parameters

- **energy_ev** (NDArray[]) – X-ray energies in eV (numpy array)
- **wavelength** (NDArray[]) – Corresponding wavelengths in meters (numpy array)
- **mass_density** (float | floating[Any] | NDArray[]) – Material density in g/cm³
- **molecular_weight** (float | floating[Any] | NDArray[]) – Molecular weight in g/mol
- **element_data** (list[tuple[float, *InterpolatorProtocol*, *InterpolatorProtocol*]]) – List of tuples (count, f1_interp, f2_interp) for each element; the interpolators must come from `create_scattering_factor_interpolators`

Return type
tuple[NDArray[], NDArray[], NDArray[], NDArray[]]

Returns
Tuple of (dispersion, absorption, f1_total, f2_total) NumPy arrays; NaN where an energy lies outside an element's table.

Raises

- **TypeError** – If an interpolator is not a `TableInterpolator` (an arbitrary callable is rejected before any entry is evaluated)
- **ValueError** – If an f1/f2 pair does not belong to one element

Mathematical Background: The dispersion and absorption coefficients are calculated using: - $\delta = (\lambda^2/2\pi) \times r_e \times \rho \times N_a \times (\Sigma_i n_i \times f1_i) / M$ - $\beta = (\lambda^2/2\pi) \times r_e \times \rho \times N_a \times (\Sigma_i n_i \times f2_i) / M$

Where: - λ: X-ray wavelength - r_e: Thomson scattering length - ρ: Mass density - N_a: Avogadro's number - n_i: Number of atoms of element i - $f1_i$, $f2_i$: Atomic scattering factors for element i - M: Molecular weight

calculate_derived_quantities(
wavelength,
dispersion,
absorption,
mass_density,
molecular_weight,
number_of_electrons,
)

Calculate derived X-ray optical quantities from dispersion and absorption.

Negative dispersion is physical near absorption edges (anomalous dispersion): the critical angle is 0 there, and β is clamped at `1e-30` in the attenuation length. This differs from the standalone helpers *calculate_critical_angle()* and *calculate_attenuation_length()*, which return NaN and `inf` at those points.

Parameters

- **wavelength** (NDArray[]) – X-ray wavelengths in meters (numpy array)
- **dispersion** (NDArray[]) – Dispersion coefficients δ (numpy array)
- **absorption** (NDArray[]) – Absorption coefficients β (numpy array)

- **mass_density** (float | floating[Any] | NDArray[]) – Material density in g/cm³
- **molecular_weight** (float | floating[Any] | NDArray[]) – Molecular weight in g/mol
- **number_of_electrons** (float | floating[Any] | NDArray[]) – Total electrons per molecule

Return type
tuple[float, NDArray[], NDArray[], NDArray[], NDArray[]]

Returns
Tuple of (electron_density, critical_angle, attenuation_length, re_sld, im_sld) - electron_density: Electron density in electrons/Å³ (scalar) - critical_angle: Critical angle in degrees (numpy array) - attenuation_length: Attenuation length in cm (numpy array) - re_sld: Real part of SLD in Å⁻² (numpy array) - im_sld: Imaginary part of SLD in Å⁻² (numpy array)

Raises
ValueError – If dispersion or absorption holds NaN or Inf

XRay Result

XRayResult dataclass for X-ray optical property calculations.

class XRayResult(
formula,
molecular_weight_g_mol,
total_electrons,
density_g_cm3,
electron_density_per_ang3,
energy_kev,
wavelength_angstrom,
dispersion_delta,
absorption_beta,
scattering_factor_f1,
scattering_factor_f2,
critical_angle_degrees,
attenuation_length_cm,
real_sld_per_ang2,
imaginary_sld_per_ang2,
)

Bases: object

Dataclass containing complete X-ray optical property calculations for a material.

This comprehensive data structure holds all computed X-ray properties including fundamental scattering factors, optical constants, and derived quantities like critical angles and attenuation lengths. All fields use descriptive snake_case names with clear units for maximum clarity.

The dataclass is optimized for scientific workflows, supporting both single-energy calculations and energy-dependent analysis. All array fields are automatically converted to numpy arrays for efficient numerical operations.

Legacy Compatibility: Deprecated CamelCase property aliases are available for backward compatibility but emit DeprecationWarning when accessed. Use the new snake_case field names for all new code.

Parameters

- **formula** (*str*)
- **molecular_weight_g_mol** (*float*)
- **total_electrons** (*float*)
- **density_g_cm3** (*float*)
- **electron_density_per_ang3** (*float*)

- **energy_kev** (*EnergyArray*)
- **wavelength_angstrom** (*WavelengthArray*)
- **dispersion_delta** (*OpticalConstantArray*)
- **absorption_beta** (*OpticalConstantArray*)
- **scattering_factor_f1** (*OpticalConstantArray*)
- **scattering_factor_f2** (*OpticalConstantArray*)
- **critical_angle_degrees** (*OpticalConstantArray*)
- **attenuation_length_cm** (*OpticalConstantArray*)
- **real_sld_per_ang2** (*OpticalConstantArray*)
- **imaginary_sld_per_ang2** (*OpticalConstantArray*)

Material Properties

formula

Chemical formula string exactly as provided

Type
str

molecular_weight_g_mol

Molecular weight in g/mol

Type
float

total_electrons

Total electrons per molecule (sum over all atoms)

Type
float

density_g_cm3

Mass density in g/cm^3

Type
float

electron_density_per_ang3

Electron density in electrons/Å^3

Type
float

X-ray Energy and Wavelength

energy_kev

X-ray photon energies in keV

Type
np.ndarray

wavelength_angstrom

Corresponding X-ray wavelengths in Å

Type
np.ndarray

Fundamental X-ray Properties

dispersion_delta
Dispersion coefficient δ (real part of refractive index decrement: n = 1 - δ - iβ)

Type
np.ndarray

absorption_beta
Absorption coefficient β (imaginary part of refractive index decrement)

Type
np.ndarray

scattering_factor_f1
Real part of atomic scattering factor

Type
np.ndarray

scattering_factor_f2
Imaginary part of atomic scattering factor

Type
np.ndarray

Derived Optical Properties

critical_angle_degrees
Critical angles for total external reflection in degrees (θc = √(2δ))

Type
np.ndarray

attenuation_length_cm
1/e penetration depths in cm

Type
np.ndarray

real_sld_per_ang2
Real part of scattering length density in Å⁻²

Type
np.ndarray

imaginary_sld_per_ang2
Imaginary part of scattering length density in Å⁻²

Type
np.ndarray

Physical Relationships:

- Refractive Index: n = 1 - δ - iβ where δ and β are wavelength-dependent
- Critical Angle: θc = √(2δ) for grazing incidence geometry
- Attenuation Length: μ^-1 = (4πβ/λ)^-1 for exponential decay
- Dispersion/Absorption: Related to f1, f2 via classical electron radius

Examples

Basic Property Access:

```
>>> import xraylabtool as xlt
>>> result = xlt.calculate_single_material_properties("SiO2", 10.0, 2.2)
>>> print(f"Material: {result.formula}")
Material: SiO2
```

(continues on next page)

(continued from previous page)

```
>>> print(f"MW: {result.molecular_weight_g_mol:.2f} g/mol")
MW: 60.08 g/mol
>>> print(result.critical_angle_degrees[0] > 0.1)  # Reasonable critical angle
True
```

Array Properties for Energy Scans:

```
>>> import numpy as np
>>> energies = np.linspace(8, 12, 5)
>>> result = xlt.calculate_single_material_properties("Si", energies, 2.33)
>>> print(f"Energies: {result.energy_kev}")
Energies: [ 8.  9. 10. 11. 12.]
>>> print(len(result.wavelength_angstrom))
5
```

Optical Constants Analysis:

```
>>> print(result.dispersion_delta.min() > 0)  # δ should be positive
True
>>> print(result.absorption_beta.min() >= 0)  # β should be non-negative
True
```

Derived Quantities:

```
>>> print(len(result.critical_angle_degrees))
5
>>> print(len(result.attenuation_length_cm))
5
```

Note

All numpy arrays have the same length as the input energy array. For scalar energy inputs, arrays will have length 1. Use standard numpy operations for analysis (e.g., np.min(), np.max(), np.argmin(), indexing).

See also

calculate_single_material_properties : Primary function returning this class calculate_xray_properties : Function returning Dict[str, XRayResult]

formula: str

molecular_weight_g_mol: float

total_electrons: float

density_g_cm3: float

electron_density_per_ang3: float

energy_kev: NDArray[float64]

wavelength_angstrom: NDArray[float64]

dispersion_delta: NDArray[float64]

absorption_beta: NDArray[float64]

scattering_factor_f1: NDArray[float64]

scattering_factor_f2: NDArray[float64]

critical_angle_degrees: NDArray[float64]

attenuation_length_cm: NDArray[float64]

real_sld_per_ang2: NDArray[float64]

imaginary_sld_per_ang2: NDArray[float64]

__post_init__()

Post-initialization to handle any setup after object creation.

Return type

None

property energy_ev

property delta

property beta

property critical_angle_mrad

property linear_absorption_coefficient

property Formula: str

Use 'formula' instead.

Type

Deprecated

property MW: float

Use 'molecular_weight_g_mol' instead.

Type

Deprecated

property Number_Of_Electrons: float

Use 'total_electrons' instead.

Type

Deprecated

property Density: float

Use 'density_g_cm3' instead.

Type

Deprecated

property Electron_Density: float

Use 'electron_density_per_ang3' instead.

Type

Deprecated

property Energy: ndarray

Use 'energy_kev' instead.

Type

Deprecated

property Wavelength: ndarray

Use 'wavelength_angstrom' instead.

Type

Deprecated

property Dispersion: ndarray

Use 'dispersion_delta' instead.

Type
Deprecated

property Absorption: ndarray

Use 'absorption_beta' instead.

Type
Deprecated

property f1: ndarray

Use 'scattering_factor_f1' instead.

Type
Deprecated

property f2: ndarray

Use 'scattering_factor_f2' instead.

Type
Deprecated

property Critical_Angle: ndarray

Use 'critical_angle_degrees' instead.

Type
Deprecated

property Attenuation_Length: ndarray

Use 'attenuation_length_cm' instead.

Type
Deprecated

property reSLD: ndarray

Use 'real_sld_per_ang2' instead.

Type
Deprecated

property imSLD: ndarray

Use 'imaginary_sld_per_ang2' instead.

Type
Deprecated

classmethod from_legacy(
formula=None,
mw=None,
number_of_electrons=None,
density=None,
electron_density=None,
energy=None,
wavelength=None,
dispersion=None,
absorption=None,
f1=None,
f2=None,
critical_angle=None,
attenuation_length=None,
re_sld=None,
im_sld=None,
***kwargs*,
)

Create XRayResult from legacy field names (for internal use).

Return type
XRayResult

Parameters

- **formula** (*str | None*)
- **mw** (*float | None*)
- **number_of_electrons** (*float | None*)
- **density** (*float | None*)
- **electron_density** (*float | None*)
- **energy** (*ndarray | None*)
- **wavelength** (*ndarray | None*)
- **dispersion** (*ndarray | None*)
- **absorption** (*ndarray | None*)
- **f1** (*ndarray | None*)
- **f2** (*ndarray | None*)
- **critical_angle** (*ndarray | None*)
- **attenuation_length** (*ndarray | None*)
- **re_sld** (*ndarray | None*)
- **im_sld** (*ndarray | None*)
- **kwargs** (*Any*)

__init__(
formula,
molecular_weight_g_mol,
total_electrons,
density_g_cm3,
electron_density_per_ang3,
energy_kev,
wavelength_angstrom,
dispersion_delta,
absorption_beta,
scattering_factor_f1,
scattering_factor_f2,
critical_angle_degrees,
attenuation_length_cm,
real_sld_per_ang2,
imaginary_sld_per_ang2,
)

Parameters

- **formula** (*str*)
- **molecular_weight_g_mol** (*float*)
- **total_electrons** (*float*)
- **density_g_cm3** (*float*)
- **electron_density_per_ang3** (*float*)
- **energy_kev** (*EnergyArray*)
- **wavelength_angstrom** (*WavelengthArray*)
- **dispersion_delta** (*OpticalConstantArray*)
- **absorption_beta** (*OpticalConstantArray*)

- **scattering_factor_f1** (*OpticalConstantArray*)
- **scattering_factor_f2** (*OpticalConstantArray*)
- **critical_angle_degrees** (*OpticalConstantArray*)
- **attenuation_length_cm** (*OpticalConstantArray*)
- **real_sld_per_ang2** (*OpticalConstantArray*)
- **imaginary_sld_per_ang2** (*OpticalConstantArray*)

Return type
None

Calculation Cache

Cache infrastructure for scattering factor data and interpolators.

class TableInterpolator(*element*, *z*, *field*)

Bases: object

f1 or f2 of one element, read from a row of the resident device tables.

Instances come from *create_scattering_factor_interpolators()*. Calling one evaluates the same PCHIP as the device program on an energy in eV (a scalar, a host array or a device array) and returns a device array, NaN outside the element's table. A host query costs one *xraylabtool.backend.runtime.upload()*, a device query uploads only the row and field indices; nothing is fetched. Each distinct query length compiles once.

Parameters

- **element** (*str*)
- **z** (*int*)
- **field** (*int*)

element

Element symbol.

Type
str

z

Row of the element in the tables (atomic number minus one).

Type
int

field

0 for f1, 1 for f2.

Type
int

__init__(*element*, *z*, *field*)

Parameters

- **element** (*str*)
- **z** (*int*)
- **field** (*int*)

Return type
None

get_cached_elements()

Get list of elements currently cached in the scattering factor cache.

Return type
list[str]

Returns
List of element symbols currently loaded in cache

get_bulk_atomic_data(*elements_tuple*)

Bulk load atomic data for multiple elements with high-performance caching.

This optimization uses a preloaded cache of common elements to eliminate expensive database queries to the Mendeleev library during runtime.

Parameters
elements_tuple (tuple[str, ...]) – Tuple of element symbols to load data for

Return type
MappingProxyType[str, MappingProxyType[str, float]]

Returns
Dictionary mapping element symbols to their atomic data

clear_scattering_factor_cache()

Clear the module-level scattering factor cache.

This function removes all cached scattering factor data from memory. Useful for testing or memory management.

Return type
None

is_element_cached(*element*)

Check if scattering factor data for an element is already cached.

Parameters
element (str) – Element symbol to check

Return type
bool

Returns
True if element data is cached, False otherwise

create_scattering_factor_interpolators(*element*)

Create the f1 and f2 interpolators of an element.

The returned callables read the element's PCHIP row of the resident device tables (see *TableInterpolator*); the `.nff` file is read and validated once, which also fills the element cache.

Parameters
element (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Ge')

Return type
tuple[*TableInterpolator*, *TableInterpolator*]

Returns
Tuple of (f1_interpolator, f2_interpolator) where each is a callable that takes energies in eV and returns interpolated scattering factors

Raises

- **FileNotFoundError** – If the .nff file for the element is not found
- **ValueError** – If the element symbol is invalid or data is insufficient

Examples

```
>>> from xraylabtool.calculators.cache import create_scattering_factor_interpolators
>>> import numpy as np
>>> f1_interp, f2_interp = create_scattering_factor_interpolators('Si')
>>> energy = 100.0  # eV
>>> f1_value = f1_interp(energy)
>>> bool(np.isfinite(np.asarray(f1_value)).all())  # a device array
True
>>> f2_value = f2_interp(energy)
>>> bool(np.isfinite(np.asarray(f2_value)).all())
True
>>> # Can also handle arrays
>>> energies = np.array([100.0, 200.0, 300.0])
>>> f1_values = f1_interp(energies)
>>> len(f1_values) == 3
True
```

Scattering Data

Scattering factor data loading and element path management.

load_scattering_factor_data(*element*)

Load f1/f2 scattering factor data for a specific element from .nff files.

This function reads .nff files using CSV parsing and caches the results in a module-level dictionary keyed by element symbol. Returns a pandas-compatible data structure for accessing columns E, f1, f2.

Parameters

element (str) – Element symbol (e.g., ‘H’, ‘C’, ‘N’, ‘O’, ‘Si’, ‘Ge’)

Returns

E (energy), f1, f2

Return type

Any

Raises

- **FileNotFoundError** – If the .nff file for the element is not found
- **ValueError** – If the element symbol is invalid, empty, or file format is invalid

Examples

```
>>> from xraylabtool.calculators.scattering_data import load_scattering_factor_data
>>> data = load_scattering_factor_data('Si')
>>> print(data.columns)
['E', 'f1', 'f2']
>>> print(len(data) > 100)  # Verify we have enough data points
True
```

class ScatteringData(*data_array*, *column_names*)

Bases: object

Pandas-like interface for scattering factor data arrays.

Parameters

- **data_array** (*np.ndarray*)
- **column_names** (*list[str]*)

__init__(*data_array*, *column_names*)

Parameters

- **data_array** (*ndarray*)
- **column_names** (*list[str]*)

Return type
None

class AtomicScatteringFactor

Bases: object

Class for handling atomic scattering factors.

This class loads and manages atomic scattering factor data from NFF files using the module-level cache.

__init__()

Return type
None

load_element_data(*element*)

Load scattering factor data for a specific element.

Parameters
element (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Ge')

Returns
E, f1, f2

Return type
Any

Raises

- **FileNotFoundError** – If the .nff file for the element is not found
- **ValueError** – If the element symbol is invalid

get_scattering_factor(*_element*, *q_values*)

Calculate scattering factors for given q values.

Parameters

- **element** – Element symbol
- **q_values** (ndarray) – Array of momentum transfer values
- **_element** (*str*)

Return type
ndarray

Returns
Array of scattering factor values

class CrystalStructure(*lattice_parameters*)

Bases: object

Class for representing and manipulating crystal structures.

Parameters
lattice_parameters (*tuple[float, float, float, float, float, float]*)

__init__(*lattice_parameters*)

Initialize crystal structure.

Parameters
lattice_parameters (tuple[float, float, float, float, float, float]) – (a, b, c, alpha, beta, gamma) in Angstroms and degrees

add_atom(*element*, *position*, *occupancy=1.0*)

Add an atom to the crystal structure.

Parameters

- **element** (str) – Element symbol
- **position** (tuple[float, float, float]) – Fractional coordinates (x, y, z)
- **occupancy** (float) – Site occupancy factor

Return type
None

calculate_structure_factor(*hkl*)

Calculate structure factor for given Miller indices.

F(hkl) = sum_j f_j * occ_j * exp(2*pi*i*(h*x_j + k*y_j + l*z_j))

where the sum runs over all atoms in the unit cell. The atomic form factor f_j is currently taken as 1.0 for all atoms (geometric structure factor).

Parameters
hkl (tuple[int, int, int]) – Miller indices (h, k, l)

Return type
complex

Returns
Complex structure factor

load_data_file(*filename*)

Load data from various file formats commonly used in X-ray analysis.

Parameters
filename (str) – Path to the data file

Return type
Any

Returns
DataFrame containing the loaded data

Resident Tables

Resident, device-side scattering-factor tables.

`xraylabtool/data/scattering_tables.npz` (built by `scripts/gen_scattering_tables.py` from the `.nff` files) is read once, uploaded once and cached for the process. The tables are a program *argument*, never a closed-over constant.

This module imports `jax` at module level, so callers import it inside functions; importing it has no side effect (no `ensure_x64()`).

References

Design: `docs/superpowers/specs/2026-10-26-package-jax-first-design.md` §2.

class ScatteringTables(*x*, *f1*, *f2*, *d1*, *d2*, *n*, *coef*, *symbols*)

Bases: object

Padded PCHIP tables for the 92 elements, in atomic-number order.

Parameters

- **x** (*Any*)
- **f1** (*Any*)

- **f2** (*Any*)
- **d1** (*Any*)
- **d2** (*Any*)
- **n** (*Any*)
- **coef** (*Any*)
- **symbols** (*tuple[str, ...]*)

x, f1, f2, d1, d2

Energy knots (eV), f1, f2 and their PCHIP node slopes. Past `n[z]` a row holds strictly increasing dummy knots, the last value and zero slopes, so a padded row evaluates like the unpadded one on `[x[0], x[n-1]]`.

Type
Array, float64, shape `[Z, N]`

n

Real knots per row.

Type
Array, int32, shape `[Z]`

coef

Per interval `i` of each row: `x[i]` and the power-basis cubic `c0 + c1 t + c2 t² + c3 t³` (`t = e - x[i]`) of f1 (entries 1-4) and f2 (entries 5-8), derived from the knots, values and slopes above so one gather fetches everything an energy needs. The device programs read only `x`, `coef` and `n`; `f1`, `f2`, `d1` and `d2` stay as the source of `coef` and for the `TableInterpolator` shims, so editing them without rebuilding `coef` changes nothing on the device.

Type
Array, float64, shape `[Z, N-1, 9]`

symbols

Element symbols (host metadata, not a pytree leaf).

Type
tuple of str

x: Any

f1: Any

f2: Any

d1: Any

d2: Any

__init__(*x*, *f1*, *f2*, *d1*, *d2*, *n*, *coef*, *symbols*)

Parameters

- **x** (*Any*)
- **f1** (*Any*)
- **f2** (*Any*)
- **d1** (*Any*)
- **d2** (*Any*)
- **n** (*Any*)
- **coef** (*Any*)
- **symbols** (*tuple[str, ...]*)

Return type
None

clear_tables()

Drop the resident tables; the next *load_tables()* reloads them.

Return type
None

element_index(*symbol*)

Row of `symbol` in the tables (atomic number minus one).

Raises
UnknownElementError – If `symbol` is not one of the 92 tabulated elements.

Return type
int

Parameters
symbol (*str*)

Examples

```
>>> from xraylabtool.calculators.tables import element_index
>>> element_index("Si")
13
```

load_tables()

Return the resident tables, reading and uploading them on first call.

Thread-safe: concurrent first calls perform a single `runtime.upload`.

References

Design spec §2.

Examples

```
>>> from xraylabtool.calculators.tables import load_tables
>>> load_tables().x.shape
(92, 449)
```

Return type
ScatteringTables

Device Programs

Pure JAX optics: PCHIP lookup of f1/f2 and the derived X-ray quantities.

The functions here are pure and `jit`/`vmap`/`grad`-compatible with x64 as a precondition (callers run *xraylabtool.backend.runtime.ensure_x64()* and the programs inside `jax.enable_x64(True)`). The scattering tables are a program *argument* (see *xraylabtool.calculators.tables*), never a closed-over constant. *encode_formula()* is the host-side half: it turns a formula into the integer/float arrays the device programs consume.

This module imports `jax` at module level, so callers import it inside functions; importing it has no side effect (no `ensure_x64()`).

References

Design: `docs/superpowers/specs/2026-10-26-package-jax-first-design.md` §4-§5. Fritsch, F. N. and Carlson, R. E. (1980), SIAM J. Numer. Anal. 17, 238 (PCHIP).

```
class OpticalProps(
    wavelength,
    dispersion,
    absorption,
    f1,
    f2,
    critical_angle,
    attenuation_length,
    re_sld,
    im_sld,
    electron_density,
)
```

Bases: `object`

Optical properties of one material at `M` energies.

Parameters

- **wavelength** (*Any*)
- **dispersion** (*Any*)
- **absorption** (*Any*)
- **f1** (*Any*)
- **f2** (*Any*)
- **critical_angle** (*Any*)
- **attenuation_length** (*Any*)
- **re_sld** (*Any*)
- **im_sld** (*Any*)
- **electron_density** (*Any*)

wavelength

Wavelength in angstrom.

Type
Array, shape `[M]`

dispersion, absorption

The refractive-index decrement δ and absorption index β.

Type
Array, shape `[M]`

f1, f2

Count-weighted sums of the atomic scattering factors.

Type
Array, shape `[M]`

critical_angle

Critical angle in degrees (0 where $\delta < 0$).

Type
Array, shape `[M]`

attenuation_length

1/e attenuation length in cm (β clamped at `1e-30`).

Type
Array, shape `[M]`

re_sld, im_sld

Real and imaginary scattering-length density in Å⁻².

Type
Array, shape [M]

electron_density

Electrons per Å³.

Type
Array, shape ()

dispersion: Any

absorption: Any

f1: Any

f2: Any

re_sld: Any

im_sld: Any

__init__(
wavelength,
dispersion,
absorption,
f1,
f2,
critical_angle,
attenuation_length,
re_sld,
im_sld,
electron_density,
)

Parameters

- **wavelength** (*Any*)
- **dispersion** (*Any*)
- **absorption** (*Any*)
- **f1** (*Any*)
- **f2** (*Any*)
- **critical_angle** (*Any*)
- **attenuation_length** (*Any*)
- **re_sld** (*Any*)
- **im_sld** (*Any*)
- **electron_density** (*Any*)

Return type
None

class ScatteringTables(*x*, *f1*, *f2*, *d1*, *d2*, *n*, *coef*, *symbols*)

Bases: object

Padded PCHIP tables for the 92 elements, in atomic-number order.

Parameters

- **x** (*Any*)
- **f1** (*Any*)

- **f2** (*Any*)
- **d1** (*Any*)
- **d2** (*Any*)
- **n** (*Any*)
- **coef** (*Any*)
- **symbols** (*tuple[str, ...]*)

x, f1, f2, d1, d2

Energy knots (eV), f1, f2 and their PCHIP node slopes. Past `n[z]` a row holds strictly increasing dummy knots, the last value and zero slopes, so a padded row evaluates like the unpadded one on `[x[0], x[n-1]]`.

Type
Array, float64, shape `[Z, N]`

n

Real knots per row.

Type
Array, int32, shape `[Z]`

coef

Per interval `i` of each row: `x[i]` and the power-basis cubic `c0 + c1 t + c2 t² + c3 t³` (`t = e - x[i]`) of f1 (entries 1-4) and f2 (entries 5-8), derived from the knots, values and slopes above so one gather fetches everything an energy needs. The device programs read only `x`, `coef` and `n`; `f1`, `f2`, `d1` and `d2` stay as the source of `coef` and for the `TableInterpolator` shims, so editing them without rebuilding `coef` changes nothing on the device.

Type
Array, float64, shape `[Z, N-1, 9]`

symbols

Element symbols (host metadata, not a pytree leaf).

Type
tuple of str

x: Any

f1: Any

f2: Any

d1: Any

d2: Any

__init__(*x*, *f1*, *f2*, *d1*, *d2*, *n*, *coef*, *symbols*)

Parameters

- **x** (*Any*)
- **f1** (*Any*)
- **f2** (*Any*)
- **d1** (*Any*)
- **d2** (*Any*)
- **n** (*Any*)
- **coef** (*Any*)
- **symbols** (*tuple[str, ...]*)

Return type
None

attenuation_length_from_beta(*wavelength_angstrom*, *beta*)

1/e attenuation length in cm, `λ/(4πβ)`; `inf` for β = 0.

Return type
Any

Parameters

- **wavelength_angstrom** (*Any*)
- **beta** (*Any*)

critical_angle_from_delta(*delta*)

Critical angle in degrees, `sqrt(2δ)·180/π`; NaN for δ < 0.

Return type
Any

Parameters
delta (*Any*)

derive(*energy_kev*, *f1_sum*, *f2_sum*, *rho*, *mw*, *n_electrons*)

Optical properties from count-weighted f1/f2 totals.

Reproduces, operation for operation, the former NumPy `core._properties_from_totals` and derived-quantity kernel: δ = ((λ²·S·ρ)·f1)/M, β likewise with f2, with `S = SCATTERING_FACTOR`. The legacy `λ²·(S·ρ/M)·f` is reassociated so that the division by `M` comes last: for the physical range, neither a huge `M` (`S·ρ/M` or `f/M` subnormal, flushed to zero by XLA) nor a tiny `f` underflows an intermediate. At the opposite extreme the product `(λ²·S·ρ)·f` can overflow (a factor sum of about 1e300 or more with a large `ρ`); the status check then reports the non-finite failure instead of returning a zero.

Parameters

- **energy_kev** (Any) – Energies in keV.
- **f1_sum** (Any) – `Σ counts·f1` and `Σ counts·f2`.
- **f2_sum** (Any) – `Σ counts·f1` and `Σ counts·f2`.
- **rho** (Any) – Density (g/cm³), molecular weight (g/mol) and electrons per formula unit.
- **mw** (Any) – Density (g/cm³), molecular weight (g/mol) and electrons per formula unit.
- **n_electrons** (Any) – Density (g/cm³), molecular weight (g/mol) and electrons per formula unit.

Return type
OpticalProps

encode_formula(*formula*)

Encode a formula for the device programs.

The molecular weight and electron count are accumulated as Python floats in the order of `core._calculate_molecular_properties`, so they are bit-identical to the values the NumPy-era path produced.

Parameters
formula (str) – Chemical formula, e.g. `"Ca5(PO4)3OH"`.

Return type
tuple[ndarray, ndarray, float, float]

Returns

- **z_idx** (ndarray, int32, shape `[K]`) – Table rows (atomic number minus one), one per distinct element.

- **counts** (ndarray, float64, shape `[K]`)
- **molecular_weight** (*float*) – g/mol.
- **number_of_electrons** (*float*)

Raises

- **`FormulaError`** – For an empty or malformed formula.
- **`UnknownElementError`** – For a symbol without tabulated data (the legacy message, where the element appears twice, comes from the atomic-data lookup).

Examples

```
>>> from xraylabtool.calculators.optics import encode_formula
>>> z, counts, mw, ne = encode_formula("SiO2")
>>> z.tolist(), counts.tolist(), ne
([13, 7], [1.0, 2.0], 30.0)
```

interpolate_f(*tables*, *z*, *counts*, *e_ev*)

Count-weighted f1 and f2 sums of a material at `e_ev`.

Parameters

- **`tables`** (*ScatteringTables*) – Resident tables from *load_tables()*.
- **`z`** (`Any`) – Table rows (atomic number minus one); padding rows use 0.
- **`counts`** (`Any`) – Atoms per formula unit; padding rows use 0 and contribute nothing, even where their own row would be NaN.
- **`e_ev`** (`Any`) – Energies in eV.

Returns

`Σ_k counts[k]·f1_k` (row 0) and `Σ_k counts[k]·f2_k` (row 1); NaN where any real element is outside its table.

Return type

`Any`

load_tables()

Return the resident tables, reading and uploading them on first call.

Thread-safe: concurrent first calls perform a single `runtime.upload`.

References

Design spec §2.

Examples

```
>>> from xraylabtool.calculators.tables import load_tables
>>> load_tables().x.shape
(92, 449)
```

Return type

ScatteringTables

optical_properties(
tables,
z,
counts,
rho,
mw,
n_electrons,
energy_kev,
)

Optical properties of one material; `vmap` over its arguments for a batch.

Pure and `jit`/`vmap`/`grad`-compatible with x64 as a precondition. With the other arguments fixed, `δ` is linear in `rho`, so `grad(rho -> optical_properties(...)[0].dispersion.sum())` equals `dispersion.sum() / rho` (the NaN-outside-range branch is not differentiated; gradients w.r.t. energy are not part of the contract).

Parameters

- **tables** (*ScatteringTables*) – From *load_tables()*.
- **z** (Any)
- **counts** (Any)
- **rho** (Any)
- **mw** (Any)
- **n_electrons** (Any)
- **energy_kev** (Any)

Return type

tuple[*OpticalProps*, Any]

Returns

- **props** (*OpticalProps*)
- **ok** (*Array, bool scalar*) – `all(isfinite(δ)) & all(isfinite(β))` and `S·ρ/M` finite.

References

Design spec §4-§5.

pack_materials(*encoded*, *densities*, *f_pad*, *k_pad*)

`[f_pad, row_width(k_pad)]` float64 rows for `_optical_program`.

Padded materials get `mw = rho = 1` and no electrons, padded elements `z = 0` with zero count. Atomic numbers are exact in float64.

Raises

ValueError – If a material has more than `k_pad` elements (it would overwrite its own counts) or there are more than `f_pad` materials.

Return type

ndarray

Parameters

- **encoded** (*Sequence[Any]*)
- **densities** (*Sequence[float]*)
- **f_pad** (*int*)
- **k_pad** (*int*)

transmission(*thickness_cm*, *attenuation_length_cm*)

Transmission `exp(-t/L)` through `thickness_cm` of material.

Return type

Any

Parameters

- **thickness_cm** (*Any*)
- **attenuation_length_cm** (*Any*)

Physics Background

The calculations are based on the complex refractive index for X-rays:

$$n = 1 - \delta - i\beta$$

Where:

- **δ (delta)**: Real part of the refractive index decrement, related to phase shifts
- **β (beta)**: Imaginary part, related to absorption

The critical angle for total external reflection is:

$$\theta_c = \sqrt{2\delta}$$

The linear absorption coefficient is:

$$\mu = \frac{4\pi\beta}{\lambda}$$

Usage Examples

Single Energy Calculation:

```
from xraylabtool.calculators.core import calculate_single_material_properties

result = calculate_single_material_properties("Si", 8.0, 2.33)  # formula, keV, g/cm³

print(f"Critical angle: {result.critical_angle_degrees[0]:.3f}°")
print(f"Attenuation length: {result.attenuation_length_cm[0]:.4f} cm")
```

Energy Array Calculation:

```
import numpy as np
from xraylabtool.calculators.core import calculate_single_material_properties

energies = np.logspace(0, np.log10(30), 100)  # 1 keV to 30 keV

# One call; every per-energy field comes back as an array of length 100
result = calculate_single_material_properties("Si", energies, 2.33)
print(result.critical_angle_degrees.shape)  # (100,)
```

Multiple Materials:

```
from xraylabtool.calculators.core import calculate_xray_properties

results = calculate_xray_properties(
    ["Si", "SiO2", "Al"], 8.0, [2.33, 2.20, 2.70]
)  # dict[str, XRayResult] keyed by formula
```

3.10.2 Data Handling Module

The data_handling module provides atomic data caching and batch processing capabilities.

Atomic Data Cache

High-performance atomic data cache system.

This module provides a pre-populated cache of atomic data for common elements to eliminate expensive database queries to the Mendeleev library during runtime.

get_atomic_data_fast(*element*)

Fast atomic data lookup with preloaded cache and fallback to Mendeleev.

This function first checks the preloaded cache, then the runtime cache, and only falls back to expensive Mendeleev queries as a last resort.

Parameters
element (str) – Element symbol (e.g., 'H', 'C', 'Si')

Return type
MappingProxyType[str, float]

Returns
Dictionary with 'atomic_number' and 'atomic_weight' keys

Raises
ValueError – If element symbol is not recognized

get_bulk_atomic_data_fast(*elements_tuple*)

High-performance bulk atomic data loader with caching.

This function loads atomic data for multiple elements efficiently, using the preloaded cache to avoid expensive database queries.

Parameters
elements_tuple (tuple[str, ...]) – Tuple of element symbols

Return type
MappingProxyType[str, MappingProxyType[str, float]]

Returns
Read-only mapping of element symbols to their atomic data (immutable views). The mapping is shared by every caller through the LRU cache, so it must not be mutable.

warm_up_cache(*elements*)

Pre-warm the cache with specific elements.

Parameters
elements (list[str]) – List of element symbols to preload

Return type
None

warm_cache_for_compounds(
formulas,
include_similar=True,
include_family=True,
timing_info=False,
)

Intelligently warm cache for compounds and their related elements.

This function performs intelligent cache warming by analyzing compound formulas, extracting their constituent elements, and pre-loading both atomic data and scattering factor interpolators. It can also include similar compounds and compound families for comprehensive warming.

Parameters

- **formulas** (list[str]) – List of chemical formulas to warm cache for
- **include_similar** (bool) – Whether to include similar compounds
- **include_family** (bool) – Whether to include compound family members
- **timing_info** (bool) – Whether to return timing information

Return type
dict[str, Any]

Returns
Dictionary with warming results and statistics

Examples

```
>>> from xraylabtool.data_handling.atomic_cache import warm_cache_for_compounds
>>> result = warm_cache_for_compounds(["SiO2", "Al2O3"])
>>> {"Si", "O", "Al"} <= set(result["elements_warmed"])  # plus related elements
True
>>> result["success_rate"] > 0.9
True
```

get_cache_stats()
Get cache statistics for monitoring.

Return type
dict[str, int]

Returns
Dictionary with cache statistics

is_element_preloaded(*element*)
Check if an element is in the preloaded cache.

Parameters
element (str) – Element symbol

Return type
bool

Returns
True if element is preloaded, False otherwise

class FastAtomicDataProvider
Bases: object

High-performance atomic data provider implementing AtomicDataProvider protocol.

This implementation uses preloaded atomic data and interpolated scattering factors for maximum performance in X-ray calculations.

__init__()
Initialize the atomic data provider.

Return type
None

get_scattering_factors(*element*, *energies*)
Get atomic scattering factors for element at given energies.

This method loads scattering factor data and interpolates it to the requested energies, returning complex scattering factors (f1 + if2).

Parameters
- **element** (str) – Chemical element symbol (e.g., 'Si', 'O')
- **energies** (NDArray[]) – X-ray energies in keV

Returns
Complex scattering factors (f1 + if2)

Return type
NDArray[]

is_element_cached(*element*)

Check if element data is cached for fast access.

Parameters

element (str) – Element symbol to check

Returns

True if element is cached for fast access

Return type

bool

preload_elements(*elements*)

Preload scattering factor data for elements.

Parameters

elements (list[str]) – List of element symbols to preload

Return type

None

get_atomic_properties(*element*)

Get basic atomic properties for an element.

Parameters

element (str) – Element symbol

Returns

Immutable mapping with atomic properties

Return type

MappingProxyType[str, float]

get_atomic_data_provider()

Get the global atomic data provider instance.

Returns

Shared atomic data provider instance

Return type

FastAtomicDataProvider

Usage Example

```
from xraylabtool.data_handling.atomic_cache import (
    get_atomic_data_fast,
    get_cache_stats,
    is_element_preloaded,
    warm_up_cache,
)

si = get_atomic_data_fast("Si")          # read-only mapping
print(si["atomic_number"], si["atomic_weight"])

print(is_element_preloaded("Si"))        # True; all 92 elements load at import
warm_up_cache(["Si", "O", "Al"])          # host atomic metadata only; xraylabtool.warm_up()␣
↪precompiles
print(get_cache_stats())
# {'preloaded_elements': 92, 'runtime_cached_elements': 0, 'total_cached_elements': 92}
```

Compound Analysis

Element combination analysis for intelligent cache warming.

This module analyzes common chemical compounds and their constituent elements to enable intelligent cache pre-loading based on typical usage patterns in materials science applications.

parse_chemical_formula(*formula*)

Parse a chemical formula into element counts.

Delegates to the canonical *xraylabtool.utils.parse_formula()* and rounds counts to integers.

Parameters
formula (str) – Chemical formula string (e.g., “Ca5(PO4)3OH”)

Return type
dict[str, int]

Returns
Dictionary mapping element symbols to their integer counts

Examples

```
>>> from xraylabtool.data_handling.compound_analysis import parse_chemical_formula
>>> parse_chemical_formula("SiO2")
{'Si': 1, 'O': 2}
>>> parse_chemical_formula("Ca5(PO4)3OH")
{'Ca': 5, 'P': 3, 'O': 13, 'H': 1}
```

get_elements_for_compound(*formula*)

Get the list of elements for a given compound formula.

Parameters
formula (str) – Chemical formula string

Return type
list[str]

Returns
List of unique element symbols in the compound

Examples

```
>>> from xraylabtool.data_handling.compound_analysis import get_elements_for_compound
>>> get_elements_for_compound("SiO2")
['Si', 'O']
>>> get_elements_for_compound("CaAl2Si2O8")
['Ca', 'Al', 'Si', 'O']
```

get_compound_frequency_score(*formula*)

Get a frequency score for a compound based on its commonality in materials science.

Parameters
formula (str) – Chemical formula string

Return type
float

Returns
Frequency score (0.0 to 1.0, higher = more common)

find_similar_compounds(*formula*, *similarity_threshold=0.5*)

Find compounds with similar element composition.

Parameters

- **formula** (str) – Target chemical formula
- **similarity_threshold** (float) – Minimum similarity score (0.0 to 1.0)

Return type
list[str]

Returns
List of similar compound formulas

get_compound_family(*formula*)

Determine which compound family a formula belongs to.

Parameters
formula (str) – Chemical formula string

Return type
str | None

Returns
Family name or None if not found

get_recommended_elements_for_warming(
recent_compounds,
max_elements=20,
)

Get recommended elements for cache warming based on recent compound usage.

Parameters

- **recent_compounds** (list[str]) – List of recently used compound formulas
- **max_elements** (int) – Maximum number of elements to recommend

Return type
list[str]

Returns
List of element symbols ranked by importance for warming

analyze_element_associations(*compound_list*)

Analyze which elements are frequently associated together.

Parameters
compound_list (list[str]) – List of compound formulas to analyze

Return type
dict[str, list[str]]

Returns
Dictionary mapping elements to their frequently associated elements

get_compound_complexity_score(*formula*)

Calculate a complexity score for a compound (more complex = higher score).

Parameters
formula (str) – Chemical formula string

Return type
float

Returns
Complexity score (higher values indicate more complex compounds)

Drives formula-aware cache warming: groups compounds into families (silicates, oxides, ...) and recommends which elements to preload for a set of recently used formulas.

```
from xraylabtool.data_handling.atomic_cache import warm_cache_for_compounds
from xraylabtool.data_handling.compound_analysis import (
    get_compound_family,
    get_recommended_elements_for_warming,
)

get_compound_family("SiO2")                              # 'silicates'
get_recommended_elements_for_warming(["SiO2", "Al2O3"])    # ['O', 'Si', 'Al', 'N', ...]
warm_cache_for_compounds(["SiO2", "Al2O3"], include_similar=True)
```

Batch Processing

High-performance batch processing module for X-ray calculations.

This module provides batch processing with memory management and progress tracking for large-scale X-ray property calculations. Materials run as `F`-blocks of one JAX device program (a `vmap` over materials); there are no worker pools.

class BatchConfig(
max_workers=None,
chunk_size=100,
memory_limit_gb=4.0,
enable_progress=True,
cache_results=False,
)

Bases: `object`

Configuration for batch processing operations.

Parameters

- **max_workers** (`int` | `None`) – Accepted and ignored (batches run on the device, not in worker pools); kept so existing configurations keep working
- **chunk_size** (`int`) – Number of calculations per chunk, and the upper bound of materials per device block (default: 100)
- **memory_limit_gb** (`float`) – Memory limit in GB before forcing garbage collection
- **enable_progress** (`bool`) – Whether to show progress bars
- **cache_results** (`bool`) – Whether to cache intermediate results

max_workers: int | None = None

chunk_size: int = 100

memory_limit_gb: float = 4.0

enable_progress: bool = True

cache_results: bool = False

__post_init__()

Initialize the configuration after object creation.

Return type
`None`

__init__(
max_workers=None,
chunk_size=100,
memory_limit_gb=4.0,
enable_progress=True,
cache_results=False,
)

Parameters

- **max_workers** (*int | None*)
- **chunk_size** (*int*)
- **memory_limit_gb** (*float*)
- **enable_progress** (*bool*)
- **cache_results** (*bool*)

Return type
None

class MemoryMonitor(*limit_gb=4.0*)

Bases: object

Memory usage monitor for batch operations.

Parameters
limit_gb (*float*)

__init__(*limit_gb=4.0*)

Initialize the memory monitor.

Parameters
limit_gb (*float*)

check_memory()

Check if memory usage is below limit.

Return type
bool

Returns
True if within limits, False if exceeded

get_memory_usage_mb()

Get current memory usage in MB.

Return type
float

Returns
Memory usage in megabytes

force_gc()

Run garbage collection.

The resident scattering tables and the per-formula encoding memo are kept: dropping them would force a table re-upload mid-batch.

Return type
None

chunk_iterator(*data*, *chunk_size*)

Yield successive chunks of data.

Parameters

- **data** (list[tuple[Any, ...]]) – List of data tuples to chunk
- **chunk_size** (int) – Size of each chunk

Yields
list[tuple[Any, ...]] – Lists of data tuples of specified chunk size

Return type
Iterator[list[tuple[*Any*, ...]]]

process_single_calculation(*formula*, *energies*, *density*)

Process a single X-ray calculation.

Parameters

- **formula** (str) – Chemical formula
- **energies** (ndarray) – Energy array
- **density** (float) – Material density

Return type
tuple[str, XRayResult | None]

Returns
Tuple of (formula, XRayResult)

process_batch_chunk(*chunk*, *config*, *progress_bar=None*)

Process a chunk of calculations as device F-blocks.

Items that share their energies run together (grouped by element bucket, then cut into blocks of at most `config.chunk_size` materials); results are returned in the same order as `chunk`. A failing item warns and yields `None`, like *process_single_calculation()*.

Parameters

- **chunk** (list[tuple[str, ndarray, float]]) – List of (formula, energies, density) tuples
- **config** () – Batch processing configuration
- **progress_bar** () – Optional object with `update(n)`, advanced per block

Return type

list[tuple[str, XRayResult | None]]

Returns

List of (formula, result) tuples

calculate_batch_properties(*formulas*, *energies*, *densities*, *config=None*)

Calculate X-ray properties for multiple materials with optimized batch processing.

This function processes large batches of calculations efficiently using chunking, device F-blocks (one JAX program call per block), and memory management. `config.max_workers` is accepted and ignored.

Parameters

- **formulas** (list[str]) – List of chemical formulas
- **energies** (float | list[float] | ndarray) – Energy values (shared across all materials)
- **densities** (list[float]) – List of material densities
- **config** (*BatchConfig* | None) – Batch processing configuration (optional)

Return type

dict[str, XRayResult | None]

Returns

Dictionary mapping formulas to XRayResult objects

Raises

ValueError – If input validation fails

Examples

```
>>> import numpy as np
>>> from xraylabtool.data_handling.batch_processing import calculate_batch_properties
>>> formulas = ["SiO2", "SiO2", "Al2O3"]  # Same formula with different densities
>>> energies = np.linspace(5, 15, 101)  # 101 energy points
>>> densities = [2.2, 2.5, 3.95]  # Different densities for SiO2
>>> results = calculate_batch_properties(formulas, energies, densities)
>>> print(f"Processed {len(results)} materials")
Processed 3 materials
```

save_batch_results(*results*, *output_file*, *format='csv'*, *fields=None*)

Save batch calculation results to file.

Parameters

- **results** (dict[str, XRayResult | None]) – Dictionary of calculation results
- **output_file** (str | Path) – Output file path
- **format** (str) – Output format ('csv', 'json', 'parquet')

- **fields** (list[str] | None) – List of fields to include (default: all)

Raises

- **ValueError** – If format is not supported
- **IOError** – If file cannot be written

Return type

None

load_batch_input(*input_file*, *formula_column='formula'*, *density_column='density'*, *energy_column=None*)

Load batch input data from file.

Parameters

- **input_file** (str | Path) – Input file path
- **formula_column** (str) – Name of formula column
- **density_column** (str) – Name of density column
- **energy_column** (str | None) – Name of energy column (optional, for per-material energies)

Return type

tuple[list[str], list[float], list[ndarray] | None]

Returns

Tuple of (formulas, densities, energies) where energies is either None or a list of numpy arrays

Raises

- **FileNotFoundError** – If input file doesn't exist
- **ValueError** – If required columns are missing

Usage Example

```
from xraylabtool.data_handling.batch_processing import (
    BatchConfig,
    calculate_batch_properties,
    load_batch_input,
    save_batch_results,
)

formulas, densities, _ = load_batch_input("materials.csv")  # formula,density columns
config = BatchConfig(chunk_size=100, enable_progress=True)  # max_workers is accepted and ignored

results = calculate_batch_properties(formulas, [5.0, 8.0, 10.0], densities, config=config)
# dict keyed "formula@density", value XRayResult (or None on failure)

save_batch_results(results, "results.csv")
```

Each chunk runs as blocks of materials through one JAX device program (one upload and one fetch per block); there are no process or thread pools, and `BatchConfig.max_workers` is accepted and ignored. `BatchConfig.chunk_size` bounds the materials per block and `BatchConfig.memory_limit_gb` triggers garbage collection between chunks.

Data Sources

Atomic scattering factor data is sourced from:

1. **CXRO Database**: Center for X-ray Optics, Lawrence Berkeley National Laboratory
2. **NIST Database**: National Institute of Standards and Technology
3. **Henke Tables**: Widely used X-ray optical constants

The data files are in Henke format (.nff files); XRayLabTool exposes the 0.03–30 keV range with PCHIP interpolation between tabulated values.

3.10.3 Interfaces Module

The interfaces module provides command-line interface and shell completion functionality.

Command Line Interface

Command Line Interface for XRayLabTool.

This package provides a comprehensive CLI for calculating X-ray optical properties of materials, including single material calculations, batch processing, utility functions for X-ray analysis, and shell completion installation.

Available Commands:

calc Calculate X-ray properties for a single material batch Process multiple materials from CSV file compare Compare X-ray properties between multiple materials convert Convert between energy and wavelength units kk Kramers-Kronig f1 from tabulated f2 and near-edge data formula Parse and analyze chemical formulas atomic Look up atomic data for elements bragg Calculate Bragg angles for diffraction list List available data and information install-completion Install shell completion for xraylabtool uninstall-completion Remove shell completion for xraylabtool

The CLI supports various output formats (table, CSV, JSON), field filtering, precision control, and comprehensive shell completion for enhanced usability.

Command parsing lives in `parser.py`, output formatting in `formatting.py`, shared helpers in `utils.py`, and each subcommand's handler in its own module under `commands/`. This module wires them together and re-exports the public names that used to live in the single `cli.py` module, for backward compatibility.

cmd_atomic(*args*)

Handle the 'atomic' command.

Return type

int

Parameters

args (*Any*)

cmd_batch(*args*)

Handle the 'batch' command.

Return type

int

Parameters

args (*Any*)

cmd_bragg(*args*)

Handle the 'bragg' command.

Return type

int

Parameters

args (*Any*)

cmd_calc(*args*)

Handle the 'calc' command.

Return type
int

Parameters
args (*Any*)

cmd_compare(*args*)

Handle the 'compare' command for material comparison.

Return type
int

Parameters
args (*Any*)

cmd_completion(*args*)

Handle the 'completion' command for the new completion system.

Return type
int

Parameters
args (*Any*)

cmd_convert(*args*)

Handle the 'convert' command.

Return type
int

Parameters
args (*Any*)

cmd_formula(*args*)

Handle the 'formula' command.

Return type
int

Parameters
args (*Any*)

cmd_install_completion(*args*)

Handle the 'install-completion' command.

Return type
int

Parameters
args (*Any*)

cmd_kk(*args*)

Handle the 'kk' command.

Return type
int

Parameters
args (*Any*)

cmd_list(*args*)

Handle the 'list' command.

Return type
int

Parameters
args (*Any*)

cmd_uninstall_completion(*args*)

Handle the 'uninstall-completion' command.

Return type
int

Parameters
args (*Any*)

create_parser()

Create the main argument parser with all subcommands.

Return type
ArgumentParser

format_xray_result(*result*, *format_type*, *precision=6*, *fields=None*)

Format XRayResult for output.

Return type
str

Parameters

- **result** (*Any*)
- **format_type** (*str*)
- **precision** (*int*)
- **fields** (*list[str] | None*)

main()

Execute the main CLI application.

Return type
int

parse_energy_string(*energy_str*)

Parse energy string and return numpy array.

Return type
ndarray

Parameters
energy_str (*str*)

Argument Parser

Argument parser construction for the xraylabtool CLI.

create_parser()

Create the main argument parser with all subcommands.

Return type
ArgumentParser

add_calc_command(*subparsers*)

Add the 'calc' subcommand for single material calculations.

Return type
None

Parameters
subparsers (*Any*)

add_batch_command(*subparsers*)

Add the 'batch' subcommand for processing multiple materials.

Return type
None

Parameters
subparsers (*Any*)

add_convert_command(*subparsers*)
Add the 'convert' subcommand for unit conversions.

Return type
None

Parameters
subparsers (*Any*)

add_kk_command(*subparsers*)
Add the 'kk' subcommand for Kramers-Kronig f1 calculations.

Return type
None

Parameters
subparsers (*Any*)

add_formula_command(*subparsers*)
Add the 'formula' subcommand for formula parsing.

Return type
None

Parameters
subparsers (*Any*)

add_atomic_command(*subparsers*)
Add the 'atomic' subcommand for atomic data lookup.

Return type
None

Parameters
subparsers (*Any*)

add_bragg_command(*subparsers*)
Add the 'bragg' subcommand for Bragg angle calculations.

Return type
None

Parameters
subparsers (*Any*)

add_list_command(*subparsers*)
Add the 'list' subcommand for listing available data.

Return type
None

Parameters
subparsers (*Any*)

add_install_completion_command(*subparsers*)
Add the 'install-completion' subcommand for shell completion setup.

Return type
None

Parameters
subparsers (*Any*)

add_completion_command(*subparsers*)
Add the 'completion' subcommand for the new completion system.

Return type
None

Parameters
subparsers (*Any*)

add_uninstall_completion_command(*subparsers*)
Add the 'uninstall-completion' subcommand for shell completion removal.

Return type
None

Parameters
subparsers (*Any*)

add_compare_command(*subparsers*)
Add the 'compare' subcommand for material comparison.

Return type
None

Parameters
subparsers (*Any*)

Output Formatting

Result formatting (table/CSV/JSON) for the 'calc' command output.

format_xray_result(*result*, *format_type*, *precision=6*, *fields=None*)
Format XRayResult for output.

Return type
str

Parameters
- **result** (*Any*)
- **format_type** (*str*)
- **precision** (*int*)
- **fields** (*list[str] | None*)

CLI Utilities

Small shared helpers used across CLI command modules.

create_batch_progress_tracker(***kwargs*)
Stub implementation for the removed monitoring/performance progress module.

Return type

Parameters
kwargs (*Any*)

parse_energy_string(*energy_str*)
Parse energy string and return numpy array.

Return type
ndarray

Parameters
energy_str (*str*)

Command Implementations

Implementation of the 'calc' command.

cmd_calc(*args*)

Handle the ‘calc’ command.

Return type
int

Parameters
args (*Any*)

Implementation of the ‘batch’ command.

cmd_batch(*args*)

Handle the ‘batch’ command.

Return type
int

Parameters
args (*Any*)

Implementation of the ‘compare’ command.

cmd_compare(*args*)

Handle the ‘compare’ command for material comparison.

Return type
int

Parameters
args (*Any*)

Implementation of the ‘convert’ command.

cmd_convert(*args*)

Handle the ‘convert’ command.

Return type
int

Parameters
args (*Any*)

Implementation of the ‘kk’ command.

cmd_kk(*args*)

Handle the ‘kk’ command.

Return type
int

Parameters
args (*Any*)

Implementation of the ‘formula’ command.

cmd_formula(*args*)

Handle the ‘formula’ command.

Return type
int

Parameters
args (*Any*)

Implementation of the ‘atomic’ command.

cmd_atomic(*args*)

Handle the ‘atomic’ command.

Return type
int

Parameters
args (*Any*)

Implementation of the 'bragg' command.

cmd_bragg(*args*)

Handle the 'bragg' command.

Return type
int

Parameters
args (*Any*)

Implementation of the 'list' command.

cmd_list(*args*)

Handle the 'list' command.

Return type
int

Parameters
args (*Any*)

Implementation of the 'completion', 'install-completion', and 'uninstall-completion' commands.

cmd_install_completion(*args*)

Handle the 'install-completion' command.

Return type
int

Parameters
args (*Any*)

cmd_uninstall_completion(*args*)

Handle the 'uninstall-completion' command.

Return type
int

Parameters
args (*Any*)

cmd_completion(*args*)

Handle the 'completion' command for the new completion system.

Return type
int

Parameters
args (*Any*)

CLI Commands Overview

XRayLabTool provides 12 CLI commands (see *CLI Reference* for every option):

Command	Description
`calc`	Calculate X-ray properties for a single material
`batch`	Process multiple materials from CSV file
`compare`	Compare X-ray properties between multiple materials
`convert`	Convert between energy and wavelength units
`kk`	Kramers-Kronig f1 from f2, with optional near-edge data and refinement
`formula`	Parse and analyze chemical formulas
`atomic`	Look up atomic scattering factor data
`bragg`	Calculate Bragg diffraction angles
`list`	Display reference information and constants
`completion`	Virtual environment-centric shell completion (install/uninstall/status/list/info)
`install-completion`	Install shell completion (bash, zsh, fish, PowerShell)
`uninstall-completion`	Remove shell completion

Command Examples

All energies are in keV.

Single Material Calculation:

```
xraylabtool calc Si -e 8.0 -d 2.33
xraylabtool calc SiO2 -e 5.0,8.0,10.0 -d 2.2
xraylabtool calc Al -e 1-20:100 -d 2.70
xraylabtool calc Si -e 8.0 -d 2.33 --format csv
xraylabtool calc Si -e 8.0 -d 2.33 -o si.json
```

Batch Processing:

```
xraylabtool batch materials.csv -o results.csv
xraylabtool batch materials.csv -o results.json --progress
```

Material Comparison:

```
xraylabtool compare SiO2,2.2 Si3N4,3.2 -e 8.0,10.0 --properties dispersion_delta,absorption_beta
```

Unit Conversions:

```
xraylabtool convert energy 8.0 --to wavelength
xraylabtool convert wavelength 1.55 --to energy
xraylabtool convert energy 5.0,8.0,10.0 --to wavelength
```

Formula Analysis:

```
xraylabtool formula SiO2
xraylabtool formula "Ca5(PO4)3F"
```

Atomic Data Lookup:

```
xraylabtool atomic Si
xraylabtool atomic Si,Al,O -o elements.csv
```

Bragg Diffraction:

```
xraylabtool bragg -d 3.14 -e 8.0
xraylabtool bragg -d 3.14,1.92,1.64 -e 8.0
xraylabtool bragg -d 3.14 -w 1.55 --order 2
```

Reference Information:

```
xraylabtool list constants
xraylabtool list fields
xraylabtool list examples
```

Output Formats

`calc` and `compare` accept `--format {table,csv,json}` for stdout; `-o/--output FILE` writes CSV or JSON by file extension.

Shell Completion v2

The `completion_v2` sub-package is the current virtual-environment-centric completion system (the legacy `completion.py` bridge was removed). Virtual environment-centric completion installer.

This module handles installation and management of shell completion that activates/deactivates with virtual environment changes.

class CompletionInstaller

Bases: `object`

Handles virtual environment-centric completion installation.

__init__()

Return type
None

install(*shell=None*, *target_env=None*, *force=False*)

Install completion in virtual environment.

Parameters

- **shell** (`str` | `None`) – Shell type (bash, zsh, fish, powershell). Auto-detected if None.
- **target_env** (`str` | `None`) – Target environment name. Current environment if None.
- **force** (`bool`) – Force reinstallation if already installed.

Return type
`bool`

Returns
True if installation succeeded, False otherwise.

uninstall(*target_env=None*, *all_envs=False*)

Uninstall completion from environment(s).

Parameters

- **target_env** (`str` | `None`) – Target environment name. Current environment if None.
- **all_envs** (`bool`) – Remove from all environments if True.

Return type
`bool`

Returns
True if uninstallation succeeded, False otherwise.

list_environments()

List all detected environments with completion status.

Return type
`None`

status()

Show completion status for current environment.

Return type

None

install_bash_completion(***kwargs*)

Install bash completion (backward compatibility).

Return type

bool

uninstall_bash_completion(***kwargs*)

Uninstall bash completion (backward compatibility).

Return type

bool

uninstall_completion(
shell_type=None,
cleanup_session=True,
***kwargs*,
)

Uninstall completion (backward compatibility).

Return type

bool

test_completion()

Test completion functionality (backward compatibility).

Return type

bool

get_bash_completion_dir()

Get bash completion directory (backward compatibility).

Return type

get_user_bash_completion_dir()

Get user bash completion directory (backward compatibility).

Return type

Shell-specific completion script generators.

This module provides modular completion script generation for different shells, replacing the large hardcoded strings with template-based generation.

class CompletionGenerator(*command_name='xraylabtool'*, *aliases=None*)

Bases: ABC

Base class for shell completion generators.

Parameters

- **command_name** (*str*)
- **aliases** (*list[str] | None*)

__init__(*command_name='xraylabtool'*, *aliases=None*)

Parameters

- **command_name** (*str*)
- **aliases** (*list[str] | None*)

abstractmethod generate(*commands*, *global_options*)

Generate completion script for this shell.

Return type

str

Parameters

- **commands** (*dict[str, dict[str, Any]]*)
- **global_options** (*list[str]*)

abstract property shell_name: str

Name of the shell this generator supports.

abstract property file_extension: str

File extension for completion scripts.

class BashCompletionGenerator(*command_name='xraylabtool'*, *aliases=None*)

Bases: *CompletionGenerator*

Generates Bash completion scripts.

Parameters

- **command_name** (*str*)
- **aliases** (*list[str] | None*)

property shell_name: str

Name of the shell this generator supports.

property file_extension: str

File extension for completion scripts.

generate(*commands*, *global_options*)

Generate Bash completion script.

Return type

str

Parameters

- **commands** (*dict[str, dict[str, Any]]*)
- **global_options** (*list[str]*)

class ZshCompletionGenerator(*command_name='xraylabtool'*, *aliases=None*)

Bases: *CompletionGenerator*

Generates native Zsh completion scripts.

Parameters

- **command_name** (*str*)
- **aliases** (*list[str] | None*)

property shell_name: str

Name of the shell this generator supports.

property file_extension: str

File extension for completion scripts.

generate(*commands*, *global_options*)

Generate native Zsh completion script.

Return type

str

Parameters

- **commands** (*dict[str, dict[str, Any]]*)

- **global_options** (*list[str]*)

class FishCompletionGenerator(*command_name='xraylabtool', aliases=None*)

Bases: *CompletionGenerator*

Generates Fish shell completion scripts.

Parameters

- **command_name** (*str*)
- **aliases** (*list[str] | None*)

property shell_name: str

Name of the shell this generator supports.

property file_extension: str

File extension for completion scripts.

generate(*commands, global_options*)

Generate Fish completion script.

Return type

str

Parameters

- **commands** (*dict[str, dict[str, Any]]*)
- **global_options** (*list[str]*)

class PowerShellCompletionGenerator(
command_name='xraylabtool',
aliases=None,
)

Bases: *CompletionGenerator*

Generates PowerShell completion scripts.

Parameters

- **command_name** (*str*)
- **aliases** (*list[str] | None*)

property shell_name: str

Name of the shell this generator supports.

property file_extension: str

File extension for completion scripts.

generate(*commands, global_options*)

Generate PowerShell completion script.

Return type

str

Parameters

- **commands** (*dict[str, dict[str, Any]]*)
- **global_options** (*list[str]*)

class CompletionManager

Bases: object

Manages completion script generation for all supported shells.

__init__()

Return type
None

get_supported_shells()

Get list of supported shell types.

Return type
list[str]

generate_completion(*shell*, *commands*, *global_options*)

Generate completion script for specified shell.

Return type
str

Parameters

- **shell** (*str*)
- **commands** (*dict[str, dict[str, Any]]*)
- **global_options** (*list[str]*)

get_filename(*shell*, *command_name='xraylabtool'*)

Get the appropriate filename for a completion script.

Return type
str

Parameters

- **shell** (*str*)
- **command_name** (*str*)

get_xraylabtool_commands()

Get the XRayLabTool command definitions for completion.

Return type
dict[str, dict[str, Any]]

get_global_options()

Get global options available for all commands.

Return type
list[str]

Environment detection and management for shell completion.

This module provides robust detection of Python virtual environments and manages completion activation/deactivation based on environment state.

class EnvironmentType

Bases: object

Constants for environment types.

SYSTEM = 'system'

VENV = 'venv'

VIRTUALENV = 'virtualenv'

CONDA = 'conda'

MAMBA = 'mamba'

PIPENV = 'pipenv'

POETRY = 'poetry'

class EnvironmentInfo(
env_type,
path,
name,
is_active=False,
python_version=None,
has_completion=False,
)

Bases: object

Information about a detected environment.

Parameters

- **env_type** (*str*)
- **path** (*Path*)
- **name** (*str*)
- **is_active** (*bool*)
- **python_version** (*str | None*)
- **has_completion** (*bool*)

__init__(
env_type,
path,
name,
is_active=False,
python_version=None,
has_completion=False,
)

Parameters

- **env_type** (*str*)
- **path** (*Path*)
- **name** (*str*)
- **is_active** (*bool*)
- **python_version** (*str | None*)
- **has_completion** (*bool*)

to_dict()

Convert to dictionary for serialization.

Return type
dict[str, Any]

classmethod from_dict(*data*)

Create from dictionary.

Return type
EnvironmentInfo

Parameters
data (*dict[str, Any]*)

class EnvironmentDetector

Bases: object

Detects and manages Python virtual environments.

__init__()

Return type
None

get_current_environment()

Get information about the currently active environment.

Return type
EnvironmentInfo | None

discover_all_environments(*use_cache=True*)

Discover all available Python environments.

Return type
list[*EnvironmentInfo*]

Parameters
use_cache (*bool*)

clear_cache()

Clear the environment cache.

Return type
None

Integration layer for the new completion system.

This module provides backward compatibility and integration with the existing CLI while gradually migrating to the new virtual environment-centric completion system.

legacy_install_completion_main(*args*)

Legacy install completion main function with new system backend.

This function maintains backward compatibility with the existing CLI while using the new completion system internally.

Return type
int

Parameters
args (*Namespace*)

legacy_uninstall_completion_main(*args*)

Legacy uninstall completion main function with new system backend.

Return type
int

Parameters
args (*Any*)

test_completion_installation()

Test if completion is working in the current environment.

Return type
int

handle_new_completion_command(*args*)

Handle new 'completion' subcommand.

Return type
int

Parameters
args (*list[str]*)

install_completion_main(*args*)

Legacy install completion main function with new system backend.

This function maintains backward compatibility with the existing CLI while using the new completion system internally.

Return type
int

Parameters
args (*Namespace*)

uninstall_completion_main(*args*)

Legacy uninstall completion main function with new system backend.

Return type
int

Parameters
args (*Any*)

Command-line interface for the new completion system.

This module provides CLI commands for managing the virtual environment-centric shell completion system.

create_completion_parser()

Create argument parser for completion commands.

Return type
ArgumentParser

handle_completion_command(*args*)

Handle completion subcommands.

Return type
int

Parameters
args (*Namespace*)

show_completion_info()

Show information about the completion system.

Return type
None

completion_main(*args=None*)

Main entry point for completion commands.

Return type
int

Parameters
args (*Any*)

install_completion_main(*args*)

Legacy install completion main function.

Return type
int

Parameters
args (*Namespace*)

uninstall_completion_main(*args*)

Legacy uninstall completion main function.

Return type
int

Parameters
args (*Any*)

Bash Completion Features

The shell completion system provides:

- **Command completion**: All 11 CLI commands
- **Option completion**: Command flags and parameters
- **File completion**: Input/output file paths
- **Element completion**: Chemical element symbols
- **Unit completion**: Energy and wavelength units

Installation

```
# Install completion for current user
xraylabtool install-completion

# Install system-wide (requires sudo)
sudo xraylabtool install-completion --system

# Per-virtual-environment install (preferred)
xraylabtool completion install --shell zsh
```

Usage

After installation, completion is available by pressing Tab:

```
xraylabtool [TAB]          # Shows all commands
xraylabtool calc [TAB]     # Shows calc command options
xraylabtool atomic S[TAB]  # Completes to supported elements starting with 'S'
```

Uninstallation

```
# Remove completion
xraylabtool uninstall-completion

# Remove system-wide completion
sudo xraylabtool uninstall-completion --system
```

Platform Support

Shell completion is supported for:

- **Bash**: Full support on Linux and macOS (requires bash-completion)
- **Zsh**: Full support on Linux and macOS (requires zsh-completions)
- **Fish**: Native support with built-in completion system
- **PowerShell**: Full support on Windows, macOS, and Linux

Error Handling

The CLI provides error handling with helpful messages:

```
# Invalid formula
$ xraylabtool calc XYZ -e 8.0 -d 1.0
Error: Invalid chemical formula 'XYZ': Unknown element symbol: X: 'XYZ'

# Missing required parameter
$ xraylabtool calc Si -e 8.0
error: the following arguments are required: -d/--density

# Invalid energy range
$ xraylabtool calc Si -e 0 -d 2.33
Error: All energies must be positive
```

Integration Examples

With Python Scripts:

```
import subprocess
import json

# Call CLI from Python
result = subprocess.run([
    "xraylabtool", "calc", "Si",
    "-e", "8.0",
    "-d", "2.33",
    "--format", "json"
], capture_output=True, text=True, check=True)

data = json.loads(result.stdout)
print(f"Critical angle: {data['critical_angle_degrees'][0]}")
```

With Shell Scripts:

```
#!/bin/bash

# Process multiple materials
for material in Si Al Cu; do
    echo "Processing $material..."
    xraylabtool calc $material -e 8.0 -d 2.33 --format csv >> results.csv
done
```

3.10.4 Validation Module

The validation module provides input validation, error handling, and custom exception classes.

Exception Hierarchy

Custom exceptions for XRayLabTool.

This module defines domain-specific exception classes that provide better error handling and more informative error messages for X-ray analysis operations.

exception AtomicDataError(*message*, *element=None*)

Bases: `XRayLabToolError`

Exception raised when atomic data cannot be retrieved or processed.

Parameters

- **message** (*str*)
- **element** (*str | None*)

__init__(*message*, *element=None*)

Initialize AtomicDataError with message and optional element context.

Parameters

- **message** (*str*)
- **element** (*str | None*)

exception BatchProcessingError(*message*, *failed_items=None*, *total_items=None*)

Bases: XRayLabToolError

Exception raised during batch processing operations.

Parameters

- **message** (*str*)
- **failed_items** (*list[Any] | None*)
- **total_items** (*int | None*)

__init__(
message,
failed_items=None,
total_items=None,
)

Initialize BatchProcessingError with message and optional failure context.

Parameters

- **message** (*str*)
- **failed_items** (*list[Any] | None*)
- **total_items** (*int | None*)

exception CalculationError(*message*, *formula=None*, *energy=None*)

Bases: XRayLabToolError

Exception raised when X-ray property calculations fail.

Parameters

- **message** (*str*)
- **formula** (*str | None*)
- **energy** (*float | None*)

__init__(*message*, *formula=None*, *energy=None*)

Initialize CalculationError with message and optional context.

Parameters

- **message** (str) – The error message
- **formula** (str | None) – Optional chemical formula context
- **energy** (float | None) – Optional energy value context in keV

exception DataFileError(*message*, *filename=None*)

Bases: XRayLabToolError

Exception raised when data files cannot be loaded or processed.

Parameters

- **message** (*str*)
- **filename** (*str | None*)

__init__(*message*, *filename=None*)

Initialize DataFileError with message and optional filename context.

Parameters

- **message** (*str*)
- **filename** (*str | None*)

exception EnergyError(*message*, *energy=None*, *valid_range=None*)

Bases: XRayLabToolError

Exception raised for invalid energy values or ranges.

Parameters

- **message** (*str*)
- **energy** (*float | None*)
- **valid_range** (*str | None*)

__init__(*message*, *energy=None*, *valid_range=None*)

Initialize EnergyError with message and optional energy context.

Parameters

- **message** (*str*)
- **energy** (*float | None*)
- **valid_range** (*str | None*)

exception FormulaError(*message*, *formula=None*)

Bases: XRayLabToolError

Exception raised for invalid chemical formulas.

Parameters

- **message** (*str*)
- **formula** (*str | None*)

__init__(*message*, *formula=None*)

Initialize FormulaError with message and optional formula context.

Parameters

- **message** (str) – The error message
- **formula** (str | None) – Optional chemical formula that caused the error

exception NearEdgeDataError(*message*, *parameter=None*, *value=None*)

Bases: ValidationError

Exception raised when near-edge absorption data cannot be loaded or merged.

Covers a malformed or non-monotonic spectrum, a spectrum that does not overlap the tabulated energy range, invalid merge points, and degenerate (flat or non-finite) normalisation against the tabulated data.

Parameters

- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

exception UnknownElementError(*element*)

Bases: AtomicDataError

Exception raised when an unknown element symbol is provided.

Parameters
element (*str*)

__init__(*element*)
Initialize UnknownElementError with element symbol.

Parameters
element (*str*)

exception ValidationError(*message*, *parameter=None*, *value=None*)
Bases: XRayLabToolError

Exception raised when input validation fails.

Parameters
- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

__init__(*message*, *parameter=None*, *value=None*)
Initialize ValidationError with message and optional parameter context.

Parameters
- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

exception XRayLabToolError
Bases: Exception

Base exception for all XRayLabTool-related errors.

Base Exception

class XRayLabToolError
Bases: Exception

Base exception for all XRayLabTool-related errors.

The base exception class for all XRayLabTool-specific errors. All custom exceptions inherit from this class, making it easy to catch any XRayLabTool-related error.

Example:

```
try:
    result = calculate_single_material_properties("InvalidFormula", 8.0, 1.0)
except XRayLabToolError as e:
    print(f"XRayLabTool error: {e}")
```

Specific Exceptions

class ValidationError(*message*, *parameter=None*, *value=None*)
Bases: XRayLabToolError

Exception raised when input validation fails.

Raised when input validation fails.

Parameters
- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

__init__(*message*, *parameter=None*, *value=None*)

Initialize ValidationError with message and optional parameter context.

Parameters

- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

class FormulaError(*message*, *formula=None*)

Bases: XRayLabToolError

Exception raised for invalid chemical formulas.

Raised when chemical formula parsing or validation fails.

Common causes: - Unknown chemical elements - Invalid formula syntax - Unsupported element combinations

Example:

```
try:
    from xraylabtool.utils import parse_formula
    composition = parse_formula("XYZ123")  # Invalid element
except FormulaError as e:
    print(f"Formula error: {e}")
```

Parameters

- **message** (*str*)
- **formula** (*str | None*)

__init__(*message*, *formula=None*)

Initialize FormulaError with message and optional formula context.

Parameters

- **message** (str) – The error message
- **formula** (str | None) – Optional chemical formula that caused the error

class EnergyError(*message*, *energy=None*, *valid_range=None*)

Bases: XRayLabToolError

Exception raised for invalid energy values or ranges.

Raised when energy values are invalid or out of supported range.

Common causes: - Negative or zero energy values - Energy outside supported range (typically 10 eV - 100 keV) - Invalid energy array or range specification

Example:

```
try:
    result = calculate_single_material_properties("Si", 2.33, -1000)  # Negative energy
except EnergyError as e:
    print(f"Energy error: {e}")
```

Parameters

- **message** (*str*)
- **energy** (*float | None*)
- **valid_range** (*str | None*)

__init__(*message*, *energy=None*, *valid_range=None*)

Initialize EnergyError with message and optional energy context.

Parameters

- **message** (*str*)
- **energy** (*float | None*)
- **valid_range** (*str | None*)

class CalculationError(*message*, *formula=None*, *energy=None*)

Bases: XRayLabToolError

Exception raised when X-ray property calculations fail.

Raised when X-ray property calculations fail due to numerical issues or invalid parameters.

Common causes: - Numerical instability in calculations - Invalid density values - Missing atomic data - Convergence failures

Parameters

- **message** (*str*)
- **formula** (*str | None*)
- **energy** (*float | None*)

__init__(*message*, *formula=None*, *energy=None*)

Initialize CalculationError with message and optional context.

Parameters

- **message** (str) – The error message
- **formula** (str | None) – Optional chemical formula context
- **energy** (float | None) – Optional energy value context in keV

Input Validators

Validation functions for XRayLabTool.

This module contains functions for validating input parameters, chemical formulas, energy ranges, and other data used in calculations.

validate_energy_range(*energies*, *min_energy=0.03*, *max_energy=30.0*)

Validate X-ray energy values.

Parameters

- **energies** (float | ndarray) – Energy value(s) in keV
- **min_energy** (float) – Minimum allowed energy in keV
- **max_energy** (float) – Maximum allowed energy in keV

Return type

ndarray

Returns

Validated energy array

Raises

EnergyError – If energies are invalid

validate_chemical_formula(*formula*)

Validate and parse a chemical formula.

Parameters

formula (str) – Chemical formula string (e.g., “SiO2”, “Ca0.5Sr0.5TiO3”)

Return type
dict[str, float]

Returns
Dictionary mapping element symbols to their quantities

Raises
FormulaError – If formula is invalid

validate_density(*density*, *min_density=0.001*, *max_density=30.0*)

Validate material density value.

Parameters

- **density** (float) – Density in g/cm³
- **min_density** (float) – Minimum allowed density
- **max_density** (float) – Maximum allowed density

Return type
float

Returns
Validated density value

Raises
ValidationError – If density is invalid

validate_calculation_parameters(*formula*, *energies*, *density*)

Validate all parameters for X-ray calculations.

Parameters

- **formula** (str) – Chemical formula
- **energies** (float | ndarray) – Energy values in keV
- **density** (float) – Material density in g/cm³

Return type
tuple[str, ndarray, float]

Returns
Tuple of validated (formula, energies, density)

Raises
ValidationError – If any parameters are invalid

Validation Functions

validate_energy_range(*energies*, *min_energy=0.03*, *max_energy=30.0*)

Validate X-ray energy values.

Parameters

- **energies** (float | ndarray) – Energy value(s) in keV
- **min_energy** (float) – Minimum allowed energy in keV
- **max_energy** (float) – Maximum allowed energy in keV

Return type
ndarray

Returns
Validated energy array

Raises
EnergyError – If energies are invalid

Validate X-ray energy values.

Parameters: - `energies` (float or np.ndarray): Energy value(s) in keV - `min_energy` (float): Minimum allowed energy in keV (default: 0.1) - `max_energy` (float): Maximum allowed energy in keV (default: 100.0)

Returns: - `np.ndarray`: Validated energy array

Raises: - `EnergyError`: If energies are invalid

Example:

```
from xraylabtool.validation.validators import validate_energy_range
import numpy as np

# Valid energies
energies = validate_energy_range(8.0)  # Single energy
energies = validate_energy_range([5.0, 8.0, 10.0])  # Multiple energies
energies = validate_energy_range(np.array([1.0, 2.0, 3.0]))  # Array

# Invalid energies will raise EnergyError
# validate_energy_range(-1.0)  # Negative energy
# validate_energy_range(200.0)  # Above max energy
```

validate_chemical_formula(*formula*)

Validate and parse a chemical formula.

Parameters
formula (str) – Chemical formula string (e.g., “SiO2”, “Ca0.5Sr0.5TiO3”)

Return type
dict[str, float]

Returns
Dictionary mapping element symbols to their quantities

Raises
FormulaError – If formula is invalid

Validate and parse a chemical formula.

Parameters: - `formula` (str): Chemical formula string (e.g., “SiO2”, “Ca0.5Sr0.5TiO3”)

Returns: - `dict`: Dictionary mapping element symbols to their quantities

Raises: - `FormulaError`: If formula is invalid

Example:

```
from xraylabtool.validation.validators import validate_chemical_formula

# Valid formulas
elements = validate_chemical_formula("SiO2")
print(elements)  # {'Si': 1.0, 'O': 2.0}

elements = validate_chemical_formula("Al2O3")
print(elements)  # {'Al': 2.0, 'O': 3.0}

# Fractional compositions
elements = validate_chemical_formula("Ca0.5Sr0.5TiO3")
print(elements)  # {'Ca': 0.5, 'Sr': 0.5, 'Ti': 1.0, 'O': 3.0}
```

validate_density(*density*, *min_density=0.001*, *max_density=30.0*)

Validate material density value.

Parameters

- **density** (float) – Density in g/cm³

- **min_density** (float) – Minimum allowed density
- **max_density** (float) – Maximum allowed density

Return type
float

Returns
Validated density value

Raises
ValidationError – If density is invalid

Validate material density value.

Parameters: - `density` (float): Density in g/cm³ - `min_density` (float): Minimum allowed density (default: 0.001) - `max_density` (float): Maximum allowed density (default: 30.0)

Returns: - `float`: Validated density value

Raises: - `ValidationError`: If density is invalid

Example:

```
from xraylabtool.validation.validators import validate_density

# Valid densities
density = validate_density(2.33)    # Silicon
density = validate_density(8.96)    # Copper
density = validate_density(0.001)   # Gas-phase materials

# Invalid densities will raise ValidationError
# validate_density(-1.0)     # Negative density
# validate_density(0.0)      # Zero density
```

validate_calculation_parameters(*formula*, *energies*, *density*)

Validate all parameters for X-ray calculations.

Parameters

- **formula** (str) – Chemical formula
- **energies** (float | ndarray) – Energy values in keV
- **density** (float) – Material density in g/cm³

Return type
tuple[str, ndarray, float]

Returns
Tuple of validated (formula, energies, density)

Raises
ValidationError – If any parameters are invalid

Validate all parameters for X-ray calculations.

Parameters: - `formula` (str): Chemical formula - `energies` (float or np.ndarray): Energy values in keV - `density` (float): Material density in g/cm³

Returns: - `tuple`: Validated (formula, energies, density)

Raises: - `ValidationError`: If any parameters are invalid

Example:

```
from xraylabtool.validation.validators import validate_calculation_parameters

# Validate all parameters at once
formula, energies, density = validate_calculation_parameters(
    "SiO2", 8.0, 2.20
```

(continues on next page)

(continued from previous page)

```
)
print(f"Formula: {formula}, Energy: {energies}, Density: {density}")
```

Error Context and Suggestions

XRayLabTool exceptions carry the offending input as an attribute alongside the message:

```
try:
    result = calculate_single_material_properties("si02", 8.0, 2.33)
except FormulaError as e:
    print(f"Error: {e}")
    print(f"Formula: {e.formula}")
    # Output:
    # Error: Invalid characters in formula: 'si02': 'si02'
    # Formula: si02
```

Internal Validation Utilities

The validation module also includes internal helper functions that are used by the main validation functions but are not part of the public API:

- `_parse_formula()`: Internal formula parsing logic
- `_get_valid_element_symbols()`: Returns a set of valid element symbols for validation

Best Practices

1. Always Validate Input:

```
from xraylabtool.validation.validators import validate_chemical_formula, validate_energy_range

def safe_calculation(formula, energy_kev, density):
    validate_chemical_formula(formula)
    validate_energy_range(energy_kev)
    # Proceed with calculation...
```

2. Handle Specific Exceptions:

```
from xraylabtool.exceptions import FormulaError, EnergyError

try:
    result = calculate_single_material_properties(formula, energy_kev, density)
except FormulaError:
    print("Please check your chemical formula")
except EnergyError:
    print("Please provide a valid energy value")
except Exception as e:
    print(f"Unexpected error: {e}")
```

3. Use Complete Parameter Validation:

```
from xraylabtool.validation.validators import validate_calculation_parameters

materials = [
    {"formula": "Si", "density": 2.33},
    {"formula": "InvalidElement", "density": 1.0}
]

# Validate all materials first
valid_materials = []
```

(continues on next page)

(continued from previous page)

```
for material in materials:
    try:
        formula, energies, density = validate_calculation_parameters(
            material["formula"], 8.0, material["density"]
        )
        valid_materials.append({
            "formula": formula,
            "density": density,
            "energies": energies
        })
    except (FormulaError, ValidationError) as e:
        print(f"Skipping invalid material: {e}")

# Process only valid materials
results = calculate_xray_properties(valid_materials, energy=8000)
```

3.10.5 I/O Operations Module

The I/O module handles file operations, data import/export, and format conversions. XRayLabTool I/O Module.

This module contains input/output operations, file handling, and data persistence.

File Operations

File operations for XRayLabTool.

This module contains functions for loading and saving data files, including atomic scattering factor data and calculation results.

load_data_file(*filename*)

Load data file with error handling.

Parameters

filename (str) – Path to the data file

Return type

ndarray

Returns

Numpy array containing the loaded data

Raises

DataFileError – If file cannot be loaded or parsed

save_calculation_results(*results*, *filename*, *format_type='csv'*)

Save calculation results to file.

Parameters

- **results** () – Calculation results to save
- **filename** (str) – Output file path
- **format_type** (str) – Output format (‘csv’, ‘json’)

Return type

None

export_to_csv(*data*, *filename*, *fields=None*, ***_kwargs*)

Export data to CSV format.

Return type

None

Parameters

- **data** (*Any*)
- **filename** (*str*)
- **fields** (*list[str] | None*)
- **_kwargs** (*Any*)

export_to_json(*data*, *filename*, ***_kwargs*)

Export data to JSON format.

Return type
None

Parameters

- **data** (*Any*)
- **filename** (*str*)
- **_kwargs** (*Any*)

Data Export and Formatting

Data export and formatting utilities for XRayLabTool.

This module contains functions for formatting and exporting calculation results in various formats suitable for different use cases.

format_xray_result(
result,
format_type='table',
fields=None,
precision=6,
)

Format XRayResult for display or export.

Parameters

- **result** () – XRayResult object to format
- **format_type** (str) – Output format ('table', 'csv', 'json')
- **fields** (list[str] | None) – Specific fields to include (None for all)
- **precision** (int) – Decimal precision for numeric values

Return type
str

Returns
Formatted string representation

format_calculation_summary(*results*, *format_type='table'*)

Format a summary of multiple calculation results.

Parameters

- **results** (list[Any]) – List of XRayResult objects
- **format_type** (str) – Output format type

Return type
str

Returns
Formatted summary string

Available Functions

File Loading and Saving

load_data_file(*filename*)

Load data file with error handling.

Parameters

filename (str) – Path to the data file

Return type

ndarray

Returns

Numpy array containing the loaded data

Raises

DataFileError – If file cannot be loaded or parsed

Load data file with error handling.

Example:

```
from xraylabtool.io.file_operations import load_data_file

data = load_data_file("atomic_data.nff")
print(data.head())
```

save_calculation_results(*results*, *filename*, *format_type='csv'*)

Save calculation results to file.

Parameters

- **results** () – Calculation results to save
- **filename** (str) – Output file path
- **format_type** (str) – Output format ('csv', 'json')

Return type

None

Save calculation results to file.

Example:

```
from xraylabtool.io.file_operations import save_calculation_results

# Save as CSV
save_calculation_results(results, "output.csv", format_type="csv")

# Save as JSON
save_calculation_results(results, "output.json", format_type="json")
```

Export Functions

export_to_csv(*data*, *filename*, *fields=None*, ***_kwargs*)

Export data to CSV format.

Return type

None

Parameters

- **data** (*Any*)
- **filename** (*str*)
- **fields** (*list[str] | None*)

- **_kwargs** (*Any*)

Export data to CSV format.

Example:

```
from xraylabtool.io.file_operations import export_to_csv

export_to_csv(calculation_results, "results.csv")
```

export_to_json(*data*, *filename*, ***_kwargs*)

Export data to JSON format.

Return type
None

Parameters

- **data** (*Any*)
- **filename** (*str*)
- **_kwargs** (*Any*)

Export data to JSON format.

Example:

```
from xraylabtool.io.file_operations import export_to_json

export_to_json(calculation_results, "results.json")
```

Data Formatting

format_xray_result(
result,
format_type='table',
fields=None,
precision=6,
)

Format XRayResult for display or export.

Parameters

- **result** () – XRayResult object to format
- **format_type** (*str*) – Output format (‘table’, ‘csv’, ‘json’)
- **fields** (*list*[*str*] | None) – Specific fields to include (None for all)
- **precision** (*int*) – Decimal precision for numeric values

Return type
str

Returns
Formatted string representation

Format XRayResult for display or export.

Example:

```
from xraylabtool.io.data_export import format_xray_result

# Format as table
table_output = format_xray_result(result, format_type="table")
print(table_output)

# Format as JSON
```

(continues on next page)

(continued from previous page)

```
json_output = format_xray_result(result, format_type="json")

# Format as CSV
csv_output = format_xray_result(result, format_type="csv")
```

format_calculation_summary(*results*, *format_type='table'*)

Format a summary of multiple calculation results.

Parameters

- **results** (list[Any]) – List of XRayResult objects
- **format_type** (str) – Output format type

Return type

str

Returns

Formatted summary string

Format a summary of multiple calculation results.

Example:

```
from xraylabtool.io.data_export import format_calculation_summary

summary = format_calculation_summary(results_list, format_type="table")
print(summary)
```

Supported File Formats

Input Formats: - **Space-separated (.nff)**: Atomic scattering factor data files - **CSV**: General comma-separated data files

Output Formats: - **CSV**: Standard comma-separated format - **JSON**: Pretty-printed JSON with proper numeric precision

Usage Examples

Basic File Operations:

```
from xraylabtool.io.file_operations import load_data_file, save_calculation_results
from xraylabtool.io.data_export import format_xray_result

# Load atomic data
atomic_data = load_data_file("element.nff")

# Save calculation results
save_calculation_results(results, "output.csv")

# Format results for display
formatted_result = format_xray_result(result, format_type="table", precision=4)
print(formatted_result)
```

Error Handling

The I/O module provides error handling:

```
from xraylabtool.io.file_operations import load_data_file
from xraylabtool.exceptions import DataFileError

try:
```

(continues on next page)

(continued from previous page)

```
    data = load_data_file("data.nff")
except FileNotFoundError:
    print("Input file not found")
except DataFileError as e:
    print(f"Error loading data file: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Performance Considerations

File Loading: - Automatic detection of .nff format for space-separated data - Efficient pandas-based CSV parsing - Proper error handling for malformed files

Data Export: - Configurable numeric precision for output formatting - Support for both single results and result collections - Memory-efficient processing for large datasets

3.10.6 Utilities Module

The utils module provides utility functions for formula parsing, unit conversions, and mathematical operations.

Device Detection

GPU detection and diagnostics (System CUDA).

get_system_cuda_version()

Detect system CUDA version from nvcc.

Returns (full_version, major_version) or (None, None). Example: ("13.1", 13)

Return type

tuple[str | None, int | None]

get_gpu_info()

Detect GPU name and SM version.

Returns (gpu_name, sm_version) or (None, None). Example: ("NVIDIA GeForce RTX 4090", 8.9)

Return type

tuple[str | None, float | None]

check_plugin_conflicts()

Check for known JAX CUDA plugin conflicts.

Returns list of issue descriptions (empty = no issues).

Return type

list[str]

check_gpu_availability(*warn=True*)

Check if GPU is available and being used by JAX.

If GPU hardware is detected but JAX is in CPU mode, prints a diagnostic warning with installation instructions.

Returns True if GPU is being used by JAX, False otherwise.

Return type

bool

Parameters

warn (*bool*)

get_recommended_package()

Get recommended JAX package based on system CUDA and GPU.

Returns "jax[cuda12-local]", "jax[cuda13-local]", or None.

Return type
str | None

get_device_info()

Get comprehensive device information as a dictionary.

Return type
dict[str, Any]

Logging Utilities

Centralized logging utilities for XRayLabTool.

This module provides a single entry point to configure logging across the library, CLI, and GUI. It keeps setup lightweight (stdlib-only) while still offering:

- Rotating file logs (default location: ~/.cache/xraylabtool/logs/xraylabtool.log)
- Optional console output (stderr) with readable timestamps
- Environment-variable overrides for level, file path, and rotation
- Quieting of noisy third-party libraries (matplotlib, asyncio)

Environment variables

- XRAYLABTOOL_LOG_LEVEL: Logging level (DEBUG, INFO, WARNING, ERROR). Default: INFO.
- XRAYLABTOOL_LOG_FILE: Path to log file. If empty, file logging is disabled.
- XRAYLABTOOL_LOG_DIR: Directory for default log file (overrides cache path).
- XRAYLABTOOL_LOG_MAX_BYTES: Max size per log file before rotation (bytes). Default: 5_000_000.
- XRAYLABTOOL_LOG_BACKUPS: Number of rotated backups to keep. Default: 3.
- XRAYLABTOOL_LOG_CONSOLE: 1/0 toggle for console logging. Default: on.

Library consumers can opt in by calling configure_logging() early. The CLI and GUI call it automatically.

configure_logging(
*,
level=None,
log_file=None,
console=None,
force=False,
)

Configure package-wide logging once and return the base logger.

Parameters

- **level** (str | int | None) – Logging level (DEBUG/INFO/...). Defaults to XRAYLABTOOL_LOG_LEVEL or INFO.
- **log_file** (str | PathLike[str] | None) – Path to log file. If "" or None after env resolution, file logging is disabled. Defaults to XRAYLABTOOL_LOG_FILE or ~/.cache/xraylabtool/logs/xraylabtool.log.
- **console** (bool | None) – Whether to emit logs to stderr. Defaults to XRAYLABTOOL_LOG_CONSOLE (on).
- **force** (bool) – If True, reconfigure even if logging was already set up (useful in tests).

Return type
Logger

get_logger(*name=None*)

Return a child logger under the xraylabtool namespace.

Calling this function ensures logging is configured once.

Return type
Logger

Parameters
name (*str | None*)

get_log_file_path()

Return the configured log file path if file logging is enabled.

Return type
str | None

log_environment(*logger*, *, *component*, *extra_keys=None*)

Emit a single structured line capturing runtime context.

Return type
None

Parameters

- **logger** (*Logger*)
- **component** (*str*)
- **extra_keys** (*Iterable[tuple[str, str]] | None*)

reset_logging()

Reset logging configuration (intended for tests).

Return type
None

Typing Extensions

Enhanced type definitions for XRayLabTool.

This module provides performance-optimized type aliases and definitions specifically designed for scientific computing applications. It includes specialized NumPy array types, protocol definitions, and type helpers that enable both type safety and high-performance vectorized operations.

Performance Note: - All type definitions use TYPE_CHECKING to avoid runtime overhead - NumPy array types specify dtypes for optimal memory layout - Protocol definitions enable duck typing without inheritance overhead

class AtomicDataProvider

Bases: object

class InterpolatorProtocol

Bases: object

class CalculationEngine

Bases: object

class PerformanceMetrics

Bases: object

validate_energy_array(*energies*)

Validate that input is a proper energy array.

Parameters
energies (Any) – Input to validate

Returns
True if valid energy array

Return type
bool

validate_complex_array(*array*)

Validate that input is a proper complex scattering factor array.

Parameters
array (Any) – Input to validate

Returns
True if valid complex array

Return type
bool

ensure_float64_array(*array*)

Ensure input is converted to float64 array for performance.

Parameters
array (Sequence[float] | NDArray[Any]) – Input array or sequence

Returns
Converted float64 array

Return type
NDArray[]

ensure_complex128_array(*array*)

Ensure input is converted to complex128 array for performance.

Parameters
array (Sequence[float] | NDArray[Any]) – Input array or sequence

Returns
Converted complex128 array

Return type
NDArray[]

is_energy_array(*obj*)

Type guard for energy arrays.

Return type
bool

Parameters
obj (*Any*)

is_complex_array(*obj*)

Type guard for complex arrays.

Return type
bool

Parameters
obj (*Any*)

is_real_array(*obj*)

Type guard for real arrays.

Return type
bool

Parameters
obj (*Any*)

optimize_array_memory_layout(*array*)

Optimize array memory layout for performance.

Parameters

array (NDArray) – Input array

Returns

Memory-optimized array

Return type

NDArray

get_optimal_chunk_size(*array_length*, *memory_limit_mb=100.0*)

Calculate optimal chunk size for batch processing.

Parameters

- **array_length** (int) – Total array length
- **memory_limit_mb** (float) – Memory limit in MB

Returns

Optimal chunk size

Return type

int

Utility functions for XRayLabTool.

This module contains helper functions for data processing, unit conversions, mathematical operations, and other common tasks in X-ray analysis.

angle_from_q(*q*, *wavelength*)

Calculate scattering angle from momentum transfer q.

Parameters

- **q** (float | floating[Any] | NDArray[]) – Momentum transfer in Å^{-1}
- **wavelength** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms

Return type

float

Returns

Scattering angle (2θ) in degrees

background_subtraction(*x*, *y*, *method='linear'*)

Perform background subtraction on diffraction data.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data (used by `'polynomial'` only; one-dimensional, same length as `y`)
- **y** (Sequence[float] | NDArray[Any]) – y-axis data. `'polynomial'` needs one dimension; `'linear'` also accepts an `(N, k)` array of `k` series in columns and subtracts each column's own line (one device call)
- **method** (str) – Background subtraction method ('linear', 'polynomial')

Return type

ndarray

Returns

Background-subtracted y data, float64, the shape of `y`

bragg_angle(*d_spacing*, *wavelength*, *order=1*)

Calculate Bragg angle for given d-spacing and wavelength.

Parameters

- **d_spacing** (float | floating[Any] | NDArray[]) – d-spacing in Angstroms

- **wavelength** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms
- **order** (int) – Diffraction order (default: 1)

Return type
float

Returns
Bragg angle in degrees

d_spacing_cubic(*h*, *k*, *miller_l*, *a*)

Calculate d-spacing for cubic crystal system.

Parameters

- **h** (int) – Miller indices
- **k** (int) – Miller indices
- **l** – Miller indices
- **a** (float | floating[Any] | NDArray[]) – Lattice parameter in Angstroms
- **miller_l** (*int*)

Return type
float

Returns
d-spacing in Angstroms

d_spacing_orthorhombic(*h*, *k*, *miller_l*, *a*, *b*, *c*)

Calculate d-spacing for orthorhombic crystal system.

Parameters

- **h** (int) – Miller indices
- **k** (int) – Miller indices
- **l** – Miller indices
- **a** (float | floating[Any] | NDArray[]) – Lattice parameters in Angstroms
- **b** (float | floating[Any] | NDArray[]) – Lattice parameters in Angstroms
- **c** (float | floating[Any] | NDArray[]) – Lattice parameters in Angstroms
- **miller_l** (*int*)

Return type
float

Returns
d-spacing in Angstroms

d_spacing_tetragonal(*h*, *k*, *miller_l*, *a*, *c*)

Calculate d-spacing for tetragonal crystal system.

Parameters

- **h** (int) – Miller indices
- **k** (int) – Miller indices
- **l** – Miller indices
- **a** (float | floating[Any] | NDArray[]) – a lattice parameter in Angstroms
- **c** (float | floating[Any] | NDArray[]) – c lattice parameter in Angstroms
- **miller_l** (*int*)

Return type
float

Returns
d-spacing in Angstroms

energy_to_wavelength(*energy*, *units='angstrom'*)

Convert X-ray energy to wavelength.

Parameters

- **energy** (float | floating[Any] | NDArray[]) – Energy in keV
- **units** (str) – Desired units of wavelength ('angstrom', 'nm', 'm')

Return type
float

Returns
Wavelength in specified units

find_peaks(*x*, *y*, *prominence=0.1*, *distance=10*)

Find peaks in diffraction data.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data (angle or q values)
- **y** (Sequence[float] | NDArray[Any]) – y-axis data (intensity)
- **prominence** (float | floating[Any] | NDArray[]) – Minimum peak prominence
- **distance** (int) – Minimum distance between peaks in data points

Return type
tuple[ndarray, dict[str, Any]]

Returns
Tuple of (peak_indices, peak_properties)

get_atomic_data(*element_symbol*)

Get comprehensive atomic data for given element symbol with LRU caching.

Parameters
element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Returns

- symbol: Element symbol
- atomic_number: Atomic number
- atomic_weight: Atomic weight in u
- name: Element name
- density: Density in g/cm³ (if available)

Return type
dict[str, Any]

Raises

- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_data
>>> data = get_atomic_data('Si')
>>> data['atomic_number']
14
>>> data['symbol']
'Si'
```

get_atomic_number(*element_symbol*)

Get atomic number for given element symbol with LRU caching.

Parameters
element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Return type
int

Returns
Atomic number as integer

Raises
- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_number
>>> get_atomic_number('H')
1
>>> get_atomic_number('C')
6
>>> get_atomic_number('Si')
14
```

get_atomic_weight(*element_symbol*)

Get atomic weight for given element symbol with LRU caching.

Parameters
element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Return type
float

Returns
Atomic weight in u (atomic mass units)

Raises
- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_weight
>>> round(get_atomic_weight('H'), 3)
1.008
>>> round(get_atomic_weight('C'), 3)
12.011
>>> round(get_atomic_weight('O'), 3)
15.999
```

load_atomic_data(*element_symbol*)

Get comprehensive atomic data for given element symbol with LRU caching.

Parameters
element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Returns
- symbol: Element symbol
- atomic_number: Atomic number

- atomic_weight: Atomic weight in u
- name: Element name
- density: Density in g/cm³ (if available)

Return type
dict[str, Any]

Raises

- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_data
>>> data = get_atomic_data('Si')
>>> data['atomic_number']
14
>>> data['symbol']
'Si'
```

normalize_intensity(*y*, *method='max'*)

Normalize intensity data.

Parameters

- **y** (Sequence[float] | NDArray[Any]) – Intensity data. `'max'` and `'standard'` accept any shape (global maximum, mean and standard deviation) and a 0-d input; `'area'` needs one dimension (`ValueError` otherwise)
- **method** (str) – Normalization method ('max', 'area', 'standard')

Return type
ndarray

Returns
Normalized intensity data, float64, the shape of `y` (a NumPy scalar for a 0-d input)

parse_formula(*formula_str*)

Parse a chemical formula string into element symbols and their counts.

Canonical formula parser for the entire codebase. Supports: - Element symbols with integer or decimal stoichiometry (`SiO2`, `H0.5He0.5`) - Parentheses, including nested (`Ca5(PO4)3OH`, `Ca10(PO4)6(OH)2`)

Parameters
formula_str (str) – Chemical formula string (e.g., "SiO2", "Al2O3", "Ca5(PO4)3OH")

Returns

- element_symbols: List of element symbols as strings
- element_counts: List of corresponding stoichiometric counts as floats

Return type
tuple[list[str], list[float]]

Raises
FormulaError – If formula string is empty, contains unmatched parentheses, or no elements are found.

Examples

```
>>> from xraylabtool.utils import parse_formula
>>> parse_formula("SiO2")
(['Si', 'O'], [1.0, 2.0])
>>> parse_formula("Ca5(PO4)3OH")
(['Ca', 'P', 'O', 'H'], [5.0, 3.0, 13.0, 1.0])
```

progress_bar(*iterable*, *desc='Processing'*)

Create a progress bar for iterations.

Parameters

- **iterable** (Any) – Iterable to wrap
- **desc** (str) – Description for progress bar

Return type

Any | Iterator[Any]

Returns

tqdm progress bar

q_from_angle(*two_theta*, *wavelength*)

Calculate momentum transfer q from scattering angle.

Parameters

- **two_theta** (float | floating[Any] | NDArray[]) – Scattering angle (2θ) in degrees
- **wavelength** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms

Return type

float

Returns

Momentum transfer q in Å⁻¹

save_processed_data(*x*, *y*, *filename*, *header='# X-ray diffraction data'*)

Save processed data to file.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data
- **y** (Sequence[float] | NDArray[Any]) – y-axis data
- **filename** (str) – Output filename
- **header** (str) – Header comment for file

Return type

None

smooth_data(*x*, *y*, *window_size=5*)

Apply moving average smoothing to one-dimensional data (`jax.numpy` convolution).

Edges are replicated; a window of at least `len(y)` returns the mean.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data (not used but kept for consistency)
- **y** (Sequence[float] | NDArray[Any]) – y-axis data to smooth; must be one-dimensional (`ValueError` otherwise)
- **window_size** (int) – Size of smoothing window

Return type

ndarray

Returns
Smoothed y data

`wavelength_to_energy`(*wavelength*, *units='angstrom'*)

Convert X-ray wavelength to energy.

Parameters

- **wavelength** (float | floating[Any] | NDArray[]) – Wavelength value
- **units** (str) – Units of wavelength ('angstrom', 'nm', 'm')

Return type
float

Returns
Energy in keV

Chemical Formula Parsing

`parse_formula`(*formula_str*)

Parse a chemical formula string into element symbols and their counts.

Canonical formula parser for the entire codebase. Supports: - Element symbols with integer or decimal stoichiometry (`SiO2`, `H0.5He0.5`) - Parentheses, including nested (`Ca5(PO4)3OH`, `Ca10(PO4)6(OH)2`)

Parameters
formula_str (str) – Chemical formula string (e.g., "SiO2", "Al2O3", "Ca5(PO4)3OH")

Returns

- element_symbols: List of element symbols as strings
- element_counts: List of corresponding stoichiometric counts as floats

Return type
tuple[list[str], list[float]]

Raises
FormulaError – If formula string is empty, contains unmatched parentheses, or no elements are found.

Examples

```
>>> from xraylabtool.utils import parse_formula
>>> parse_formula("SiO2")
(['Si', 'O'], [1.0, 2.0])
>>> parse_formula("Ca5(PO4)3OH")
(['Ca', 'P', 'O', 'H'], [5.0, 3.0, 13.0, 1.0])
```

Parse a chemical formula string into element symbols and their counts.

Parameters: - `formula_str` (str): Chemical formula string (e.g., "SiO2", "Al2O3", "H0.5He0.5")

Returns: - `tuple`: (element_symbols, element_counts) where element_symbols is a list of element symbols and element_counts is a list of corresponding stoichiometric counts

Examples:

```
from xraylabtool.utils import parse_formula

# Simple compounds
symbols, counts = parse_formula("SiO2")
print(symbols, counts)  # ['Si', 'O'] [1.0, 2.0]
```

(continues on next page)

(continued from previous page)

```
# Complex compounds
symbols, counts = parse_formula("Al2O3")
print(symbols, counts)  # ['Al', 'O'] [2.0, 3.0]

# Fractional compositions
symbols, counts = parse_formula("H0.5He0.5")
print(symbols, counts)  # ['H', 'He'] [0.5, 0.5]
```

Atomic Data Functions

get_atomic_number(*element_symbol*)

Get atomic number for given element symbol with LRU caching.

Parameters

element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Return type

int

Returns

Atomic number as integer

Raises

- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_number
>>> get_atomic_number('H')
1
>>> get_atomic_number('C')
6
>>> get_atomic_number('Si')
14
```

Get atomic number for given element symbol with LRU caching.

Example:

```
from xraylabtool.utils import get_atomic_number

atomic_num = get_atomic_number('Si')
print(atomic_num)  # 14
```

get_atomic_weight(*element_symbol*)

Get atomic weight for given element symbol with LRU caching.

Parameters

element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Return type

float

Returns

Atomic weight in u (atomic mass units)

Raises

- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_weight
>>> round(get_atomic_weight('H'), 3)
1.008
>>> round(get_atomic_weight('C'), 3)
12.011
>>> round(get_atomic_weight('O'), 3)
15.999
```

Get atomic weight for given element symbol with LRU caching.

Example:

```
from xraylabtool.utils import get_atomic_weight

atomic_weight = get_atomic_weight('Si')
print(f"Silicon atomic weight: {atomic_weight:.3f} u")
```

get_atomic_data(*element_symbol*)

Get comprehensive atomic data for given element symbol with LRU caching.

Parameters

element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Returns

- symbol: Element symbol
- atomic_number: Atomic number
- atomic_weight: Atomic weight in u
- name: Element name
- density: Density in g/cm³ (if available)

Return type

dict[str, Any]

Raises

- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_data
>>> data = get_atomic_data('Si')
>>> data['atomic_number']
14
>>> data['symbol']
'Si'
```

Get atomic data for given element symbol with LRU caching.

Example:

```
from xraylabtool.utils import get_atomic_data

data = get_atomic_data('Si')
print(f"Symbol: {data['symbol']}")
print(f"Atomic number: {data['atomic_number']}")
print(f"Atomic weight: {data['atomic_weight']}")
```

load_atomic_data(*element_symbol*)

Get comprehensive atomic data for given element symbol with LRU caching.

Parameters

element_symbol (str) – Element symbol (e.g., 'H', 'C', 'N', 'O', 'Si', 'Al')

Returns

- symbol: Element symbol
- atomic_number: Atomic number
- atomic_weight: Atomic weight in u
- name: Element name
- density: Density in g/cm³ (if available)

Return type

dict[str, Any]

Raises

- **UnknownElementError** – If element symbol is not recognized
- **AtomicDataError** – If there's an issue loading atomic data

Examples

```
>>> from xraylabtool.utils import get_atomic_data
>>> data = get_atomic_data('Si')
>>> data['atomic_number']
14
>>> data['symbol']
'Si'
```

Backward compatibility alias for get_atomic_data.

Unit Conversions

energy_to_wavelength(*energy*, *units='angstrom'*)

Convert X-ray energy to wavelength.

Parameters

- **energy** (float | floating[Any] | NDArray[]) – Energy in keV
- **units** (str) – Desired units of wavelength ('angstrom', 'nm', 'm')

Return type

float

Returns

Wavelength in specified units

Convert X-ray energy to wavelength.

Parameters: - energy (float): Energy in keV - units (str): Desired units of wavelength ('angstrom', 'nm', 'm')

Returns: - float: Wavelength in specified units

Example:

```
from xraylabtool.utils import energy_to_wavelength

# Convert 8 keV to wavelength in Angstroms
wavelength = energy_to_wavelength(8.0)  # 8 keV
print(f"8 keV = {wavelength:.3f} Å")
```

wavelength_to_energy(*wavelength*, *units='angstrom'*)

Convert X-ray wavelength to energy.

Parameters

- **wavelength** (float | floating[Any] | NDArray[]) – Wavelength value
- **units** (str) – Units of wavelength ('angstrom', 'nm', 'm')

Return type
float

Returns
Energy in keV

Convert X-ray wavelength to energy.

Parameters: - `wavelength` (float): Wavelength value - `units` (str): Units of wavelength ('angstrom', 'nm', 'm')

Returns: - `float`: Energy in keV

Example:

```
from xraylabtool.utils import wavelength_to_energy

# Convert 1.55 Å to energy
energy = wavelength_to_energy(1.55)  # Copper K-alpha
print(f"1.55 Å = {energy:.1f} keV")
```

Crystallographic and Diffraction Functions

bragg_angle(*d_spacing*, *wavelength*, *order=1*)

Calculate Bragg angle for given d-spacing and wavelength.

Parameters

- **d_spacing** (float | floating[Any] | NDArray[]) – d-spacing in Angstroms
- **wavelength** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms
- **order** (int) – Diffraction order (default: 1)

Return type
float

Returns
Bragg angle in degrees

Calculate Bragg angle for given d-spacing and wavelength.

Example:

```
from xraylabtool.utils import bragg_angle

# Calculate Bragg angle for Si(111) reflection at 8 keV
d_spacing = 3.14  # Angstroms for Si(111)
wavelength = 1.55  # Angstroms (8 keV)
angle = bragg_angle(d_spacing, wavelength)
print(f"Bragg angle: {angle:.2f} degrees")
```

d_spacing_cubic(*h*, *k*, *miller_l*, *a*)

Calculate d-spacing for cubic crystal system.

Parameters

- **h** (int) – Miller indices
- **k** (int) – Miller indices
- **l** – Miller indices

- **a** (float | floating[Any] | NDArray[]) – Lattice parameter in Angstroms
- **miller_l** (*int*)

Return type
float

Returns
d-spacing in Angstroms

Calculate d-spacing for cubic crystal system.

Example:

```
from xraylabtool.utils import d_spacing_cubic

# Si(111) d-spacing
d = d_spacing_cubic(1, 1, 1, 5.43)  # Si lattice parameter
print(f"Si(111) d-spacing: {d:.3f} Å")
```

d_spacing_tetragonal(*h*, *k*, *miller_l*, *a*, *c*)

Calculate d-spacing for tetragonal crystal system.

Parameters

- **h** (int) – Miller indices
- **k** (int) – Miller indices
- **l** – Miller indices
- **a** (float | floating[Any] | NDArray[]) – a lattice parameter in Angstroms
- **c** (float | floating[Any] | NDArray[]) – c lattice parameter in Angstroms
- **miller_l** (*int*)

Return type
float

Returns
d-spacing in Angstroms

Calculate d-spacing for tetragonal crystal system.

d_spacing_orthorhombic(*h*, *k*, *miller_l*, *a*, *b*, *c*)

Calculate d-spacing for orthorhombic crystal system.

Parameters

- **h** (int) – Miller indices
- **k** (int) – Miller indices
- **l** – Miller indices
- **a** (float | floating[Any] | NDArray[]) – Lattice parameters in Angstroms
- **b** (float | floating[Any] | NDArray[]) – Lattice parameters in Angstroms
- **c** (float | floating[Any] | NDArray[]) – Lattice parameters in Angstroms
- **miller_l** (*int*)

Return type
float

Returns
d-spacing in Angstroms

Calculate d-spacing for orthorhombic crystal system.

q_from_angle(*two_theta*, *wavelength*)

Calculate momentum transfer q from scattering angle.

Parameters

- **two_theta** (float | floating[Any] | NDArray[]) – Scattering angle (2θ) in degrees
- **wavelength** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms

Return type
float

Returns
Momentum transfer q in Å⁻¹

Calculate momentum transfer q from scattering angle.

Example:

```
from xraylabtool.utils import q_from_angle

q = q_from_angle(28.4, 1.55)  # 2theta, wavelength
print(f"Momentum transfer: {q:.3f} Å⁻¹")
```

angle_from_q(*q*, *wavelength*)

Calculate scattering angle from momentum transfer q.

Parameters

- **q** (float | floating[Any] | NDArray[]) – Momentum transfer in Å⁻¹
- **wavelength** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms

Return type
float

Returns
Scattering angle (2θ) in degrees

Calculate scattering angle from momentum transfer q.

Data Processing and Analysis

smooth_data(*x*, *y*, *window_size=5*)

Apply moving average smoothing to one-dimensional data (jax.numpy convolution).

Edges are replicated; a window of at least len(y) returns the mean.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data (not used but kept for consistency)
- **y** (Sequence[float] | NDArray[Any]) – y-axis data to smooth; must be one-dimensional (ValueError otherwise)
- **window_size** (int) – Size of smoothing window

Return type
ndarray

Returns
Smoothed y data

Apply moving average smoothing to data using optimized NumPy convolution.

Example:

```
from xraylabtool.utils import smooth_data
import numpy as np

x = np.linspace(0, 10, 100)
y = np.sin(x) + 0.1 * np.random.randn(100)  # Noisy data
smoothed = smooth_data(x, y, window_size=5)
```

find_peaks(*x*, *y*, *prominence=0.1*, *distance=10*)

Find peaks in diffraction data.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data (angle or q values)
- **y** (Sequence[float] | NDArray[Any]) – y-axis data (intensity)
- **prominence** (float | floating[Any] | NDArray[]) – Minimum peak prominence
- **distance** (int) – Minimum distance between peaks in data points

Return type

tuple[ndarray, dict[str, Any]]

Returns

Tuple of (peak_indices, peak_properties)

Find peaks in diffraction data.

Example:

```
from xraylabtool.utils import find_peaks
import numpy as np

x = np.linspace(0, 50, 1000)
y = np.exp(-((x-20)**2)/10) + np.exp(-((x-30)**2)/5)  # Two peaks
peaks, properties = find_peaks(x, y, prominence=0.1)
print(f"Found {len(peaks)} peaks at x = {properties['x_values']}")
```

background_subtraction(*x*, *y*, *method='linear'*)

Perform background subtraction on diffraction data.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data (used by `'polynomial'` only; one-dimensional, same length as `y`)
- **y** (Sequence[float] | NDArray[Any]) – y-axis data. `'polynomial'` needs one dimension; `'linear'` also accepts an `(N, k)` array of `k` series in columns and subtracts each column's own line (one device call)
- **method** (str) – Background subtraction method ('linear', 'polynomial')

Return type

ndarray

Returns

Background-subtracted y data, float64, the shape of `y`

Perform background subtraction on diffraction data.

normalize_intensity(*y*, *method='max'*)

Normalize intensity data.

Parameters

- **y** (Sequence[float] | NDArray[Any]) – Intensity data. `'max'` and `'standard'` accept any shape (global maximum, mean and standard deviation) and a 0-d input; `'area'` needs one dimension (`ValueError` otherwise)
- **method** (str) – Normalization method ('max', 'area', 'standard')

Return type
ndarray

Returns
Normalized intensity data, float64, the shape of `y` (a NumPy scalar for a 0-d input)

Normalize intensity data.

Example:

```
from xraylabtool.utils import normalize_intensity
import numpy as np

intensity = np.array([100, 200, 150, 300, 250])
normalized = normalize_intensity(intensity, method="max")
print(normalized)  # [0.33, 0.67, 0.5, 1.0, 0.83]
```

Utility Functions

progress_bar(*iterable*, *desc='Processing'*)

Create a progress bar for iterations.

Parameters

- **iterable** (Any) – Iterable to wrap
- **desc** (str) – Description for progress bar

Return type
Any | Iterator[Any]

Returns
tqdm progress bar

Create a progress bar for iterations.

Example:

```
from xraylabtool.utils import progress_bar

for i in progress_bar(range(100), desc="Processing"):
    # Your processing code here
    pass
```

save_processed_data(*x*, *y*, *filename*, *header='# X-ray diffraction data'*)

Save processed data to file.

Parameters

- **x** (Sequence[float] | NDArray[Any]) – x-axis data
- **y** (Sequence[float] | NDArray[Any]) – y-axis data
- **filename** (str) – Output filename
- **header** (str) – Header comment for file

Return type
None

Save processed data to file.

Example:

```
from xraylabtool.utils import save_processed_data
import numpy as np

x = np.linspace(0, 10, 100)
```

(continues on next page)

(continued from previous page)

```
y = np.sin(x)
save_processed_data(x, y, "sine_data.txt", header="# x, sin(x) data")
```

3.10.7 Constants Module

The constants module provides physical constants and conversion factors used throughout XRayLabTool. Physical constants for X-ray calculations.

This module contains fundamental physical constants used throughout X-ray analysis calculations, translated from the Julia implementation with preserved numerical precision. All values are provided with explanatory docstrings describing their physical meaning and units.

The constants are defined with high precision to ensure accurate calculations in X-ray optical property computations.

ANGSTROM_TO_METER: Final[float] = 1e-10

Conversion factor from Angstroms to meters.

Value: 10^{-10} Units: m/Å Usage: length_m = length_angstrom * ANGSTROM_TO_METER

AVOGADRO: Final[float] = 6.02214199e+23

Avogadro's number.

The number of constituent particles (atoms or molecules) per mole. Used in calculations relating molecular properties to bulk material properties.

Value: $6.02214199 \times 10^{23}$ mol^{-1} Units: per mole (mol^{-1}) Reference: CODATA 2018 recommended values

Note: This value maintains compatibility with the Julia implementation. For the most current CODATA value, see: 6.02214076e23 mol^{-1} (exact since 2019).

CM_TO_METER: Final[float] = 0.01

Conversion factor from centimeters to meters.

Value: 0.01 Units: m/cm Usage: length_m = length_cm * CM_TO_METER

DEGREES_TO_RADIANS: Final[float] = 0.017453292519943295

Conversion factor from degrees to radians.

Value: $\pi/180 \approx 0.017453292519943295$ Units: rad/° Usage: angle_rad = angle_deg * DEGREES_TO_RADIANS

ELEMENT_CHARGE: Final[float] = 1.60217646e-19

Elementary charge.

The electric charge of a single proton, equal in magnitude to the charge of an electron. Used in X-ray calculations for energy unit conversions between Joules and eV.

Value: $1.60217646 \times 10^{-19}$ C Units: coulombs (C) Reference: CODATA 2018 recommended values

Note: This value maintains compatibility with the Julia implementation. For the most current CODATA value, see: 1.602176634e-19 C (exact since 2019).

ENERGY_TO_WAVELENGTH_FACTOR: Final[float] = 1.2398417166827828e-09

Pre-computed factor for energy-to-wavelength conversion.

This constant combines fundamental constants to efficiently convert X-ray energies in keV to wavelengths in meters using: λ = ENERGY_TO_WAVELENGTH_FACTOR / E_keV

Mathematical derivation: λ = hc/E = (hc/e) / E_eV = (hc/e) / (1000 × E_keV)

Value: (SPEED_OF_LIGHT × PLANCK / ELEMENT_CHARGE) / 1000.0
= $(2.998 \times 10^{8} \times 6.626 \times 10^{-34} / 1.602 \times 10^{-19}) / 1000 \approx 1.240 \times 10^{-6}$ m·keV

Units: meter-keV (m·keV) Usage: wavelength_m = ENERGY_TO_WAVELENGTH_FACTOR / energy_keV

EV_TO_KEV: Final[float] = 0.001

Conversion factor from eV to keV.

Value: 0.001 Units: keV/eV Usage: energy_keV = energy_eV * EV_TO_KEV

KEV_TO_EV: Final[float] = 1000.0

Conversion factor from keV to eV.

Value: 1000 Units: eV/keV Usage: energy_eV = energy_keV * KEV_TO_EV

METER_TO_ANGSTROM: Final[float] = 10000000000.0

Conversion factor from meters to Angstroms.

Value: 10^{10} Units: Å/m Usage: length_angstrom = length_m * METER_TO_ANGSTROM

METER_TO_CM: Final[float] = 100.0

Conversion factor from meters to centimeters.

Value: 100 Units: cm/m Usage: length_cm = length_m * METER_TO_CM

PI: Final[float] = 3.141592653589793

The mathematical constant π.

The ratio of a circle's circumference to its diameter, appearing frequently in X-ray scattering calculations and Fourier transforms.

Value: 3.141592653589793... Units: dimensionless

PLANCK: Final[float] = 6.626068e-34

Planck constant.

The quantum of action, relating the energy of a photon to its frequency. Essential for X-ray energy-wavelength conversions through E = hc/λ.

Value: 6.626068×10^{-34} J·s Units: joule-seconds (J·s) Reference: CODATA 2018 recommended values

Note: This value maintains compatibility with the Julia implementation. For the most current CODATA value, see: 6.62607015e-34 J·s (exact since 2019).

RADIANS_TO_DEGREES: Final[float] = 57.29577951308232

Conversion factor from radians to degrees.

Value: $180/\pi \approx 57.29577951308232$ Units: °/rad Usage: angle_deg = angle_rad * RADIANS_TO_DEGREES

SCATTERING_FACTOR: Final[float] = 270086523204316.84

Pre-computed factor for X-ray scattering calculations.

This constant combines the Thomson scattering length with Avogadro's number and unit conversion factors, optimized for calculating dispersion and absorption coefficients in X-ray optical property calculations.

Mathematical derivation: Factor = $(r_e \times N_a \times 10^6) / (2\pi)$

Where: - r_e: Thomson scattering length (classical electron radius) - N_a: Avogadro's number - 10^6: Unit conversion factor (cm^3/m^3 to account for density units) - 2π: Geometric factor from X-ray scattering theory

Value: THOMPSON × AVOGADRO × 10^6 / (2π)
≈ 2.70×10^{14} $m^{-1} \cdot mol^{-1}$

Units: per meter per mole ($m^{-1} \cdot mol^{-1}$) Usage: Used in calculating $\delta = (\lambda^2/2\pi) \times r_e \times \rho \times N_a \times (\Sigma f_1) / M$

SPEED_OF_LIGHT: Final[float] = 299792458.0

Speed of light in vacuum.

The speed of electromagnetic radiation in vacuum, a fundamental physical constant that appears in many X-ray calculations involving wavelength-energy conversions.

Value: 299,792,458 m/s (exact by definition) Units: meters per second (m/s) Reference: SI base unit definition

SQRT_2: Final[float] = 1.4142135623730951

Square root of 2.

Mathematical constant that appears in geometric calculations and normalizations in X-ray crystallography.

Value: 1.4142135623730951... Units: dimensionless

TABULATED_MAX_KEV: Final[float] = 30.0

Upper end of the tabulated CXRO/NIST energy range (keV).

TABULATED_MIN_KEV: Final[float] = 0.03

Lower end of the tabulated CXRO/NIST energy range (keV).

THOMPSON: Final[float] = 2.8179403227e-15

Thomson scattering length (classical electron radius).

The Thomson scattering length represents the classical radius of an electron, which is the length scale characterizing the scattering of electromagnetic radiation by a free electron in the classical limit.

Value: $2.8179403227 \times 10^{-15}$ m Units: meters (m) Reference: CODATA 2018 recommended values

TWO_PI: Final[float] = 6.283185307179586

Mathematical constant 2π.

Commonly appears in X-ray calculations involving angular frequencies, momentum transfer, and scattering theory.

Value: 6.283185307179586... Units: dimensionless

attenuation_length_cm(*wavelength_angstrom*, *absorption*)

Calculate X-ray attenuation length from wavelength and absorption coefficient.

Parameters

- **wavelength_angstrom** (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms
- **absorption** (float | floating[Any] | NDArray[]) – Absorption coefficient β (dimensionless)

Return type

float

Returns

Attenuation length (1/e length) in centimeters

Example

```
>>> from xraylabtool.constants import attenuation_length_cm
>>> length = attenuation_length_cm(1.24, 1e-7)
>>> print(f"Attenuation length: {length:.2f} cm")
Attenuation length: 0.01 cm
```

critical_angle_degrees(*dispersion*)

Calculate critical angle for total external reflection from dispersion coefficient.

Parameters

dispersion (float | floating[Any] | NDArray[]) – Dispersion coefficient δ (dimensionless)

Return type

float

Returns
Critical angle in degrees

Example

```
>>> from xraylabtool.constants import critical_angle_degrees
>>> theta_c = critical_angle_degrees(1e-6)
>>> print(f"Critical angle: {theta_c:.4f}°")
Critical angle: 0.0810°
```

energy_to_wavelength_angstrom(*energy_kev*)

Convert X-ray energy in keV to wavelength in Angstroms.

Parameters
energy_kev (float | floating[Any] | NDArray[]) – X-ray energy in keV

Return type
float

Returns
Wavelength in Angstroms

Example

```
>>> from xraylabtool.constants import energy_to_wavelength_angstrom
>>> wavelength = energy_to_wavelength_angstrom(10.0)  # 10 keV
>>> print(f"Wavelength: {wavelength:.4f} Å")
Wavelength: 1.2398 Å
```

validate_constants()

Validate that all constants are properly defined and have reasonable values.

Return type
bool

Returns
True if all constants pass validation

Raises
ValueError – If any constant fails validation

wavelength_angstrom_to_energy(*wavelength_angstrom*)

Convert X-ray wavelength in Angstroms to energy in keV.

Parameters
wavelength_angstrom (float | floating[Any] | NDArray[]) – X-ray wavelength in Angstroms

Return type
float

Returns
Energy in keV

Example

```
>>> from xraylabtool.constants import wavelength_angstrom_to_energy
>>> energy = wavelength_angstrom_to_energy(1.2398)  # Cu Kα₁
>>> print(f"Energy: {energy:.4f} keV")
Energy: 10.0003 keV
```

Usage Examples

Prefer the helpers in `xraylabtool.utils` (`energy_to_wavelength`, `wavelength_to_energy`) over recomputing from raw constants. The constants are exposed for custom derivations:

Energy-Wavelength Conversion:

```
from xraylabtool.constants import ENERGY_TO_WAVELENGTH_FACTOR, METER_TO_ANGSTROM

def energy_to_wavelength_angstrom(energy_kev):
    # ENERGY_TO_WAVELENGTH_FACTOR = h*c/e / 1000, in m·keV
    return ENERGY_TO_WAVELENGTH_FACTOR / energy_kev * METER_TO_ANGSTROM

energy_to_wavelength_angstrom(8.0)  # 1.5498 Å
```

Critical Angle in Degrees:

```
import numpy as np
from xraylabtool.constants import RADIANS_TO_DEGREES

def critical_angle_degrees(delta):
    return np.sqrt(2 * delta) * RADIANS_TO_DEGREES
```

Unit Conversions:

```
from xraylabtool.constants import ANGSTROM_TO_METER, CM_TO_METER, EV_TO_KEV

wavelength_m = 1.55 * ANGSTROM_TO_METER  # Å -> m
length_m = 9.84 * CM_TO_METER            # cm -> m
energy_kev = 8048 * EV_TO_KEV            # eV -> keV
```

Scattering Length Density Prefactor:

```
from xraylabtool.constants import AVOGADRO, THOMPSON

# THOMPSON is the classical electron radius r_e in metres; the δ/β kernels use
# SCATTERING_FACTOR = THOMPSON * AVOGADRO * 1e6 / (2π)  (g/cm³ -> kg/m³ folded in).
```

Constants Reference Table

Table 3: Physical Constants (module attribute, value)

Constant	Attribute	Value	Unit
Planck constant	`PLANCK`	6.626068e-34	J·s
Speed of light	`SPEED_OF_LIGHT`	2.99792458e8	m/s
Elementary charge	`ELEMENT_CHARGE`	1.60217646e-19	C
Avogadro constant	`AVOGADRO`	6.02214199e23	mol^{-1}
Classical electron radius	`THOMPSON`	2.8179403227e-15	m
hc/e ÷ 1000	`ENERGY_TO_WAVELENGTH_FACTOR`	1.23984e-9	m·keV

Table 4: Conversion Factors

Conversion	Attribute	Factor
Å → m / m → Å	`ANGSTROM_TO_METER` / `METER_TO_ANGSTROM`	1e-10 / 1e10
cm → m / m → cm	`CM_TO_METER` / `METER_TO_CM`	1e-2 / 1e2
keV → eV / eV → keV	`KEV_TO_EV` / `EV_TO_KEV`	1e3 / 1e-3
rad → deg / deg → rad	`RADIANS_TO_DEGREES` / `DEGREES_TO_RADIANS`	57.2958 / 0.0174533

CODATA Standards

All physical constants are based on the 2018 CODATA internationally recommended values, ensuring compatibility with modern scientific standards and other physics software packages.

3.10.8 Analysis

Basic analysis functions for X-ray optical properties.

Simplified analysis functionality focused on core scientific calculations.

class MaterialComparator

Bases: object

Compare X-ray properties between multiple materials.

__init__()

Return type
None

compare_materials(
formulas,
densities,
energies,
properties=None,
)

Compare X-ray properties across multiple materials.

Parameters

- **formulas** (list[str]) – List of chemical formulas
- **densities** (list[float]) – List of material densities in g/cm³
- **energies** (list[float]) – List of X-ray energies in keV
- **properties** (list[str] | None) – Properties to compare (uses defaults if None)

Return type

Returns
ComparisonResult with comparison data

create_comparison_table(*result*)

Create a pandas DataFrame from comparison results.

Parameters
result () – ComparisonResult object

Return type
DataFrame

Returns
DataFrame with comparison data

generate_comparison_report(*result*)

Generate a detailed text report from comparison results.

Parameters
result () – ComparisonResult object

Return type
str

Returns
Formatted text report

compare_materials(*results*, *property_name='critical_angle_degrees'*)

Simple material comparison for a given property.

Parameters

- **results** (list[XRayResult]) – list of XRayResult objects to compare
- **property_name** (str) – Property to compare across materials

Return type

dict

Returns

Dictionary with basic statistics

find_absorption_edges(*energies*, *f2_values*, *threshold=0.1*)

Simple absorption edge detection using f2 derivative.

Parameters

- **energies** (ndarray) – Energy array in keV
- **f2_values** (ndarray) – Imaginary scattering factor values
- **threshold** (float) – Minimum derivative threshold for edge detection

Return type

list[tuple[float, float]]

Returns

list of (energy, derivative_magnitude) tuples for detected edges

Material Comparator

Material comparison functionality for X-ray properties analysis.

class ComparisonResult(
materials,
energies,
properties,
data,
summary_stats,
recommendations,
)

Bases: object

Result container for material comparisons.

Parameters

- **materials** (*list[str]*)
- **energies** (*list[float]*)
- **properties** (*list[str]*)
- **data** (*dict[str, Any]*)
- **summary_stats** (*dict[str, dict[str, float]]*)
- **recommendations** (*list[str]*)

materials: list[str]

energies: list[float]

properties: list[str]

data: dict[str, Any]

summary_stats: dict[str, dict[str, float]]

recommendations: list[str]

__init__(
materials,
energies,
properties,
data,
summary_stats,
recommendations,
)

Parameters

- **materials** (*list[str]*)
- **energies** (*list[float]*)
- **properties** (*list[str]*)
- **data** (*dict[str, Any]*)
- **summary_stats** (*dict[str, dict[str, float]]*)
- **recommendations** (*list[str]*)

Return type
None

class MaterialComparator

Bases: object

Compare X-ray properties between multiple materials.

__init__()

Return type
None

compare_materials(
formulas,
densities,
energies,
properties=None,
)

Compare X-ray properties across multiple materials.

Parameters

- **formulas** (list[str]) – List of chemical formulas
- **densities** (list[float]) – List of material densities in g/cm³
- **energies** (list[float]) – List of X-ray energies in keV
- **properties** (list[str] | None) – Properties to compare (uses defaults if None)

Return type

Returns
ComparisonResult with comparison data

create_comparison_table(*result*)

Create a pandas DataFrame from comparison results.

Parameters
result () – ComparisonResult object

Return type
DataFrame

Returns
DataFrame with comparison data

generate_comparison_report(*result*)

Generate a detailed text report from comparison results.

Parameters
result () – ComparisonResult object

Return type
str

Returns
Formatted text report

3.10.9 GUI

GUI entry for XRayLabTool.

This package provides a lightweight Qt-based desktop interface for running single-material and multi-material X-ray optical property calculations and plotting results using PyQtGraph.

Main Window

class MainWindow(*theme_manager=None*)

Bases: *SingleTabMixin*, *MultiTabMixin*, *KKTabMixin*, *SharedHelpersMixin*, QMainWindow

Parameters
theme_manager (*Any | None*)

__init__(*theme_manager=None*)

Parameters
theme_manager (*Any | None*)

Return type
None

staticMetaObject = PySide6.QtCore.QMetaObject("MainWindow" inherits "QMainWindow":)

class Toast(*parent*)

Bases: QLabel

Lightweight, non-blocking toast overlay.

Parameters
parent (*QWidget*)

__init__(*parent*)

Parameters
parent (*QWidget*)

Return type
None

show_toast(*message*, *kind='info'*, *duration_ms=None*)

Return type
None

Parameters

- **message** (*str*)
- **kind** (*str*)
- **duration_ms** (*int | None*)

staticMetaObject = PySide6.QtCore.QMetaObject("Toast" inherits "QLabel":)

Main Window Tabs

class SingleTabMixin

Bases: object

Widgets and behavior for the ‘Single Material’ tab.

threadpool: QThreadPool | None

start_core_warm_up()

Precompile the programs of the Compute buttons on an idle thread.

Called by the launcher once the window is shown, with the formula currently in the Single tab. The worker is not tracked and has no result handlers: it must not touch the window.

Return type
None

Parameters
self (*MainWindowProtocol*)

class MultiTabMixin

Bases: object

Widgets and behavior for the ‘Multiple Materials’ tab.

threadpool: QThreadPool | None

Widgets and behavior for the ‘Kramers-Kronig’ tab (view layer only).

class KKTabMixin

Bases: object

Widgets and behavior for the ‘Kramers-Kronig’ tab.

threadpool: QThreadPool | None

start_kk_warm_up()

Precompile the programs of the tab’s default Compute click on an idle thread.

Called by the launcher once the window is shown. It uses the formula and energy grid currently in the form, so the first click with those values compiles nothing. The worker is not tracked and has no result handlers: it must not touch the KK tab.

Return type
None

Parameters
self (*MainWindowProtocol*)

class SharedHelpersMixin

Bases: object

Window-lifecycle, layout-tuning, and export helpers shared by both tabs.

closeEvent(*event*)

Drop queued-but-unstarted compute jobs so the app shuts down promptly instead of blocking on the global QThreadPool destructor while a large sweep is in flight.

Return type
None

Parameters
event (*QCloseEvent*)

resizeEvent(*event*)

Return type
None

Parameters
event (*Any*)

class Toast(*parent*)

Bases: QLabel

Lightweight, non-blocking toast overlay.

Parameters
parent (*QWidget*)

__init__(*parent*)

Parameters
parent (*QWidget*)

Return type
None

show_toast(*message*, *kind='info'*, *duration_ms=None*)

Return type
None

Parameters

- **message** (*str*)
- **kind** (*str*)
- **duration_ms** (*int | None*)

staticMetaObject = PySide6.QtCore.QMetaObject("Toast" inherits "QLabel":)

Services

Helper utilities for the GUI layer.

class EnergyConfig(*start_kev=8.0*, *end_kev=12.0*, *points=50*, *logspace=False*)

Bases: object

Parameters

- **start_kev** (*float*)
- **end_kev** (*float*)
- **points** (*int*)
- **logspace** (*bool*)

start_kev: float = 8.0

end_kev: float = 12.0

points: int = 50

logspace: bool = False

to_array()

Return type
ndarray

__init__(
start_kev=8.0,
end_kev=12.0,
points=50,
logspace=False,
)

Parameters

- **start_kev** (*float*)
- **end_kev** (*float*)
- **points** (*int*)
- **logspace** (*bool*)

Return type
None

compute_single(*formula*, *density*, *energy_cfg*)

Parameters

- **formula** (*str*)
- **density** (*float*)
- **energy_cfg** (EnergyConfig)

compute_multiple(*formulas*, *densities*, *energy_cfg*, *progress_cb=None*)

Parameters

- **formulas** (*Iterable[str]*)
- **densities** (*Iterable[float]*)
- **energy_cfg** (EnergyConfig)
- **progress_cb** (*Callable[[int], None] | None*)

warm_up_core(*formula='SiO2'*)

Precompile the core programs of the Compute buttons (no Qt, safe in a worker).

Runs `xraylabtool.warm_up()` for `formula` and the two conversion programs behind the Single tab's wavelength/energy fields, so the first click with up to 64 energies compiles nothing. Best effort: a failure is logged, never raised, because nothing the user asked for depends on it.

Return type
None

Parameters
formula (*str*)

linear_absorption_cm(*inv_length_cm*)

Return type
float | None

Parameters
inv_length_cm (*float | None*)

class KKComputation(*result*, *cxro_energy_kev*, *cxro_f1*)

Bases: object

Kramers-Kronig result plus the CXRO f1 totals to compare it with.

Parameters

- **result** (KKResult)
- **cxro_energy_kev** (*NDArray[np.float64]*)
- **cxro_f1** (*NDArray[np.float64]*)

result: *KKResult*

Output of `xraylabtool.kk.kk_calculate_real()`.

cxro_energy_kev: NDArray[float64]

Energies (keV) of `result` inside 0.03-30 keV, shape (k,); empty if none.

__init__(*result*, *cxro_energy_kev*, *cxro_f1*)

Parameters

- **result** (KKResult)
- **cxro_energy_kev** (*NDArray[np.float64]*)
- **cxro_f1** (*NDArray[np.float64]*)

Return type
None

cxro_f1: NDArray[float64]
CXRO f1 per formula unit (electrons) at `cxro_energy_kev`, shape (k,).

warm_up_kk(*formula='SiO2'*, *energies_kev=None*, *refine_tolerance=0.01*)
Precompile the programs of the KK tab's Compute click (no Qt, safe in a worker).

Runs the same public call the tab makes (`kk_calculate_real` with an explicit energy grid), once without and once with refinement, so whatever buckets that call uses are the ones compiled. Best effort: a failure is logged, never raised, because nothing the user asked for depends on it.

Return type
None

Parameters

- **formula** (*str*)
- **energies_kev** (*ArrayLike | None*)
- **refine_tolerance** (*float*)

compute_kk(
formula,
nearedge_path,
input_type,
energy_unit,
merge_points,
refine_tolerance,
energies_kev=None,
)
Run the KK transform and the CXRO f1 lookup (no Qt, safe in a worker).

Parameters

- **formula** (str) – Chemical formula, e.g. `"SiO2"`.
- **nearedge_path** (str | None) – Near-edge data file; `None` uses the tabulated spectrum alone.
- **input_type** (str | None) – Kind of near-edge data (`"nexafs"`, ...); required with a file.
- **energy_unit** (str) – Unit of the near-edge file energies and of `merge_points`.
- **merge_points** (tuple[float, float] | None) – `(low, high)` merge energies in `energy_unit`.
- **refine_tolerance** (float | None) – Adaptive-refinement tolerance; `None` disables refinement.
- **energies_kev** (ArrayLike | None) – Output energies in keV; `None` uses the default grid.

Returns
The transform result and CXRO f1 on its energies within 0.03-30 keV.

Return type
KKComputation

Raises

- **XRayLabToolError** – For an invalid formula, energies or near-edge data, including DataFileError for an unreadable near-edge file.
- **ValueError** – If energy_unit is not "eV" or "keV".
- **TypeError** – If near-edge arguments are inconsistent.

Theme Manager

class ThemeManager(*app*)

Bases: QObject

Manages application theme state and persistence.

Parameters
app (*QApplication*)

theme_changed

__init__(*app*)

Parameters
app (*QApplication*)

Return type
None

property current_palette: *ColorPalette*

Get the active ColorPalette object.

set_theme(*mode*)

Set the active theme mode.

Return type
None

Parameters
mode (*str*)

get_theme()

Get current theme mode string.

Return type
str

toggle_theme()

Toggle between light and dark modes.

Return type
None

apply(*app_instance*)

Apply current theme to application and global resources.

Return type
None

Parameters
app_instance (*QApplication*)

staticMetaObject = PySide6.QtCore.QMetaObject("ThemeManager" inherits "QObject": Methods: #4 type=Signal, signature=theme_changed(QString), parameters=QString)

Workers

Threaded workers for GUI calculations.

class WorkerSignals

Bases: QObject

finished

error

progress

staticMetaObject = PySide6.QtCore.QMetaObject("WorkerSignals" inherits "QObject": Methods: #4 type=Signal, signature=finished(PyObject), parameters=PyObject #5 type=Signal, signature=error(QString), parameters=QString #6 type=Signal, signature=progress(int), parameters=int)

class CalculationWorker(*fn*, **args*, ***kwargs*)

Bases: QRunnable

Parameters

- **fn** (*Callable[..., Any]*)
- **args** (*Any*)
- **kwargs** (*Any*)

__init__(*fn*, **args*, ***kwargs*)

Parameters

- **fn** (*Callable[[...], Any]*)
- **args** (*Any*)
- **kwargs** (*Any*)

Return type

None

run()

Return type

None

Logging Filters

suppress_qt_noise()

Filter noisy Qt offscreen stderr lines (e.g., propagateSizeHints).

Keeps other stderr intact so real warnings still surface.

Return type

Generator[None, None, None]

enable_offscreen_quiet_env()

Set env vars that quiet common Qt offscreen noise without muting real warnings.

Prefers the quieter “minimal” platform when available, otherwise falls back to “offscreen”. Does not override an explicit user choice.

Return type

None

Widgets

Custom widgets for the XRayLabTool GUI.

current_palette()

Detect the active palette by checking the application background color.

Return type
ColorPalette

legend_kwargs(*palette=None*)

Return `addLegend` styling: theme text and a translucent plot-colored box.

Return type
dict[str, Any]

Parameters
palette (ColorPalette | *None*)

apply_palette_to_widget(*pw*, *palette*)

Apply background and foreground colors from palette to a PlotWidget.

Return type
None

Parameters

- **pw** (*Any*)
- **palette** (ColorPalette)

apply_palette_to_item(*pi*, *palette*)

Apply background/foreground colors from palette to a PlotItem.

Return type
None

Parameters

- **pi** (*Any*)
- **palette** (ColorPalette)

Material Form

Input widgets for material definitions and energy ranges.

class MaterialInputForm(*parent=None*)

Bases: `QWidget`

Parameters
parent (*QWidget* | *None*)

__init__(*parent=None*)

Parameters
parent (*QWidget* | *None*)

Return type
None

energy_config()

Return type
EnergyConfig

values()

Return type
tuple[str, float, *EnergyConfig*]

staticMetaObject = PySide6.QtCore.QMetaObject("MaterialInputForm" inherits "QWidget":)

Material Table

Table widget for managing multiple materials.

class MaterialTable(*parent=None*)

Bases: QTableWidget

Parameters
parent (*QWidget | None*)

__init__(*parent=None*)

Parameters
parent (*QWidget | None*)

Return type
None

add_material(*formula*, *density*)

Return type
None

Parameters

- **formula** (*str*)
- **density** (*float*)

remove_selected()

Return type
None

materials()

Return type
tuple[list[str], list[float]]

staticMetaObject = PySide6.QtCore.QMetaObject("MaterialTable" inherits "QTableWidget":)

Plot Canvas

PyQtGraph-based plot canvas widget.

class PlotCanvas(*parent=None*)

Bases: QWidget

Single-panel plot widget backed by PyQtGraph.

Parameters
parent (*QWidget | None*)

__init__(*parent=None*)

Parameters
parent (*QWidget | None*)

Return type
None

set_scales(*log_x*, *log_y*)

Return type
None

Parameters

- **log_x** (*bool*)
- **log_y** (*bool*)

clear()

Return type
None

plot_single(*result*, *property_name*, *ylabel=None*)

Return type
None

Parameters

- **result** (*Any*)
- **property_name** (*str*)
- **ylabel** (*str | None*)

plot_multi(*results*, *property_name*, *ylabel=None*)

Return type
None

Parameters

- **results** (*Mapping[str, Any]*)
- **property_name** (*str*)
- **ylabel** (*str | None*)

update_theme()

Re-apply colors from the currently active palette.

Return type
None

staticMetaObject = PySide6.QtCore.QMetaObject("PlotCanvas" inherits "QWidget":)

Sweep Plots

Scattering factor plot widgets backed by PyQtGraph.

This module contains small PyQtGraph-based widgets used by the GUI.

class F1F2Plot(*parent=None*)

Bases: QWidget

Single-panel plot showing f1 and f2 vs energy for one material.

Parameters
parent (*QWidget | None*)

__init__(*parent=None*)

Parameters
parent (*QWidget | None*)

Return type
None

clear()

Return type
None

render_result(*result*)

Return type
None

Parameters
result (*Any*)

update_theme()

Re-apply colors from the currently active palette.

Return type
None

staticMetaObject = PySide6.QtCore.QMetaObject("F1F2Plot" inherits "QWidget":)

class MultiF1F2Plot(*parent=None*)

Bases: QWidget

Two-panel plot comparing f1 (top) and f2 (bottom) for multiple materials.

Parameters
parent (*QWidget | None*)

__init__(*parent=None*)

Parameters
parent (*QWidget | None*)

Return type
None

clear()

Return type
None

render_multi(*results*)

Render f1 and f2 vs energy for multiple materials.

Parameters
results (Mapping[str, Any]) – Mapping of formula -> XRayResult-like objects.

Return type
None

update_theme()

Re-apply colors from the currently active palette.

Return type
None

staticMetaObject = PySide6.QtCore.QMetaObject("MultiF1F2Plot" inherits "QWidget":)

Kramers-Kronig Plot

Kramers-Kronig plot widget backed by PyQtGraph.

class KKPlot(*parent=None*)

Bases: QWidget

Plot of measured/spliced f2, KK f1 and CXRO f1, each on its own grid.

Parameters
parent (*QWidget | None*)

__init__(*parent=None*)

Parameters
parent (*QWidget | None*)

Return type
None

set_data(*computation*)

Draw the four curves; curves without data are hidden.

Parameters

computation (*KKComputation*) – Result and CXRO comparison; every curve keeps its own energy grid.

Return type

None

update_theme()

Re-apply colors from the currently active palette.

Return type

None

staticMetaObject = PySide6.QtCore.QMetaObject("KKPlot" inherits "QWidget":)

Scrollbar Helper

Overlay scrollbar margin helper for QScrollArea widgets.

class OverlayScrollbarMarginHelper(*parent*, *target*)

Bases: QObject

Adjusts viewport margins so overlay scrollbars don't cover content.

Install on a QScrollArea to automatically add right margin when the vertical scrollbar overlaps the viewport (common with overlay-style scrollbars on macOS).

Parameters

- **parent** (*QObject*)
- **target** (*QScrollArea*)

__init__(*parent*, *target*)

Parameters

- **parent** (*QObject*)
- **target** (*QScrollArea*)

Return type

None

eventFilter(*watched*, *event*)

Return type

bool

Parameters

- **watched** (*QObject*)
- **event** (*QEvent*)

apply_margins()

Return type

None

staticMetaObject = PySide6.QtCore.QMetaObject("OverlayScrollbarMarginHelper" inherits "QObject":)

Styling and Table Formatting

Shared Qt styling for the XRayLabTool GUI.

class ColorPalette(
name,
window_bg,
panel_bg,

input_bg,
text_primary,
text_secondary,
border,
border_focus,
accent,
accent_hover,
accent_text,
error,
error_bg,
success,
plot_bg,
plot_cycle,
)

Bases: `object`

Semantic color tokens for the GUI.

Parameters

- **name** (*str*)
- **window_bg** (*str*)
- **panel_bg** (*str*)
- **input_bg** (*str*)
- **text_primary** (*str*)
- **text_secondary** (*str*)
- **border** (*str*)
- **border_focus** (*str*)
- **accent** (*str*)
- **accent_hover** (*str*)
- **accent_text** (*str*)
- **error** (*str*)
- **error_bg** (*str*)
- **success** (*str*)
- **plot_bg** (*str*)
- **plot_cycle** (*list[str]*)

name: str

window_bg: str

panel_bg: str

input_bg: str

text_primary: str

text_secondary: str

border: str

border_focus: str

accent: str

accent_hover: str

accent_text: str

error: str

error_bg: str

success: str

plot_bg: str

plot_cycle: list[str]

to_qpalette()

Convert to QPalette.

Return type

QPalette

__init__(
name,
window_bg,
panel_bg,
input_bg,
text_primary,
text_secondary,
border,
border_focus,
accent,
accent_hover,
accent_text,
error,
error_bg,
success,
plot_bg,
plot_cycle,
)

Parameters

- **name** (*str*)
- **window_bg** (*str*)
- **panel_bg** (*str*)
- **input_bg** (*str*)
- **text_primary** (*str*)
- **text_secondary** (*str*)
- **border** (*str*)
- **border_focus** (*str*)
- **accent** (*str*)
- **accent_hover** (*str*)
- **accent_text** (*str*)
- **error** (*str*)
- **error_bg** (*str*)
- **success** (*str*)
- **plot_bg** (*str*)
- **plot_cycle** (*list[str]*)

Return type
None

get_qss(*t*)

Generate QSS string from palette.

Return type
str

Parameters
t (ColorPalette)

apply_theme(*app*, *theme*)

Apply palette and stylesheet to the QApplication.

Return type
None

Parameters

- **app** (*QApplication*)
- **theme** (ColorPalette)

apply_styles(*app*)

Entry point: Defaults to Dark Industrial Theme.

Return type
None

Parameters
app (*QApplication*)

apply_pyqtgraph_theme(*theme=DARK_INDUSTRIAL*)

Apply a PyQtGraph theme aligned with the GUI palette.

Return type
None

Parameters
theme (ColorPalette)

Shared table-cell formatting for X-ray result rows.

class TableFormatter

Bases: object

Formats X-ray calculation results into display-ready cell strings.

static format_single_row(*result*, *index*)

Format one energy point of a single-material result.

Returns a list of strings matching the single-material table columns: Energy, Wavelength, delta, beta, Critical Angle, mrad, Atten Length, mu, f1, f2, Re(SLD), Im(SLD).

Return type
list[str]

Parameters

- **result** (*Any*)
- **index** (*int*)

static format_multi_row(*formula*, *result*, *index*)

Format one energy point of a multi-material result.

Returns a list of strings matching the multi-material table columns: Formula, Density, Energy, Wavelength, delta, beta, Critical Angle, mrad, Atten Length, mu, f1, f2, Re(SLD), Im(SLD).

Return type
list[str]

Parameters

- **formula** (*str*)
- **result** (*Any*)
- **index** (*int*)

static format_summary(*result*)

Format the single-material summary row.

Returns: [formula, molecular_weight, density, electron_density, total_electrons].

Return type
list[str]

Parameters
result (*Any*)

3.10.10 Backend

The backend package holds the JAX runtime helpers, the shape buckets that bound compile counts, and the in-tree PCHIP interpolator. XRayLabTool runs on JAX only (ADR-006); `set_backend("jax")` is a no-op, `set_backend("numpy")` raises `ValueError`, and `get_backend()` returns `"jax"`. Backend helpers: JAX runtime, shape buckets and the in-tree PCHIP interpolator.

The NumPy backend and the `ops` dispatch layer were removed (ADR-006); this package keeps `set_backend`/`get_backend` as explicit stubs so existing callers fail loudly instead of silently running a different code path. It imports nothing heavy: `runtime`, `shapes` and `interpolation` are loaded by their users.

get_backend()

Return the name of the only backend, `"jax"`.

Return type
str

set_backend(*name*)

Accept `"jax"` (a no-op); reject every other name.

Parameters
name (str) – Backend name. Only `"jax"` is valid.

Raises
ValueError – For any name other than `"jax"`, including `"numpy"`.

Return type
None

Runtime

JAX runtime policy: x64, host/device transfers, compile cache, worker guard.

Everything that crosses the host/device boundary goes through *upload()*, *fetch_status()* or *fetch()*; each opens a narrow `jax.transfer_guard("allow")` scope and bumps a counter, which is what the transfer-budget tests assert.

Importing this module has no side effect: it does not call *ensure_x64()*, enable the compile cache or query a device. Callers opt in.

References

Design: `docs/superpowers/specs/2026-10-26-package-jax-first-design.md` §3; `docs/architecture/005-adr-kk-jax-only.md`.

enable_persistent_cache()

Turn on JAX's persistent compile cache in the OS user-cache directory.

Called by the `calc`, `batch`, `compare` and `kk` CLI commands and the GUI launcher; the library calls it only when `XRAYLABTOOL_JAX_CACHE=1` is set (a library must not change the host application's global JAX configuration unasked). Does nothing when `XRAYLABTOOL_NO_JAX_CACHE=1` or when the user already set `jax_compilation_cache_dir`. When it enables the cache it also sets `jax_persistent_cache_min_compile_time_secs` to 0 (JAX's default of 1 s would skip most kk programs) unless the user configured that threshold; the lock makes concurrent calls single-flight.

Returns

Whether this call enabled the cache.

Return type

bool

References

Design spec §10 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.backend.runtime import enable_persistent_cache
>>> isinstance(enable_persistent_cache.__doc__, str)
True
```

ensure_x64()

Enable 64-bit JAX arrays (a no-op when already on).

Called at every concrete public entry before any upload or trace: with x64 off a float64 buffer would be canonicalised to float32 inside a later trace. The first call also enables the persistent compile cache when the process sets `XRAYLABTOOL_JAX_CACHE=1` (see *enable_persistent_cache()*).

References

Design spec §4 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.backend.runtime import ensure_x64
>>> ensure_x64()
```

Return type

None

fetch[*T*](*tree*)

`jax.device_get` of the final output pytree: one counted transfer.

Return type

T

Parameters

tree (*T*)

fetch_status[*T*](*tree*)

`jax.device_get` of a small status pytree: one counted transfer.

Return type

T

Parameters

tree (*T*)

reset_transfer_counts()

Set all boundary counters to zero.

Return type
None

thread_transfer_counts()

Boundary calls made by the calling thread so far, by kind.

Unlike *transfer_counts()* it is not disturbed by other threads (GUI workers), so a wrapper can assert its own per-call budget from the difference of two readings. *reset_transfer_counts()* does not reset it.

Return type
dict[str, int]

transfer_counts()

Number of boundary calls so far, by kind (`upload`, `fetch_status`, `fetch`).

Return type
dict[str, int]

upload[*T*](*tree*)

`jax.device_put` of a whole pytree: one counted host-to-device transfer.

Return type
T

Parameters
tree (*T*)

worker_guard()

The transfer-guard scope a worker entry opens around a compute call under tests.

The guard is thread-local, so every worker thread opens its own. It is on when `XRAYLABTOOL_TRANSFER_GUARD` is set (the test suite sets it in `tests/conftest.py`; subprocesses inherit it) and a no-op otherwise.

References

Design spec §1 and §10 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.backend.runtime import worker_guard
>>> with worker_guard():
...     pass
```

Return type
AbstractContextManager[Any]

Shapes

Static-shape buckets for the JAX device programs.

Compiled JAX programs specialise on array shapes, so host wrappers round every true size up to one of a few bucket sizes and carry the true count alongside. The ladder is a deterministic recurrence (nothing is hard-coded): b_0 is the minimum and $b_{k+1} = 8 \cdot \lfloor 1.25 \cdot b_k / 8 \rfloor$, so every bucket is a multiple of 8 and, above the minimum, padding wastes at most 25 %. The core device programs pad the energy axis with *bucket_energy()* instead: powers of two up to 4096 (a compile costs far more than the padded work on a small axis), the same ladder above.

Importing this module has no side effect.

References

Design: `docs/superpowers/specs/2026-10-25-kk-jax-first-design.md` §3.

block_for(*chunk_size*, *capacity*, *m*, *elements=CHUNK_ELEMENTS*)

Energies per block: a ladder or power-of-two value never above the request.

The upper bound is `min(chunk_size, elements // capacity, m)` (at least 1); the result is the largest power of two or *bucket()* ladder value that does not exceed it, so a requested `chunk_size` is never exceeded.

Parameters

- **chunk_size** (`int | None`) – Requested maximum block size; `None` leaves only the memory bound.
- **capacity** (`int`) – Segment capacity of the spectrum (columns of each temporary).
- **m** (`int`) – Number of energies to evaluate.
- **elements** (`int`) – Element budget of one `(block, capacity)` temporary.

Returns

Block size, at least 1.

Return type

`int`

References

Design spec §3-§4 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.backend.shapes import block_for
>>> block_for(7, 1228, 1000), block_for(None, 1228, 1)
(4, 1)
```

bucket(*n*, *minimum=512*)

Smallest ladder value that is at least `n`.

Parameters

- **n** (`int`) – True size (non-negative).
- **minimum** (`int`) – First rung of the ladder; a multiple of 8 and at least 64 (the recurrence stalls at 24 and below; 64 is a conservative floor).

Returns

`b ≥ n`, a multiple of 8; `b ≤ 1.25·n` whenever `n > minimum`.

Return type

`int`

Raises

ValueError – If `minimum` is not a multiple of 8 or is below 64, or `n < 0`.

References

Design spec §3 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.backend.shapes import bucket
>>> [bucket(n) for n in (1, 512, 513, 1001)]
[512, 512, 640, 1248]
```

bucket_energy(*n*)

Energy-axis bucket of the core programs: coarse when small, fine when large.

A new bucket is a new compile (hundreds of milliseconds), while padding a short axis costs microseconds, so up to 4096 the rungs are powers of two (at most 7 compiles for the whole range); above, the *bucket()* ladder keeps the padding below 25 %.

Parameters

n (int) – True number of energies (non-negative).

Returns

`b ≥ max(n, 64)`, a power of two up to 4096 and a ladder rung above.

Return type

int

Raises

ValueError – If `n < 0`.

Examples

```
>>> from xraylabtool.backend.shapes import bucket_energy
>>> [bucket_energy(n) for n in (1, 64, 65, 500, 4096, 4097)]
[64, 64, 128, 512, 4096, 5120]
```

bucket_pow2(*n*, *minimum=8*)

Smallest power of two that is at least `max(n, minimum)`.

Parameters

- **n** (int) – True size.
- **minimum** (int) – Lower bound on the result.

Returns

A power of two.

Return type

int

References

Design spec §3 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.backend.shapes import bucket_pow2
>>> [bucket_pow2(n) for n in (1, 8, 9, 100)]
[8, 8, 16, 128]
```

Interpolation

class JaxPchipInterpolator(*x*, *y*)

Bases: object

Monotone piecewise-cubic Hermite (PCHIP) interpolator in `jax.numpy`.

Reproduces `scipy.interpolate.PchipInterpolator(x, y, extrapolate=False)` to float64 round-off, so it matches SciPy. Queries outside `[x[0], x[-1]]` return NaN. Evaluation is jit-compiled and the node slopes are computed once at construction.

Parameters

- **x** (Any) – Strictly increasing sample points, `n >= 2`.
- **y** (Any) – Values at `x`.

Raises

`ValueError` – If `x` and `y` are not 1-D of equal length `>= 2`, contain complex or non-finite values, or `x` is not strictly increasing.

Notes

Construction and evaluation use local x64 scopes so another caller's disabled-x64 setting cannot reduce the interpolation precision.

__init__(*x*, *y*)

Parameters

- **x** (*Any*)
- **y** (*Any*)

Return type

None

3.10.11 Kramers-Kronig (kk)

The `kk` sub-package computes the real scattering factor f1 from the tabulated imaginary part f2, optionally spliced with measured near-edge data, by the piecewise Laurent polynomial method of Watts (2014). Background: *Kramers-Kronig Transform*; usage: *Kramers-Kronig Guide*.

Public names are exported lazily from `xraylabtool.kk` (`import xraylabtool` does not import it). Public functions take energies in keV, except the low-level functions, which work in eV like the coefficient tables.

Which function implements which equation

Equation numbers refer to B. Watts, *Opt. Express* **22**(19), 23628 (2014).

Name (xraylabtool.kk)	Paper	Notes
kk_piecewise_poly	Eqs. (7), (10)	Closed-form f1, a_{-3} term with the sign of Table 1 (erratum); knot and end-point conventions from upstream (D5). Energies in eV.
kk_real_spectrum	Eq. (10)	Validated, chunked wrapper of kk_piecewise_poly (eV).
kk_f1	Eq. (10)	Device-level f1 of a padded Spectrum (eV); jit/vmap/grad compatible.
Spectrum	Eq. (8)	Padded piecewise-polynomial f2 table, a JAX pytree (eV).
relativistic_correction	Eq. (9)	Z* of a formula unit.
evaluate_f2	Eq. (8)	f2 from the coefficient table, right-continuous at knots (eV).
compound_spectrum	Eq. (8), step 1	Stoichiometric sum of elemental tables on the union of their knots.
load_asf, AsfDatabase	Eq. (8) data	Vendored kkcalc ASF.json tables, 10 eV-500 keV.
NearEdgeSpectrum, load_nearedge	step 2	Two-column near-edge file reader and validated container.
to_asf_scale	step 2	Converts NEXAFS/photoabsorption, beta or ASF data to the f2 scale.
merge_spectra	step 2	Affine endpoint matching of the near-edge data into the table.
refine_spectrum, RefineOutcome	not in the paper	Upstream improve_accuracy: adaptive midpoint insertion.
kk_calculate_real, KKResult	steps 1-6	High-level entry point (keV) and its result.
warm_up	not in the paper	Precompiles the default device programs for given formulas (the GUI warms its own Compute call the same way; see *Kramers-Kronig Guide*).
calculate_properties_from_factors	not in the paper	xraylabtool.calculators.core: δ, β and derived quantities from f1/f2 totals; used by KKResult.to_xray_result.

Package

Kramers-Kronig transform of X-ray scattering factors (kkcalc port).

Public names are imported lazily on first access, so `import xraylabtool.kk` stays cheap until a function is used.

High-level API

High-level Kramers-Kronig calculation of f1 from the tabulated f2.

Altered from the original: a re-implementation of kk_calculate_real from kkcalc/kk.py of the Kramers-Kronig Calculator (kkcalc, https://github.com/benajamin/kkcalc, commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), Copyright (c) 2013 Benjamin Watts, Daniel J. Lauk, zlib/libpng licence (full text in xraylabtool/data/kkcalc/LICENSE-kkcalc.txt; see also NOTICE). The method is described in B. Watts, Opt. Express 22(19), 23628-23639 (2014), doi:10.1364/OE.22.023628.

The public API takes energies in keV; the transform runs in eV (the tabulated coefficients are in eV) after a single conversion at the boundary.

The numerical part is a device pipeline (xraylabtool.kk._pipeline, *xraylabtool.kk.refine*): one upload of all inputs, one status fetch after assembly and merge, one per refinement pass, one final download.

kk_calculate_real(
formula,
*,
energies_kev=None,
chunk_size=None,

nearedge=None,
input_type=None,
merge_points_kev=None,
refine_tolerance=None,
refine_max_passes=50,
refine_max_points=100_000,
)

Real part f1 of the scattering factor from the tabulated f2.

Evaluates the Kramers-Kronig transform of the tabulated imaginary scattering factor of `formula` (Henke tables, 10 eV - 500 keV) and the f2 spectrum itself.

Parameters

- **formula** (str) – Chemical formula, e.g. `"SiO2"` or `"Ca5(PO4)3OH"`.
- **energies_kev** (ArrayLike | None) – Photon energies in keV, 0.01 to 500, finite, any order, repeats allowed. Default: the knots of the tabulated spectrum (union of the elemental grids) inside 0.03-30 keV; the number of points clipped away is logged at INFO. A `jax.Array` is validated on the host, so it costs one extra counted fetch (input preparation) before the calculation's own single upload.
- **chunk_size** (int | None) – Energies per block of the transform, bounding memory; the result does not depend on it beyond rounding. Also used for the refinement midpoints. Default: platform dependent, at most about 2 MiB (CPU) or 32 MiB (accelerator) per float64 temporary.
- **nearedge** (*NearEdgeSpectrum* | None) – Measured near-edge absorption data (energies in eV, see *load_nearedge()*). It is converted to the f2 scale, normalised to the tabulated f2 at the merge points and replaces the tabulated spectrum between them.
- **input_type** (Literal['nexafs', 'photoabsorption', 'xanes', 'beta', 'asf'] | None) – Kind of `nearedge` values; required with `nearedge` (see *to_asf_scale()*).
- **merge_points_kev** (tuple[float, float] | None) – `(low, high)` merge energies in keV, inside the near-edge data and the tabulated range. Default: the whole overlap of the two ranges. Only with `nearedge`.
- **refine_tolerance** (float | None) – Absolute interpolation error (electrons per formula unit), finite and positive. If given, midpoints are inserted wherever linear interpolation of f1 or f2 between neighbouring energies is off by more than this (see *refine_spectrum()*), after any near-edge merge. The output is then sorted and de-duplicated: the unique requested energies (keV values that are equal in eV count once) plus the added midpoints. Without it, the energies keep the caller's order.
- **refine_max_passes** (int) – Maximum bisection passes of the refinement; at least 1.
- **refine_max_points** (int) – Maximum number of points the refinement may add; at least 1.

Returns

`f1` and `f2` in electrons per formula unit at `energy_kev`. With near-edge data, `merge_points_kev` holds the merge energies used and `nearedge_scaled_kev_f2` the normalised near-edge data `(energy_kev, f2)` on its own grid. The default output grid is then the merged spectrum's knots. `n_refined_points` counts the points added by refinement (0 without `refine_tolerance`).

Return type

KKResult

Raises

- **FormulaError** – If the formula is not a string, is empty or malformed, contains an element outside Z = 1..92, or has a non-positive composition.

- **EnergyError** – If `energies_kev` is empty, not 1-D, not finite, or outside 0.01-500 keV (nothing is extrapolated); or, without `energies_kev`, if no knot of the (merged) spectrum lies inside 0.03-30 keV, so the default grid would be empty.
- **ValidationError** – If `chunk_size`, `refine_max_passes` or `refine_max_points` is not a positive integer, `refine_tolerance` is not a finite positive number, or `input_type` is unknown.
- **TypeError** – If `nearedge` is given without `input_type`, is not a *NearEdgeSpectrum*, or `input_type`/`merge_points_kev` is given without `nearedge`.
- **NearEdgeDataError** – If the near-edge data cannot be merged (no overlap with the tabulated range, bad merge points, flat end points, non-finite normalised values); see *merge_spectra()*.

Notes

Core energies may lie anywhere in 10 eV - 500 keV; only the conversion to an `XRayResult` is restricted to 0.03-30 keV. The transform is numerically exact for any energy but f1 below ~20 eV for heavy elements is good to about 1e-4 absolute (see *kk_piecewise_poly()*). Within a few eV of 10 eV or 500 keV, f1 is dominated by the truncation of the integral at the data range, not by the data.

Near-edge values that are negative after normalisation are kept and logged at WARNING, never clipped.

Refinement never drops a requested energy; it logs the points added per pass (DEBUG) and in total (INFO), and a WARNING if it stops on `refine_max_passes` or `refine_max_points` before the tolerance is met everywhere.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eqs. (7)-(10): f1 from the piecewise-polynomial f2. The keV interface, validation and default window are this package's additions; the call sequence follows upstream `kkcalc.kk.kk_calculate_real` (near-edge merge: see *merge_spectra()*).

Examples

```
>>> from xraylabtool.kk import NearEdgeSpectrum, kk_calculate_real
>>> result = kk_calculate_real("Si", energies_kev=[1.0, 2.0])
>>> result.energy_kev.tolist()
[1.0, 2.0]
>>> bool((result.f1 > 0).all() and (result.f2 > 0).all())
True
>>> result.backend
'jax'
>>> edge = NearEdgeSpectrum(
...     [280.0, 290.0, 300.0, 310.0, 320.0], [0.05, 0.1, 0.3, 0.45, 0.5]
... )
>>> merged = kk_calculate_real(
...     "C", nearedge=edge, input_type="nexafs", merge_points_kev=(0.28, 0.32)
... )
>>> merged.merge_points_kev
(0.28, 0.32)
>>> 0.29 in merged.energy_kev
True
>>> refined = kk_calculate_real(
...     "Si", energies_kev=[5.0, 0.1, 5.0], refine_tolerance=0.05
... )
>>> refined.n_refined_points > 0 and bool((refined.energy_kev[1:] > refined.energy_kev[:-1]).
↪all())
True
```

warm_up(*formulas=('SiO2',)*, *refine_tolerance=0.01*)

Compile the programs a default call of each formula touches.

Runs the default-grid calculation of every formula, and the refined one when `refine_tolerance` is given (the tolerance decides which active buckets the refinement visits), so later calls with the same buckets compile nothing. Idempotent; returns after the results are on the host, i.e. after everything is compiled and executed. The GUI does not call it: it warms the programs of its own Compute click on an idle worker (`gui.services.warm_up_kk`). The CLI never warms up, so a one-shot call never pays for formulas it was not asked for.

Parameters

- **formulas** (`Sequence`[`str`]) – Chemical formulas whose default calculations are compiled.
- **refine_tolerance** (`float` | `None`) – Also compile a refined run at this tolerance; `None` skips it.

Raises

FormulaError – If a formula is invalid (as for *kk_calculate_real()*).

Return type

`None`

References

Design spec §10 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.kk import warm_up
>>> warm_up(("Si",), refine_tolerance=None)
```

Result

Result container of the Kramers-Kronig calculation.

class KKResult(
formula,
energy_kev,
f1,
f2,
relativistic_correction,
merge_points_kev,
n_refined_points,
backend,
nearedge_scaled_kev_f2=None,
refine_exhausted=False,
)

Bases: `object`

Real and imaginary scattering factors of a formula unit.

The dataclass is frozen and compares by identity (`eq=False`): the array fields have no element-wise-safe `==` or `hash`.

Parameters

- **formula** (*str*)
- **energy_kev** (*NDArray[float64]*)
- **f1** (*NDArray[float64]*)
- **f2** (*NDArray[float64]*)
- **relativistic_correction** (*float*)

- **merge_points_kev** (*tuple[float, float] | None*)
- **n_refined_points** (*int*)
- **backend** (*Literal['numpy', 'jax']*)
- **nearedge_scaled_kev_f2** (*tuple[NDArray[float64], NDArray[float64]] | None*)
- **refine_exhausted** (*bool*)

formula

Chemical formula as given.

Type
str

energy_kev

Photon energies in keV, in the order they were calculated.

Type
numpy.ndarray, shape (m,)

f1

Real part of the scattering factor, electrons per formula unit.

Type
numpy.ndarray, shape (m,)

f2

Imaginary part of the scattering factor, electrons per formula unit.

Type
numpy.ndarray, shape (m,)

relativistic_correction

Relativistic correction Z* (electrons) added to the transform.

Type
float

merge_points_kev

Energies (keV) at which near-edge data was merged into the tabulated spectrum, or `None` when the tabulated spectrum alone was used. Stored as a tuple of floats.

Type
tuple of float or None

n_refined_points

Number of points added by adaptive refinement (0 if not refined).

Type
int

backend

Array backend that ran the transform. `kk_calculate_real` always reports `"jax"`; `"numpy"` is still accepted for results built directly (legacy).

Type
str

nearedge_scaled_kev_f2

`(energy_kev, f2)` of the near-edge data on the scattering-factor scale, on its own energy grid; `None` without near-edge data.

Type
tuple of numpy.ndarray or None

refine_exhausted

`True` if adaptive refinement stopped on a limit (`refine_max_passes`, `refine_max_points` or float spacing) before every interval met the tolerance; `False` when it converged or was not requested.

Type
bool

References

Upstream-only/package addition: a container for the output of the transform of B. Watts, Opt. Express 22(19), 23628 (2014), Eqs. (7)-(10) (Z* from Eq. (9)).

Examples

```
>>> from xraylabtool.kk import kk_calculate_real
>>> result = kk_calculate_real("Si", energies_kev=[1.0, 2.0])
>>> result.f1.shape == result.f2.shape == result.energy_kev.shape
True
>>> result.n_refined_points
0
```

to_xray_result(*density*)

Convert to an `XRayResult`.

Computes δ, β, the critical angle, attenuation length and SLD from `f1` and `f2` of this result, with the same code as `calculate_single_material_properties`. Negative `f1` near an absorption edge is kept (critical angle 0 there); the energy order is preserved. `XRayResult` has no field for it, so `refine_exhausted` is not carried over: check it on this result first.

Parameters
density (`float`) – Mass density in g/cm^3; finite and positive.

Returns
Optical properties at `energy_kev`, arrays of shape `(m,)`.

Return type
XRayResult

Raises

- **EnergyError** – If any energy lies outside the tabulated 0.03-30 keV range of the package. This is the only place the range is enforced; the transform itself accepts 10 eV-500 keV.
- **ValidationError** – If `density` is not finite and positive.

References

Package addition: the adapter to the repository's result type; the transform it carries is B. Watts, Opt. Express 22(19), 23628 (2014), Eqs. (7)-(10) (Z* from Eq. (9)).

Examples

```
>>> from xraylabtool.kk import kk_calculate_real
>>> kk = kk_calculate_real("Si", energies_kev=[2.0, 8.0])
>>> xr = kk.to_xray_result(2.33)
>>> xr.energy_kev.shape
(2,)
>>> bool((xr.scattering_factor_f1 == kk.f1).all())
True
```

```
__init__(
    formula,
    energy_kev,
    f1,
    f2,
    relativistic_correction,
    merge_points_kev,
    n_refined_points,
    backend,
    nearedge_scaled_kev_f2=None,
    refine_exhausted=False,
)
```

Parameters

- **formula** (*str*)
- **energy_kev** (*NDArray[float64]*)
- **f1** (*NDArray[float64]*)
- **f2** (*NDArray[float64]*)
- **relativistic_correction** (*float*)
- **merge_points_kev** (*tuple[float, float] | None*)
- **n_refined_points** (*int*)
- **backend** (*Literal['numpy', 'jax']*)
- **nearedge_scaled_kev_f2** (*tuple[NDArray[float64], NDArray[float64]] | None*)
- **refine_exhausted** (*bool*)

Return type
None

Transform kernel

Piecewise-polynomial Kramers-Kronig transform kernel.

Altered from the original: a broadcasting `jax.numpy` re-implementation of `KK_PP` and `calc_relativistic_correction` from kkcalc/kk.py of the Kramers-Kronig Calculator (kkcalc, https://github.com/benajamin/kkcalc, commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), Copyright (c) 2013 Benjamin Watts, Daniel J. Lauk, zlib/libpng licence (full text in `xraylabtool/data/kkcalc/LICENSE-kkcalc.txt`; see also `NOTICE`). The method is described in B. Watts, Opt. Express 22(19), 23628-23639 (2014), doi:10.1364/OE.22.023628.

Energies are in eV throughout this module (the tabulated coefficients are in eV); conversion from the keV public API happens in the callers.

relativistic_correction(*atomic_numbers*, *counts*)

Relativistic correction Z* added to the Kramers-Kronig transform.

Overloads

- **atomic_numbers** (Sequence[int]), **counts** (Sequence[float]) → float
- **atomic_numbers** (Array), **counts** (Sequence[float] | Array) → Array
- **atomic_numbers** (Sequence[int]), **counts** (Array) → Array

Parameters

- **atomic_numbers** (*Any*)
- **counts** (*Any*)

Return type
float | *Array*

Computes $\Sigma\ n_i\ (Z_i - (Z_i/82.5)^{2.37})$ over the elements of a formula unit, the high-energy limit of f1 used as the integration constant.

Parameters

- **atomic_numbers** (Any) – Atomic number Z of each element.
- **counts** (Any) – Number of atoms of each element per formula unit, same length. A zero count contributes exactly nothing, so padded element lists work.

Returns

Z* in electrons per formula unit: a `float` for sequence input, a float64 scalar array (traceable under `jit`) when either argument is a JAX array.

Return type

float or jax.Array

Raises

ValueError – If the two sequences differ in length.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (9).

Examples

```
>>> from xraylabtool.kk.transform import relativistic_correction
>>> round(relativistic_correction([14, 8], [1.0, 2.0]), 6)
29.977129
```

kk_piecewise_poly(*eval_ev*, *seg_ev*, *coeffs*, *rel_corr*)

Real part f1 of the scattering factor from a piecewise Laurent f2.

On segment j ($\mathrm{seg_ev}[j] \le x \le \mathrm{seg_ev}[j+1]$) the imaginary part is $f2(x) = a_1 x + a_0 + a_{-1}/x + a_{-2}/x^2 + a_{-3}/x^3$. Each segment's principal value integral has a closed form; this function sums them and adds Z*. Pure `jax.numpy` code with no data-dependent Python branching, so it can be wrapped in `jax.jit`; use *kk_f1()* for padded, chunked evaluation and *kk_real_spectrum()* for validated host arrays.

With $P_j(E) = a_1 E^2 + a_0 E + a_{-1} + a_{-2}/E + a_{-3}/E^2$ ($= E\ f2(E)$ on segment j) and $Q_j(E)$ the same with the signs of a_0 and a_{-2} flipped, segment j contributes, before division by πE:

$$
\begin{aligned}
&Q_j \ln\left|\frac{x_{j+1}+E}{x_j+E}\right| - P_j \ln\left|\frac{x_{j+1}-E}{x_j-E}\right| \\
&- 2a_1 E(x_{j+1}-x_j) + (2a_{-2}/E)\ln\left|x_{j+1}/x_j\right| \\
&+ (2a_{-3}/E)(1/x_j - 1/x_{j+1})
\end{aligned}
$$

The last term has the corrected sign; the typeset Eq. (10) of the reference prints it as $+(2a_{-3}/E)(x_{j+1}^{-1} - x_j^{-1})$, the opposite of its own Table 1. The arithmetic follows upstream `KK_PP` term by term so results agree with it to rounding.

Singular evaluation energies (exact float equality with a knot):

- Interior knot $E = x_j$ ($0 < j < n$): the two $\ln|x_j - E|$ terms of the adjacent segments are dropped and $P_j(x_j)\ \ln\left|\frac{x_{j+1}-x_j}{x_{j-1}-x_j}\right|$ is used instead, with P_j from the segment starting at x_j. Where f2 is continuous at x_j this is the principal value, continuous in E. Where f2 jumps, the ordinary principal value diverges logarithmically; the finite value returned is upstream's convention, not a limit.
- First or last knot: the boundary segment's $\ln|x - E|$ term is dropped with no compensation. This is upstream's convention as well; the integral diverges there when f2 is nonzero at the end of the data.

Every `log` sits between two `where` calls, so neither the value nor the gradient with respect to `eval_ev` is NaN at knots.

Parameters

- **eval_ev** (Any) – Photon energies in eV at which f1 is evaluated; must be positive.
- **seg_ev** (Any) – Strictly increasing, positive segment edges in eV.
- **coeffs** (Any) – Columns a_1, a_0, a_{-1}, a_{-2}, a_{-3} per segment (energies in eV).
- **rel_corr** (Any) – Relativistic correction Z* (see *relativistic_correction()*).

Returns
f1 in electrons per formula unit (float64).

Return type
Array

Notes

Raises no exceptions: inputs are not validated (that would need data-dependent branching), so non-positive energies or unsorted edges give non-finite or meaningless values. *kk_real_spectrum()* validates.

64-bit mode must be on before tracing (otherwise the float64 casts here truncate to float32); importing *xraylabtool.kk.transform* (or touching any lazy `xraylabtool.kk` name) enables it and *kk_real_spectrum()* re-enables it on every call.

The per-segment terms can exceed the result by many orders of magnitude (heavy elements at low E: terms near 8e12 summing to about 3e3 for Au at 13 eV), so the result carries an absolute rounding error of a few $\mathrm{eps} \cdot \max_j |\mathrm{term}_j| / (\pi E)$ (about 1e-4 there, below 1e-7 for light elements). Evaluations that differ only in operation order (NumPy vs XLA, another chunk size) agree to that level, not to `rtol=1e-8`.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eqs. (7) and (10); Eq. (10) with the a_{-3} term as $+(2a_{-3}/E)(x_j^{-1} - x_{j+1}^{-1})$, consistent with the paper's Table 1 and upstream `KK_PP`. The knot and endpoint conventions are upstream-only.

Examples

```
>>> import numpy as np
>>> from xraylabtool.kk.transform import kk_piecewise_poly
>>> f1 = kk_piecewise_poly(
...     np.array([1.0, 10.0]),
...     np.array([2.0, 5.0]),
...     np.array([[0.0, 1.0, 0.0, 0.0, 0.0]]),
...     0.0,
... )
>>> print(np.round(np.asarray(f1), 6))
[-0.661907  0.078578]
```

kk_f1(*eval_ev*, *spectrum*, *rel_corr*, ***, *block=None*)

Real part f1 of a padded *Spectrum*, on device.

Pure and `jit`/`vmap`/`grad`-compatible: evaluates *kk_piecewise_poly()* over blocks of energies with `jax.lax.map`, masking coefficients by `spectrum.n_segments` so the result is independent of the capacity the spectrum is padded to (up to the rounding scale documented in *kk_piecewise_poly()*). Gradients flow to the real coefficients and to the energies; padded coefficient rows get gradient 0.

No input validation is done (that would need data-dependent branching): energies must be positive and finite and the spectrum well formed, as built by `Spectrum.from_numpy()`. Inside a user `jit` the function specialises on `eval_ev.shape`; empty input is handled by a static branch. 64-bit mode must be on (see *kk_piecewise_poly()*).

Parameters

- **eval_ev** (Any) – Photon energies in eV.
- **spectrum** (*Spectrum*) – Padded f2 table.
- **rel_corr** (Any) – Relativistic correction Z* (see *relativistic_correction()*). A Python float is an implicit host-to-device transfer per call; pass a device scalar to keep a warmed call free of transfers.
- **block** (int | None) – Static upper bound on energies per block. The default follows the platform (see `energy_blocks()`): on a CPU cache-sized blocks of at most `2**18 // capacity` energies, on an accelerator one block of the fine-ladder length when it fits `2**22 // capacity`, else blocks of that size. Repeated calls share compiled programs, and blocks past the last real energy are skipped, so fewer than `m` plus one block of energies are evaluated.

Returns
f1 in electrons per formula unit.

Return type
Array

Raises
ValueError – If `block < 1`.

Notes

A padded spectrum is finite only while its pad knots `x_n·(1 + k)` do not overflow when squared (edges below about 1e152 eV; see *Spectrum*), far outside the physical range.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eqs. (7) and (10) (see *kk_piecewise_poly()*). Padding, masking and blocking are this package's additions (design spec §2-§4).

Examples

```
>>> import jax.numpy as jnp
>>> from xraylabtool.kk import Spectrum, kk_f1
>>> spec = Spectrum.from_numpy([2.0, 5.0], [[0.0, 1.0, 0.0, 0.0, 0.0]])
>>> print(jnp.round(kk_f1(jnp.array([1.0, 10.0]), spec, 0.0), 6))
[-0.661907  0.078578]
```

kk_real_spectrum(*eval_ev*, *seg_ev*, *coeffs*, *rel_corr*, *chunk_size=None*)

Validated, chunked evaluation of *kk_piecewise_poly()* on host arrays.

Pads the segment table to a shape bucket and the energies to a whole number of blocks (so compiled programs are reused across calls), uploads once, evaluates with *kk_f1()*'s block driver and downloads once. Each energy is computed independently, so chunking, order and duplicates change results only at the rounding level described in *kk_piecewise_poly()*. 64-bit mode is enabled globally first and inputs are cast to float64; an enclosing `jax.enable_x64(False)` block still wins and raises `TypeError` (ADR-006, known limitations).

Parameters

- **eval_ev** (ArrayLike) – Positive, finite photon energies in eV, in any order, repeats allowed.
- **seg_ev** (ArrayLike) – Strictly increasing, positive, finite segment edges in eV, n ≥ 1.
- **coeffs** (ArrayLike) – Finite coefficients a_1, a_0, a_{-1}, a_{-2}, a_{-3} per segment.
- **rel_corr** (float) – Relativistic correction Z* (see *relativistic_correction()*).
- **chunk_size** (int | None) – Upper bound on energies per block (rounded down to a power of two or bucket-ladder value). Default: platform dependent, at most about 2 MiB (CPU) or 32 MiB (accelerator) per float64 temporary.

Returns
f1 in electrons per formula unit, in the order of `eval_ev`.

Return type
`NDArray`[]

Raises

- `ValueError` – If an input is complex, has the wrong shape or is not finite, an energy or edge is not positive, the edges are not strictly increasing, or `chunk_size < 1`.
- `TypeError` – If `chunk_size` is a non-integer (a float such as `2.0` or `2.5`, or nan) that is not larger than the number of energies; a larger float is clamped to that number first.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eqs. (7) and (10) (see *kk_piecewise_poly()*). Validation and chunking are this package's additions; upstream evaluates all energies in one pass.

Examples

```
>>> import numpy as np
>>> from xraylabtool.kk.transform import kk_real_spectrum
>>> f1 = kk_real_spectrum(
...     [1.0, 10.0], [2.0, 5.0], [[0.0, 1.0, 0.0, 0.0, 0.0]], 0.0, chunk_size=1
... )
>>> print(np.round(f1, 6))
[-0.661907  0.078578]
```

Padded spectrum

class Spectrum(*seg_ev*, *coeffs*, *n_segments*)

Bases: `object`

Padded piecewise Laurent polynomial f2 table (a JAX pytree).

The capacity `N` is a shape bucket at least as large as the true number of segments `n`. Knots beyond the last real knot `x_n` are `x_n·(1 + k), k = 1, 2, …` (strictly increasing and finite) with all-zero coefficients, which contribute exactly nothing to the transform. `n_segments` is a traced int32 leaf, not static metadata, so spectra of different true sizes but equal capacity can be stacked and `vmap`-ed; unflattening performs no validation, so it is safe on tracers.

Parameters

- **seg_ev** (`Array`) – Segment edges in eV, padded as described above.
- **coeffs** (`Array`) – Columns a_1, a_0, a_{-1}, a_{-2}, a_{-3} per segment; rows ≥ `n_segments` are ignored by the kernels.
- **n_segments** (`Array`) – True segment count `n` (int32 scalar).

Notes

Pad knots grow as `x_n·(1 + k)` and the kernel squares them, so for absurd edges (about 1e152 eV and above, far outside the tabulated 10 eV-500 keV range) the finite pad knots overflow to `inf` and the result is NaN where the unpadded kernel is still finite. Use `capacity == n` (no padding) there.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (8) (the piecewise representation); the padding rule is this package's addition, see the design spec §2.

Examples

```
>>> import numpy as np
>>> from xraylabtool.kk import Spectrum
>>> spec = Spectrum.from_numpy([2.0, 5.0], [[0.0, 1.0, 0.0, 0.0, 0.0]], 4)
>>> spec.coeffs.shape, int(spec.n_segments)
((4, 5), 1)
```

seg_ev: Array

coeffs: Array

n_segments: Array

classmethod from_numpy(*seg_ev*, *coeffs*, *capacity=None*)

Validate a host table, pad it to `capacity` and upload it.

Parameters

- **seg_ev** (`ArrayLike`) – Strictly increasing, positive, finite edges in eV, `n ≥ 1`.
- **coeffs** (`ArrayLike`) – Finite coefficients per segment.
- **capacity** (`int | None`) – Segment capacity `N ≥ n`; default `bucket(n)`.

Returns

Device-resident, one counted `upload()`.

Return type

Spectrum

Raises

ValueError – If the table is malformed (see `validated_arrays()`) or `capacity < n`.

References

Design spec §2-§3 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.kk import Spectrum
>>> Spectrum.from_numpy([2.0, 5.0], [[0.0, 1.0, 0.0, 0.0, 0.0]]).seg_ev.shape
(513,)
```

to_numpy()

Download and slice to the true size.

Return type

`tuple[NDArray[], NDArray[]]`

Returns

- **seg_ev** (*numpy.ndarray, shape (n + 1,)*)
- **coeffs** (*numpy.ndarray, shape (n, 5)*)

Raises

CalculationError – If the true segment count exceeds the capacity (the table would be truncated).

References

Design spec §2 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.kk import Spectrum
>>> seg, c = Spectrum.from_numpy([2.0, 5.0], [[0.0, 1.0, 0.0, 0.0, 0.0]]).to_numpy()
>>> seg.tolist(), c.shape
([2.0, 5.0], (1, 5))
```

__init__(*seg_ev*, *coeffs*, *n_segments*)

Parameters

- **seg_ev** (*Array*)
- **coeffs** (*Array*)
- **n_segments** (*Array*)

Return type

None

Compound spectrum

Compound imaginary scattering-factor spectrum on a piecewise-polynomial grid.

Altered from the original: a vectorised re-implementation of `calculate_asf` and `coeffs_to_ASF` from kkcalc/data.py of the Kramers-Kronig Calculator (kkcalc, https://github.com/benajamin/kkcalc, commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), Copyright (c) 2013 Benjamin Watts, Daniel J. Lauk, zlib/libpng licence (full text in `xraylabtool/data/kkcalc/LICENSE-kkcalc.txt`; see also `NOTICE`). The method is described in B. Watts, Opt. Express 22(19), 23628-23639 (2014), doi:10.1364/OE.22.023628.

Energies are in eV throughout this module (the tabulated coefficients are in eV). The assembly (*`compound_spectrum_device()`*) and the evaluation (*`evaluate_f2_device()`*) are pure `jax.numpy` and stay on the device; *`compound_spectrum()`* and *`evaluate_f2()`* are the validated NumPy wrappers around them. The Kramers-Kronig kernel lives in *`xraylabtool.kk.transform`*.

compound_spectrum(*symbols*, *counts*)

Piecewise-polynomial f2 of a formula unit on the union of element grids.

The segment edges are the sorted union of the elemental edges; on each union segment the coefficients are the count-weighted sum of the polynomial that each element uses there ($\Sigma\, n_i\ f2_i$).

Parameters

- **symbols** (`Sequence[str]`) – Element symbols (case-sensitive, `"H"` .. `"U"`).
- **counts** (`Sequence[float]`) – Atoms of each element per formula unit; finite and positive.

Return type

`tuple[NDArray[], NDArray[]]`

Returns

- **seg_ev** (*numpy.ndarray, shape (n + 1,)*) – Strictly increasing segment edges in eV (10 eV to 500 keV).
- **coeffs** (*numpy.ndarray, shape (n, 5)*) – Columns a_1, a_0, a_{-1}, a_{-2}, a_{-3} of E^1 … E^{-3} (E in eV) for the segment `[seg_ev[i], seg_ev[i + 1])`.

Raises

FormulaError – If no elements are given, `symbols` and `counts` differ in length, a symbol is not a supported element (Z = 1..92), or a count is not finite and positive.

Notes

Elements are accumulated in the order given, so the sums are bit-identical to upstream's. All elements share the 10 eV - 500 keV range.

References

B. Watts, Opt. Express 22(19), 23628 (2014): the f2 spectrum is the stoichiometric sum of the tabulated piecewise polynomials entering Eq. (8). The union-grid summation is upstream-only (`kkcalc.data.calculate_asf`), not an equation of the paper.

Examples

```
>>> from xraylabtool.kk import compound_spectrum
>>> seg_ev, coeffs = compound_spectrum(["Si", "O"], [1.0, 2.0])
>>> seg_ev.shape[0] == coeffs.shape[0] + 1
True
>>> coeffs.shape[1]
5
>>> float(seg_ev[0]), float(seg_ev[-1])
(10.0, 500000.0)
```

compound_spectrum_device(*z*, *counts*, *n_elements*, *tables*, *, *capacity*)

Piecewise-polynomial f2 of a formula unit, assembled on the device.

Device counterpart of *compound_spectrum()*: the segment edges are the sorted union of the elemental edges, and each union segment carries the count-weighted sum of the polynomial every element uses there.

Parameters

- **z** (Array) – Atomic numbers (1..92) of the element list, padded to a bucketed `k`.
- **counts** (Array) – Atoms per formula unit (float64); padded entries are ignored.
- **n_elements** (Array) – Number of real entries in `z` (int scalar, at least 1).
- **tables** (*DeviceTables*) – Packed tables from *AsfDatabase.device_tables*.
- **capacity** (int) – Static segment capacity, at least the true segment count; a smaller value silently truncates the union. *AsfDatabase.compound_capacity* is a safe sum bound; the pipeline sizes it to the exact union instead (`api._compound_segments`), which is never larger.

Returns

Padded to `capacity` with the pad rule of `Spectrum`; `n_segments` is the true count.

Return type

Spectrum

Notes

The padding of every gathered knot row (and of padded elements) is masked to `+inf` before the union, so no dummy knot can be mistaken for a real one. Elements are accumulated in the given order, so the sums match the host reference to rounding and do not depend on `capacity`.

References

Design spec §5 (`2026-10-25-kk-jax-first-design.md`); the union-grid summation is upstream's (`kkcalc.data.calculate_asf`), not an equation of B. Watts, Opt. Express 22(19), 23628 (2014).

evaluate_f2(*energy_ev*, *seg_ev*, *coeffs*)

Evaluate the piecewise Laurent polynomial f2 at given energies.

`f2(E) = a₁E + a₀ + a₋₁/E + a₋₂/E² + a₋₃/E³` with the coefficients of the segment `[seg_ev[i], seg_ev[i + 1])` containing `E`. Segments are right-continuous, so an energy on an interior edge uses the segment that starts there; the final edge uses the last polynomial, as upstream does.

Parameters

- **energy_ev** (ArrayLike) – Photon energies in eV, within `[seg_ev[0], seg_ev[-1]]`.
- **seg_ev** (ArrayLike) – Strictly increasing segment edges in eV.
- **coeffs** (ArrayLike) – Coefficients `a₁, a₀, a₋₁, a₋₂, a₋₃` per segment.

Returns

f2 in electrons per formula unit.

Return type

NDArray[]

Raises

ValueError – If an input is complex or has the wrong shape, `energy_ev` is not finite or lies outside the segment range, or `seg_ev` is not finite and strictly increasing.

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (8): the piecewise Laurent form of f2. The right-continuous edge convention is upstream's (`kkcalc.data.coeffs_to_ASF`).

Examples

```
>>> from xraylabtool.kk import evaluate_f2
>>> seg = [1.0, 2.0, 4.0]
>>> c = [[0.0, 1.0, 0.0, 0.0, 0.0], [0.0, 0.0, 2.0, 0.0, 0.0]]
>>> evaluate_f2([1.5, 2.0, 4.0], seg, c).tolist()
[1.0, 1.0, 0.5]
```

evaluate_f2_device(*energy_ev*, *spectrum*)

Evaluate the piecewise Laurent polynomial f2 on the device.

Same convention as *evaluate_f2()*: right-continuous at interior edges, and the segment index is clamped to the last *real* segment `n_segments - 1`, so the final real edge and anything above it use the last real polynomial and the padding is never selected. No range check is made (the host wrapper validates).

Parameters

- **energy_ev** (Array) – Photon energies in eV (float64, any shape, including empty).
- **spectrum** (*Spectrum*) – Padded table, see *Spectrum*.

Returns

f2 in electrons per formula unit, same shape as `energy_ev`.

Return type

Array

References

B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (8); design spec §5 (`2026-10-25-kk-jax-first-design.md`).

Near-edge data

Near-edge absorption data: loading, conversion and merging.

Altered from the original: a re-implementation of `convert_data`, `convert_NEXAFS_to_ASF`, `convert_Beta_to_ASF` and `merge_spectra` from kkcalc/data.py of the Kramers-Kronig Calculator (kkcalc, https://github.com/benajamin/kkcalc, commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), Copyright (c) 2013 Benjamin Watts, Daniel J. Lauk, zlib/libpng licence (full text in `xraylabtool/data/kkcalc/LICENSE-kkcalc.txt`; see also `NOTICE`). The method is described in B. Watts, Opt. Express 22(19), 23628-23639 (2014), doi:10.1364/OE.22.023628.

The conversion constants come from *xraylabtool.constants*, the "add background" and "fix distortions" options of upstream are not ported, and every input is validated (a malformed spectrum, merge points outside the data, flat end points or non-finite values raise `NearEdgeDataError` instead of producing `inf` or silently wrong coefficients).

Energies are in eV throughout this module (the tabulated coefficients are in eV). Loading, validation and merge-point resolution are host-side NumPy; the numerical part (*to_asf_scale_device()*, *merge_device()*) is pure `jax.numpy` and stays on the device, with *raise_or_log()* turning its status pytree into the data errors and the D6 warning on the host.

value_exponent(*values*)

Power-of-two exponent that brings tiny near-edge `values` to order one.

`0` (no rescaling, bit-identical results) unless the largest finite magnitude is below `2**-500`. Multiplying by `2**exponent` is exact, and every later step (conversion, the affine normalisation of the merge) is homogeneous in the values, so the merged spectrum does not depend on it.

Return type
`int`

Parameters
values (*NDArray[float64]*)

class NearEdgeSpectrum(*energy_ev*, *values*)

Bases: `object`

A measured absorption spectrum over a limited energy range.

Construction validates the data exactly as *load_nearedge()* does, so a spectrum built directly from arrays cannot bypass the checks. The arrays are stored as read-only float64 copies.

Parameters

- **energy_ev** (NDArray[]) – Photon energies in eV, finite, positive and strictly increasing, `n >= 2`.
- **values** (NDArray[]) – Absorption values in the units of the input type (see *to_asf_scale()*), finite.

Raises
NearEdgeDataError – If the arrays are not real, 1-D and of equal length, hold fewer than two points, contain NaN or inf, or the energies are not positive and strictly increasing.

References

Package addition: a validated container for the near-edge data that upstream kkcalc keeps as a bare two-column array; the merge it feeds is described in B. Watts, Opt. Express 22(19), 23628 (2014).

Examples

```
>>> from xraylabtool.kk import NearEdgeSpectrum
>>> spec = NearEdgeSpectrum([280.0, 290.0, 300.0], [0.1, 0.4, 0.3])
>>> spec.energy_ev.tolist()
[280.0, 290.0, 300.0]
```

(continues on next page)

(continued from previous page)

```
>>> NearEdgeSpectrum([2.0, 1.0], [0.0, 0.0])
Traceback (most recent call last):
    ...
xraylabtool.exceptions.NearEdgeDataError: Near-edge energies must be strictly increasing, but␣
↪energy[0]=2 >= energy[1]=1
```

energy_ev: NDArray[float64]

values: NDArray[float64]

__init__(*energy_ev*, *values*)

Parameters

- **energy_ev** (*NDArray[float64]*)
- **values** (*NDArray[float64]*)

Return type

None

load_nearedge(*path*, *, *energy_unit='eV'*)

Read a near-edge spectrum from a whitespace-separated text file.

The first column is the photon energy and the last column the absorption value; columns in between are ignored. Non-numeric lines before the first data row (headers) and lines starting with `#`, `%` or `;` are skipped; any other non-numeric line after the data has started is an error, never silently dropped.

Parameters

- **path** (`str` | `Path`) – Text file with at least two numeric columns.
- **energy_unit** (`Literal['eV', 'keV']`) – Unit of the first column. Default `"eV"`, the beamline convention; keV values are converted to eV once here.

Returns

Energies in eV, shape `(n,)`, and the raw absorption values.

Return type

NearEdgeSpectrum

Raises

- **DataFileError** – If the file cannot be read as text.
- **ValidationError** – If `energy_unit` is not `"eV"` or `"keV"`.
- **NearEdgeDataError** – If the file holds no numeric rows, rows with differing column counts or fewer than two columns, a stray non-numeric line inside the data, or data that fail the *NearEdgeSpectrum* checks (fewer than two points, NaN/inf, non-positive or non-increasing energies).

References

Package addition. Upstream `kkcalc.data.load_data` reads the same two columns but silently skips every unparsable line and applies no checks.

Examples

```
>>> from pathlib import Path
>>> from xraylabtool.kk import load_nearedge
>>> import tempfile, os
>>> with tempfile.TemporaryDirectory() as d:
...     name = os.path.join(d, "edge.txt")
...     _ = Path(name).write_text("% header\n280.0 0.1\n290.0 0.4\n")
```

(continues on next page)

(continued from previous page)

```
...     spec = load_nearedge(name)
>>> spec.energy_ev.tolist(), spec.values.tolist()
([280.0, 290.0], [0.1, 0.4])
```

to_asf_scale(*spec*, *input_type*, ***, *density=None*, *formula_mass=None*)

Convert near-edge absorption data to the scattering-factor f2 scale.

Parameters

- **spec** (*NearEdgeSpectrum*) – Energies in eV and absorption values in the units of `input_type`.
- **input_type** (`Literal['nexafs', 'photoabsorption', 'xanes', 'beta', 'asf']`) – `"nexafs"`, `"photoabsorption"` and `"xanes"` are the same linear absorption coefficient (mass or cross-section scale), which is multiplied by `E / (2 r_e hc)`. `"beta"` is the imaginary part of the refractive index, converted with the number density `density * N_A / formula_mass`. `"asf"` is already f2 and is returned unchanged.
- **density** (`float | None`) – Mass density in g/cm^3, `"beta"` only. Default 1 (logged at INFO).
- **formula_mass** (`float | None`) – Formula mass in g/mol, `"beta"` only. Default 100 (logged at INFO).

Returns

Same energies (eV), values on the f2 scale (up to the scale factor that the affine normalisation of *merge_spectra()* removes).

Return type

NearEdgeSpectrum

Raises

- **ValidationError** – If `input_type` is unknown, or `density`/`formula_mass` is not positive and finite.
- **NearEdgeDataError** – If the converted values overflow to a non-finite number.

Notes

merge_spectra() rescales the data to the tabulated f2 at the merge points, so a constant factor (the absolute scale of `nexafs` data, the density and formula mass of `beta` data) drops out of the merged spectrum; only the shape of the measured curve matters. Data whose largest magnitude is below `2**-500` are scaled by an exact power of two for the device calculation (XLA flushes subnormals) and the result is scaled back, so the outcome does not depend on the absolute scale.

References

Upstream-only conversions (`convert_NEXAFS_to_ASF` and `convert_Beta_to_ASF` of kkcalc), using the package constants; the paper (Opt. Express 22(19), 23628) describes the normalisation to the tabulated scattering factors but not these unit factors.

Examples

```
>>> from xraylabtool.kk import NearEdgeSpectrum, to_asf_scale
>>> spec = NearEdgeSpectrum([280.0, 290.0], [0.1, 0.2])
>>> to_asf_scale(spec, "asf").values.tolist()
[0.1, 0.2]
>>> out = to_asf_scale(spec, "nexafs")
>>> bool(out.values[1] / out.values[0] > 2.0)  # E * mu: ratio 2 * 290/280
True
```

to_asf_scale_device(
energy_ev,
values,
input_type,
***,
density=None,
formula_mass=None,
)

Convert near-edge absorption values to the f2 scale on the device.

Device counterpart of *to_asf_scale()*: the same formulas, element by element, so padded entries do not affect real ones. The host wrapper owns the argument validation and the INFO log of the `beta` defaults; invalid arithmetic (overflow) yields `inf` and is reported by the host wrapper.

Parameters

- **energy_ev** (`Array`) – Photon energies in eV (float64), any 1-D length.
- **values** (`Array`) – Absorption values in the units of `input_type`, same shape.
- **input_type** (`str`) – Static (a different value compiles a separate program); see *to_asf_scale()*.
- **density** (`float` | `Array` | `None`) – Mass density in g/cm^3, `"beta"` only. Default 1. Pass a device scalar inside a transfer-guarded region (a Python float is an implicit transfer).
- **formula_mass** (`float` | `Array` | `None`) – Formula mass in g/mol, `"beta"` only. Default 100; same note.

Returns
Values on the f2 scale, same shape as `values`.

Return type
`Array`

Raises
ValueError – If `input_type` is not one of the supported kinds (at trace time).

References

Upstream `kkcalc.data.convert_NEXAFS_to_ASF` and `convert_Beta_to_ASF` with the package constants; design spec §6 (`2026-10-25-kk-jax-first-design.md`).

class MergeStatus(
n_segments,
v_low,
v_high,
a_low,
a_high,
measured_step,
tabulated_step,
nonfinite,
n_negative,
f2_min,
n_inside,
)

Bases: `NamedTuple`

Scalars the host needs to raise or log after a device merge.

All fields are device scalars (NumPy scalars after `fetch_status()`); padded near-edge entries take no part in any of them.

Parameters

- **n_segments** (*Array*)

- **v_low** (*Array*)
- **v_high** (*Array*)
- **a_low** (*Array*)
- **a_high** (*Array*)
- **measured_step** (*Array*)
- **tabulated_step** (*Array*)
- **nonfinite** (*Array*)
- **n_negative** (*Array*)
- **f2_min** (*Array*)
- **n_inside** (*Array*)

n_segments
: True segment count of the merged spectrum.

  Type
  int32

v_low, v_high
: Measured near-edge values at the two merge points.

  Type
  float

a_low, a_high
: Tabulated f2 at the two merge points.

  Type
  float

measured_step, tabulated_step
: `v_high - v_low` and `a_high - a_low`.

  Type
  float

nonfinite
: Any normalised near-edge value is not finite.

  Type
  bool

n_negative
: Normalised values below zero strictly inside the merge window.

  Type
  int32

f2_min
: Their minimum over the window (`inf` when the window holds none).

  Type
  float

n_inside
: Near-edge points strictly inside the merge window.

  Type
  int32

References

Design spec §6 (`2026-10-25-kk-jax-first-design.md`).

v_low: Array
Alias for field number 1

v_high: Array
Alias for field number 2

a_low: Array
Alias for field number 3

a_high: Array
Alias for field number 4

measured_step: Array
Alias for field number 5

tabulated_step: Array
Alias for field number 6

class MergeResult(*spectrum*, *scaled_f2*, *used_points_ev*, *status*)

Bases: `NamedTuple`

Output of *merge_device()*.

Parameters

- **spectrum** (`Spectrum`)
- **scaled_f2** (*Array*)
- **used_points_ev** (*Array*)
- **status** (`MergeStatus`)

spectrum
Merged table at the requested capacity.

Type
Spectrum

scaled_f2
The full normalised near-edge curve, including the points outside the replaced window; entries past the true count are `0`.

Type
jax.Array, shape (m,)

used_points_ev
The `(low, high)` merge points the splice used.

Type
jax.Array, shape (2,)

status
Everything *raise_or_log()* needs.

Type
MergeStatus

merge_device(
spec,
ne_energy_ev,
ne_f2,
n_ne,
low_ev,
high_ev,
*,

capacity,
)

Normalise near-edge data to the tabulated f2 and splice it in, on the device.

Device counterpart of *merge_spectra()* (the merge points are already resolved and snapped by the host). Between `low_ev` and `high_ev` the tabulated segments are replaced by the piecewise-linear curve through the normalised near-edge points; outside they are copied unchanged. No data check is raised here: the flatness and finiteness conditions are returned in *MergeStatus* for *raise_or_log()*, and computed without trapping on invalid arithmetic.

Parameters

- **spec** (*Spectrum*) – Tabulated table (`n` true segments).
- **ne_energy_ev** (`Array`) – Near-edge energies in eV, strictly increasing over the first `n_ne` entries; the rest is padding and is masked.
- **ne_f2** (`Array`) – Near-edge values on the f2 scale (*to_asf_scale_device()*).
- **n_ne** (`Array`) – Number of real near-edge points (int scalar, at least 2).
- **low_ev** (`float` | `Array`) – Merge points, `low_ev < high_ev`. Inside a `jax.transfer_guard("disallow_explicit")` region they must already be device scalars: converting a Python float is an implicit transfer.
- **high_ev** (`float` | `Array`) – Merge points, `low_ev < high_ev`. Inside a `jax.transfer_guard("disallow_explicit")` region they must already be device scalars: converting a Python float is an implicit transfer.
- **capacity** (`int`) – Static output segment capacity; use `bucket(n_segments_upper_bound + n_near_edge + 2)`, a smaller value silently truncates the knot union.

Returns

Merged *Spectrum* at `capacity`, the full normalised near-edge curve, the merge points used and the status.

Return type

MergeResult

Notes

Candidate knots are the real tabulated knots outside `[low, high]`, `low`, the near-edge energies strictly inside the window and `high`; every excluded or padded entry is sent to `+inf` before the union, so the result does not depend on `capacity` or on the padding. The products that feed sums are pinned with `jax.lax.optimization_barrier` so the rounding is that of the NumPy reference rather than an FMA's.

References

B. Watts, Opt. Express 22(19), 23628 (2014): normalisation of the near-edge data to the tabulated scattering factors; design spec §6 (`2026-10-25-kk-jax-first-design.md`).

raise_or_log(*status*, *low*, *high*, *exponent=0*)

Raise or log what a device merge reported (host side).

Parameters

- **status** (*MergeStatus*) – The fetched status (NumPy scalars, see `fetch_status()`).
- **low** (`float`) – The merge points in eV, for the messages.
- **high** (`float`) – The merge points in eV, for the messages.
- **exponent** (`int`) – The *value_exponent()* the values were rescaled by before the merge; the reported end-point values are shown unscaled.

Raises

NearEdgeDataError – If the near-edge values at the merge points are (nearly) equal, the tabulated f2 is equal there, or a normalised value is not finite.

Return type
None

Notes

Normalised values below zero inside the window are kept and logged at WARNING (count, window size and minimum), never clipped.

References

Design spec §1 and §6 (`2026-10-25-kk-jax-first-design.md`).

merge_spectra(*spec*, *seg_ev*, *coeffs*, *, *merge_points_ev*)

Normalise near-edge data to the tabulated f2 and splice it in.

The near-edge values are rescaled and shifted (affine endpoint matching) so that they equal the tabulated f2 at the two merge points; between them the tabulated segments are replaced by the piecewise-linear near-edge curve. Outside the merge points the tabulated segments are unchanged.

Parameters

- **spec** (*NearEdgeSpectrum*) – Near-edge data on the f2 scale (see *to_asf_scale()*), energies in eV.
- **seg_ev** (NDArray[]) – Segment edges of the tabulated spectrum in eV, strictly increasing.
- **coeffs** (NDArray[]) – Polynomial coefficients (E^1, E^0, E^{-1}, E^{-2}, E^{-3}) of each segment.
- **merge_points_ev** (tuple[float, float] | None) – `(low, high)` in eV with `low < high`, both inside the near-edge data and the tabulated range. `None` merges over the whole overlap of the two ranges.

Return type
tuple[NDArray[], NDArray[]]

Returns

- **seg_ev** (*numpy.ndarray, shape (m+1,)*) – Segment edges of the merged spectrum in eV, strictly increasing; the merge points and the near-edge energies between them are edges.
- **coeffs** (*numpy.ndarray, shape (m, 5)*) – Coefficients of the merged spectrum, ready for *kk_real_spectrum()*.

Raises

- **NearEdgeDataError** – If the near-edge data do not overlap the tabulated range (the message states both ranges), the merge points are not two finite increasing energies inside both ranges, the near-edge values at the merge points are (nearly) equal so the matching would divide by ~0, the tabulated f2 is equal at the merge points, or the normalised values are not finite.
- **ValidationError** – If `seg_ev` and `coeffs` do not have shapes `(n+1,)` and `(n, 5)`, or are complex.
- **ValueError** – If `seg_ev` is not finite or not strictly increasing.

Notes

Normalised values below zero are kept and logged at WARNING (count and minimum), never clipped: a noisy pre-edge can legitimately dip below the tabulated level. At a tabulated edge that coincides with a merge point the segment that starts there is used, as in upstream.

References

Upstream `kkcalc.data.merge_spectra` without the "add background" and "fix distortions" options; the validation and the conditioning guard are package additions. The normalisation to the tabulated scattering factors is described in B. Watts, Opt. Express 22(19), 23628 (2014).

Examples

```
>>> from xraylabtool.kk import NearEdgeSpectrum, merge_spectra
>>> from xraylabtool.kk.spectrum import compound_spectrum
>>> seg, coeffs = compound_spectrum(["C"], [1.0])
>>> edge = NearEdgeSpectrum([280.0, 290.0, 300.0, 310.0], [0.1, 0.2, 0.4, 0.5])
>>> merged, merged_coeffs = merge_spectra(
...     edge, seg, coeffs, merge_points_ev=(280.0, 310.0)
... )
>>> bool(merged.size == merged_coeffs.shape[0] + 1)
True
>>> bool({290.0, 300.0} <= set(merged.tolist()))
True
```

Refinement

Adaptive refinement of a sampled scattering-factor spectrum.

Altered from the original: a re-implementation of `improve_accuracy` from kkcalc/kk.py of the Kramers-Kronig Calculator (kkcalc, https://github.com/benajamin/kkcalc, commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), Copyright (c) 2013 Benjamin Watts, Daniel J. Lauk, zlib/libpng licence (full text in `xraylabtool/data/kkcalc/LICENSE-kkcalc.txt`; see also `NOTICE`).

Differences from upstream: the grid need not coincide with the segment edges of the tabulated spectrum (f2 is evaluated at each midpoint from the segment that contains it), an interval next to a tabulated range end is left alone where f2 exceeds the tolerance there (upstream skips its first interval unconditionally, see *refine_spectrum()*), the pass and point budgets are explicit and reported, and an interval that float spacing cannot halve ends the loop instead of repeating.

Energies are in eV and the tolerance is absolute, in electrons per formula unit. The passes are host-orchestrated and device-resident: the number of new points is data dependent, so the host runs one jitted pass per bisection round on bucketed device buffers and reads back a handful of integer counters per pass (the continue/stop/rebucket decision and the per-pass log line); the sorted union is downloaded once at the end (design spec §7).

class RefineOutcome(*energy_ev*, *f1*, *f2*, *n_added*, *exhausted*, *n_pole_skipped=0*)

Bases: `object`

Result of *refine_spectrum()*.

The dataclass is frozen and compares by identity (`eq=False`): the array fields have no element-wise-safe `==` or `hash`.

Parameters

- **energy_ev** (*NDArray[float64]*)
- **f1** (*NDArray[float64]*)
- **f2** (*NDArray[float64]*)
- **n_added** (*int*)
- **exhausted** (*bool*)
- **n_pole_skipped** (*int*)

energy_ev

Strictly increasing energies in eV: the input grid plus the added midpoints.

Type
numpy.ndarray, shape (m + n_added,)

f1

Real part at `energy_ev`, electrons per formula unit.

Type
numpy.ndarray, shape (m + n_added,)

f2

Imaginary part at `energy_ev`, electrons per formula unit.

Type
numpy.ndarray, shape (m + n_added,)

n_added

Number of midpoints added.

Type
int

exhausted

`True` if refinement stopped on a limit (`max_passes`, `max_points` or float spacing) before every interval met the tolerance.

Type
bool

n_pole_skipped

Number of intervals next to a tabulated range end, narrower than 2 % of their energy, that were left unrefined because f1 is logarithmically singular there (see *refine_spectrum()*).

Type
int

References

Upstream-only: `improve_accuracy` of kkcalc is not part of B. Watts, Opt. Express 22(19), 23628 (2014).

Examples

```
>>> import numpy as np
>>> from xraylabtool.kk import RefineOutcome
>>> out = RefineOutcome(np.array([1.0]), np.array([2.0]), np.array([3.0]), 0, False)
>>> out.n_added, out.exhausted, out.n_pole_skipped
(0, False, 0)
```

__init__(
energy_ev,
f1,
f2,
n_added,
exhausted,
n_pole_skipped=0,
)

Parameters

- **energy_ev** (*NDArray[float64]*)
- **f1** (*NDArray[float64]*)
- **f2** (*NDArray[float64]*)
- **n_added** (*int*)

- **exhausted** (*bool*)
- **n_pole_skipped** (*int*)

Return type
None

refine_spectrum(
energy_ev,
f1,
f2,
seg_ev,
coeffs,
rel_corr,
*,
tolerance,
max_passes=50,
max_points=100_000,
chunk_size=None,
)

Insert midpoints where linear interpolation of f1 or f2 is inaccurate.

Each interval between neighbouring energies is bisected. The midpoint's f1 (Kramers-Kronig transform) and f2 (tabulated polynomial) are computed and compared with the average of the interval's end values; if either differs by more than `tolerance` the midpoint is kept and both halves are checked in the next pass. Intervals that pass are final. Only midpoints are tested, so a feature narrower than an interval can still be missed. Input points are never dropped or changed.

Parameters

- **energy_ev** (`ArrayLike`) – Strictly increasing, unique, positive energies in eV inside `[seg_ev[0], seg_ev[-1]]`.
- **f1** (`ArrayLike`) – Real part at `energy_ev`, electrons per formula unit.
- **f2** (`ArrayLike`) – Imaginary part at `energy_ev`, electrons per formula unit.
- **seg_ev** (`ArrayLike`) – Segment edges of the spectrum in eV, as for *kk_real_spectrum()*.
- **coeffs** (`ArrayLike`) – Polynomial coefficients per segment.
- **rel_corr** (`float`) – Relativistic correction Z* (see *relativistic_correction()*).
- **tolerance** (`float`) – Largest accepted interpolation error, absolute, in electrons per formula unit; finite and positive.
- **max_passes** (`int`) – Maximum number of bisection passes (an interval can be halved at most this many times); at least 1.
- **max_points** (`int`) – Maximum number of points to add; at least 1. When a pass would exceed it, only the first `max_points - added` failing intervals of that pass, in internal order (not necessarily the lowest energies), get their midpoints and refinement stops.
- **chunk_size** (`int | None`) – Upper bound on the energies evaluated per block of the midpoint kernel (rounded down to a power of two or bucket-ladder value, never up); `None` leaves only the memory bound of `block_for()`. At least 1.

Returns
The sorted union of the input energies and the added midpoints with f1 and f2, the number of points added and whether a limit stopped refinement. Points added per pass are logged at DEBUG, the total at INFO and an exhausted limit at WARNING (logger `xraylabtool.kk`).

Return type
RefineOutcome

Raises

- **ValidationError** – If an array is empty, not 1-D, of unequal length or not finite, `energy_ev` is not strictly increasing, positive and inside the tabulated range, `tolerance` is not finite and positive, or a limit is not a positive integer.
- **ValueError** – If `seg_ev`, `coeffs` or `rel_corr` are invalid, or `chunk_size` is not a positive integer.

Notes

An interval whose midpoint equals one of its ends in floating point cannot be split further; it is left as is and reported as exhausted.

The inputs are uploaded once, the bisection passes run on the device (host-orchestrated, one small status read per pass) and the sorted union is downloaded once. Midpoint f1 may differ from a separate *kk_real_spectrum()* call at the rounding level documented there; a midpoint whose error sits within that rounding of `tolerance` can therefore be accepted or rejected differently.

f1 diverges logarithmically at a tabulated range end `seg_ev[0]` or `seg_ev[-1]` where f2 is nonzero, so bisecting towards it does not converge once `|f2|` there exceeds `tolerance`. For such an end the interval touching it is bisected as usual (a failing midpoint is kept), but a piece next to the end that is narrower than 2 % of its energy is left unrefined. Pieces dropped this way are counted in `n_pole_skipped` and logged: at INFO when each is no wider than the tabulated segment at that end (the expected case), at WARNING otherwise. Upstream instead never splits the first interval of its grid, which is the same as this for the lower end of every element (the first tabulated segment is 1.6 % wide) and leaves the upper end to its recursion limit. On the segment grid at tolerance 0.05 this gives the identical point set for Au (f2 = 0.17 at 500 keV; the chain converges), and for Pb (f2 = 0.20) and U (0.36) 44 fewer points (Pb: 461 against 505): upstream's chain towards 500 keV does not converge and adds points until its recursion limit, here it stops at the 2 % width.

References

Upstream-only: this is kkcalc's `improve_accuracy` (not in B. Watts, Opt. Express 22(19), 23628 (2014)).

Examples

```
>>> import numpy as np
>>> from xraylabtool.kk import compound_spectrum, evaluate_f2, refine_spectrum
>>> from xraylabtool.kk import kk_real_spectrum, relativistic_correction
>>> seg, coeffs = compound_spectrum(["Si"], [1.0])
>>> rel = relativistic_correction([14], [1.0])
>>> e = np.array([100.0, 1000.0, 10000.0])
>>> out = refine_spectrum(
...     e,
...     kk_real_spectrum(e, seg, coeffs, rel),
...     evaluate_f2(e, seg, coeffs),
...     seg,
...     coeffs,
...     rel,
...     tolerance=0.1,
... )
>>> bool(out.n_added > 0 and out.energy_ev.size == 3 + out.n_added)
True
```

Database

Henke atomic scattering factor (ASF) database vendored from kkcalc.

The tables are piecewise-polynomial fits of the imaginary scattering factor `f2(E) = a1*E + a0 + a-1/E + a-2/E**2 + a-3/E**3` per element (`E` in eV) over 10 eV - 500 keV, taken unaltered from upstream

kkcalc `ASF.json` and repacked into `data/kkcalc/asf.npz` by `scripts/build_kk_data.py`.

References

B. Watts, Opt. Express 22(19), 23628-23639 (2014), doi:10.1364/OE.22.023628 (upstream-only: data loading has no counterpart equation).

class DeviceTables(*knots*, *coeffs*, *n_knots*)

Bases: `object`

Packed per-element ASF tables resident on the device (a JAX pytree).

Rows are indexed by `Z - 1`. Knots beyond an element's last real knot `x` are `x·(1 + k)`, `k = 1, 2, …` (finite and strictly increasing, so a `searchsorted` over a whole row is well defined) and the matching coefficient rows are exactly zero.

Parameters

- **knots** (*Array*)
- **coeffs** (*Array*)
- **n_knots** (*Array*)

knots

Segment edges in eV, padded as described above.

Type
jax.Array, shape (92, Kmax)

coeffs

a_1, a_0, a_{-1}, a_{-2}, a_{-3} per segment, zero past the true length.

Type
jax.Array, shape (92, Kmax - 1, 5)

n_knots

True knot count of each element (int32).

Type
jax.Array, shape (92,)

References

Design spec §1 and §5 (`2026-10-25-kk-jax-first-design.md`); the tabulated data are upstream kkcalc's, B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (8).

__init__(*knots*, *coeffs*, *n_knots*)

Parameters

- **knots** (*Array*)
- **coeffs** (*Array*)
- **n_knots** (*Array*)

Return type
None

class AsfDatabase(
symbols,
mass,
_energy,
_energy_offsets,
_coeffs,
_coeff_offsets,
)

Bases: `object`

Per-element piecewise-polynomial `f2` tables (energies in eV).

Parameters

- **symbols** (*tuple[str, ...]*)
- **mass** (*NDArray[float64]*)
- **_energy** (*NDArray[float64]*)
- **_energy_offsets** (*NDArray[int64]*)
- **_coeffs** (*NDArray[float64]*)
- **_coeff_offsets** (*NDArray[int64]*)

symbols

Element symbols for Z = 1..92.

Type
tuple of str

mass

Atomic masses (g/mol) for Z = 1..92.

Type
ndarray, shape (92,)

References

Upstream-only: data handling of kkcalc, not an equation of B. Watts, Opt. Express 22(19), 23628 (2014); the table form is its Eq. (8).

Examples

```
>>> from xraylabtool.kk import load_asf
>>> db = load_asf()
>>> db.symbols[0], db.mass.shape
('H', (92,))
```

energy_ev(*z*)

Segment knot energies of element `z` (1-based).

Parameters
z (`int`) – Atomic number, 1..92.

Returns
Knot energies in eV, strictly increasing from 10 eV to 500 keV.

Return type
`NDArray`[]

Raises
`ValueError` – If `z` is not an integer in 1..92.

References

Upstream-only (kkcalc `ASF.json` layout); the knots are the `x_j` of B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (10).

Examples

```
>>> from xraylabtool.kk import load_asf
>>> knots = load_asf().energy_ev(14)
>>> float(knots[0]), float(knots[-1])
(10.0, 500000.0)
```

im_coeffs(*z*)

Polynomial coefficients of `f2` for element `z` (1-based).

Parameters

z (`int`) – Atomic number, 1..92.

Returns

Columns `a1, a0, a-1, a-2, a-3` of `E**1 ... E**-3` for the segment `[energy_ev(z)[i], energy_ev(z)[i + 1])`.

Return type

`NDArray`[]

Raises

ValueError – If `z` is not an integer in 1..92.

References

Upstream-only (kkcalc `ASF.json` layout); the columns are the coefficients of B. Watts, Opt. Express 22(19), 23628 (2014), Eq. (8).

Examples

```
>>> from xraylabtool.kk import load_asf
>>> db = load_asf()
>>> db.im_coeffs(14).shape == (db.energy_ev(14).size - 1, 5)
True
```

compound_capacity(*z*)

Conservative segment capacity for a compound of the elements `z`.

The union of the elemental knot sets cannot hold more than $\Sigma\ (n_i - 1)$ segments, so this bucketed sum is an upper bound on the true segment count and overflow is excluded by construction.

Parameters

z (`Sequence`[`int`]) – Atomic numbers (1..92); repeats are allowed.

Returns

bucket(Σ (n_knots - 1)).

Return type

`int`

Raises

ValueError – If an atomic number is outside 1..92.

References

Design spec §3 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.kk import load_asf
>>> load_asf().compound_capacity([14, 8]) >= 512
True
```

device_tables()

Packed ASF tables on the default device, built once and cached.

The host packing is I/O preparation; the arrays are uploaded in one `upload()`. Construction is single-flight under a lock and the cache is keyed on the default device.

Returns
Knots, coefficients and true knot counts of Z = 1..92.

Return type
DeviceTables

References

Design spec §1 and §5 (`2026-10-25-kk-jax-first-design.md`).

Examples

```
>>> from xraylabtool.kk import load_asf
>>> load_asf().device_tables().knots.shape[0]
92
```

__init__(
symbols,
mass,
_energy,
_energy_offsets,
_coeffs,
_coeff_offsets,
)

Parameters

- **symbols** (*tuple[str, ...]*)
- **mass** (*NDArray[float64]*)
- **_energy** (*NDArray[float64]*)
- **_energy_offsets** (*NDArray[int64]*)
- **_coeffs** (*NDArray[float64]*)
- **_coeff_offsets** (*NDArray[int64]*)

Return type
None

load_asf()

Load the vendored ASF database (cached after the first call).

Single-flight: concurrent first callers share one database object (the device-table cache is keyed on its identity, so two objects would upload the tables twice).

Returns
Tables for Z = 1..92.

Return type
AsfDatabase

References

Upstream-only: kkcalc's data loading, not an equation of B. Watts, Opt. Express 22(19), 23628 (2014).

Examples

```
>>> from xraylabtool.kk import load_asf
>>> db = load_asf()
>>> db.symbols[13]
'Si'
>>> float(db.energy_ev(14)[0])
10.0
```

Optical properties from f1 and f2

calculate_properties_from_factors(
formula,
energy_kev,
density,
f1_total,
f2_total,
)

Build an `XRayResult` from supplied scattering factors.

Computes δ, β and the derived quantities from per-formula-unit `f1` and `f2` that the caller provides instead of reading the tabulated `.nff` files. It runs the same code as `calculate_single_material_properties()` after its table lookup. This function is a package addition, not part of the cited databases or papers.

Parameters

- **formula** (`str`) – Chemical formula of one formula unit, e.g. `"SiO2"`.
- **energy_kev** (`NDArray`[]) – Photon energies in keV. Finite; the tabulated 0.03-30 keV range is not enforced here.
- **density** (`float`) – Mass density in g/cm^3; finite and positive.
- **f1_total** (`NDArray`[]) – Real scattering factor, electrons per formula unit. Negative values (anomalous dispersion near an edge) are kept: δ becomes negative and the critical angle is 0 there.
- **f2_total** (`NDArray`[]) – Imaginary scattering factor, electrons per formula unit. Negative values are kept; the attenuation length clamps β at `1e-30`.

Returns

All fields are arrays of shape `(m,)` except the scalar material properties; the order of `energy_kev` is preserved.

Return type

`XRayResult`

Raises

- **ValidationError** – If the three arrays are not non-empty 1-D arrays of equal length, any value is non-finite, or `density` is not finite and positive.
- **FormulaError** – If `formula` cannot be parsed or names an unknown element.

References

Package addition. The optical constants follow the CXRO/NIST convention used by `calculate_single_material_properties()`.

Examples

```
>>> import numpy as np
>>> from xraylabtool.calculators.core import calculate_properties_from_factors
>>> res = calculate_properties_from_factors(
```

(continues on next page)

(continued from previous page)

```
...     "Si", np.array([8.0]), 2.33, np.array([14.2]), np.array([0.5])
... )
>>> res.scattering_factor_f1.shape
(1,)
>>> bool(res.dispersion_delta[0] > 0)
True
```

References

- B. Watts, "Calculation of the Kramers-Kronig transform of X-ray spectra by a piecewise Laurent polynomial method", *Opt. Express* **22**(19), 23628–23639 (2014), doi:10.1364/OE.22.023628.
- kkcalc (commit f331ca31a654af946f7d550a22cf4e9c11fa8afc), zlib/libpng licence, https://github.com/benajamin/kkcalc; see *Kramers-Kronig Guide* for the attribution.

3.10.12 High-Level Interface

The primary entry points for most users:

Main Calculation Functions

calculate_single_material_properties(
formula,
energy_keV=None,
density=None,
*,
energy=None,
)

Calculate X-ray optical properties for a single material composition.

This is a pure function that calculates comprehensive X-ray optical properties for a single chemical formula at given energies and density. It returns an XRayResult dataclass containing all computed properties.

Parameters

- **formula** (str) – Chemical formula string (e.g., "SiO2", "Al2O3", "CaCO3")
- **energy_keV** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any] | None) – X-ray energies in keV. Valid range: 0.03-30.0 keV
- **density** (float | floating[Any] | NDArray[] | None) – Material mass density in g/cm³ (must be positive)
- **energy** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any] | None) – Optional backward-compatible alias (eV). If provided, interpreted as electron-volts and converted to keV.

Returns

Dataclass containing all calculated X-ray properties.

Return type

XRayResult

Raises

- **FormulaError** – If chemical formula cannot be parsed or contains invalid elements
- **EnergyError** – If energy values are outside valid range (0.03-30.0 keV)
- **ValidationError** – If density is not positive or other validation failures

Examples

```
>>> import xraylabtool as xlt
>>> result = xlt.calculate_single_material_properties("SiO2", 8.0, 2.2)
>>> print(f"Formula: {result.formula}")
Formula: SiO2
```

See also

calculate_xray_properties : Calculate properties for multiple materials XRayResult : Complete documentation of returned dataclass

calculate_xray_properties(*formulas*, *energies*, *densities*)

Calculate X-ray optical properties for multiple material compositions.

Parameters

- **formulas** (list[str]) – List of chemical formula strings
- **energies** (float | floating[Any] | NDArray[] | Sequence[float] | NDArray[Any]) – X-ray energies in keV applied to all materials
- **densities** (list[float]) – List of material mass densities in g/cm³

Returns

Dictionary mapping formula strings to XRayResult objects.

Return type

dict[str, XRayResult]

Raises

- **ValueError** – If formulas or densities are empty or not lists, their lengths differ, a formula is not a non-empty string, or a density is not a positive number.
- **RuntimeError** – If no formulas were processed successfully

See also

calculate_single_material_properties : Single material calculations XRayResult : Documentation of returned data structure

3.10.13 Quick Reference

Single Material Calculation:

```
import xraylabtool as xrt

result = xrt.calculate_single_material_properties("Si", 8.0, 2.33)  # formula, keV, g/cm³
```

Batch Calculation:

```
results = xrt.calculate_xray_properties(
    ["Si", "Al"], [5.0, 8.0, 10.0], [2.33, 2.70]
)  # dict[formula, XRayResult]
```

Formula Parsing:

```
from xraylabtool.utils import parse_formula

composition = parse_formula("SiO2")
# Returns: {"Si": 1, "O": 2}
```

Unit Conversions:

```
from xraylabtool.utils import energy_to_wavelength, wavelength_to_energy

wavelength = energy_to_wavelength(8.0)  # keV -> Å (1.5498)
energy = wavelength_to_energy(1.55)     # Å -> keV (7.999)
```

3.10.14 Exception Hierarchy

class XRayLabToolError

Bases: Exception

Base exception for all XRayLabTool-related errors.

class ValidationError(*message*, *parameter=None*, *value=None*)

Bases: XRayLabToolError

Exception raised when input validation fails.

Parameters

- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

__init__(*message*, *parameter=None*, *value=None*)

Initialize ValidationError with message and optional parameter context.

Parameters

- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

class FormulaError(*message*, *formula=None*)

Bases: XRayLabToolError

Exception raised for invalid chemical formulas.

Parameters

- **message** (*str*)
- **formula** (*str | None*)

__init__(*message*, *formula=None*)

Initialize FormulaError with message and optional formula context.

Parameters

- **message** (str) – The error message
- **formula** (str | None) – Optional chemical formula that caused the error

class EnergyError(*message*, *energy=None*, *valid_range=None*)

Bases: XRayLabToolError

Exception raised for invalid energy values or ranges.

Parameters

- **message** (*str*)
- **energy** (*float | None*)
- **valid_range** (*str | None*)

__init__(*message*, *energy=None*, *valid_range=None*)

Initialize EnergyError with message and optional energy context.

Parameters

- **message** (*str*)
- **energy** (*float | None*)
- **valid_range** (*str | None*)

class CalculationError(*message*, *formula=None*, *energy=None*)

Bases: XRayLabToolError

Exception raised when X-ray property calculations fail.

Parameters

- **message** (*str*)
- **formula** (*str | None*)
- **energy** (*float | None*)

__init__(*message*, *formula=None*, *energy=None*)

Initialize CalculationError with message and optional context.

Parameters

- **message** (str) – The error message
- **formula** (str | None) – Optional chemical formula context
- **energy** (float | None) – Optional energy value context in keV

class DataFileError(*message*, *filename=None*)

Bases: XRayLabToolError

Exception raised when data files cannot be loaded or processed.

Parameters

- **message** (*str*)
- **filename** (*str | None*)

__init__(*message*, *filename=None*)

Initialize DataFileError with message and optional filename context.

Parameters

- **message** (*str*)
- **filename** (*str | None*)

class AtomicDataError(*message*, *element=None*)

Bases: XRayLabToolError

Exception raised when atomic data cannot be retrieved or processed.

Parameters

- **message** (*str*)
- **element** (*str | None*)

__init__(*message*, *element=None*)

Initialize AtomicDataError with message and optional element context.

Parameters

- **message** (*str*)
- **element** (*str | None*)

class UnknownElementError(*element*)

Bases: AtomicDataError

Exception raised when an unknown element symbol is provided.

Parameters
element (*str*)

__init__(*element*)

Initialize UnknownElementError with element symbol.

Parameters
element (*str*)

class BatchProcessingError(*message*, *failed_items=None*, *total_items=None*)

Bases: XRayLabToolError

Exception raised during batch processing operations.

Parameters

- **message** (*str*)
- **failed_items** (*list[Any] | None*)
- **total_items** (*int | None*)

__init__(
message,
failed_items=None,
total_items=None,
)

Initialize BatchProcessingError with message and optional failure context.

Parameters

- **message** (*str*)
- **failed_items** (*list[Any] | None*)
- **total_items** (*int | None*)

class NearEdgeDataError(*message*, *parameter=None*, *value=None*)

Bases: ValidationError

Exception raised when near-edge absorption data cannot be loaded or merged.

Covers a malformed or non-monotonic spectrum, a spectrum that does not overlap the tabulated energy range, invalid merge points, and degenerate (flat or non-finite) normalisation against the tabulated data.

Parameters

- **message** (*str*)
- **parameter** (*str | None*)
- **value** (*Any | None*)

3.11 Architecture

Design decisions and architectural documentation for XRayLabTool.

3.11.1 JAX Architecture & Design

Audience: Developers and users wanting deep understanding of the v0.5.0 design.

Purpose: Explain why JAX, how it works, and how XRayLabTool uses it.

Why JAX?

What is JAX?

JAX is a Python library for numerical computing with:

1. **JIT (Just-In-Time) Compilation** - Convert Python → compiled machine code
2. **Automatic Differentiation** - Compute gradients with `grad()`
3. **Vectorization** - Apply functions across batches with `vmap()`
4. **GPU/TPU Support** - Automatic device acceleration

XRayLabTool uses features 1, 3 and 4. Feature 2 is public for the core as well: *`xraylabtool.calculators.optics.optical_properties()`* and `kk_f1` are `jit`/`vmap`/`grad`-compatible.

NumPy vs JAX

```
import numpy as np
import jax
import jax.numpy as jnp

# NumPy: eager evaluation
x_np = np.array([1.0, 2.0, 3.0])
y_np = np.sin(x_np)  # Computed immediately
print(y_np)  # [0.841, 0.909, 0.141]

# JAX: lazy evaluation + JIT
x_jax = jnp.array([1.0, 2.0, 3.0])
y_jax = jnp.sin(x_jax)  # Not computed yet, just recorded

# Explicit compilation
sin_jit = jax.jit(jnp.sin)
y_compiled = sin_jit(x_jax)  # Computed and cached
```

The core calculators and the Kramers-Kronig package are JAX-only. Neither has a NumPy twin. For `xraylabtool.kk` the single `jax.numpy` kernel is 5-14 times faster on CPU than the removed NumPy implementation at every size measured (*ADR-005: The Kramers-Kronig Package Is JAX-Only*). For the core calculators the old JAX backend lost to NumPy because of shape-keyed eager compilation and host round trips, not because of JAX; the device program of ADR-006 removes both. A small single call is still slower than the removed NumPy path (launch latency), which ADR-006 records as accepted; batches are faster. `set_backend("jax")` is a no-op and `set_backend("numpy")` raises `ValueError`; see *ADR-006: The Core Calculators Are JAX-Only* for the evidence and *ADR-004: Host-Device Transfer Minimization Strategy* for the transfer policy.

Performance Benefits

The figures and snippets in this section are generic illustrations of JAX, not measurements of XRayLabTool; the measured gates and the accepted trade-offs are in *ADR-006: The Core Calculators Are JAX-Only*.

1. JIT Compilation: 10-100x speedup

```
NumPy (eager):        ■■■■■■■■■■■■■■■■■■■■■ (1.0x baseline)
JAX (eager):          ■■■■■■■■■■■■■■■■■■■■■ (0.95x, ~same)
JAX (JIT compiled):   ■ (0.01-0.1x, 10-100x faster)
```

2. Vectorization: Automatic batching with `vmap()`

```
# NumPy: manual loop
energies = np.linspace(1000, 20000, 1000)
results = []
```

(continues on next page)

(continued from previous page)

```
for e in energies:
    result = calculate_property(e)  # 1000 calls
    results.append(result)

# JAX: automatic vectorization
calculate_batch = jax.vmap(calculate_property)
results = calculate_batch(energies)  # Single compiled call
```

3. GPU Acceleration: Automatic offloading

```
CPU:        ■■■■■■■■■■■■■■■■■■■■ (1.0x baseline)
GPU (V100): ■ (0.02x, 50x faster)
GPU (A100): ■ (0.01x, 100x faster)
```

Design Philosophy

JAX adoption strategy for XRayLabTool:

1. **Transparent to users** - API stays the same
2. **Same public API** - public function names and results are kept; `set_backend("numpy")`, `ops` and the worker pools are removed (see the changelog)
3. **Gradual adoption** - Don't rewrite everything at once
4. **Future-proof** - Enable autodiff and hardware acceleration later

JAX Fundamentals

JIT Compilation

What is JIT?

JIT converts Python functions into compiled machine code:

```
import jax
import jax.numpy as jnp

# Normal function (eager evaluation)
def compute(x):
    return jnp.sin(x) ** 2 + jnp.cos(x) ** 2

# First call: Python interpreter runs
result1 = compute(1.0)  # ~0.1 ms (with overhead)

# Compiled version
compute_jit = jax.jit(compute)

# First call: JAX compiles function (expensive)
result2 = compute_jit(1.0)  # ~50 ms (includes compilation)

# Subsequent calls: use cached compiled code (fast)
result3 = compute_jit(1.0)  # ~0.001 ms
```

Cost-Benefit:

- Compilation cost: 50-100 ms (one-time)
- Per-call speedup: 100-1000x
- Break-even: 100+ calls

Why this matters for XRayLabTool:

- Calculation functions run thousands of times (batch processing)

- One-time compilation cost is negligible
- Per-call speedup dominates overall performance

Traces and Shapes

JAX compiles based on **input shapes**, not values:

```
import jax
import jax.numpy as jnp

@jax.jit
def process(x):
    return jnp.sum(x) * 2

# First call with shape (3,)
result1 = process(jnp.array([1.0, 2.0, 3.0]))
# JAX traces and compiles for shape (3,)

# Second call with shape (3,) - reuses compilation
result2 = process(jnp.array([4.0, 5.0, 6.0]))
# Cache hit, fast

# Third call with shape (5,) - recompiles!
result3 = process(jnp.array([1.0, 2.0, 3.0, 4.0, 5.0]))
# New shape = recompilation (slow)
```

Implication for XRayLabTool:

- Single material calculations always use same shape
- Batch processing with consistent batch sizes benefits from caching
- Different input sizes trigger recompilation

Automatic Differentiation

Future capability (enabled by JAX, not yet used):

```
import jax
import jax.numpy as jnp

def scattering_factor(energy):
    """Calculate f1 component."""
    return jnp.exp(-energy / 1000)  # Simplified

# Compute gradient dβ/dE
gradient_fn = jax.grad(scattering_factor)
dfdE = gradient_fn(8000.0)  # Automatic differentiation
```

Future use cases:

- Optimization (find optimal angles, densities)
- Uncertainty quantification
- Inverse problems (given results, find parameters)

Vectorization with vmap

Automatic batching. XRayLabTool's batch path uses it: `optics._optical_program` is a `vmap` of `optics.optical_properties` over a block of materials, and `host.run_optical_blocks` runs the blocks (generic illustration below):

```
import jax
import jax.numpy as jnp

def calculate_one(energy):
    """Single energy calculation."""
    return energy ** 2

# Vectorized version
calculate_batch = jax.vmap(calculate_one)

energies = jnp.array([1.0, 5.0, 8.0, 10.0])  # keV
results = calculate_batch(energies)  # Compiled as single operation
```

Current approach (ADR-006):

A batch is a `vmap` over materials inside one jitted program (`optics._optical_program`); `calculate_xray_properties` groups the materials by element-count bucket and runs blocks of them through that program, one upload and one fetch per block.

XRayLabTool Implementation

Architecture Overview

```
User Code
   |
   ├─→ API Layer (xraylabtool.__init__)
   |       └─→ validate inputs on the host
   |           one upload (runtime.upload)
   |
   ├─→ Calculation Layer (calculators/)
   |       └─→ @jax.jit compiled programs (optics.py, host.py)
   |           resident scattering tables (tables.py)
   |           physics calculations
   |
   ├─→ Data Layer (data_handling/)
   |       └─→ host atomic metadata cache
   |           batch grouping and blocks
   |
   └─→ Output
       └─→ XRayResult dataclass
           NumPy ndarrays (one fetch)
```

Compilation in XRayLabTool

Key functions decorated with @jax.jit:

```
# In xraylabtool/calculators/optics.py (abridged)
@jax.jit
def _optical_program(tables, batch, energies):
    """vmap of optical_properties over a block of F materials at M shared energies."""
    ...

# xraylabtool/calculators/host.py (run_optical, run_optical_blocks) pads the
# inputs to buckets, uploads once, calls the program and fetches once.
```

Warm-up pattern:

```
# First call triggers compilation
result = calculate_single_material_properties("Si", 8.0, 2.33)
# about 0.3 s on a CPU (loads the tables, compiles the program; ADR-006 gate P8)
```

(continues on next page)

(continued from previous page)

```
# Subsequent calls in the same shape bucket compile nothing
result = calculate_single_material_properties("Si", 8.0, 2.33)
# about 0.3 to 0.45 ms on a CPU (ADR-006 gate P1 limit: 0.5 ms)
```

Kramers-Kronig programs and shape buckets:

`xraylabtool.kk` compiles only for *bucketed* shapes: segment tables are padded to a geometric capacity ladder and energy counts to a power of two on CPU (the fine bucket ladder in equal blocks on accelerators), so a default refined `SiO2` run compiles five top-level programs (`_assemble`, `_evaluate`, `_first_pass`, `_refine_pass`, `_finalize`) and a second call with other values but the same buckets compiles none. Two helpers cut the cold start:

```
from xraylabtool.kk import warm_up

warm_up(("SiO2",))  # compiles the default and refined (0.01) programs
```

The GUI warms the programs of its own default Compute click on an idle worker after the window is shown (through the public `kk_calculate_real`, with the tab's explicit energy grid); the library never warms up by itself. The `xraylabtool kk` command and the GUI launcher also enable JAX's persistent compile cache (`xraylabtool/jax` under the OS user-cache directory, compile-time threshold 0) unless `jax_compilation_cache_dir` is already configured or `XRAYLABTOOL_NO_JAX_CACHE=1` is set. The library itself enables it only when the process sets `XRAYLABTOOL_JAX_CACHE=1` (checked on the first compute call).

Data Flow

Core calculators data flow (device-resident, ADR-006):

```
# User input: Python/NumPy
formula = "SiO2"  # str
density = 2.2     # float
energy = 8.0      # float, keV

host: validate, parse the formula, encode z_idx / counts / molecular weight
|-> upload   (one transfer of the encoded batch and the energies, padded to buckets)
|-> device: interpolate f1/f2 from the resident tables, derive delta, beta,
|            critical angle, attenuation length, SLD, status flag
\-> fetch    (one download of every output) -> XRayResult (NumPy)

# Output: NumPy arrays
result.critical_angle_degrees  # numpy.ndarray (shape: (1,))
result.attenuation_length_cm   # numpy.ndarray (shape: (1,))
```

Kramers-Kronig data flow (device-resident):

```
host: validate formula, energies, near-edge arguments; pick buckets
|-> upload   (one transfer of every input, padded to its bucket)
|-> device: assemble compound spectrum, merge near-edge data
|-> fetch_status  (one small status pytree; errors raised on the host)
|-> device: f1/f2 on the output grid
|-> [refinement] device pass -> fetch_status (one per pass)
\-> fetch  (one download of the final arrays) -> KKResult (NumPy)
```

Between the upload and the final download no array crosses the host/device boundary implicitly; the tests run that region under `jax.transfer_guard("disallow_explicit")` and count the three boundary helpers in `xraylabtool.kk._device`.

Memory Management

JAX vs NumPy memory semantics:

```
import jax.numpy as jnp

# JAX arrays are immutable
x = jnp.array([1.0, 2.0, 3.0])
y = x + 1  # Creates new array, doesn't modify x

# Lazy evaluation
z = jnp.sin(x)  # Computation recorded, not executed yet

# Materialization
z_ready = z.block_until_ready()  # Forces computation, blocks until done
```

For XRayLabTool users:

- No memory leaks (immutability prevents issues)
- Minimal memory overhead (lazy evaluation)
- Transparent memory management (automatic)

Kramers-Kronig padding: a `Spectrum` of `n` segments is stored at a capacity `N >= n` with zero-coefficient pad segments on knots `x_n * (1 + k)`, which contribute exactly nothing to the transform. Memory is bounded by `chunk_size` (energies per block, an upper bound; the default depends on the platform: cache-sized blocks of about 2 MiB per float64 temporary on a CPU, one block or blocks of about 32 MiB on an accelerator), and the true counts are traced int32 leaves so different-sized spectra of one capacity share a compiled program.

GPU Acceleration

Automatic device placement:

```
import jax

# JAX automatically detects and uses available devices
print(jax.devices())  # Shows available hardware

# No code changes needed
result = calculate_single_material_properties("Si", 8.0, 2.33)
# Automatically runs on GPU if available
```

Device hints (advanced):

```
import jax

# Force CPU
with jax.default_device(jax.devices("cpu")[0]):
    result = calculate(...)

# Force GPU
with jax.default_device(jax.devices("gpu")[0]):
    result = calculate(...)
```

Kramers-Kronig on a GPU: the kk pipeline runs on the default JAX device with the same code as on CPU. CI has no GPU, so `scripts/kk_gpu_check.py` (run by hand on the CUDA machine) repeats the oracle, additivity, transfer budget (guard live on the device), padding invariance and benchmark checks and exits 1 on a miss.

Type System

JAX array types:

```
import jax.numpy as jnp
import numpy as np

x_jax = jnp.array([1.0, 2.0])
x_np = np.array([1.0, 2.0])

print(type(x_jax))  # <class 'jaxlib.xla_extension.ArrayImpl'>
print(type(x_np))   # <class 'numpy.ndarray'>

# NumPy operations work with JAX arrays
np.sin(x_jax)  # Works!
np.concatenate([x_jax, x_np])  # Works!

# Type hints
from typing import Union
from jax import Array

def process(x: Union[np.ndarray, Array]) -> float:
    return float(jnp.mean(x))
```

Configuration & Control

JAX Configuration Options

For advanced users:

```
import jax

# Disable JIT (useful for debugging)
jax.config.update("jax_disable_jit", True)

# Disable GPU (force CPU)
jax.config.update("jax_platforms", "cpu")

# Precision control
jax.config.update("jax_default_float_dtype", jnp.float32)
```

In XRayLabTool code:

```
# Environment variables
JAX_DISABLE_JIT=1 python script.py  # Debug without JIT
JAX_PLATFORMS=cpu python script.py  # Force CPU
XRAYLABTOOL_NO_JAX_CACHE=1 xraylabtool kk Si  # no persistent compile cache
```

`set_backend("jax")` is a no-op and `set_backend("numpy")` raises `ValueError`; the core calculators and `xraylabtool.kk` are always float64 JAX (x64 is established at every public compute entry, not only at import).

Common Gotchas & Solutions

Problem: Shape-based recompilation

Issue:

```
@jax.jit
def calculate(energies):
    return jnp.sin(energies)
```

(continues on next page)

(continued from previous page)

```
# Recompilation on every call
for energy_list in many_lists:
    result = calculate(energy_list)  # Different sizes = recompile!
```

Solution:

```
# Pad to fixed size
max_size = 1000
def pad(x):
    return jnp.pad(x, (0, max_size - len(x)))

energy_fixed = pad(energy_list)
result = calculate(energy_fixed)  # Single compilation
```

Problem: Python control flow

Issue:

```
@jax.jit
def conditional_calc(x, use_gpu):
    if use_gpu:  # Won't work! (Python control flow)
        return gpu_calc(x)
    else:
        return cpu_calc(x)
```

Solution:

```
@jax.jit
def conditional_calc(x, use_gpu):
    return jnp.where(use_gpu, gpu_calc(x), cpu_calc(x))
```

Problem: Non-array inputs

Issue:

```
@jax.jit
def calc(material_name, energy):
    data = MATERIAL_DATA[material_name]  # Dict lookup won't trace
    return compute(data, energy)
```

Solution:

```
# Move non-array inputs outside @jax.jit
def calc(material_name, energy):
    data = MATERIAL_DATA[material_name]  # Python (outside JIT)
    return compute_jit(data, energy)  # JAX (inside JIT)

@jax.jit
def compute_jit(data, energy):
    return jnp.sin(energy) * data
```

Testing with JAX

Unit Testing Considerations

```
import jax
import jax.numpy as jnp
```

(continues on next page)

(continued from previous page)

```
import pytest

def test_calculation_matches_reference():
    """JAX results match expected output."""
    result = calculate_single_material_properties("Si", 8.0, 2.33)
    angle = float(result.critical_angle_degrees[0])
    assert abs(angle - 0.2248) < 0.001  # Allow small tolerance

def test_jit_consistency():
    """JIT and eager evaluation match."""
    # Compile once
    compute_jit = jax.jit(core_compute)

    x = jnp.array([1.0, 5.0, 8.0])
    result_eager = core_compute(x)  # Eager (no JIT)
    result_jit = compute_jit(x)     # Compiled

    assert jnp.allclose(result_eager, result_jit)
```

Debugging Strategies

```
import jax

# Disable JIT for debugging
jax.config.update("jax_disable_jit", True)

# Add prints (with caveats)
@jax.jit
def debug_calc(x):
    print(x.shape)  # Prints shape at compile time
    return x ** 2

# Use jax.debug.print for runtime prints
from jax import debug

@jax.jit
def debug_calc2(x):
    debug.print("Value: {}", x)
    return x ** 2
```

Performance Profiling

```
import time
import jax
import jax.numpy as jnp

# Warm up JIT
compute_fn = jax.jit(your_function)
compute_fn(example_input)

# Profile
start = time.time()
for _ in range(1000):
    result = compute_fn(input)
elapsed = (time.time() - start) / 1000
print(f"Time per call: {elapsed * 1000:.3f} ms")
```

Future Directions

Potential JAX Features for v0.5+

The snippets below are illustrative sketches. The pure device API `xraylabtool.calculators.optics.optical_properties` is already `jit`, `vmap` and `grad` compatible (ADR-006).

1. Autodiff for Optimization

```
from jax import grad

def objective(energy_kev):
    """Minimize deviation from target critical angle."""
    result = calculate_single_material_properties("Si", energy_kev, 2.33)
    return (result.critical_angle_degrees[0] - target) ** 2

gradient = grad(objective)
optimal_energy = optimize(objective, gradient, initial_guess)
```

2. vmap for Automatic Batching (shipped)

`calculate_xray_properties` already batches with `vmap` over materials inside `optics._optical_program`; formulas are encoded on the host first, so the host-side calculators themselves are not `vmap`-able.

3. Distributed Processing

```
from jax.experimental import maps

# Shard across multiple GPUs
results = maps.pmap(calculate_single_material_properties)(
    sharded_materials, densities, energy
)
```

Further Reading

- JAX Quickstart
- JAX Documentation
- JIT Mechanics
- Performance Tips

Conclusion

JAX is the single implementation of XRayLabTool's core calculators and the Kramers-Kronig package (ADR-005, ADR-006):

- **One implementation**: float64 `jax.numpy` on JAX's default device, no NumPy twin
- **Measured trade-offs** (ADR-006): a small single call is slower than the removed NumPy path (about 0.3 to 0.45 ms against 0.08 ms on a CPU), batches are about 1.8x faster on a CPU, and the same code runs on a GPU; first calls compile (about 0.3 s on a CPU)
- **Same public API**: results equal the NumPy-era results to `rtol=1e-12`; `set_backend("numpy")` and the `ops` layer are removed (see the changelog)
- **Autodiff ready**: `optics.optical_properties` is `jit`, `vmap` and `grad` compatible

3.11.2 ADR-001: JAX vs NumPy Computation Backend

Status: SUPERSEDED **Date:** 2026-04-06 **Deciders:** Architecture Team **Supersedes:** None **Superseded by:** ADR-006 (the core calculators are JAX-only; the NumPy backend, `ops` and `InterpolationFactory` were removed)

Context

pyXRayLabTool v0.3.0 performs all X-ray optical property calculations using NumPy arrays with SciPy for interpolation. The computation pipeline is:

```
parse_formula -> load_scattering_data -> PchipInterpolator(energy)
  -> calculate_scattering_factors (einsum/broadcast)
  -> calculate_derived_quantities (sqrt, exp, division)
  -> XRayResult
```

Current performance characteristics observed in the codebase:

- `calculate_scattering_factors()` dominates runtime for energy sweeps (>100 points)
- The inner loop iterates over elements, calling interpolators and accumulating weighted sums
- `optimization/vectorized_core.py` already restructures this as matrix ops (`np.einsum`)
- `optimization/optimized_core.py` provides monkey-patching for data loading (2-3x faster I/O)
- Manual SIMD heuristics in `vectorized_core.py` (AVX detection, threshold tuning) indicate the team has hit the ceiling of what NumPy can do

What JAX offers for this workload:

1. **JIT compilation:** Fuses the entire `scattering_factors + derived_quantities` pipeline into a single XLA program. Eliminates Python dispatch overhead and intermediate array allocations.
2. **vmap:** Replaces the explicit for-loop over materials in `compute_multiple()` and `batch_processing.py` with automatic vectorization.
3. **Automatic differentiation:** Enables future gradient-based optimization (NLSQ warm-start for Bayesian fitting per CLAUDE.md guidelines).
4. **Platform portability:** Same code runs on CPU (via XLA), GPU (CUDA/ROCm), and Apple Silicon (Metal plugin).

Which computations benefit most:

Function	Pattern	JIT Benefit	vmap Benefit
`calculate_scattering_factors`	Matrix multiply (einsum) + broadcast	HIGH – fuses multiply+sum+scale into single kernel	HIGH – vectorize over materials
`calculate_derived_quantities`	Element-wise sqrt, division, multiply	MODERATE – eliminates 5 intermediate arrays	LOW – already vectorized
`PchipInterpolator.__call__`	Piecewise polynomial evaluation	HIGH – fuses search+evaluate into single kernel	HIGH – vectorize over elements
`energy_to_wavelength`	Scalar division	LOW – too simple to benefit	N/A
`parse_formula`	Regex string parsing	NONE – not numerical	N/A
Multi-material batch	Loop over formulas	MODERATE	HIGH – `vmap` eliminates loop

Decision

Adopt JAX as the primary computation backend, with NumPy retained at I/O boundaries.

Specifically:

1. All numerical computation in `calculators/`, `optimization/`, and `data_handling/` will use `jax.numpy` via a backend abstraction layer.
2. The `backend/` module will provide a `Protocol`-based `ArrayBackend` interface with `NumpyBackend` and `JaxBackend` implementations.
3. `NumpyBackend` will be the default during migration (zero behavior change), switchable to `JaxBackend` via `set_backend("jax")` or environment variable.
4. After Phase 2, `JaxBackend` becomes the default with `NumpyBackend` as fallback.

5. I/O operations (file loading, CSV export, pandas DataFrames) remain NumPy/pandas.
6. The `PchipInterpolator` dependency is abstracted behind `InterpolationFactory`, allowing the scipy and `JaxPchipInterpolator` (`jax.numpy`) implementations to coexist.

Consequences

Positive

- **Performance:** JIT-compiled scattering factor calculation should achieve 2-10x speedup for energy sweeps >100 points by eliminating Python dispatch and fusing operations.
- **Simplification:** The entire `optimization/vectorized_core.py` module (700 lines of manual SIMD heuristics, AVX detection, contiguity checks) becomes unnecessary – JAX's XLA compiler handles all of this automatically.
- **Future capability:** `jax.grad` enables automatic differentiation through the full calculation pipeline, unlocking GPU-accelerated NLSQ fitting and Bayesian inference via NumPyro.
- **Batch processing:** `jax.vmap` replaces `ThreadPoolExecutor`-based parallel processing in `batch_processing.py` with hardware-vectorized batch computation.

Negative

- **Cold start:** JIT compilation adds 100ms-2s to the first calculation call. Mitigated by AOT compilation at import time for hot-path functions.
- **Debugging complexity:** JIT-compiled functions produce opaque XLA error messages. Mitigated by `jax.disable_jit()` context manager for development.
- **Dependency weight:** JAX + jaxlib adds ~200MB to the install footprint. Mitigated by making JAX optional (`pip install xraylabtool[jax]`).
- **Learning curve:** Team members need to understand JAX's functional purity constraints (no in-place mutation, no Python control flow in JIT).

Risks

- **PCHIP parity:** The JAX backend uses an in-tree `jax.numpy` PCHIP (`JaxPchipInterpolator`, Fritsch-Carlson slopes as in SciPy) instead of the `interpax` dependency it first planned; `tests/unit/test_jax_pchip.py` pins it to SciPy at 1e-12.
- **Numerical equivalence:** JAX's XLA compiler may reorder floating-point operations, producing results that differ at the ~1e-15 level. The golden test suite with 1e-12 tolerance provides a safety net.

Appendix: Functions to JIT-Compile (Priority Order)

1. `calculate_scattering_factors()` – Hot path, called once per material per calculation
2. `calculate_derived_quantities()` – Called immediately after scattering factors
3. `vectorized_interpolation_batch()` – Inner loop of multi-element processing
4. `vectorized_multi_material_batch()` – Outer batch loop (candidate for vmap)
5. `EnergyConfig.to_array()` – Trivial but called frequently

Appendix: Functions that MUST NOT be JIT-compiled

1. `load_scattering_factor_data()` – File I/O, side effects (caching)
2. `create_scattering_factor_interpolators()` – Object creation, side effects (LRU cache)
3. `parse_formula()` – Regex string processing

4. `_warm_priority_cache()` – Threading, side effects
5. Any function that raises `ValueError`/`FileNotFoundError` – JAX JIT traces through exceptions

3.11.3 ADR-002: PyQtGraph vs Matplotlib for GUI Plotting

Status: ACCEPTED **Date:** 2026-04-06 **Deciders:** Architecture Team **Supersedes:** None

---

Context

The pyXRayLabTool GUI currently uses matplotlib embedded in PySide6 via `FigureCanvasQTAgg` and `NavigationToolbar2QT`. There are three matplotlib-based plot widgets:

Widget	File	Purpose	Complexity
`PlotCanvas`	`gui/widgets/plot_canvas.py`	Primary property plot (energy vs. property)	Medium – log axes, grid, legend, markers
`F1F2Plot`	`gui/widgets/sweep_plots.py`	Single-material f1/f2 scattering factors	Simple – two line plots
`MultiF1F2Plot`	`gui/widgets/sweep_plots.py`	Multi-material f1/f2 comparison (2 subplots)	Medium – shared x-axis, dual subplot

All three widgets follow the same pattern:

1. `self.figure = Figure(figsize=...)`
2. `self.canvas = FigureCanvasQTAgg(self.figure)`
3. `self.figure.clear()` then `self.figure.add_subplot(111)`
4. `ax.plot(x, y, label=..., marker=..., linewidth=...)`
5. `ax.set_xlabel/ylabel/xscale`
6. `self.canvas.draw_idle()`

Pain points with matplotlib in the GUI:

1. **Rendering speed:** `canvas.draw_idle()` redraws the entire figure. For energy sweeps with 500+ points across 5+ materials, plot updates feel sluggish.
2. **Theme integration:** The `update_theme()` method on each widget manually iterates over axes, spines, ticks, and legend to apply Qt palette colors via `mpl.rcParams`. This is fragile and requires synchronization with the `ColorPalette` dataclass.
3. **Interactivity:** matplotlib's `NavigationToolbar` provides zoom/pan but no smooth real-time interaction (e.g., hover tooltips, cursor tracking).
4. **Memory:** Each `FigureCanvasQTAgg` embeds a full Agg renderer. Three plot widgets means three independent renderers in memory.
5. **Import time:** matplotlib is one of the heaviest imports, adding ~500ms to GUI startup.

What PyQtGraph offers:

1. **Native Qt rendering:** Renders directly via QPainter, so it shares the Qt event loop and palette natively. No Agg renderer overhead.
2. **Real-time performance:** Optimized for live data display (oscilloscope-style). Can handle 10k+ points at 60fps.
3. **Built-in interactivity:** Mouse-wheel zoom, drag-to-pan, hover crosshair, region-of-interest selection – all built in.
4. **Theme integration:** Uses Qt's QPalette directly. The existing `ColorPalette.to_qpalette()` method works without additional matplotlib synchronization.
5. **Lighter weight:** ~50MB vs matplotlib's ~100MB. Faster import.

Decision

Replace matplotlib with PyQtGraph for all interactive GUI plots. Retain matplotlib as an optional dependency for publication-quality static export.

Migration strategy:

1. Define a `PlotWidget` protocol matching the current interface (`clear()`, `plot_single()`, `plot_multi()`, `update_theme()`, `set_scales()`).
2. Implement `PyQtGraphPlotCanvas`, `PyQtGraphF1F2Plot`, `PyQtGraphMultiF1F2Plot` behind this protocol.
3. Swap imports in `gui/main_window.py` from matplotlib widgets to PyQtGraph widgets.
4. Remove `apply_matplotlib_theme()` from `gui/style.py`.
5. Add an optional `export_publication_plot()` function that uses matplotlib for high-DPI vector export (PDF/SVG).

Consequences

Positive

- **Rendering performance:** Plot updates for 500+ point energy sweeps will be near-instantaneous instead of the ~100ms matplotlib redraw.
- **Theme consistency:** PyQtGraph inherits the Qt palette automatically. The manual `update_theme()` methods (25 lines each across 3 widgets) become trivial or unnecessary.
- **Interactivity:** Users get smooth zoom/pan, hover crosshair with value readout, and real-time cursor tracking without custom code.
- **Startup time:** Removing matplotlib from the GUI import chain saves ~500ms of startup time.
- **Code reduction:** The three matplotlib widget classes (~165 lines total) can be replaced with simpler PyQtGraph equivalents (~100 lines total), since PyQtGraph handles more out of the box.

Negative

- **Log-scale axes:** PyQtGraph's `setLogMode(x=True)` works but tick formatting is less polished than matplotlib. May need custom `AxisItem` subclass for publication-quality log ticks.
- **Legend placement:** PyQtGraph's `LegendItem` has less automatic layout intelligence than matplotlib's `ax.legend()`. May need manual positioning.
- **Publication export:** PyQtGraph produces raster output by default. For vector (PDF/SVG) export, either use PyQtGraph's `exportFile()` or fall back to matplotlib.
- **Familiarity:** Team members experienced with matplotlib will need to learn PyQtGraph's API (PlotWidget, PlotItem, PlotDataItem).

Migration Details

Current matplotlib pattern:

```
# plot_canvas.py
from matplotlib.backends.backend_qtagg import FigureCanvasQTAgg
from matplotlib.figure import Figure


class PlotCanvas(QWidget):
    def __init__(self):
        self.figure = Figure(figsize=(6, 4))
        self.canvas = FigureCanvasQTAgg(self.figure)
```

(continues on next page)

(continued from previous page)

```
    def plot_single(self, result, property_name, ylabel):
        self.figure.clear()
        ax = self.figure.add_subplot(111)
        ax.plot(x, y, label=label, marker="o", linewidth=1.5)
        ax.set_xlabel("Energy (keV)")
        self.canvas.draw_idle()
```

Target PyQtGraph pattern:

```
# plot_canvas.py
import pyqtgraph as pg


class PlotCanvas(QWidget):
    def __init__(self):
        self.plot_widget = pg.PlotWidget()
        self.plot_widget.setBackground(None)  # Use Qt palette
        self.plot_widget.showGrid(x=True, y=True, alpha=0.3)

    def plot_single(self, result, property_name, ylabel):
        self.plot_widget.clear()
        self.plot_widget.plot(
            x, y, name=label, pen=pg.mkPen(width=1.5), symbol="o", symbolSize=6
        )
        self.plot_widget.setLabel("bottom", "Energy (keV)")
```

Log scale handling:

```
# PyQtGraph log mode
self.plot_widget.setLogMode(x=self.log_x, y=self.log_y)
# Custom tick formatting for nice log labels
if self.log_x:
    self.plot_widget.getAxis("bottom").setStyle(tickTextOffset=4)
```

Appendix: Feature Parity Checklist

Feature	matplotlib (current)	PyQtGraph (target)	Notes
Line plot	`ax.plot()`	`pw.plot()`	Direct equivalent
Markers	`marker='o', markersize=6`	`symbol='o', symbolSize=6`	Direct equivalent
Log axes	`ax.set_xscale('log')`	`pw.setLogMode(x=True)`	Direct equivalent
Grid	`ax.grid(True, alpha=0.3)`	`pw.showGrid(x=True, y=True, alpha=0.3)`	Direct equivalent
Legend	`ax.legend()`	`pw.addLegend()`	PyQtGraph needs manual call
Axis labels	`ax.set_xlabel()`	`pw.setLabel('bottom', ...)`	Different API
Dual subplot	`fig.add_subplot(211)`	Two `PlotWidget` in `QSplitter`	Architectural change
Theme colors	Manual `rcParams` sync	Qt QPalette (automatic)	Simplification
Zoom/Pan	`NavigationToolbar`	Built-in mouse interaction	Feature upgrade
Export	`fig.savefig()`	`pg.exportFile()` or matplotlib fallback	May need both

3.11.4 ADR-003: float64 Preservation Policy

Status: ACCEPTED **Date:** 2026-04-06 **Deciders:** Architecture Team **Supersedes:** None

---

Context

The float64 Requirement in X-ray Calculations

pyXRayLabTool calculates X-ray optical properties that span many orders of magnitude:

Quantity	Typical Range	Precision Needed
Dispersion delta	1e-5 to 1e-8	Relative 1e-6
Absorption beta	1e-6 to 1e-10	Relative 1e-6
Attenuation length (cm)	1e-4 to 1e+2	Relative 1e-4
Critical angle (degrees)	0.01 to 1.0	Absolute 1e-4
SLD (Ang^-2)	1e-5 to 1e-6	Relative 1e-6
Energy (eV)	30 to 30000	Absolute 0.1
Wavelength (m)	4e-11 to 4e-8	Relative 1e-8

The computation involves products of physical constants spanning 30+ orders of magnitude:

```
# From constants.py
THOMPSON = 2.8179403227e-15  # meters
AVOGADRO = 6.02214199e23  # mol^-1
SCATTERING_FACTOR = THOMPSON * AVOGADRO * 1e6 / (2 * pi)  # ~2.7e14

# From core.py calculate_scattering_factors():
common_factor = SCATTERING_FACTOR * mass_density / molecular_weight
wave_sq = wavelength**2  # ~(1e-10)^2 = 1e-20
dispersion = wave_sq * common_factor * f1_total
# = 1e-20 * 2.7e14 * density/MW * f1 -> ~1e-6
```

The intermediate product `wave_sq * common_factor` is ~1e-6, and the final dispersion is also ~1e-6. With float32 (7 decimal digits of precision), the relative error in dispersion would be ~1e-1, which is catastrophically wrong. **float64 is non-negotiable for correctness.**

JAX float32 Default

JAX defaults to float32 for all operations. This is a deliberate design choice for ML workloads where float32 is sufficient and 2x faster. However, JAX provides a configuration flag:

```
jax.config.update("jax_enable_x64", True)  # Enable float64 globally
```

When enabled, `jnp.array([1.0])` creates a float64 array, matching NumPy behavior.

Current Codebase float64 Usage

The codebase is consistently float64:

```
# core.py -- explicit float64 everywhere
np.zeros(n_energies, dtype=np.float64, order="C")
np.empty((n_elements, n_energies), dtype=np.float64, order="C")
np.asarray(f1_values, dtype=np.float64)

# vectorized_core.py
np.empty((n_elements, n_energies), dtype=np.float64, order="C")

# batch_processing.py
np.array([float(energies)], dtype=np.float64)
```

All array types in `typing_extensions.py` are defined as `NDArray[np.float64]`.

Decision

Enforce float64 globally via JAX configuration at application startup. Treat float32 computation as a correctness bug.

Implementation:

1. Set `jax.config.update("jax_enable_x64", True)` in `xraylabtool/__init__.py` before any JAX imports.
2. Add a runtime assertion in `backend/jax_impl.py` that verifies x64 mode is enabled.
3. All explicit `dtype=np.float64` specifications in the codebase are preserved (they become `dtype=jnp.float64`).
4. The `validate_energy_array()` function in `typing_extensions.py` already checks `dtype in [np.float32, np.float64]` – tighten this to float64-only.
5. Add a CI check that runs the test suite with `JAX_ENABLE_X64=0` and expects failures (proving that float64 is correctly enforced).

Consequences

Positive

- **Correctness guarantee:** No possibility of silent float32 precision loss in X-ray calculations.
- **Behavioral parity:** `jnp.array([1.0]).dtype == jnp.float64`, identical to NumPy.
- **Explicit contract:** The dtype policy is documented and enforced, not implicit.

Negative

- **Performance cost:** float64 operations are ~2x slower than float32 on GPU and ~1.3x slower on CPU. For this workload (X-ray property calculations, not ML training), this is an acceptable tradeoff – correctness is Priority 1 per CLAUDE.md.
- **Memory cost:** float64 arrays use 2x the memory of float32. For typical energy sweeps (50-1000 points, 1-10 materials), this means kilobytes, not a concern.
- **JAX ecosystem friction:** Some JAX libraries default to float32. Any third-party JAX code must be validated for float64 compatibility.

Enforcement Strategy

```
# xraylabtool/__init__.py (FIRST LINES, before any other imports)
import os

os.environ.setdefault("JAX_ENABLE_X64", "1")


# Verify enforcement at import time
def _verify_float64():
    """Verify that JAX float64 mode is enabled."""
    try:
        import jax
        import jax.numpy as jnp

        if not jax.config.jax_enable_x64:
            raise RuntimeError(
                "JAX float64 mode is required for X-ray calculations. "
                "Set JAX_ENABLE_X64=1 or call "
                "jax.config.update('jax_enable_x64', True)"
            )
        # Verify actual behavior
        test_val = jnp.array(1.0)
```

(continues on next page)

(continued from previous page)

```
        if test_val.dtype != jnp.float64:
            raise RuntimeError("JAX float64 mode enabled but arrays are float32")
    except ImportError:
        pass  # JAX not installed, numpy backend handles this


_verify_float64()
```

Appendix: Numerical Precision Requirements by Function

Function	Min Precision Required	float32 OK?	float64 OK?
`energy_to_wavelength`	1e-8 relative	NO (6e-7 range)	YES
`calculate_scattering_factors`	1e-12 relative	NO (1e-7 intermediate products)	YES
`calculate_derived_quantities`	1e-8 relative	NO (sqrt of 1e-8 values)	YES
`calculate_critical_angle`	1e-4 absolute	MARGINAL	YES
`calculate_attenuation_length`	1e-4 relative	NO (division by 1e-10)	YES
`parse_formula`	Exact (integer)	N/A	N/A
`PchipInterpolator`	1e-6 relative	NO (cubic polynomial coefficients)	YES

Conclusion: No function in the calculation pipeline can safely use float32.

3.11.5 ADR-004: Host-Device Transfer Minimization Strategy

Status: ACCEPTED **Date:** 2026-04-06 **Deciders:** Architecture Team **Supersedes:** None **Updated by:** ADR-006 (the shipped flow below replaces the `ops`-based sketch; the policy is unchanged)

Context

The Transfer Problem

JAX arrays live on a "device" (CPU via XLA, GPU, or TPU). NumPy arrays live on the "host" (regular Python memory). Every conversion between JAX and NumPy arrays triggers a data transfer:

```
jax_array = jnp.array(numpy_array)  # Host -> Device transfer
numpy_array = np.array(jax_array)  # Device -> Host transfer
numpy_array = jax_array.to_py()  # Device -> Host transfer (explicit)
```

On CPU-only systems (the primary deployment for pyXRayLabTool), the "device" is the same physical memory, so the transfer cost is a memcpy (~1us for small arrays, ~100us for large ones). On GPU, transfers go over PCIe and cost 10-100x more.

Where Transfers Happen in the Current Architecture

Analyzing the data flow through the codebase:

```
[LOAD: numpy] -> [CACHE: numpy dict] -> [INTERPOLATE: scipy on numpy]
  -> [COMPUTE: numpy broadcast/einsum] -> [RESULT: XRayResult(numpy)]
  -> [PLOT: matplotlib takes numpy] / [EXPORT: pandas takes numpy]
```

With a naive JAX migration, transfers would occur at every boundary:

```
[LOAD: numpy] -> HOST-DEVICE -> [CACHE: jax?] -> [INTERPOLATE: JaxPchipInterpolator on jax]
  -> [COMPUTE: jax jit] -> DEVICE-HOST -> [RESULT: XRayResult(numpy)]
  -> [PLOT: pyqtgraph takes numpy] / [EXPORT: pandas takes numpy]
```

Critical transfer points identified:

1. **Interpolator input:** Energy values (numpy) -> interpolator (scipy / JaxPchipInterpolator)
2. **Interpolator output:** f1/f2 values -> scattering factor calculation
3. **Scattering factors -> derived quantities:** Should be zero-copy within JAX
4. **Computation result -> XRayResult:** JAX arrays -> numpy for storage
5. **XRayResult -> plotting:** numpy arrays -> PyQtGraph (which accepts numpy)
6. **XRayResult -> export:** numpy arrays -> pandas DataFrame

Array Sizes in This Workload

Array	Typical Size	Bytes (float64)
Energy sweep	50-1000 points	400B - 8KB
Wavelength	same as energy	400B - 8KB
f1/f2 per element	same as energy	400B - 8KB
Dispersion/Absorption	same as energy	400B - 8KB
f1_matrix (n_elements x n_energies)	2-10 x 50-1000	800B - 80KB
Scattering factor table (loaded from .nff)	~500 rows x 3 cols	~12KB

These are small arrays. The transfer overhead matters not because of data size, but because of **frequency** – a batch of 100 materials means 100 x N transfers.

Decision

Minimize host-device transfers by keeping data as JAX arrays through the entire computation pipeline, converting to NumPy only at consumption boundaries (GUI, export, CLI output).

Transfer Policy

Boundary	Direction	Policy
File loading (.nff data)	Disk -> Host (numpy)	**Keep as numpy.** File I/O is inherently host-side.
Scattering data cache	Host -> Device	**Upload once.** `load_tables()` reads `scattering_tables.npz` and uploads the stacked tables once; they are a program argument.
Interpolator creation	Host -> Device	**No per-element interpolators.** The PCHIP nodes and slopes of all 92 elements are rows of the resident tables.
Energy input from user	Host -> Device	**One `runtime.upload`** of the encoded formula, density and energies at the entry point of `calculate_single_material_properties()`.
Computation pipeline	Device -> Device	**Zero transfers.** One compiled program (`optics.optical_properties`) covers interpolation, scattering factors and derived quantities.
XRayResult construction	Device -> Host	**One `runtime.fetch`** of the whole result tree; the host slices off padding and builds `XRayResult`.
GUI plotting	Host (numpy) -> Qt	**Zero additional transfers.** PyQtGraph accepts numpy arrays directly.
CLI output	Host (numpy) -> stdout	**Zero additional transfers.** `print()` calls `.item()` or `str()`.
Batch export	Host (numpy) -> pandas	**Zero additional transfers.** pandas accepts numpy arrays.

Implementation Pattern

The shipped flow (ADR-006). Only `runtime.upload`, `runtime.fetch_status` and `runtime.fetch` cross the boundary; each opens a narrow `jax.transfer_guard("allow")` scope and bumps a counter, and tests run the device region under `jax.transfer_guard("disallow_explicit")` and assert the counters (at most 1 upload and 1 fetch per device program call).

```
host:   validate formula, energies, density; encode z_idx, counts, mw, n_electrons
  |-> runtime.upload  ((z_idx, counts, mw, rho, n_electrons, energies_keV), one call)
  |-> device: one compiled program
  |     interpolate f1/f2 from the resident tables -> delta, beta
  |     -> critical angle, attenuation length, SLD, electron density, ok flag
  \-> runtime.fetch   ({"out": (F, 9, M), "electron_density": (F,), "ok": (F,)}, one call)
host:   slice off padding, raise on a failed status, build XRayResult (NumPy)
```

```python
# calculators/host.py (sketch of run_optical)
def run_optical(batch, energies_kev):
    tables = device_tables()  # resident, uploaded once
    device_batch = runtime.upload((*batch, energies_kev))  # SINGLE HOST -> DEVICE
    out = _optical_program(tables, device_batch)  # ALL COMPUTATION ON DEVICE
    return runtime.fetch(out)  # SINGLE DEVICE -> HOST
```

Batch Processing Optimization

For multi-material calculations, the transfer savings compound:

```python
# BEFORE (naive): N materials x 2 transfers each = 2N transfers
for formula, density in materials:
    energy_jax = jnp.asarray(energy_np)  # Transfer 1
    result = compute(energy_jax, ...)  # Device computation
    result_np = np.asarray(result)  # Transfer 2

# AFTER (shipped): one upload and one fetch per F-block of materials
device_batch = runtime.upload(
    (z_idx, counts, mw, rho, n_electrons, energies)
)  # Transfer 1
out = _optical_program(tables, device_batch)  # vmap over materials inside the program
host_out = runtime.fetch(out)  # Transfer 2 (bulk)
```

Consequences

Positive

- **Minimal overhead:** Only 2 transfers per single-material calculation (input + output), regardless of computation complexity.
- **Batch efficiency:** `vmap`-based batch processing shares the input transfer, reducing from 2N to 2 transfers for N materials.
- **JIT effectiveness:** Keeping data on-device allows XLA to fuse the entire computation pipeline into a single kernel, maximizing JIT benefits.
- **Future GPU readiness:** If GPU support is added later, the transfer minimization strategy prevents PCIe bottlenecks.

Negative

- **Conversion at XRayResult:** The `XRayResult` dataclass stores numpy arrays, requiring a bulk device-to-host conversion at result creation. This is intentional – `XRayResult` is the public API boundary, and downstream consumers (GUI, export, CLI) all expect numpy.
- **Cache storage:** The scattering tables are resident device arrays: 92 elements padded to 449 knots, five arrays, 1.7 MB in float64 – negligible.
- **Cannot mix backends in computation:** Once data enters the JAX pipeline, all intermediate operations must use JAX ops. No falling back to scipy mid-computation.

Monitoring

`xraylabtool.backend.runtime` counts uploads and fetches (`transfer_counts`, `thread_transfer_counts`, `reset_transfer_counts`); the transfer-budget tests assert them. `MemoryMonitor` in `batch_processing.py` still checks host result memory only.

Appendix: Transfer Cost Reference (CPU Backend)

Array Size	Transfer Time (est.)	Notes
100 floats (800B)	~1us	Negligible
1000 floats (8KB)	~5us	Negligible
10000 floats (80KB)	~50us	Noticeable in tight loops
100000 floats (800KB)	~500us	Should avoid in hot path

For a typical calculation with 200-point energy sweep and 3 elements:

- Input transfer: 200 floats = ~1us
- Output transfer: 200 x 10 fields = 2000 floats = ~10us
- Total transfer overhead: ~11us out of ~1ms total computation = ~1%

This confirms that the 2-transfer strategy keeps overhead well below 5% of computation time.

3.11.6 ADR-005: The Kramers-Kronig Package Is JAX-Only

Status: ACCEPTED **Date:** 2026-10-26 **Deciders:** Wei Chen **Supersedes:** The "NumPy is the default backend" part of ADR-001, for `xraylabtool.kk` only

---

Context

`xraylabtool.kk` (Kramers-Kronig f1 from tabulated f2) first shipped as a NumPy implementation with a JAX twin selected through `set_backend()`, following ADR-001. ADR-001 rests on the premise that NumPy is the faster choice for the small arrays of a CPU-only user. For this kernel that premise does not hold: the closed-form sum over 1201 segments (SiO2) at m energies is a dense broadcast that XLA fuses and threads, while NumPy materialises every temporary.

Two implementations also meant two sets of rounding, two sets of tests and a device boundary crossed on every chunk.

Decision

`xraylabtool.kk` is one implementation, written with `jax.numpy`. There is no NumPy twin of any algorithm and no `ops`-proxy dispatch inside `kk`.

1. **Device-resident pipeline.** One upload of all inputs, one status fetch after assembly and near-edge merge, one status fetch per refinement pass and one final download. The device-only region runs under `jax.transfer_guard("disallow_explicit")` in tests; the three boundary helpers (`upload`, `fetch_status`, `fetch`) in `xraylabtool.kk._device` are the only crossings and count themselves (see ADR-004 for the general policy).
2. **Padded `Spectrum` pytree.** Segment tables are padded to a capacity bucket with `x_n * (1 + k)` knots and all-zero coefficients, which add exactly nothing to the transform. The true count `n_segments` is a traced int32 leaf, so spectra of different sizes but equal capacity can be stacked and `vmap`-ed. `Spectrum` and `kk_f1` are public and `jit`/`vmap`/`grad`-compatible.

3. **Shape buckets.** Every compiled entry sees only bucketed shapes (a geometric ladder with ratio at most 1.25 for segment capacities; for energy counts a power of two on CPU, the fine bucket ladder in equal blocks on accelerators), so the set of compiled programs is small. A default refined `SiO2` run compiles five top-level programs (`_assemble`, `_evaluate`, `_first_pass`, `_refine_pass`, `_finalize`); a second call with other values and the same buckets compiles none.
4. **Status pytrees, not exceptions, on the device.** Capacity overflow, near-edge merge problems and refinement bookkeeping are returned as small status pytrees, fetched once and turned into the established exceptions and log messages on the host.
5. **float64 always.** x64 is established at every concrete public entry and every worker entry, not only at import.
6. **Persistent compile cache for the CLI and GUI only.** They call `xraylabtool.kk._device.enable_persistent_cache()`; the library does so only when `XRAYLABTOOL_JAX_CACHE=1` is set. `XRAYLABTOOL_NO_JAX_CACHE=1` opts out. The GUI also warms, on an idle worker, the programs of its own default Compute click (the public `kk_calculate_real` with the tab's explicit grid; `kk.warm_up()` does the same for the default grid); the CLI never warms up.

Evidence

The ADR-001 assumption, refuted for this kernel. CPU only (20-core machine, no GPU in the measuring session), `SiO2`, host wrapper (upload, kernel, download), median of 15 warm calls, milliseconds. The NumPy column is the historical measurement of the removed implementation (commit `1bf56d9`, recorded in `scripts/kk_bench.py`); the JAX column was measured with `scripts/kk_bench.py --platform cpu` on 2026-10-26 on a shared machine whose load average was between 1 and 7, so read the figures to within roughly 30 %.

m (energies)	NumPy (removed)	JAX on CPU	JAX faster by
100	17.7	3.5	5x
1 000	157	16.0	9.8x
10 000	1 892	139	13.6x
100 000	14 746	1 381	10.7x

`kk_calculate_real("SiO2", refine_tolerance=0.01)` on CPU (1383 points, 267 refined):

	NumPy (removed)	JAX
cold, fresh interpreter	0.29 s	1.31-1.46 s (three runs)
cold, persistent cache filled	-	0.58-0.65 s (two runs)
warm, median	0.273 s	0.154-0.158 s (min 0.096-0.119 s)
first `kk_calculate_real` after import, 1 000 energies	-	0.50 s

On the RTX 4090 laptop GPU the design measurement (2026-10-25) gave the JAX kernel at 2.2 / 3.2 / 16 / 107 ms for m = 100 / 1 000 / 10 000 / 100 000 against 11 / 145 / 1 555 / 16 649 ms for NumPy. Those GPU figures are the spec's, not repeated here; `scripts/kk_gpu_check.py` reruns them on the target machine.

Accuracy is unchanged: the upstream goldens and an independent longdouble quadrature oracle (`tests/unit/kk/test_oracle.py`) pass at tolerances that were measured, not loosened.

Consequences

- **Cold start grows.** A cold refined `SiO2` run took 0.29 s with NumPy and now takes 1.31-1.46 s when it must compile, and 0.58-0.65 s with the persistent cache filled. The persistent cache is therefore on by default in the CLI and GUI, with the compile-time threshold set to 0 because JAX's default of 1 s would skip nearly every kk program.
- **`set_backend()` has no effect on `kk`.** `set_backend("numpy")` is accepted and ignored by `kk` (no warning, no error); the rest of the package keeps its backend selection. `KKResult.backend` is always `"jax"` from `kk`; the legacy value `"numpy"` is still accepted when constructing a `KKResult`.

- `ensure_float64` is removed from `xraylabtool.backend.array_ops`; `kk` establishes x64 itself.
- Tests that were parametrised over NumPy and JAX collapse to JAX. The goldens and the oracle replace the parity comparison.
- **JAX floor is `>=0.12.0.1`.** The golden tolerances were measured on this release; releases 0.8.0 to 0.12.0 differ in the last digits of fused XLA expressions and miss some of them by a few units in the twelfth digit. The tolerances were not loosened; the dependency floor was raised instead.

Known deviations from the acceptance gates

Measured on 2026-10-26 on a shared 20-core CPU and an RTX 4090 laptop GPU (times in ms; min / median where both are given). These gates were not met and are accepted:

- **CPU refined run (`SiO2`, `refine_tolerance=0.01`) is 10-20 % slower in steady state than the forced-JAX pipeline at `1bf56d9`.** Median at HEAD 128.5 / 151.3 / 132.5 / 143.9 against 119.4 / 117.6 / 124.5 / 122.5 (four interleaved rounds). The gate "refined warm <= 0.10 s" is **not met**: the median is about 0.125-0.155 s on this machine (minimum 0.096-0.119 s).
- **GPU fixed overhead at small m** (device-side assembly plus the status fetch), minimum ms, `1bf56d9` -> HEAD: m = 10^3 2.5 -> 4.1; m = 3100 5.2 -> 7.0; m = 10^4 13.0 -> 15.8 (the kernel gate of 20 ms is met); m = 10^5 108.9 -> 115.1.
- **Bare-library GPU refined cold run is 3.7-4.4 s against the 2.5 s gate** (`1bf56d9` itself took 3.5 s). With the persistent cache primed it takes 0.54 s, which is what the CLI and GUI get.

The cause is the design itself: the spectrum is assembled and merged on the device, so the inputs go up once and the result comes down once, at the price of a status fetch and a few more small programs than the host-assembled pipeline; cold compiles on a GPU are slower than on a CPU.

x64 requirement

All kk entries need 64-bit mode and set `jax_enable_x64` globally when they are called. A surrounding thread-local `jax.enable_x64(False)` context is not overridden: inside it the call raises `TypeError` naming `jax_enable_x64`. This is a documented limitation.

Alternatives considered

- **Extend `ArrayBackend` so one `ops`-written path also runs on NumPy.** Rejected: loops (`lax.map`, compaction), `searchsorted` and `unique(size=)` have no common-denominator form, and the measurements show NumPy has no remaining advantage.
- **Keep the NumPy twin as the CPU default.** Rejected: slower at every size measured, and a second implementation to keep in step with the goldens.
- **A single `while_loop` for refinement.** Rejected in favour of host-orchestrated, device-resident passes with one status fetch each; the loop's data-dependent output size does not fit a static shape.

References

- Design: `docs/superpowers/specs/2026-10-25-kk-jax-first-design.md`
- ADR-001 (backend), ADR-003 (dtype policy), ADR-004 (transfers)
- `docs/guides/kk_guide.rst`, `docs/architecture/jax_architecture.rst`

3.11.7 ADR-006: The Core Calculators Are JAX-Only

Status: ACCEPTED (core calculators, resident tables, batch, `utils` helpers, CLI and GUI wiring; see "Scope and what follows") **Date:** 2026-10-27 **Deciders:** Wei Chen **Supersedes:** ADR-001 (JAX vs NumPy computation backend). ADR-003 (dtype policy) applies unchanged; the transfer policy of ADR-004 is unchanged (its flow diagram was updated to the shipped design).

Context

ADR-005 made `xraylabtool.kk` one `jax.numpy` implementation. The rest of the package was still NumPy-first: the core calculators ran on NumPy unless an NVIDIA GPU was present, the JAX backend was an optional twin (`NumpyBackend`, `JaxBackend`, `ops`), interpolation was one SciPy or JAX object per element, and batch runs forked process pools.

Measured on 2026-10-26 (RTX 4090 laptop, CUDA 13, 20 cores, median of 5 fresh processes, `scripts/core_bench.py`, commit `4c3e2e9`; times in ms):

Path	SiO2 x200 E	SiO2 x20 000 E	200 formulas x200 E	2000 formulas x200 E
NumPy backend, CPU	0.083	1.63	19.8	198
JAX backend (then), CPU	0.816	5.71	184	1898
JAX backend (then), GPU	3.27	3.98	728	7289

The old JAX backend was 10x slower than NumPy on CPU for single calls and batches and 40x slower on GPU, and its first batch in a fresh process cost 1.2 to 4.0 s. The causes were structural, not the physics: shape-keyed eager compilation (68 programs in the first call, 16 more per new energy count), about 60 tiny dispatches and 6 blocking syncs per call, ten separate downloads, and a batch that was the single-material path in a Python loop. ADR-001's premise (NumPy is faster on CPU for small arrays) was therefore true of the old JAX glue, not of JAX.

Decision

The core calculators are one implementation in `jax.numpy` float64. No NumPy twin of any algorithm, no `ops` dispatch, no SciPy PCHIP at runtime, no `pure_callback`, no float32. Wei's directive of 2026-10-26: the whole package is JAX-first and this is mandatory.

1. **One device program per public call.** Host: parse and validate the formula and inputs (the trust boundary), compute the per-material scalars (molecular weight, electron count) as Python floats in today's summation order, and encode `z_idx` (int32) and `counts` (float64). One `runtime.upload`, one compiled program, one `runtime.fetch` of `{"out": (F, 9, M), "electron_density": (F,), "ok": (F,)}`. The device-only region runs under `jax.transfer_guard("disallow_explicit")` in tests and the budgets (at most 1 upload and 1 fetch per device program call: per public call for single materials, per `F`-block for batches) are asserted from the `runtime` counters.
2. **Resident scattering tables.** `xraylabtool/data/scattering_tables.npz` holds `x, f1, f2` and the PCHIP slopes `d1, d2` for the 92 elements as `[92, 449]` float64 (1.7 MB), built by `scripts/gen_scattering_tables.py` from the `.nff` files (which stay the source of truth) with the package's own `_pchip_slopes`. `load_tables()` reads and uploads them once under a lock. They are a program *argument*, never a closed-over constant, so they are not in the HLO or the compile-cache key. `scripts/gen_scattering_tables.py --check` runs in CI; a test pins every row's slopes to SciPy `PchipInterpolator` at 1e-12.
3. **Pure public device API.** `xraylabtool.calculators.optics` exports `optical_properties`, `OpticalProps`, `interpolate_f`, `derive`, `encode_formula` and the standalone primitives `critical_angle_from_delta`, `attenuation_length_from_beta`, `transmission`. `optical_properties` is jit/vmap/grad-compatible (x64 precondition); `d(sum dispersion)/d(rho)` equals `sum dispersion / rho` to 1e-12. `xraylabtool.calculators.tables` exports `ScatteringTables`, `load_tables`. The tables also hold `coef`, per-interval power-basis cubics derived at load time, so a lookup is one `searchsorted` and one gather.
4. **Shape buckets and shared runtime.** `backend/runtime.py` (`ensure_x64`, `enable_persistent_cache`, `worker_guard`, `upload`, `fetch`, `fetch_status`, counters) and `backend/shapes.py` (`bucket`, `bucket_pow2`, `block_for`) are promoted from `kk` and shared; `kk` re-exports them. `bucket_energy` is new in the core (`kk` keeps `bucket`). Energies `M` use `bucket_energy(m)` (powers of two from 64 to 4096, the 1.25 ladder above: a compile costs 100-300 ms, padding a short axis microseconds, so 7 rungs cover 1-4096 energies instead of 21), elements `K` and materials `F` use powers of two. A call uploads one flat float64 buffer (material rows, then energies; one leaf is ~4x cheaper than six on the CPU). Padded rows are `z=0, counts=0, mw=1, rho=1`; padded energies repeat the last real energy. Padding invariance is tested bit-exact. Importing `runtime` or `shapes` has no side effect and `import xraylabtool` does not import JAX.

5. **Batch is `vmap` over materials.** `calculate_xray_properties`, `calculate_multiple_xray_properties`, `data_handling.batch_processing.calculate_batch_properties` and the comparator group materials by `K` bucket and run `F`-blocks through the same program. Output order equals input order (the energy sort and restore are gone: the program is order-independent). `ProcessPoolExecutor`, `ThreadPoolExecutor` and `multiprocessing` are removed from `xraylabtool/` (fork after JAX has started threads can deadlock, and `filterwarnings = error` turns the JAX fork `RuntimeWarning` into failures); an AST test keeps them out. `BatchConfig.max_workers` and the parallel switches are accepted and ignored.
6. **Removed names.** `ops`, `NumpyBackend`, `JaxBackend`, `ArrayBackend`, `InterpolationFactory`, `calculators.kernels` (`backend/array_ops.py` is deleted). `set_backend("jax")` returns `None`; any other name raises `ValueError("the NumPy backend was removed; see ADR-006")`; `get_backend()` returns `"jax"`. `calculate_scattering_factors(..., element_data)` accepts `TableInterpolator` entries only (an arbitrary callable raises `TypeError`). The objects from `create_scattering_factor_interpolators` are `TableInterpolator` shims, not SciPy PCHIP objects: calling one returns a read-only device array (item assignment raises `TypeError`) and `.x`, `.c` and `.derivative()` are gone. `calculate_attenuation_length` and `calculate_transmission` return `inf` and `0.0` for a Python-float `beta = 0` or attenuation length 0, where the NumPy path raised `ZeroDivisionError` (arrays already gave `inf` and `0.0`).
7. **Explicit warm-up.** There is no hidden first-call warm-up. `xraylabtool.warm_up(formulas=("SiO2",))` is public and explicit; the first public call after `clear_scattering_factor_cache()` only loads the resident tables and compiles the program of its own buckets.
8. **CLI.** `calc`, `batch`, `compare`, `kk`, `convert` and `bragg` enable the persistent compile cache (`runtime.enable_persistent_cache()`; `XRAYLABTOOL_NO_JAX_CACHE=1` opts out); `--help`, `--version`, `completion`, `formula`, `atomic` and `list` never import JAX. Per-command subprocess tests enforce both.

Exemptions (host code by design)

Code	Why it stays on the host
`.nff` reading, npz loading, CSV/Parquet/JSON/pandas export, `io/*`	file I/O and containers
`parse_formula` and the `mendeleev` lookup	string parsing, third-party tables
`validation/*` and input checks in `core.py`	the trust boundary, run before upload
`utils.find_peaks`, `progress_bar`, `save_processed_data`	data-dependent output length, tty, file I/O
`analysis/comparator.py` statistics and report text, `interfaces/cli/formatting.py`	reporting on already-downloaded results
GUI widgets, pyqtgraph/Matplotlib plotting, `gui/services.py` glue	view layer, takes NumPy
`XRayResult` and `KKResult` containers	frozen host containers, NumPy fields

Everything else that produces physics output runs in `jax.numpy`.

Scope and what follows

The **core calculators, the resident tables and batch** (PR 1) and the **`utils` physics and signal helpers** (PR 2, `xraylabtool/_physics.py`) are JAX-first. `convert` and `bragg` enable the persistent compile cache like `calc`, and the GUI warms the core programs (and the two conversion programs of the Single tab) on an idle worker (`gui.services.warm_up_core`), with the compute services under `runtime.worker_guard()` and a `QThreadPool` agreement test.

Host by design, stated plainly so that "JAX-first" is not read as "every number is computed on the device":

- spec section 9: `find_peaks`, `progress_bar`, `save_processed_data`, `parse_formula`;
- the scalar helpers of `constants.py` (`critical_angle_degrees`, `attenuation_length_cm`, `energy_to_wavelength_angstrom`, `wavelength_angstrom_to_energy`): plan step 2.2 ruled that they stay host functions (`math` on Python floats). Spec section 10 lists `critical_angle_degrees` among the `jnp` utilities and section 9 does not list these helpers; the discrepancy is recorded here, the plan ruling stands. The device equivalents are `calculators.optics.critical_angle_from_delta` and `attenuation_length_from_beta`;
- the three-point fit of the polynomial background (next paragraph).

A first click on the Single tab's Convert fields before the idle warm-up has imported JAX runs the conversion on the UI thread and pays the `import jax` and the compile (about 1 s) once; afterwards a conversion costs about 125 microseconds.

One deliberate host step inside a utility: the three-point quadratic fit of `background_subtraction(method="polynomial")` is `numpy.polyfit` on the host. The frozen golden contains LAPACK's rounding noise in that fit (the residual at the end points is a few ulp, not zero), and no `jax.numpy` solver reproduces it (exact rational coefficients differ from NumPy's in the last bits too); the Horner evaluation and the subtraction run on the device, with an optimisation barrier so XLA does not fuse the multiply and add into an FMA.

Accepted: single-call latency relative to NumPy

The mandate trades the NumPy row's small-call latency for one implementation. Accepted by Wei on 2026-10-26 (spec C15). A single small call on CPU is slower than the removed NumPy path (frozen Task 0 row: 0.083 ms for SiO2 x200 E): 0.446 ms measured with the device program (median of 21 fresh processes; about 5.4x, wider than the 3x the design spike suggested), and on GPU 0.59 to 0.61 ms (launch latency). The gates P1 (at most 0.5 ms) and P5 (at most 1.5 ms) are met, not waived; they bind, and P1 has 11 % headroom. The memoised per-formula encoding (commit `3a8abcc`) landed during Task 3B as an optimisation for P3 and P6, not as the spec's risk-1 mitigation (P1 did not miss). At 20 000 energies NumPy remains faster on CPU (1.63 ms against 3.56 ms measured, 2.3 to 4.5 ms across processes; gate P2 at most 6 ms). In exchange, batches are about 1.8x faster than the frozen NumPy row on CPU (P3 11.3 ms against 19.8 ms, P4 111 ms against 198 ms) and the same code runs on GPU.

Gate evidence

Authoritative measurement of Task 5B on 2026-10-27, HEAD `bacf45c` plus the `core_gpu_check` commits, machine: Linux laptop, RTX 4090 Laptop GPU (16 GB), JAX 0.12.0.1, x64. The unrelated Julia jobs that loaded the host earlier in the work had finished; every run started after the 1-minute load average had fallen below 2.0 (CPU core rows 0.78, 1.99, 1.79; CLI rows 1.04, 1.05; GPU rows 0.86 to 0.97) with the GPU idle (0 % utilisation, 15 MiB). The load average printed by each script run (start and end) is in `task-5b-report.md`; the end values are higher because the run itself loads the host. CPU rows run with `JAX_PLATFORMS=cpu` (so the "cpu-host" rows use the CPU device even on this CUDA machine); GPU rows use the default device. Every number is the median of fresh interpreter processes (no persistent compile cache), 21 processes for the CPU core rows and for the GPU timings of `core_gpu_check.py`, 14 runs per CLI command. P3 and P4 are relative to the frozen NumPy row recorded in the spec's "Task 0 baselines" (19.8 ms, 198 ms), because the NumPy path was deleted and the row can no longer be re-measured.

#	Gate	Spec value	Measured (median, range over processes)	Status
P1	SiO2 x200 E, CPU	at most 0.5 ms	0.446 ms (0.416 to 1.11)	met, 11 % headroom
P2	SiO2 x20 000 E, CPU	at most 6 ms	3.56 ms (2.31 to 4.46)	met
P3	200 formulas x200 E, CPU	at most 19.8 ms	11.3 ms (9.4 to 17.7)	met
P4	2000 formulas x200 E, CPU	at most 198 ms	111 ms (93.7 to 128)	met
P5	SiO2 x200 E, GPU	at most 1.5 ms	0.607 ms (0.594 to 0.618)	met
P6	200 formulas x200 E, GPU	at most 5 ms	**7.29 ms (6.03 to 9.90)**	**MISS by 2.3 ms (46 %)**, see below
P7	SiO2 x20 000 E, GPU	at most 2 ms	1.70 ms (1.30 to 3.24)	met; thin, single processes reach 3.2 ms
P8	first call incl. compile, CPU	at most 1.0 s	0.289 s (0.279 to 0.307)	met
P9	CLI `calc`, cold / warm persistent cache, CPU host	at most 2.0 s	0.75 s / 0.61 s	met
P9	CLI `convert`, cold / warm, CPU host	at most 2.0 s	0.51 to 0.52 s / 0.47 to 0.48 s	met
P9	CLI `bragg`, cold / warm, CPU host	at most 2.0 s	0.50 to 0.52 s / 0.48 s	met
P9	CLI `calc`, cold / warm, GPU host	at most 3.3 s	1.29 s / 1.01 s	met
P9	CLI `convert`, cold / warm, GPU host	at most 3.3 s	1.02 to 1.05 s / 0.96 to 0.98 s	met
P9	CLI `bragg`, cold / warm, GPU host	at most 3.3 s	1.03 to 1.08 s / 0.97 to 1.00 s	met
P10	`--version`, `--help`, `formula`, `list`, `completion`	at most 0.9 s, no JAX	0.10 to 0.15 s on both hosts; `import xraylabtool` 15 to 16 ms, no JAX import	met
P10	`atomic` (mendeleev import, not JAX)	at most 1.3 s	0.91 s on both hosts (0.90 to 0.95)	met
P11	transfers per device program call	at most 1 upload, 1 fetch	GPU, guard live: single 1/1, from factors 1/1, batch of 200 (one block) 1/1, mixed-K batch (three blocks) 3/3; CPU asserted by the counters in tests	met
P12	compiles after warm-up, same buckets	0	0 (CPU 21 processes, GPU 42)	met
P13	compiles for a new energy count in a bucket	0	0 (200 to 201, 201 to 216; CPU and GPU)	met

The goldens (`rtol=1e-12`) pass on the GPU for all 24 families (worst relative error 2.0e-14, `utils_signal`; the core families stay below 3.6e-15), and the four padding-invariance tests are bit-exact on the GPU.

Also measured: first-call compiles 1 (single material) and 2 (batch); GPU first call 0.8 s (single) and 1.4 s (batch of 200) in a fresh process.

P6 is a miss and is recorded as a deviation, not relaxed. `core_gpu_check.py` (three runs of 7 fresh processes, medians 6.92, 7.45 and 7.33 ms) and `core_bench.py --device gpu` (two runs of 7, medians 6.89 and 6.65 ms) agree on 6.6 to 7.5 ms. The earlier “best of 5.50 ms” was a best single reading; the quiet host does not bring the median under 5 ms. A first version of the script that kept a CUDA context resident in the parent process while the children were timed read 5.3 to 5.5 ms (21 processes, minimum 4.4 ms) and a first-call time 200 ms shorter; with the parent holding the GPU the driver and clocks stay up (the laptop GPU idles at 210 MHz, performance state P8), so that figure does not model a fresh user process and the script now times first. The spread

across processes is wide (6.0 to 9.9 ms), and one warm call is mostly host work around a short device program. No optimisation was applied in this task. The two options on record are views of one per-item copy for the nine result rows (changes the "fresh arrays" semantics of public results) and merging small `K` groups into one block (changes the grouping design); a decision on either, or on relaxing the gate with this evidence, belongs to Wei.

Consequences

- **Cold start grows for compute commands.** `import jax` costs about 0.7 s and the first program compiles; the persistent cache (CLI and GUI by default; the library only with `XRAYLABTOOL_JAX_CACHE=1`) brings `calc` back to the P9 figures above.
- **One implementation, one set of tests.** The goldens (frozen from the NumPy path in Task 0, `rtol=1e-12`, never loosened; worst measured relative error 4.4e-15 for the core families on CPU, 3.6e-15 on GPU, 2.0e-14 for `utils_signal` on GPU) replace the NumPy/JAX parity tests. The one exact-zero floor is the `background_subtraction|polynomial` residual at its three fit nodes (the host `np.polyfit` of R30 rounds differently per LAPACK build, giving 0, -9e-16 or +5e-15): every entry of that array also allows `1e-12` times its largest magnitude (`ZERO_FLOOR_KEYS` in `tests/characterization/core/cases.py`); `rtol` is untouched. `scripts/gen_core_golden.py` and `scripts/gen_kk_prejax_baseline.py` need the deleted NumPy path; each carries a note naming the commit to check out.
- **Break for downstream code.** `set_backend("numpy")`, `ops`, `NumpyBackend`, `JaxBackend`, `InterpolationFactory` and `calculators.kernels` are gone and raise or fail to import. The changelog marks it; released as 0.5.0.
- **Benchmarks.** `scripts/core_bench.py` (CPU) and `scripts/core_gpu_check.py` (manual, CUDA) are standalone gates, not collected by pytest. The Task 0 "cpu-host" rows ran on the CPU through the NumPy backend and are not like-for-like with the later row that pins JAX to the CPU (`JAX_PLATFORMS=cpu`).

Amendments to the design spec, recorded here

- `_status` (the device-side `ok` flag) also requires `isfinite(SCATTERING_FACTOR * rho / mw)`, which keeps the legacy failure for tiny molecular weights. `derive` and the programs compute `((wavelength^2 * (SCATTERING_FACTOR * rho)) * f_sum) / mw` (the molecular weight divides last, R25), so no subnormal intermediate is formed.
- `calculate_derived_quantities` now accepts plain lists (the NumPy-era base raised `TypeError`).
- Standalone `calculate_scattering_factors` with a tiny molecular weight returns the finite value (the NumPy-era base returned `inf`).
- P3 and P4 bind against the frozen NumPy numbers of the spec's Task 0 baselines (19.8 ms, 198 ms).
- The spec's "cpu-host" definition is the CPU device: CPU rows run with `JAX_PLATFORMS=cpu`, so a CUDA machine's cpu-host row is the CPU device.

Known limitations

- `TableInterpolator.__call__` does not use the shape buckets: every distinct host query length compiles one small program (the only host path that does), so a loop over many different lengths grows the compile cache. The shim exists for the legacy `create_scattering_factor_interpolators` API; the calculators never call it. Pad the query to a fixed length to avoid the growth.
- The `kk` entry points need `jax_enable_x64` in the calling context. Inside an explicit `jax.enable_x64(False)` block they raise `TypeError` (and warn about a float32 truncation); the core wraps its programs in `enable_x64(True)` and is not affected. Never silent float32.
- A molecular weight above about 4.5e307 g/mol (for example `"H1" + "0" * 308`) gives `delta = beta = 0` with an accepted status: XLA rewrites `x / m` as `x * (1 / m)` and `1 / m` is flushed as a subnormal. It is physically unreachable and parked; a status guard (fail when `1 / mw` is subnormal) is the fix if wanted.

- The molecular weight is divided out last (`((λ²·S·ρ)·f)/M`), so the product `(λ²·S·ρ)·f` overflows for a factor sum of about 1e300 or more with a large density (for example `"H1" + "0" * 305` at `rho = 1e4`); the status check then raises the non-finite `ValueError` rather than returning a zero. Non-physical and accepted; the `f / M`-first order underflowed at the opposite extreme.
- A single material whose own padded `K x M` exceeds the block budget (for example 9 distinct elements at roughly 300k energies, or 65 at 40k) runs as one `F = 1` block, up to about 1.3x over the planning bound. The budget is a conservative bound on `F`; splitting one material would need a second program call per public call, which the transfer contract (one upload and one fetch per call) forbids.
- A device exception (for example out-of-memory) aborts the whole `calculate_xray_properties` or batch call. Skip-and-log is per formula only (spec section 6); the `F`-block budget makes device failures unlikely.
- `calculate_scattering_factors(..., element_data)` no longer accepts arbitrary callables.

3.12 Contributing Guide

Contributions welcome: code, documentation, scientific validation, and community support.

3.12.1 Development Setup

Prerequisites: Python 3.12+, Git, uv

```
git clone https://github.com/imewei/pyXRayLabTool.git
cd pyXRayLabTool
uv sync
uv run pre-commit install
uv run pytest tests/ -v
```

3.12.2 Development Workflow

Branch naming: feature/description, fix/description, docs/description

```
git checkout main && git pull origin main
git checkout -b feature/my-new-feature
# Make changes, add tests, update docs
uv run ruff format .
uv run ruff check .
uv run mypy xraylabtool
uv run pytest tests/ -v --cov=xraylabtool
make verify    # lint, type check, unit and characterization tests in one step
git add . && git commit -m "feat: descriptive message"
git push origin feature/my-new-feature
```

`make verify` runs ruff, mypy (advisory), the unit tests and the characterization tests, which is what to check before pushing. `make verify-fast` runs only the lint and type checks (no tests).

Tool versions. ruff is pinned to the version locked in `uv.lock` in three other places: the `rev` in `.pre-commit-config.yaml`, `RUFF_VERSION` in `.github/workflows/ci.yml` and the `ruff==` install in `.github/workflows/security.yml`. When bumping ruff, update all of them together, or a rule that is stable in one version can fail in another. Dependency vulnerabilities are checked with `pip-audit` (`security.yml` and `dependencies.yml`).

3.12.3 Code Standards

Style: PEP 8, 88 char lines, NumPy docstrings, type hints required

```python
def calculate_critical_angle(delta: float) -> tuple[float, float, float]:
    """Calculate critical angle from refractive index decrement.

    Parameters
    ----------
    delta : float
        Real part of refractive index decrement.

    Returns
    -------
    tuple[float, float, float]
        Critical angle in (radians, degrees, milliradians).
    """
    import numpy as np
    theta_rad = np.sqrt(2 * delta)
    return theta_rad, theta_rad * 180 / np.pi, theta_rad * 1000

# Error handling
from xraylabtool.exceptions import FormulaError, EnergyError

def validate_inputs(formula: str, energy: float) -> None:
    if not formula.strip():
        raise FormulaError("Formula cannot be empty")
    if energy <= 0:
        raise EnergyError(f"Energy must be positive, got {energy} keV")
```

3.12.4 Testing Guidelines

Structure: tests/unit/, tests/integration/, tests/performance/

```python
# Unit test example
import pytest
from xraylabtool.calculators.core import calculate_single_material_properties
from xraylabtool.exceptions import UnknownElementError

def test_silicon_properties():
    result = calculate_single_material_properties("Si", 8.0, 2.33)  # formula, keV, g/cm³
    assert result.formula == "Si"
    assert abs(result.critical_angle_degrees[0] - 0.2248) < 0.001

def test_invalid_formula():
    with pytest.raises(UnknownElementError):
        calculate_single_material_properties("XYZ", 8.0, 1.0)
```

```
pytest tests/ -v                    # All tests
pytest tests/unit/ -v               # Unit tests
pytest tests/ --cov=xraylabtool     # With coverage
```

3.12.5 Documentation Standards

Docstrings: NumPy style for all public functions

```python
def calculate_properties(formula: str, energy: float) -> dict:
    """Calculate X-ray properties for a material.

    Parameters
    ----------
    formula : str
```

(continues on next page)

(continued from previous page)

```
        Chemical formula (e.g., "SiO2").
    energy : float
        X-ray energy in eV.

    Returns
    -------
    dict
        Properties including critical angle and attenuation.
    """
```

3.12.6 Performance Requirements

- Single calculations: < 0.1 ms
- Batch processing: > 100,000 calculations/second
- Cache efficiency: > 90% hit rate

```
# Benchmark example
import time
start_time = time.time()
for _ in range(1000):
    calculate_single_material_properties("Si", 8.0, 2.33)
avg_time = (time.time() - start_time) / 1000
assert avg_time < 0.01
```

3.12.7 Review Process

Requirements: - [ ] Code follows style guidelines - [ ] Tests included and passing - [ ] Documentation updated - [ ] Performance acceptable

Process: Automated checks → reviewer approval → maintainer approval → squash merge

3.12.8 Contributing Atomic Data

Requirements: Source citation, energy range, precision estimates, HDF5/CSV format

```
atomic_data = {
    'element': 'Si',
    'atomic_number': 14,
    'energies': np.array([...]),     # eV
    'f1_values': np.array([...]),    # Real factors
    'f2_values': np.array([...]),    # Imaginary factors
    'source': 'Henke et al. (1993)'
}
```

3.12.9 Getting Help

Communication: GitHub Issues (bugs/features), GitHub Discussions (questions)

When asking for help: - Include Python/OS versions - Provide minimal reproducible examples - Share error messages and stack traces

Contributors are recognized in AUTHORS.md and release notes.

3.13 Performance and Optimization

3.13.1 Where the time goes

- **Atomic data**: the f1, f2 and PCHIP slopes of all 92 elements are one stacked table (*xraylabtool.calculators.tables*), read from `scattering_tables.npz` and uploaded to the JAX device once per process. It is the only thing the first call has to load.
- **Per-call work**: formula parsing and validation on the host, then one compiled device program (*xraylabtool.calculators.optics*) that interpolates f1/f2 and derives δ, β and the derived quantities, with one upload and one download. Cost scales with the number of energy points, not the number of calls, so pass an energy array instead of looping.
- **Backend**: everything runs in JAX float64 on JAX's default device (the CPU, or the GPU of a CUDA install); there is no NumPy path (ADR-006, *ADR-006: The Core Calculators Are JAX-Only*). A small single call costs a few tenths of a millisecond (launch latency), more than the removed NumPy path did, and on CPU NumPy was also faster at 20 000 energies; batches (and, on a GPU, large energy grids) are where the device program pays off.
- **Compilation**: shapes are padded to buckets (the core energy axis to powers of two from 64 to 4096, then a 1.25 ladder), so a program compiled for one energy count serves every count in the same bucket and a repeat call compiles nothing. The first call compiles; `JAX_PLATFORMS=cpu` selects the CPU device.

Measure before tuning:

```
import time
import numpy as np
import xraylabtool as xrt

energies = np.logspace(0, np.log10(30), 500)
xrt.calculate_single_material_properties("SiO2", energies, 2.2)  # warm-up

start = time.perf_counter()
xrt.calculate_single_material_properties("SiO2", energies, 2.2)
print(f"{(time.perf_counter() - start) * 1e3:.2f} ms for {energies.size} energies")
```

3.13.2 Optimization Strategies

Energy arrays, not loops

```
# Good: one call, vectorised over 100 energies
result = xrt.calculate_single_material_properties("Si", np.linspace(5, 15, 100), 2.33)

# Slow: 100 calls, each re-parsing the formula and re-interpolating
for e in np.linspace(5, 15, 100):
    xrt.calculate_single_material_properties("Si", e, 2.33)
```

Many materials

`xraylabtool.calculate_xray_properties()` runs the materials as blocks through one device program (grouped by element-count bucket, results in input order) and returns `dict[formula, XRayResult]`:

```
results = xrt.calculate_xray_properties(["Si", "SiO2", "Al2O3"], 8.0, [2.33, 2.2, 3.95])
```

For large CSV-driven jobs use *xraylabtool.data_handling.batch_processing.calculate_batch_properties()*, which chunks the input (`BatchConfig.chunk_size`, default 100, also the most materials per device block) and watches memory against `BatchConfig.memory_limit_gb`. There are no worker pools: `BatchConfig.max_workers` and `--workers` are accepted and ignored.

```
from xraylabtool.data_handling.batch_processing import (
    BatchConfig,
    calculate_batch_properties,
```

(continues on next page)

(continued from previous page)

```
    load_batch_input,
    save_batch_results,
)

formulas, densities, _ = load_batch_input("materials.csv")  # formula,density columns
config = BatchConfig(chunk_size=200, enable_progress=True)
results = calculate_batch_properties(formulas, [5.0, 8.0, 10.0], densities, config=config)
save_batch_results(results, "results.csv")
```

The same path is available from the shell: `xraylabtool batch materials.csv -o results.csv`.

Warming up

The first compute call loads the resident tables and compiles the program of its own shape buckets. To pay that cost up front, call `xraylabtool.warm_up` (explicit; nothing warms up behind your back):

```
import xraylabtool as xrt

xrt.warm_up(("SiO2", "Al2O3"))  # compiles the default programs for these element counts
xrt.warm_up(("SiO2",), energy_counts=(1, 500, 2000))  # also the energy buckets 64, 512 and 2048
```

A new energy bucket costs a 100-300 ms compile, so pass the grid sizes you will use as `energy_counts`.

The CLI and GUI also keep JAX's persistent compile cache, so later processes skip the compile (`XRAYLABTOOL_NO_JAX_CACHE=1` opts out). The library does not touch JAX's global configuration unasked; set `XRAYLABTOOL_JAX_CACHE=1` to enable the cache for your own scripts (a warm cache saves about 170 ms on a fresh process's first call). *`xraylabtool.calculators.cache.clear_scattering_factor_cache()`* drops the resident tables and the encoding memo (rarely needed).

Staying on the device

`xraylabtool.calculators.optics.optical_properties()` is the pure device function behind the calculators. It works under `jax.jit`, `jax.vmap` and `jax.grad` and avoids the host round trip entirely when you already hold device arrays; see *JAX Architecture & Design*.

3.13.3 Energy grids

Fewer, well-placed points beat dense uniform grids:

```
# Logarithmic spacing over the tabulated range (keV)
energies = np.logspace(np.log10(0.03), np.log10(30), 100)

# Dense only around an absorption edge (Si K edge at 1.839 keV)
low = np.logspace(np.log10(0.03), np.log10(1.8), 30)
edge = np.linspace(1.80, 1.88, 50)
high = np.logspace(np.log10(1.9), np.log10(30), 30)
energies = np.concatenate([low, edge, high])
```

3.13.4 Troubleshooting

- **First call slow**: it loads the tables and compiles a program. Call `xraylabtool.warm_up()` first, or rely on the persistent compile cache in the CLI and GUI.
- **Small single calls are slower than before**: a few tenths of a millisecond per call is the launch latency of a device program (ADR-006). Pass energy arrays or batches instead of looping.
- **Memory growth in batch jobs**: lower `BatchConfig.chunk_size` or `memory_limit_gb`.

3.14 Testing Guide

Guide to testing XRayLabTool code and ensuring quality.

3.14.1 Testing Philosophy

XRayLabTool follows a comprehensive testing strategy:

Test Categories: - **Unit Tests**: Test individual functions and classes - **Integration Tests**: Test complete workflows and CLI commands - **Performance Tests**: Ensure performance requirements are met - **Characterization Tests**: 202 golden-value assertions for migration safety - **Physics Tests**: Validate scientific accuracy

Testing Principles: - Quick feedback: Most tests run in milliseconds - Measured coverage: reported to Codecov on every CI run (see *Test Coverage*) - Reliable: Tests pass consistently across platforms - Clear failures: Descriptive error messages

3.14.2 Test Organization

Directory Structure

```
tests/
├── conftest.py                # Pytest configuration and fixtures
├── test_code_quality.py       # Code quality checks (naming, imports, types)
├── unit/                      # Unit tests
│   ├── test_core.py           # Core calculation tests
│   ├── test_utils.py          # Utility function tests
│   ├── test_backend_dispatch.py # Backend abstraction tests
│   └── ...
├── integration/               # Integration tests
│   ├── test_integration.py    # End-to-end workflow tests
│   └── test_completion_installer.py
├── characterization/          # Golden-value migration safety tests
│   ├── test_golden_constants.py
│   ├── test_golden_interpolation.py
│   ├── test_golden_molecular.py
│   ├── test_golden_pipeline.py
│   └── ...                    # 202 assertions total
└── performance/               # Performance regression tests
    ├── test_performance_benchmarks.py
    ├── test_memory_management.py
    └── ...
```

3.14.3 Running Tests

Basic Test Execution

```
# Run all tests (using uv)
uv run pytest tests/ -v

# Run specific test categories
uv run pytest tests/unit/ -v              # Unit tests only
uv run pytest tests/integration/ -v       # Integration tests only
uv run pytest tests/characterization/ -v  # Golden-value tests
uv run pytest tests/performance/ -v       # Performance tests only

# Run with coverage
uv run pytest tests/ --cov=xraylabtool --cov-report=html

# Run tests matching pattern
```

(continues on next page)

(continued from previous page)

```
uv run pytest tests/ -k "test_silicon" -v

# Or use Makefile shortcuts
make test          # Tests with coverage
make test-all      # Full suite
```

3.14.4 Writing Tests

Unit Test Example

```
import pytest
from xraylabtool.calculators.core import calculate_single_material_properties
from xraylabtool.exceptions import EnergyError, UnknownElementError

class TestSingleMaterialCalculations:
    """Test single material property calculations."""

    def test_silicon_properties(self):
        """Test silicon properties at 8 keV."""
        result = calculate_single_material_properties("Si", 8.0, 2.33)

        assert result.formula == "Si"
        assert result.density_g_cm3 == 2.33
        assert result.energy_kev[0] == 8.0
        assert abs(result.critical_angle_degrees[0] - 0.2248) < 0.001

    def test_invalid_formula(self):
        """Test error handling for invalid formulas."""
        with pytest.raises(UnknownElementError, match="Unknown element"):
            calculate_single_material_properties("XYZ", 8.0, 1.0)

    @pytest.mark.parametrize("energy", [0, -1.0])
    def test_invalid_energy(self, energy):
        """Test error handling for invalid energies."""
        with pytest.raises(EnergyError):
            calculate_single_material_properties("Si", energy, 2.33)
```

Integration Test Example

```
import subprocess
import json

def test_cli_calc_command():
    """Test the calc CLI command."""
    result = subprocess.run([
        "xraylabtool", "calc", "Si",
        "-e", "8.0",
        "-d", "2.33",
        "--format", "json"
    ], capture_output=True, text=True)

    assert result.returncode == 0
    data = json.loads(result.stdout)
    assert data["formula"] == "Si"
    assert abs(data["critical_angle_degrees"][0] - 0.2248) < 0.001
```

Performance Test Example

```
import time
import pytest

def test_batch_processing_performance():
    """Test that batch processing meets performance requirements."""
    formulas = ["Si", "SiO2", "Al2O3"] * 100
    densities = [2.33, 2.20, 3.95] * 100

    start_time = time.time()
    results = calculate_xray_properties(formulas, 8.0, densities)
    end_time = time.time()

    assert (end_time - start_time) < 5.0
    assert set(results) == {"Si", "SiO2", "Al2O3"}  # keyed by formula
```

3.14.5 Test Configuration

Pytest Configuration

The `conftest.py` file contains shared test configuration:

```
import pytest
import numpy as np

@pytest.fixture
def sample_materials():
    """Common test materials."""
    return [
        {"formula": "Si", "density": 2.33},
        {"formula": "SiO2", "density": 2.20},
        {"formula": "Al2O3", "density": 3.95}
    ]

@pytest.fixture
def energy_range():
    """Common energy range for testing."""
    return np.logspace(0, np.log10(30), 10)  # 1 keV to 30 keV
```

Test Utilities

The `fixtures/` directory contains helper functions:

```
def assert_result_valid(result):
    """Assert that an XRayResult is valid."""
    assert result.formula is not None
    assert (result.energy_kev > 0).all()
    assert (result.critical_angle_degrees > 0).all()
    assert (result.attenuation_length_cm > 0).all()

def create_test_material(formula="Si", energy_kev=8.0, density=2.33):
    """Create a test material for consistent testing."""
    return calculate_single_material_properties(formula, energy_kev, density)
```

3.14.6 Performance Testing

Performance Requirements

Tests ensure performance standards:

- **Single calculations**: a few ms with a warm atomic-data cache
- **Batch processing**: blocks of materials run through one JAX device program (no worker pools)
- **Memory usage**: Reasonable scaling with dataset size

Benchmarking Code

```
import time
from xraylabtool.calculators.core import calculate_single_material_properties

def benchmark_single_calculation():
    """Benchmark single material calculation."""
    n_iterations = 1000

    start_time = time.time()
    for _ in range(n_iterations):
        calculate_single_material_properties("Si", 8.0, 2.33)
    end_time = time.time()

    avg_time = (end_time - start_time) / n_iterations
    assert avg_time < 0.01  # a few ms with a warm cache
```

3.14.7 Test Coverage

Coverage Requirements

- **CI gate**: the full test suite must reach at least 22% (`--cov-fail-under` in `.github/workflows/ci.yml`). The floor is deliberately low because CI attributes far less coverage to the largest module (the CLI) than a local run does.
- **Measured coverage**: about 62% of lines on Codecov as of 2026-10-23. Local runs report a different figure, because coverage config omits `__init__.py` files and some specialized modules.
- **Patch coverage**: Codecov's `codecov/patch` check compares the coverage of the lines a commit changes against the project's coverage. Every touched executable line counts, including a line where only a trailing comment was removed, so changes to code that CI cannot reach need a test (for example, the GPU path of the JAX device programs; CI has no GPU, so it is checked manually with `scripts/core_gpu_check.py` and everything else runs on the CPU).
- **Exception paths**: error conditions should have tests.

Generating Coverage Reports

```
# Generate HTML coverage report
pytest tests/ --cov=xraylabtool --cov-report=html

# Generate terminal coverage report
pytest tests/ --cov=xraylabtool --cov-report=term-missing

# Coverage with branch checking
pytest tests/ --cov=xraylabtool --cov-branch
```

3.14.8 Continuous Integration

Tests run automatically on:

- **Pushes and pull requests** to `main` and `develop` (changes that only touch Markdown or `docs/` skip the test workflow)
- **Manual dispatch** of the Continuous Integration workflow

Security Scanning also runs on pushes and pull requests, and weekly on Mondays at 06:00 UTC. Dependency Management (dependency audit and update check) runs weekly on Mondays at 09:00 UTC and on manual dispatch.

Test Matrix

Tests run across multiple configurations:

- **Python versions**: 3.12, 3.13 and 3.14
- **Operating systems**: Ubuntu, macOS, Windows
- **Toolchain**: ruff (lint + format), mypy (type checking, advisory in CI), pytest (testing), pip-audit (dependency vulnerabilities)
- **CI**: GitHub Actions with SHA-pinned action versions

3.14.9 Contributing Tests

When contributing code:

1. **Write tests first** (TDD approach)
2. **Ensure all tests pass** before submitting
3. **Cover the lines you change**: Codecov's patch check expects changed lines to be covered at least as well as the project
4. **Add performance tests** for new features
5. **Include integration tests** for CLI changes

Test Guidelines

Good Test Practices: - Test one thing per test function - Use descriptive test names - Include both positive and negative test cases - Use appropriate assertions - Mock external dependencies

Test Naming Convention: - `test_function_behavior_condition()` - Example: `test_calculate_properties_invalid_formula()`

For more testing examples and patterns, see the existing test suite in the `tests/` directory.

3.15 Changelog

Changes to XRayLabTool are documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

3.15.1 Version Categories

- **Added**: New features
- **Changed**: Changes in existing functionality
- **Deprecated**: Soon-to-be removed features
- **Removed**: Now removed features
- **Fixed**: Bug fixes
- **Security**: Vulnerability fixes

3.15.2 v0.5.0 (2026-10-28)

Added

- `benchmarks/perf_benchmarks.py --stat mean` averages the per-round medians (with their standard deviation) instead of taking the best round; `benchmarks/RESULTS_cpu.md` and `RESULTS_gpu.md` are the v0.4.8 vs v0.5.0 means of 10 rounds.
- `xraylabtool.kk`: Kramers-Kronig transform of the imaginary scattering factor f2 to f1 by the piecewise Laurent polynomial method of Watts (2014), a JAX port of kkcalc's `KK_PP` with its tabulated f2 data (10 eV-500 keV, vendored as `data/kkcalc/asf.npz`), optional merging of measured near-edge spectra (`load_nearedge`, `to_asf_scale`, `merge_spectra`), adaptive refinement (`refine_spectrum`) and `kk_calculate_real` returning `KKResult` (with `to_xray_result`). Lazily imported; `import xraylabtool` does not load it.
- `xraylabtool kk FORMULA` CLI subcommand (`--nearedge`, `--input-type`, `--energy-unit`, `--merge`, `--density`, `--refine`, `--energies`, `-o`, `--format`) with shell completion.
- GUI "Kramers-Kronig" tab: near-edge file, merge points, refinement, plot of measured/spliced f2, KK f1 and CXRO f1, CSV export.
- `NearEdgeDataError` (a `ValidationError`), also exported from `xraylabtool`.
- Docs: `physics/kramers_kronig` (equations, the Eq. (10) a_{-3} sign erratum, accuracy, 30 keV truncation), `api/kk` and `guides/kk_guide`; `NOTICE` with the kkcalc Zlib attribution.
- `xraylabtool.kk` is JAX-only: one `jax.numpy` implementation, a single device-resident pipeline (one upload, one status fetch after assembly and merge, one per refinement pass, one download) over bucketed shapes, so a default refined run compiles five programs and a second call with the same buckets compiles none. See ADR-005.
- `xraylabtool.kk.Spectrum` (padded piecewise-polynomial f2 table, a JAX pytree) and `xraylabtool.kk.kk_f1`: `jit`/`vmap`/`grad`-compatible device-level f1.
- `xraylabtool.kk.warm_up(formulas=("SiO2",), refine_tolerance=0.01)` precompiles the default programs. The GUI warms the programs of its own Compute click on an idle worker; the CLI never warms up.
- The `xraylabtool kk` command and the GUI launcher enable JAX's persistent compile cache under the OS user-cache directory (`xraylabtool/jax`, compile-time threshold 0) unless `jax_compilation_cache_dir` is already set or `XRAYLABTOOL_NO_JAX_CACHE=1`; the library never enables it.
- `KKResult.refine_exhausted` (default `False`) is `True` when adaptive refinement stopped on `refine_max_passes`, `refine_max_points` or float spacing before every interval met the tolerance; the `kk` CLI prints a warning then.
- `scripts/kk_gpu_check.py`: manual GPU gate (oracle, additivity, transfer budget with the guard live, padding invariance, f2 at the Si edge, benchmark) for the CUDA machine.
- `docs/architecture/005-adr-kk-jax-only.md` (with the section "Known deviations from the acceptance gates": CPU refined run 10-20 % slower than the forced-JAX baseline, GPU small-m overhead of 2-4 ms, bare-library GPU cold start 3.7-4.4 s); `jax_architecture` and the kk guide describe the device pipeline.
- `xraylabtool.calculators.optics`: the pure `jax.numpy` device programs behind the calculators, `optical_properties` and `OpticalProps` (`jit`/`vmap`/`grad`-compatible, x64 precondition), `interpolate_f`, `derive`, `encode_formula`, and the standalone primitives `critical_angle_from_delta`, `attenuation_length_from_beta` and `transmission`.
- `xraylabtool.calculators.tables` (`ScatteringTables`, `load_tables`) and `xraylabtool/data/scattering_tables.npz`: the f1, f2 and PCHIP slopes of the 92 elements as one `[92, 449]` float64 table, uploaded to the device once and passed to the programs as an argument. `scripts/gen_scattering_tables.py` builds it from the `.nff` files and `--check` (run in the CI test job) fails on drift.
- `xraylabtool.backend.runtime` (`ensure_x64`, `enable_persistent_cache`, `worker_guard`, counted `upload`/`fetch`/`fetch_status`) and `xraylabtool.backend.shapes` (`bucket`, `bucket_pow2`, `block_for`), promoted from `kk` (which re-exports them); importing either has no side effect.
- `xraylabtool.warm_up(formulas=("SiO2",))`: public, explicit precompile of the default core programs (nothing warms up implicitly).

- `scripts/core_gpu_check.py` and `make core-gpu-check`: the manual CUDA gate (goldens, P5 to P7, P11 to P13, a verdict per gate; refuses without a CUDA device).
- `scripts/core_bench.py` (CPU gates P1 to P4, P8 to P10, P12, P13) and the frozen golden outputs under `tests/characterization/core/` (generated from the NumPy path before it was deleted; `rtol=1e-12`).
- `docs/architecture/006-adr-core-jax-only.md` (supersedes ADR-001; the gate table was measured on a quiet host; GPU gate P6 is recorded there as a miss) and an updated ADR-004.
- `xraylabtool/_physics.py`: `energy_to_wavelength`, `wavelength_to_energy`, `bragg_angle`, `d_spacing_*`, `q_from_angle`, `angle_from_q`, `smooth_data`, `background_subtraction` and `normalize_intensity` are pure `jax.numpy` programs behind the validating `utils` wrappers (same messages and Python float / NumPy array returns, edge-case departures listed under Changed; signal series are padded to shape buckets so lengths in one bucket share a compile). `find_peaks`, `progress_bar`, `save_processed_data` and `parse_formula` stay host code.
- `xraylabtool convert` and `bragg` enable the persistent compile cache like `calc` (`XRAYLABTOOL_NO_JAX_CACHE=1` opts out); with a warm cache they take about 0.5 s on the CPU host and 1.0 s on the GPU host. `--help`, `--version`, `formula`, `atomic`, `list` and `completion` still import no JAX.
- GUI: the launcher warms the core programs (`gui.services.warm_up_core`) on an idle worker next to the Kramers-Kronig warm-up; the Compute services run under `runtime.worker_guard()`.
- `xraylabtool.calculators` re-exports `OpticalProps`, `ScatteringTables`, `encode_formula`, `load_tables` and `optical_properties` lazily (importing the package still does not load JAX).

Changed

- `utils` signal and conversion helpers (`xraylabtool/_physics.py`) keep their results but differ in a few edge cases from the NumPy implementation: float32 input is computed in float64 (results are float64; for `energy_to_wavelength(np.float32(10))` this corrects the float32 rounding, 1.23984170 to 1.23984172); `smooth_data` accepts one-dimensional data only and raises `ValueError("Signal data must be one-dimensional")` otherwise (it raised `TypeError` for an unsized input and returned a meaningless result for N-D); `normalize_intensity(..., "area")` on other than 1-D data raises that `ValueError` (was `TypeError`); `background_subtraction(..., "polynomial")` with `x` and `y` of different length raises `ValueError("x and y must have the same length")` (was a NumPy broadcast `ValueError` or `IndexError`); `energy_to_wavelength([10.0])` still raises `TypeError`, with NumPy's conversion message. Complex input is rejected rather than cast: a complex scalar or array passed to a conversion, Bragg, d-spacing or q helper raises `TypeError` (a Python complex scalar already raised it for `energy_to_wavelength`, `wavelength_to_energy` and the cubic `d_spacing`; in the tetragonal and orthorhombic `d_spacing` and the `q_from_angle` wavelength argument, and for NumPy complex scalars and arrays, the imaginary part used to be dropped with a `ComplexWarning`), and a complex signal array raises `TypeError("complex signal data is not supported")` (the signal helpers used to compute complex results); no `ComplexWarning` is emitted. `smooth_data` keeps the old window validation (integers only; `2.9`, `3.0` and `True` raise `TypeError`; a window of at least `len(y)` returns the mean of any numeric type). `normalize_intensity "standard"` and the mean returned by `smooth_data` (a window of at least `len(y)`) sum offsets from the first sample, so a constant series is exactly constant (standardising it gives NaN, as NumPy) and data with a large offset keep their spread; windowed smoothing sums the raw values as before. Window types: `np.uint8` (and other unsigned NumPy integers) is now accepted, and a 1-element array such as `np.array([3])` now raises `TypeError`; the multi-column linear background runs as one device call. Further edge cases: `background_subtraction(..., "linear")` on data of three or more dimensions raises `ValueError("Signal data must be one-dimensional")` (it computed an N-D result); `linear` on a 0-d signal and on a `(0, k)` array, and `normalize_intensity(..., "area")` on a 0-d signal, raise that `ValueError` (they raised `IndexError`); `polynomial` on an empty `y` raises `IndexError` (it raised `ValueError`); `polynomial` on N-D `y` raises that `ValueError` (it raised a broadcast `ValueError` for 2-D and `TypeError` for 3-D); `wavelength_to_energy([...])` raises `TypeError` with NumPy's conversion message (like `energy_to_wavelength`). Unchanged: `normalize_intensity "max"`/`"standard"` on any shape and 0-d, and `background_subtraction(..., "linear")` on an `(N, k)` array (column-wise).
- The JAX backend's PCHIP interpolation is an in-tree `jax.numpy` implementation, `backend.interpolation.JaxPchipInterpolator` (SciPy's Fritsch-Carlson slopes, jit-compiled evaluation, NaN outside the table), replacing `interpax`; it matches SciPy's `PchipInterpolator` to float64 round-off.

- Licence expression is `MIT AND Zlib` in `pyproject.toml` and `conda-recipe/meta.yaml`; `LICENSE-kkcalc.txt` ships in the sdist and wheel.
- `calculators.core.calculate_properties_from_factors(formula, energy_kev, density, f1_total, f2_total)`: new public helper computing δ, β and the derived quantities from given f1/f2 totals; the lookup path shares its code.
- GUI: the main window opens maximized.
- `kk_calculate_real(chunk_size=...)`: `bool`, `float` and `str` now raise `ValueError` (a positive integer, including NumPy integers, is required; it is an upper bound on the energies per block).
- `refine_spectrum` validates the spectrum table and `chunk_size` up front; one-point grids and sliver-only passes no longer skip validation.
- Near-edge conversion overflow now surfaces after the merge-point validation (precedence deviation from the pre-JAX pipeline: same exception type, different message; also when the data do not overlap the table).
- `set_backend()` has no effect on `xraylabtool.kk`; `KKResult.backend` is always `"jax"` from `kk` (the legacy value `"numpy"` is still accepted when constructing a `KKResult`).
- NumPy integer values are accepted for the point and chunk limits (`refine_max_passes`, `refine_max_points`, `chunk_size`).
- Minimum JAX is `>=0.12.0.1` (`pyproject.toml`, `uv.lock`): the golden tolerances were measured on it, and releases 0.8.0 to 0.12.0 differ in the last digits of fused XLA expressions.
- `tests/test_code_quality.py::test_naming_conventions` ignores physical unit symbols (`eV`, `keV`, …) in its camelCase check; the threshold is unchanged.
- `docs/api/kk.rst` no longer documents `MergeStatus`, `MergeResult` and `DeviceTables` fields twice (duplicate-object warnings).
- **Breaking:** the core calculators are one JAX float64 implementation with no NumPy twin (ADR-006). A public call does host validation, one upload, one compiled program and one download; shapes are padded to buckets so a repeat call compiles nothing. Results equal the previous NumPy path to `rtol=1e-12`. A small single call on CPU is slower than before (about 0.45 ms against 0.08 ms; GPU about 0.6 ms), batches of 200 and 2000 formulas are faster on CPU (about 1.8x against the frozen NumPy-era row, measured on a quiet host; see ADR-006).
- `calculate_xray_properties`, `calculate_multiple_xray_properties`, `calculate_batch_properties` and the comparator run materials as blocks through one device program (a `vmap` over materials). `calculate_xray_properties` no longer sorts and restores the energy order: the output follows the input order. `BatchConfig.max_workers` and the CLI `--workers` are accepted and ignored.
- `set_backend("jax")` returns `None`; any other name raises `ValueError("the NumPy backend was removed; see ADR-006")`; `get_backend()` returns `"jax"`.
- `import xraylabtool` no longer sets `JAX_ENABLE_X64` or imports JAX; `runtime.ensure_x64()` runs at every compute entry. `--help`, `--version`, `completion`, `formula`, `atomic` and `list` never import JAX. The `calc`, `batch` and `compare` commands enable the persistent compile cache like `kk` (`XRAYLABTOOL_NO_JAX_CACHE=1` opts out).
- `calculate_scattering_factors(..., element_data)` accepts `TableInterpolator` entries only (anything else raises `TypeError`); with a tiny molecular weight it returns the finite value (it returned `inf`). `calculate_derived_quantities` accepts plain lists (it raised `TypeError`).
- The first public call after `clear_scattering_factor_cache()` only loads the tables; the hidden first-call warm-up is gone (use `warm_up`).
- Known limitation: a molecular weight above about 4.5e307 g/mol gives `delta = beta = 0` with an accepted status (XLA flushes `1 / mw` as a subnormal); a device exception such as out-of-memory aborts the whole batch call, skip-and-log stays per formula.

Removed

- `ensure_float64` from `xraylabtool.backend.array_ops` (its only user was `kk`, which now establishes x64 itself) and the NumPy twin of the kk kernel, assembly, merge and refinement.

- `calculators.derived_quantities.RADIANS_TO_DEGREES` (a re-imported constant; import it from `xraylabtool.constants`).
- `xraylabtool.backend.ops`, `NumpyBackend`, `JaxBackend`, `ArrayBackend`, `InterpolationFactory`, `calculators.kernels` and `backend/array_ops.py`; the SciPy PCHIP is no longer used at runtime. `scripts/gen_core_golden.py` and `scripts/gen_kk_prejax_baseline.py` need the removed path and say which commit to check out.
- `ProcessPoolExecutor`, `ThreadPoolExecutor` and `multiprocessing` from `xraylabtool/` (an AST test keeps them out), and the batch chunk stall timeout that only existed for the executors.

Fixed

- `JaxPchipInterpolator` preserves float64 when x64 is disabled after construction or in a thread-local JAX context, and rejects complex tables instead of discarding their imaginary parts.
- GUI: table headers no longer render uppercase, which turned δ, β and μ into Δ, B and M.
- GUI: the single-material plot now spans the energy-wavelength converter row and, with the multi-material plot, resizes to fit its panel instead of being forced to 720 px; the scrollbar remains for windows too short to fit it.
- GUI: toast notifications are sized after their stylesheet is applied, so the message is no longer clipped ("calculation do…").
- GUI: plot legends use the theme text color on a translucent plot-colored box, stay readable over curves and restyle on a live theme switch; the multi-material results table no longer stretches its last column.
- GUI: plot axis titles use the theme text color instead of pyqtgraph's near-white default, so they stay readable in the Light theme.

3.15.3 v0.4.8 (2026-10-24)

Added

- Short console-script aliases `xlt` (for `xraylabtool`) and `xlt-gui` (for `xraylabtool-gui`); shell completion registers `xlt` alongside `xtool` for bash, zsh, fish, and PowerShell.
- `conda-recipe/meta.yaml`: noarch conda recipe. The package is published on the `imewei` channel (`conda install -c imewei -c conda-forge xraylabtool`) with the core runtime dependencies and CPU-only JAX; the CUDA extras are pip/uv only.
- `tests/unit/test_completion_aliases.py`: checks that every shell completion generator registers both CLI aliases.

Changed

- The README and the index, getting-started and completion guides document the conda installation.
- `check-yaml` excludes the Jinja-templated `conda-recipe/meta.yaml`.

3.15.4 v0.4.7 (2026-10-23)

Added

- `tests/unit/test_deep_debug_regressions.py`: regression tests for each defect fixed from the deep-debugging review (lazy imports, energy contract, anomalous dispersion, cache immutability, data integrity for all `.nff` files, exports, backend state, batch ordering/timeouts/fallback, parser, validators, repeated materials, pandas loader, GUI error slots and worker, utility validation).
- Python 3.12 in the CI test matrix (Ubuntu, macOS, Windows); the 3.12 suite passes (740 tests) against the locked dependencies.
- `tests/unit/test_jax_backend_parity.py`: JAX-vs-NumPy parity tests (SiO2, H2O, Fe2O3; `rtol=1e-12`), an `InterpolationFactory.create_pchip` check against `interpax`, and tests for the `float()` fallback in the two scalar-energy converters.

- scripts/check_version.py (checks that the nine files recording the version agree; --expect, --installed, --print), scripts/test_install.py (installs a wheel or a PyPI/TestPyPI release into a fresh venv and runs one calculation) and scripts/gpu_check.py (JAX backend, devices and CUDA plugin conflicts); the check-version pre-commit hook; make test-characterization and make test-gui.

Changed

- jax/jaxlib no longer have an upper bound (>=0.8.0); the <0.11 ceiling added in 0.4.5 is removed from pyproject.toml and from the cuda12-local/cuda13-local Makefile targets. The suite and a JAX-vs-NumPy comparison pass on 0.12.0.1 (maximum relative difference 2.5e-16), and uv.lock moves jax/jaxlib from 0.10.2 to 0.12.0.1, matching the locked CUDA plugin packages.
- ruff is pinned to 0.17.2 in .pre-commit-config.yaml, ci.yml and security.yml, matching uv.lock. PLR0917 (stable in ruff 0.17) is ignored alongside PLR0913 instead of making public signatures keyword-only.
- Refreshed uv.lock dependency versions (including typer 0.27.3).
- Machine-readable output (format_xray_result CSV/JSON, xraylabtool.io) now keeps full precision; precision applies to table output only.
- export_to_csv raises ValueError for results with different numbers of energies, and a scalar field (for example density_g_cm3) has one column named after the field instead of one per energy.
- energy_to_wavelength, wavelength_to_energy and the d_spacing_* functions raise ValueError for non-positive energies, wavelengths and lattice parameters and for Miller indices (0, 0, 0) instead of returning negative or infinite values; parse_energy_string raises ValueError for unknown spacing (only linear, lin and log) and for extra : fields.
- pyproject.toml matches the code: unused dependency groups are gone (API server, scikit-learn/statsmodels, profilers, Jupyter, seaborn/plotly, openpyxl/jinja2, pbr, codecov and others; uv.lock has 185 packages), PySide6/pyqtgraph/matplotlib and psutil are documented as intentional core dependencies, setuptools>=77.0 is the only build requirement, and the ruff/mypy/pre-commit floors equal the CI pins. Only the 9 markers that tests use are registered (--strict-markers stays on), testpaths is tests, mypy checks the package only and replaces the deprecated strict_concatenate with extra_checks, and the ruff per-file-ignores list only codes that fire on the files they match.
- .gitignore rebuilt in sections with anchored directory rules so a source package named cache/ or lib/ cannot be ignored by accident. graphify-out/ tracks GRAPH_REPORT.md, graph.html and graph.json; the other graphify files (manifest, cost, root/interpreter paths) are untracked.
- .pre-commit-config.yaml: pre-commit-hooks v6.0.0, gitleaks v8.30.1, the ruff-check hook id, generated graphify-out/ and docs/_build/ excluded, and the version-consistency hook.
- Makefile: runs against ./.venv without activation; category test targets pass --no-cov and --run-performance where the conftest gate requires it; GUI tests (all four files CI ignores) run serially after the parallel run; build uses uv build plus twine check; version-check, test-install-* and gpu-* call the new scripts; the GPU install targets use uv sync --extra gpu_cuda12|gpu_cuda13; verify also checks formatting and fails on type errors; release-check adds the doctest build and a clean-venv wheel install. make claude enforces a 27% coverage floor (measured 28.7%) instead of 42%.

Removed

- safety dev dependency, its [tool.safety] table and its steps in the security.yml and dependencies.yml workflows. pip-audit is now the only dependency vulnerability gate. The lock also drops the 11 packages only safety required (nltk, authlib, marshmallow, tenacity, …).
- repomix-output.xml (a 9.6 MB snapshot of the repository from 2026-09-17, now out of date), scripts/create_favicon.py (unreferenced one-off generator) and the root-level .graphify_uncached.txt (a graphify temp file that was committed by mistake and contained another machine's absolute paths).
- The 2026-04 audit reports docs/audit_remediation_plan.md, docs/security_audit_cicd.md and docs/test_coverage_audit.md, which were not part of the documentation build, and their entries in docs/conf.py exclude_patterns.

- Dead ruff `per-file-ignores` for files that no longer exist (`xraylabtool/core/*.py`, `**/interfaces/completion.py`).
- `docs/_static/logo-light.jpg`, a copy of the logo that nothing referenced (the docs theme uses `logo-light.svg` and `logo-dark.svg`).

Fixed

- OSTI GitLab mirror: tags are force-pushed in a separate command, so a re-tagged release (such as `v0.4.6`) no longer fails the job with "tag already exists".
- `mypy xraylabtool/` reports no errors on mypy 2.4.0 and on the CI-pinned 1.20.0 (23 errors fixed: generic `dict` annotations, batch executor typing, a list rebound to an ndarray in `analysis`, stale `type: ignore` comments).
- `test_documented_import_examples` executed prose from `CLAUDE.md` ("so `import xraylabtool` stays fast; …") and always failed; it now matches only import statements and also scans `README.md`.
- `xraylabtool.utils`, `.constants`, `.validation`, `.analysis`, `.io`, `.calculators` and `.data_handling` raised `RecursionError` when first accessed as attributes of the package in a fresh interpreter (`import xraylabtool; xraylabtool.utils`); the lazy loader now uses `importlib`, and the lookups for the nonexistent `cleanup`/`export` modules are gone.
- Energy input: 2-D arrays silently produced wrong delta/beta/f1/f2, 0-d NumPy arrays crashed, complex energies lost their imaginary part (silently in the batch API for scalars), and NaN raised a generic `ValueError`. Energies must now be a scalar or 1-D real array; otherwise `EnergyError`. `XRayResult.energy_kev` is a copy and no longer aliases the caller's array.
- Negative delta near absorption edges (anomalous dispersion) is no longer rejected as "physically unrealistic": 20 elements, including C, Na, Ca, Ag and Pb, failed at the 0.03 keV limit. The critical angle is 0 where delta < 0; absorption and attenuation are unaffected.
- `get_bulk_atomic_data` returned a shared mutable dict, so editing it changed later calculations in the process and `clear_scattering_factor_cache()` did not undo that; the mapping is read-only and the cache clear now includes the inner loader.
- `data/AtomicScatteringFactor/f.nff`: first row energy was `129.3` instead of `29.3`, which corrupted fluorine f1/f2 near 129 eV after sorting.
- Exports: wide CSV shifted values into the wrong columns for unequal energy grids or scalar fields; dictionary CSV split strings into characters and depended on key order; a dict of `XRayResult` was written as its `repr()` in CSV and JSON; CSV/JSON formatters rounded to a fixed number of decimals, turning small beta and SLD values into 0.
- `set_backend()` left backend-specific interpolators cached, so the JAX backend could keep using scipy interpolators (and the reverse) depending on call order.
- Batch API: results were returned in completion order (thread path), the documented per-calculation timeout could never fire, and a process pool that cannot start (for example without semaphores) aborted the batch instead of falling back to threads.
- JAX scatter kernel recompiled for every distinct density/material because `common_factor` was a static argument.
- `parse_formula`: invalid text inside a group (`Si(O-2)`, `Si(O2+)`, `Si()`) was silently dropped, and small fractional counts inside a group (`(H0.00001)O`) failed after expansion.
- Repeated formulas with different densities overwrote each other in `calculate_xray_properties`, `calculate_multiple_xray_properties` and the GUI service; later entries now get a `formula@density` key.
- `validate_chemical_formula` rejected Ac, Fr, Pa, Ra, Th and U although scattering data exists for them.
- `scattering_data.load_data_file` failed on pandas 3 (`delim_whitespace` was removed).
- GUI: both calculation-error slots raised `KeyError` while logging (reserved `message` field), so the error dialog never appeared; `CalculationWorker` crashed on `functools.partial`/builtin callables and mistook a local variable named `progress_cb` for a parameter.

- The `perf-baseline`, `perf-compare` and `perf-report` Makefile targets pointed at `tests/test_integration.py`, which moved to `tests/integration/test_integration.py`, so they collected nothing.
- Makefile targets that could not work: `info` and `test-install-*` held multi-line `python -c` strings that the shell split; `gpu-diagnose` ran a `for` loop after a semicolon (a syntax error hidden by `2>/dev/null`); `install-jax-gpu` read an `nvidia-smi` error message as the compute capability; `type-check*` and `claude` printed "mypy not available" when mypy failed; `claude` ignored ruff failures and its performance phase collected nothing without `--run-performance`; `test-memory`, `test-edge`, `test-ci`, `test-nightly` and `test-smoke` selected markers that no longer exist; `upload` used `read -p` under `sh`; `clean` descended into `.venv`.
- `mypy xraylabtool/ --strict` reports no errors (`MaterialComparator.__init__` annotated; untyped `jax.config.update` covered by `untyped_calls_exclude`).
- Four stale performance tests: `MemoryMonitor` only catches `psutil.Error`, batch results keep repeated formulas under `formula@density` keys, and `get_bulk_atomic_data` returns a read-only mapping.
- Ruff reported RUF001/RUF003/PLC2401 on `docs/conf.py` and the example notebooks when run on the whole repository (as pre-commit does); `docs/**` now ignores those typography rules.
- `scripts/test_docs.py` (`make docs-test-all`) ran Sphinx on the nonexistent `docs/source` and so failed every build step; it now uses `docs/`. The old Makefile guard (`command -v python scripts/test_docs.py`) was always false, which hid this by running the plain `docs-test`/`docs-doctest`/`docs-linkcheck` fallback instead.

Security

- `filelock` 3.32.7 to 4.2.4 (CVE-2026-112940, fixed in 4.2.1; build/dev tooling only). The update was blocked by `safety`, which required `filelock<4`.

Documentation

- API reference now covers the CLI `parser`, `formatting` and `utils` modules and the GUI main-window tab, widget, style and table-formatter modules; the backend classes, `get_backend`, `set_backend` and `InterpolationFactory` have docstrings.
- Testing guide: replaced the unsupported ">95% coverage" targets with the actual CI gate (22%) and the Codecov project/patch behavior, and corrected the CI triggers and Python matrix. Contributing guide: `make verify` and where the ruff version is pinned. Getting started: supported JAX versions.
- Python code blocks in Markdown formatted with ruff 0.17.2.

3.15.5 v0.4.6 (2026-09-17)

Changed

- `xraylabtool.interfaces.cli` is now a package (`parser`, `formatting`, `utils`, one module per command under `commands/`); `main()` and prior public names are re-exported, so the console-script entry point and existing imports are unaffected
- `xraylabtool.gui.main_window` is now a package of per-tab mixins (`single_tab`, `multi_tab`, `shared`, `toast`) combined into `MainWindow`
- Batch calculation error-path warnings now go through the structured logger instead of `print()`
- `psutil` failures in batch processing are caught as `psutil.Error` instead of a bare `Exception`
- Internal refactors (no behavior change): `parse_formula`, `save_calculation_results`, `warm_cache_for_compounds`, `get_compound_family`, `MaterialComparator`, `cmd_compare`, and the Zsh/Fish completion generators split into smaller helpers

Removed

- Legacy `xraylabtool.interfaces.completion` bridge module; import from `xraylabtool.interfaces.completion_v2` instead

- Automated `release.yml` workflow; releases are now bumped/tagged by hand and published via `publish.yml` (see `.github/workflows/README.md`)

Fixed

- CI dependency audit no longer fails on the unpatched `nltk` CVE (PYSEC-2026-3740) pulled in by `safety` itself

Documentation

- Correct energy units (keV), argument order `(formula, energy_keV, density)`, CLI flags, install extras, and removed references to non-existent functions/constants across guides, API reference, notebooks, and developer docs
- CLI reference rewritten from actual `--help` output

3.15.6 v0.4.5 (2026-08-12)

Fixed

- GUI now applies the persisted/default theme immediately on launch instead of staying unstyled until the first theme toggle
- Light theme is now a real palette (`LIGHT_INDUSTRIAL`) tuned for WCAG contrast on white, instead of aliasing the dark palette's neon accents
- GUI layout: consistent control spacing in the converter/presets/energy-range rows, the converter row no longer stretches full-width, and the multi-material table has a minimum height

Changed

- `jax`/`jaxlib` dependency bounds constrained to `<0.11` to avoid untested upstream releases

3.15.7 v0.4.4 (2026-06-25)

Added

- Python 3.14 support: `pyproject.toml` classifier and CI matrix coverage (ubuntu/macOS/windows)

Fixed

- `parse_formula` now rejects trailing/invalid characters (e.g. `SiO2junk`, `Si-O2`) instead of silently ignoring them
- Complex energy inputs are rejected with a clear `EnergyError` rather than being coerced to their real part
- Empty energy arrays now raise `EnergyError` instead of producing empty results
- Batch processing disambiguates colliding result keys so no result is silently dropped
- `calculate_scattering_length_density` reconciled with the pipeline kernel (SLD = $2\pi\cdot\delta/\lambda^2$; real part positive for $\delta > 0$)
- `validate_energy_range` defaults corrected to the supported 0.03–30 keV range
- GUI: density spinbox range aligned with `validate_density` (0.001–30 g/cm³); non-numeric density cells are skipped instead of crashing
- GUI "Save PNG" now exports the plot via pyqtgraph's `ImageExporter` (the PyQtGraph widgets have no matplotlib figure, so the previous `savefig` path always errored)
- `xraylabtool batch` no longer crashes on the default non-progress path, where the progress tracker yielded `None`

Changed

- Removed unused `EnhancedValidator` / `ErrorRecoveryManager` machinery from the batch CLI path
- Migrated class-scoped pytest fixtures to `@classmethod` for pytest 9 compatibility

3.15.8 v0.4.3 (2026-05-08)

Changed

- Upgraded pip to >=26.1 in CI to resolve CVE-2026-6357
- Expanded `.gitignore` with comprehensive uv/venv, CI, and tooling patterns
- Updated Sphinx API docs to cover all public modules (calculators, gui, interfaces, utils)
- Fixed Sphinx build output path in README development commands

Security

- CI: pin `pip>=26.1` in dependency-audit job to prevent CVE-2026-6357 false positives

3.15.9 v0.4.2 (2026-04-30)

Changed

- UI/UX Pro Max upgrade applied to PySide6 interface
- Upgraded form validation to use non-blocking toast notifications instead of blocking message boxes
- Enhanced touch target sizes for all inputs and buttons
- Standardized UI spacing and interactive states (loading buttons, disabled opacities)

3.15.10 v0.4.1 (2026-04-07)

Fixed

- Version bump for patch release

3.15.11 v0.4.0 (2026-04-07)

Added

- Backend abstraction layer (`xraylabtool.backend`) supporting NumPy and JAX backends
- `set_backend("jax")` / `set_backend("numpy")` for runtime backend switching
- `InterpolationFactory` for backend-agnostic PCHIP interpolation (scipy ↔ interpax)
- PyQtGraph-based interactive plotting widgets (replaces matplotlib in GUI)
- JAX float64 auto-configuration at package import time
- 202 characterization tests with golden-value assertions for migration safety
- Optional `[jax]` dependency group: `pip install xraylabtool[jax]`
- Optional `[plots]` dependency group for matplotlib (publication plots)
- Architecture Decision Records in `docs/architecture/`
- Security scanning workflow and license compliance auditing

Changed

- All core computation modules now use backend ops instead of direct NumPy
- GUI plotting migrated from matplotlib to PyQtGraph for interactive performance
- `ColorPalette.mpl_cycle` renamed to `ColorPalette.plot_cycle`
- `apply_matplotlib_theme()` replaced with `apply_pyqtgraph_theme()`
- `scipy.constants` eliminated from utils.py (uses constants.py directly)
- matplotlib moved from required to optional dependency
- ReadTheDocs migrated to uv-based builds on Ubuntu 24.04 / Python 3.13
- CI updated to Python 3.13, latest GitHub Actions with SHA-pinned versions

Deprecated

- `optimization/vectorized_core.py` manual SIMD heuristics (use JAX backend instead)
- `optimization/optimized_core.py` monkey-patching (use `set_backend('jax')`)

Removed

- `ScalarFriendlyArray` numpy subclass (replaced with plain numpy arrays)
- Direct matplotlib dependency (now optional via `[plots]` extra)
- Direct scipy.constants usage in computation path

3.15.12 v0.3.0 (2025-12-15)

Added

- GUI modernization: dark mode toggle with persistent preferences
- New theme engine supporting light and dark themes

Security

- Replaced python-jose with PyJWT to address Minerva attack vulnerability

Fixed

- Plot clipping issues with scrollbars
- Single-point plot visibility in energy sweeps
- Log path toggle contrast
- GUI smoke test made non-blocking for CI reliability

Maintenance

- Removed obsolete workflows (security.yml, performance-monitoring.yml, dependabot.yml)
- Simplified lint job to use ruff only (removed isort/black)
- Updated tool versions in CI
- Added explicit permissions to all GitHub Actions workflows

3.15.13 v0.2.x (2025)

See the full history in CHANGELOG.md for detailed v0.2.0 through v0.2.7 release notes covering:

- Enhanced CLI with shell completion support
- Batch processing from CSV files
- Performance benchmarking suite (150,000+ calculations/sec)
- Cross-platform shell completion (bash, zsh, fish, PowerShell)
- Mamba support and environment isolation

3.15.14 v0.1.0 (2024)

Added

- Core X-ray optical properties calculations
- Support for single materials and compounds
- Complex refractive index calculations (delta, beta)
- Critical angle and attenuation length calculations
- Chemical formula parsing
- Energy-wavelength conversions

- Basic command-line interface
- Atomic scattering factor data for 92 elements

3.15.15 Compatibility Matrix

Table 5: Platform and Python Version Support

Version	Python 3.12	Python 3.13	Windows	macOS/Linux
v0.4.1	✔	✔	✔	✔
v0.4.0	✔	✔	✔	✔
v0.3.0	✔	✔	✔	✔
v0.2.x	✔	✔	✔	✔
v0.1.0	✔	✘	✔	✔

CHAPTER

FOUR

PERFORMANCE HIGHLIGHTS

Operation	Speed	Use Case
Single calculation (warm, 500 energies, CPU)	about 0.3 ms	Interactive analysis
Batch processing (200 materials, 100 energies, CPU)	about 7 ms (about 6 ms on a GPU)	High-throughput screening
Energy array (500 points, CPU)	< 1 ms	Spectroscopy analysis

Warm timings from `benchmarks/RESULTS_cpu.md` and `RESULTS_gpu.md` (v0.5.0, mean of 10 rounds); the first call in a process also compiles (see *Performance and Optimization*).

CHAPTER

FIVE

TARGET AUDIENCE

- **Synchrotron Scientists**: Beamline optimization and experimental planning
- **Materials Researchers**: X-ray characterization and property prediction
- **X-ray Optics Developers**: Mirror and multilayer design
- **Students & Educators**: Learning X-ray physics and optics

CHAPTER

SIX

CITATION

If you use XRayLabTool in your research, please cite:

```
@software{xraylabtool,
  title={XRayLabTool: High-Performance X-ray Optical Properties Calculator},
  author={Wei Chen},
  url={https://github.com/imewei/pyXRayLabTool},
  version={0.5.0},
  year={2024--2026}
}
```

CHAPTER

SEVEN

INDICES AND TABLES

- genindex
- modindex
- search

PYTHON MODULE INDEX

X

xraylabtool.analysis, 145
xraylabtool.analysis.comparator, 146
xraylabtool.backend, 163
xraylabtool.backend.interpolation, 167
xraylabtool.backend.runtime, 163
xraylabtool.backend.shapes, 165
xraylabtool.calculators.cache, 66
xraylabtool.calculators.derived_quantities, 57
xraylabtool.calculators.optics, 72
xraylabtool.calculators.scattering_data, 68
xraylabtool.calculators.tables, 70
xraylabtool.constants, 140
xraylabtool.data_handling.atomic_cache, 79
xraylabtool.data_handling.batch_processing, 85
xraylabtool.data_handling.compound_analysis, 82
xraylabtool.device, 121
xraylabtool.gui, 148
xraylabtool.gui.logging_filters, 154
xraylabtool.gui.main_window, 148
xraylabtool.gui.main_window.kk_tab, 149
xraylabtool.gui.main_window.multi_tab, 149
xraylabtool.gui.main_window.shared, 149
xraylabtool.gui.main_window.single_tab, 149
xraylabtool.gui.main_window.toast, 150
xraylabtool.gui.services, 150
xraylabtool.gui.style, 159
xraylabtool.gui.table_formatter, 162
xraylabtool.gui.theme_manager, 153
xraylabtool.gui.widgets, 155
xraylabtool.gui.widgets.kk_plot, 158
xraylabtool.gui.widgets.material_form, 155
xraylabtool.gui.widgets.material_table, 156
xraylabtool.gui.widgets.plot_canvas, 156
xraylabtool.gui.widgets.scrollbar_helper, 159
xraylabtool.gui.widgets.sweep_plots, 157
xraylabtool.gui.workers, 154
xraylabtool.interfaces.cli, 89
xraylabtool.interfaces.cli.commands.atomic, 94
xraylabtool.interfaces.cli.commands.batch, 94
xraylabtool.interfaces.cli.commands.bragg, 95
xraylabtool.interfaces.cli.commands.calc, 93
xraylabtool.interfaces.cli.commands.compare, 94
xraylabtool.interfaces.cli.commands.completion, 95
xraylabtool.interfaces.cli.commands.convert, 94
xraylabtool.interfaces.cli.commands.formula, 94
xraylabtool.interfaces.cli.commands.kk, 94
xraylabtool.interfaces.cli.commands.list_cmd, 95
xraylabtool.interfaces.cli.formatting, 93
xraylabtool.interfaces.cli.parser, 91
xraylabtool.interfaces.cli.utils, 93
xraylabtool.interfaces.completion_v2.cli, 104
xraylabtool.interfaces.completion_v2.environment, 101
xraylabtool.interfaces.completion_v2.installer, 97
xraylabtool.interfaces.completion_v2.integration, 103
xraylabtool.interfaces.completion_v2.shells, 98
xraylabtool.io, 116
xraylabtool.kk, 169
xraylabtool.kk.api, 169
xraylabtool.kk.database, 195
xraylabtool.kk.nearedge, 184
xraylabtool.kk.refine, 192
xraylabtool.kk.result, 172
xraylabtool.kk.spectrum, 181
xraylabtool.kk.transform, 175
xraylabtool.logging_utils, 122
xraylabtool.typing_extensions, 123

INDEX

Non-alphabetical

__init__() (*AsfDatabase method*), 199
__init__() (*AtomicScatteringFactor method*), 69
__init__() (*BatchConfig method*), 85
__init__() (*CalculationWorker method*), 154
__init__() (*ColorPalette method*), 161
__init__() (*ComparisonResult method*), 147
__init__() (*CompletionGenerator method*), 98
__init__() (*CompletionInstaller method*), 97
__init__() (*CompletionManager method*), 100
__init__() (*CrystalStructure method*), 69
__init__() (*DeviceTables method*), 196
__init__() (*EnergyConfig method*), 150
__init__() (*EnvironmentDetector method*), 102
__init__() (*EnvironmentInfo method*), 102
__init__() (*F1F2Plot method*), 157
__init__() (*FastAtomicDataProvider method*), 81
__init__() (*JaxPchipInterpolator method*), 168
__init__() (*KKComputation method*), 151
__init__() (*KKPlot method*), 158
__init__() (*KKResult method*), 174
__init__() (*MainWindow method*), 148
__init__() (*MaterialComparator method*), 145, 147
__init__() (*MaterialInputForm method*), 155
__init__() (*MaterialTable method*), 156
__init__() (*MemoryMonitor method*), 86
__init__() (*MultiF1F2Plot method*), 158
__init__() (*NearEdgeSpectrum method*), 185
__init__() (*OpticalProps method*), 74
__init__() (*OverlayScrollbarMarginHelper method*), 159
__init__() (*PlotCanvas method*), 156
__init__() (*RefineOutcome method*), 193
__init__() (*ScatteringData method*), 68
__init__() (*ScatteringTables method*), 71, 75
__init__() (*Spectrum method*), 181
__init__() (*TableInterpolator method*), 66
__init__() (*ThemeManager method*), 153
__init__() (*Toast method*), 148, 150
__post_init__() (*BatchConfig method*), 85

A

a_high (*MergeStatus attribute*), 189
a_low (*MergeStatus attribute*), 189
absorption (*OpticalProps attribute*), 74
accent (*ColorPalette attribute*), 160
accent_hover (*ColorPalette attribute*), 160
accent_text (*ColorPalette attribute*), 160
add_atom() (*CrystalStructure method*), 70
add_atomic_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_batch_command() (*in module xraylabtool.interfaces.cli.parser*), 91
add_bragg_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_calc_command() (*in module xraylabtool.interfaces.cli.parser*), 91
add_compare_command() (*in module xraylabtool.interfaces.cli.parser*), 93
add_completion_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_convert_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_formula_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_install_completion_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_kk_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_list_command() (*in module xraylabtool.interfaces.cli.parser*), 92
add_material() (*MaterialTable method*), 156
add_uninstall_completion_command() (*in module xraylabtool.interfaces.cli.parser*), 93
analyze_element_associations() (*in module xraylabtool.data_handling.compound_analysis*), 84
angle_from_q() (*in module xraylabtool.utils*), 137
ANGSTROM_TO_METER (*in module xraylabtool.constants*), 140
apply() (*ThemeManager method*), 153
apply_margins() (*OverlayScrollbarMarginHelper method*), 159
apply_palette_to_item() (*in module xraylabtool.gui.widgets*), 155
apply_palette_to_widget() (*in module xraylabtool.gui.widgets*), 155
apply_pyqtgraph_theme() (*in module xraylabtool.gui.style*), 162
apply_styles() (*in module xraylabtool.gui.style*), 162
apply_theme() (*in module xraylabtool.gui.style*), 162
AsfDatabase (*class in xraylabtool.kk.database*), 196
AtomicDataProvider (*class in xraylabtool.typing_*

extensions), 123
AtomicScatteringFactor (*class in xraylabtool.calculators.scattering_data*), 69
attenuation_length (*OpticalProps attribute*), 73
attenuation_length_cm() (*in module xraylabtool.constants*), 142
attenuation_length_from_beta() (*in module xraylabtool.calculators.optics*), 76
AVOGADRO (*in module xraylabtool.constants*), 140

B

backend (*KKResult attribute*), 173
background_subtraction() (*in module xraylabtool.utils*), 138
BashCompletionGenerator (*class in xraylabtool.interfaces.completion_v2.shells*), 99
BatchConfig (*class in xraylabtool.data_handling.batch_processing*), 85
block_for() (*in module xraylabtool.backend.shapes*), 166
border (*ColorPalette attribute*), 160
border_focus (*ColorPalette attribute*), 160
bragg_angle() (*in module xraylabtool.utils*), 135
bucket() (*in module xraylabtool.backend.shapes*), 166
bucket_energy() (*in module xraylabtool.backend.shapes*), 166
bucket_pow2() (*in module xraylabtool.backend.shapes*), 167

C

cache_results (*BatchConfig attribute*), 85
calculate_attenuation_length() (*in module xraylabtool.calculators.derived_quantities*), 57
calculate_batch_properties() (*in module xraylabtool.data_handling.batch_processing*), 87
calculate_critical_angle() (*in module xraylabtool.calculators.derived_quantities*), 57
calculate_derived_quantities() (*in module xraylabtool.calculators.derived_quantities*), 58
calculate_scattering_factors() (*in module xraylabtool.calculators.derived_quantities*), 57
calculate_scattering_length_density() (*in module xraylabtool.calculators.derived_quantities*), 57
calculate_structure_factor() (*CrystalStructure method*), 70
calculate_transmission() (*in module xraylabtool.calculators.derived_quantities*), 57
CalculationEngine (*class in xraylabtool.typing_extensions*), 123
CalculationWorker (*class in xraylabtool.gui.workers*), 154
check_gpu_availability() (*in module xraylabtool.device*), 121
check_memory() (*MemoryMonitor method*), 86
check_plugin_conflicts() (*in module xraylabtool.device*), 121
chunk_iterator() (*in module xraylabtool.data_handling.batch_processing*), 86
chunk_size (*BatchConfig attribute*), 85
clear() (*F1F2Plot method*), 157
clear() (*MultiF1F2Plot method*), 158
clear() (*PlotCanvas method*), 156
clear_cache() (*EnvironmentDetector method*), 103
clear_scattering_factor_cache() (*in module xraylabtool.calculators.cache*), 67
clear_tables() (*in module xraylabtool.calculators.tables*), 72
closeEvent() (*SharedHelpersMixin method*), 149
CM_TO_METER (*in module xraylabtool.constants*), 140
cmd_atomic() (*in module xraylabtool.interfaces.cli*), 89
cmd_atomic() (*in module xraylabtool.interfaces.cli.commands.atomic*), 94
cmd_batch() (*in module xraylabtool.interfaces.cli*), 89
cmd_batch() (*in module xraylabtool.interfaces.cli.commands.batch*), 94
cmd_bragg() (*in module xraylabtool.interfaces.cli*), 89
cmd_bragg() (*in module xraylabtool.interfaces.cli.commands.bragg*), 95
cmd_calc() (*in module xraylabtool.interfaces.cli*), 89
cmd_calc() (*in module xraylabtool.interfaces.cli.commands.calc*), 93
cmd_compare() (*in module xraylabtool.interfaces.cli*), 90
cmd_compare() (*in module xraylabtool.interfaces.cli.commands.compare*), 94
cmd_completion() (*in module xraylabtool.interfaces.cli*), 90
cmd_completion() (*in module xraylabtool.interfaces.cli.commands.completion*), 95
cmd_convert() (*in module xraylabtool.interfaces.cli*), 90
cmd_convert() (*in module xraylabtool.interfaces.cli.commands.convert*), 94
cmd_formula() (*in module xraylabtool.interfaces.cli*), 90
cmd_formula() (*in module xraylabtool.interfaces.cli.commands.formula*), 94
cmd_install_completion() (*in module xraylabtool.interfaces.cli*), 90
cmd_install_completion() (*in module xraylabtool.interfaces.cli.commands.completion*), 95
cmd_kk() (*in module xraylabtool.interfaces.cli*), 90
cmd_kk() (*in module xraylabtool.interfaces.cli.commands.kk*), 94
cmd_list() (*in module xraylabtool.interfaces.cli*), 90

cmd_list() (*in module xraylabtool.interfaces.cli.commands.list_cmd*), 95
cmd_uninstall_completion() (*in module xraylabtool.interfaces.cli*), 90
cmd_uninstall_completion() (*in module xraylabtool.interfaces.cli.commands.completion*), 95
coef (*ScatteringTables attribute*), 71, 75
coeffs (*DeviceTables attribute*), 196
coeffs (*Spectrum attribute*), 180
ColorPalette (*class in xraylabtool.gui.style*), 159
compare_materials() (*in module xraylabtool.analysis*), 145
compare_materials() (*MaterialComparator method*), 145, 147
ComparisonResult (*class in xraylabtool.analysis.comparator*), 146
completion_main() (*in module xraylabtool.interfaces.completion_v2.cli*), 104
CompletionGenerator (*class in xraylabtool.interfaces.completion_v2.shells*), 98
CompletionInstaller (*class in xraylabtool.interfaces.completion_v2.installer*), 97
CompletionManager (*class in xraylabtool.interfaces.completion_v2.shells*), 100
compound_capacity() (*AsfDatabase method*), 198
compound_spectrum() (*in module xraylabtool.kk.spectrum*), 181
compound_spectrum_device() (*in module xraylabtool.kk.spectrum*), 182
compute_kk() (*in module xraylabtool.gui.services*), 152
compute_multiple() (*in module xraylabtool.gui.services*), 151
compute_single() (*in module xraylabtool.gui.services*), 151
CONDA (*EnvironmentType attribute*), 101
configure_logging() (*in module xraylabtool.logging_utils*), 122
create_batch_progress_tracker() (*in module xraylabtool.interfaces.cli.utils*), 93
create_comparison_table() (*MaterialComparator method*), 145, 147
create_completion_parser() (*in module xraylabtool.interfaces.completion_v2.cli*), 104
create_parser() (*in module xraylabtool.interfaces.cli*), 91
create_parser() (*in module xraylabtool.interfaces.cli.parser*), 91
create_scattering_factor_interpolators() (*in module xraylabtool.calculators.cache*), 67
critical_angle (*OpticalProps attribute*), 73
critical_angle_degrees() (*in module xraylabtool.constants*), 142
critical_angle_from_delta() (*in module xraylabtool.calculators.optics*), 76
CrystalStructure (*class in xraylabtool.calculators.scattering_data*), 69
current_palette (*ThemeManager property*), 153
current_palette() (*in module xraylabtool.gui.widgets*), 155
cxro_energy_kev (*KKComputation attribute*), 151
cxro_f1 (*KKComputation attribute*), 152

D

d1 (*ScatteringTables attribute*), 71, 75
d2 (*ScatteringTables attribute*), 71, 75
d_spacing_cubic() (*in module xraylabtool.utils*), 135
d_spacing_orthorhombic() (*in module xraylabtool.utils*), 136
d_spacing_tetragonal() (*in module xraylabtool.utils*), 136
data (*ComparisonResult attribute*), 146
DEGREES_TO_RADIANS (*in module xraylabtool.constants*), 140
derive() (*in module xraylabtool.calculators.optics*), 76
device_tables() (*AsfDatabase method*), 198
DeviceTables (*class in xraylabtool.kk.database*), 196
discover_all_environments() (*EnvironmentDetector method*), 103
dispersion (*OpticalProps attribute*), 74

E

electron_density (*OpticalProps attribute*), 74
element (*TableInterpolator attribute*), 66
ELEMENT_CHARGE (*in module xraylabtool.constants*), 140
element_index() (*in module xraylabtool.calculators.tables*), 72
enable_offscreen_quiet_env() (*in module xraylabtool.gui.logging_filters*), 154
enable_persistent_cache() (*in module xraylabtool.backend.runtime*), 163
enable_progress (*BatchConfig attribute*), 85
encode_formula() (*in module xraylabtool.calculators.optics*), 76
end_kev (*EnergyConfig attribute*), 150
energies (*ComparisonResult attribute*), 146
energy_config() (*MaterialInputForm method*), 155
energy_ev (*NearEdgeSpectrum attribute*), 185
energy_ev (*RefineOutcome attribute*), 192
energy_ev() (*AsfDatabase method*), 197
energy_kev (*KKResult attribute*), 173
energy_to_wavelength() (*in module xraylabtool.utils*), 134
energy_to_wavelength_angstrom() (*in module xraylabtool.constants*), 143
ENERGY_TO_WAVELENGTH_FACTOR (*in module xraylabtool.constants*), 140
EnergyConfig (*class in xraylabtool.gui.services*), 150
ensure_complex128_array() (*in module xraylabtool.typing_extensions*), 124
ensure_float64_array() (*in module xraylabtool.typing_extensions*), 124

ensure_x64() (*in module xraylabtool.backend.runtime*), 164
EnvironmentDetector (*class in xraylabtool.interfaces.completion_v2.environment*), 102
EnvironmentInfo (*class in xraylabtool.interfaces.completion_v2.environment*), 102
EnvironmentType (*class in xraylabtool.interfaces.completion_v2.environment*), 101
error (*ColorPalette attribute*), 161
error (*WorkerSignals attribute*), 154
error_bg (*ColorPalette attribute*), 161
EV_TO_KEV (*in module xraylabtool.constants*), 140
evaluate_f2() (*in module xraylabtool.kk.spectrum*), 182
evaluate_f2_device() (*in module xraylabtool.kk.spectrum*), 183
eventFilter() (*OverlayScrollbarMarginHelper method*), 159
exhausted (*RefineOutcome attribute*), 193
export_to_csv() (*in module xraylabtool.io.file_operations*), 118
export_to_json() (*in module xraylabtool.io.file_operations*), 119

F

f1 (*KKResult attribute*), 173
f1 (*OpticalProps attribute*), 74
f1 (*RefineOutcome attribute*), 193
f1 (*ScatteringTables attribute*), 71, 75
F1F2Plot (*class in xraylabtool.gui.widgets.sweep_plots*), 157
f2 (*KKResult attribute*), 173
f2 (*OpticalProps attribute*), 74
f2 (*RefineOutcome attribute*), 193
f2 (*ScatteringTables attribute*), 71, 75
f2_min (*MergeStatus attribute*), 188
FastAtomicDataProvider (*class in xraylabtool.data_handling.atomic_cache*), 81
fetch() (*in module xraylabtool.backend.runtime*), 164
fetch_status() (*in module xraylabtool.backend.runtime*), 164
field (*TableInterpolator attribute*), 66
file_extension (*BashCompletionGenerator property*), 99
file_extension (*CompletionGenerator property*), 99
file_extension (*FishCompletionGenerator property*), 100
file_extension (*PowerShellCompletionGenerator property*), 100
file_extension (*ZshCompletionGenerator property*), 99
find_absorption_edges() (*in module xraylabtool.analysis*), 146
find_peaks() (*in module xraylabtool.utils*), 138
find_similar_compounds() (*in module xraylabtool.data_handling.compound_analysis*), 83
finished (*WorkerSignals attribute*), 154
FishCompletionGenerator (*class in xraylabtool.interfaces.completion_v2.shells*), 100
force_gc() (*MemoryMonitor method*), 86
format_calculation_summary() (*in module xraylabtool.io.data_export*), 120
format_multi_row() (*TableFormatter static method*), 162
format_single_row() (*TableFormatter static method*), 162
format_summary() (*TableFormatter static method*), 163
format_xray_result() (*in module xraylabtool.interfaces.cli*), 91
format_xray_result() (*in module xraylabtool.interfaces.cli.formatting*), 93
format_xray_result() (*in module xraylabtool.io.data_export*), 119
formula (*KKResult attribute*), 173
from_dict() (*EnvironmentInfo class method*), 102
from_numpy() (*Spectrum class method*), 180

G

generate() (*BashCompletionGenerator method*), 99
generate() (*CompletionGenerator method*), 98
generate() (*FishCompletionGenerator method*), 100
generate() (*PowerShellCompletionGenerator method*), 100
generate() (*ZshCompletionGenerator method*), 99
generate_comparison_report() (*MaterialComparator method*), 145, 148
generate_completion() (*CompletionManager method*), 101
get_atomic_data() (*in module xraylabtool.utils*), 133
get_atomic_data_fast() (*in module xraylabtool.data_handling.atomic_cache*), 79
get_atomic_data_provider() (*in module xraylabtool.data_handling.atomic_cache*), 82
get_atomic_number() (*in module xraylabtool.utils*), 132
get_atomic_properties() (*FastAtomicDataProvider method*), 82
get_atomic_weight() (*in module xraylabtool.utils*), 132
get_backend() (*in module xraylabtool.backend*), 163
get_bash_completion_dir() (*CompletionInstaller method*), 98
get_bulk_atomic_data() (*in module xraylabtool.calculators.cache*), 67
get_bulk_atomic_data_fast() (*in module xraylabtool.data_handling.atomic_cache*), 80
get_cache_stats() (*in module xraylabtool.data_handling.atomic_cache*), 81

get_cached_elements() (*in module xraylabtool.calculators.cache*), 66
get_compound_complexity_score() (*in module xraylabtool.data_handling.compound_analysis*), 84
get_compound_family() (*in module xraylabtool.data_handling.compound_analysis*), 84
get_compound_frequency_score() (*in module xraylabtool.data_handling.compound_analysis*), 83
get_current_environment() (*EnvironmentDetector method*), 103
get_device_info() (*in module xraylabtool.device*), 122
get_elements_for_compound() (*in module xraylabtool.data_handling.compound_analysis*), 83
get_filename() (*CompletionManager method*), 101
get_global_options() (*in module xraylabtool.interfaces.completion_v2.shells*), 101
get_gpu_info() (*in module xraylabtool.device*), 121
get_log_file_path() (*in module xraylabtool.logging_utils*), 123
get_logger() (*in module xraylabtool.logging_utils*), 123
get_memory_usage_mb() (*MemoryMonitor method*), 86
get_optimal_chunk_size() (*in module xraylabtool.typing_extensions*), 125
get_qss() (*in module xraylabtool.gui.style*), 162
get_recommended_elements_for_warming() (*in module xraylabtool.data_handling.compound_analysis*), 84
get_recommended_package() (*in module xraylabtool.device*), 121
get_scattering_factor() (*AtomicScatteringFactor method*), 69
get_scattering_factors() (*FastAtomicDataProvider method*), 81
get_supported_shells() (*CompletionManager method*), 101
get_system_cuda_version() (*in module xraylabtool.device*), 121
get_theme() (*ThemeManager method*), 153
get_user_bash_completion_dir() (*CompletionInstaller method*), 98
get_xraylabtool_commands() (*in module xraylabtool.interfaces.completion_v2.shells*), 101

H

handle_completion_command() (*in module xraylabtool.interfaces.completion_v2.cli*), 104
handle_new_completion_command() (*in module xraylabtool.interfaces.completion_v2.integration*), 103

I

im_coeffs() (*AsfDatabase method*), 198
im_sld (*OpticalProps attribute*), 74
input_bg (*ColorPalette attribute*), 160
install() (*CompletionInstaller method*), 97
install_bash_completion() (*CompletionInstaller method*), 98
install_completion_main() (*in module xraylabtool.interfaces.completion_v2.cli*), 104
install_completion_main() (*in module xraylabtool.interfaces.completion_v2.integration*), 103
interpolate_f() (*in module xraylabtool.calculators.optics*), 77
InterpolatorProtocol (*class in xraylabtool.typing_extensions*), 123
is_complex_array() (*in module xraylabtool.typing_extensions*), 124
is_element_cached() (*FastAtomicDataProvider method*), 81
is_element_cached() (*in module xraylabtool.calculators.cache*), 67
is_element_preloaded() (*in module xraylabtool.data_handling.atomic_cache*), 81
is_energy_array() (*in module xraylabtool.typing_extensions*), 124
is_real_array() (*in module xraylabtool.typing_extensions*), 124

J

JaxPchipInterpolator (*class in xraylabtool.backend.interpolation*), 167

K

KEV_TO_EV (*in module xraylabtool.constants*), 141
kk_calculate_real() (*in module xraylabtool.kk.api*), 169
kk_f1() (*in module xraylabtool.kk.transform*), 177
kk_piecewise_poly() (*in module xraylabtool.kk.transform*), 176
kk_real_spectrum() (*in module xraylabtool.kk.transform*), 178
KKComputation (*class in xraylabtool.gui.services*), 151
KKPlot (*class in xraylabtool.gui.widgets.kk_plot*), 158
KKResult (*class in xraylabtool.kk.result*), 172
KKTabMixin (*class in xraylabtool.gui.main_window.kk_tab*), 149
knots (*DeviceTables attribute*), 196

L

legacy_install_completion_main() (*in module xraylabtool.interfaces.completion_v2.integration*), 103
legacy_uninstall_completion_main() (*in module xraylabtool.interfaces.completion_v2.integration*), 103
legend_kwargs() (*in module xraylabtool.gui.widgets*), 155

linear_absorption_cm() (*in module xraylabtool.gui.services*), 151
list_environments() (*CompletionInstaller method*), 97
load_asf() (*in module xraylabtool.kk.database*), 199
load_atomic_data() (*in module xraylabtool.utils*), 133
load_batch_input() (*in module xraylabtool.data_handling.batch_processing*), 88
load_data_file() (*in module xraylabtool.calculators.scattering_data*), 70
load_data_file() (*in module xraylabtool.io.file_operations*), 118
load_element_data() (*AtomicScatteringFactor method*), 69
load_nearedge() (*in module xraylabtool.kk.nearedge*), 185
load_scattering_factor_data() (*in module xraylabtool.calculators.scattering_data*), 68
load_tables() (*in module xraylabtool.calculators.optics*), 77
load_tables() (*in module xraylabtool.calculators.tables*), 72
log_environment() (*in module xraylabtool.logging_utils*), 123
logspace (*EnergyConfig attribute*), 150

M

main() (*in module xraylabtool.interfaces.cli*), 91
MainWindow (*class in xraylabtool.gui.main_window*), 148
MAMBA (*EnvironmentType attribute*), 101
mass (*AsfDatabase attribute*), 197
MaterialComparator (*class in xraylabtool.analysis*), 145
MaterialComparator (*class in xraylabtool.analysis.comparator*), 147
MaterialInputForm (*class in xraylabtool.gui.widgets.material_form*), 155
materials (*ComparisonResult attribute*), 146
materials() (*MaterialTable method*), 156
MaterialTable (*class in xraylabtool.gui.widgets.material_table*), 156
max_workers (*BatchConfig attribute*), 85
measured_step (*MergeStatus attribute*), 189
memory_limit_gb (*BatchConfig attribute*), 85
MemoryMonitor (*class in xraylabtool.data_handling.batch_processing*), 85
merge_device() (*in module xraylabtool.kk.nearedge*), 189
merge_points_kev (*KKResult attribute*), 173
merge_spectra() (*in module xraylabtool.kk.nearedge*), 191
MergeResult (*class in xraylabtool.kk.nearedge*), 189
MergeStatus (*class in xraylabtool.kk.nearedge*), 187
METER_TO_ANGSTROM (*in module xraylabtool.constants*), 141
METER_TO_CM (*in module xraylabtool.constants*), 141
module
    xraylabtool.analysis, 145
    xraylabtool.analysis.comparator, 146
    xraylabtool.backend, 163
    xraylabtool.backend.interpolation, 167
    xraylabtool.backend.runtime, 163
    xraylabtool.backend.shapes, 165
    xraylabtool.calculators.cache, 66
    xraylabtool.calculators.derived_quantities, 57
    xraylabtool.calculators.optics, 72
    xraylabtool.calculators.scattering_data, 68
    xraylabtool.calculators.tables, 70
    xraylabtool.constants, 140
    xraylabtool.data_handling.atomic_cache, 79
    xraylabtool.data_handling.batch_processing, 85
    xraylabtool.data_handling.compound_analysis, 82
    xraylabtool.device, 121
    xraylabtool.gui, 148
    xraylabtool.gui.logging_filters, 154
    xraylabtool.gui.main_window, 148
    xraylabtool.gui.main_window.kk_tab, 149
    xraylabtool.gui.main_window.multi_tab, 149
    xraylabtool.gui.main_window.shared, 149
    xraylabtool.gui.main_window.single_tab, 149
    xraylabtool.gui.main_window.toast, 150
    xraylabtool.gui.services, 150
    xraylabtool.gui.style, 159
    xraylabtool.gui.table_formatter, 162
    xraylabtool.gui.theme_manager, 153
    xraylabtool.gui.widgets, 155
    xraylabtool.gui.widgets.kk_plot, 158
    xraylabtool.gui.widgets.material_form, 155
    xraylabtool.gui.widgets.material_table, 156
    xraylabtool.gui.widgets.plot_canvas, 156
    xraylabtool.gui.widgets.scrollbar_helper, 159
    xraylabtool.gui.widgets.sweep_plots, 157
    xraylabtool.gui.workers, 154
    xraylabtool.interfaces.cli, 89
    xraylabtool.interfaces.cli.commands.atomic, 94
    xraylabtool.interfaces.cli.commands.batch, 94
    xraylabtool.interfaces.cli.commands.bragg, 95
    xraylabtool.interfaces.cli.commands.calc, 93
    xraylabtool.interfaces.cli.commands.compare, 94
    xraylabtool.interfaces.cli.commands.completion, 95
    xraylabtool.interfaces.cli.commands.convert, 94
    xraylabtool.interfaces.cli.commands.formula, 94
    xraylabtool.interfaces.cli.commands.kk, 94
    xraylabtool.interfaces.cli.commands.list_cmd, 95
    xraylabtool.interfaces.cli.formatting, 93
    xraylabtool.interfaces.cli.parser, 91

xraylabtool.interfaces.cli.utils, 93
xraylabtool.interfaces.completion_v2.cli, 104
xraylabtool.interfaces.completion_v2.environment, 101
xraylabtool.interfaces.completion_v2.installer, 97
xraylabtool.interfaces.completion_v2.integration, 103
xraylabtool.interfaces.completion_v2.shells, 98
xraylabtool.io, 116
xraylabtool.kk, 169
xraylabtool.kk.api, 169
xraylabtool.kk.database, 195
xraylabtool.kk.nearedge, 184
xraylabtool.kk.refine, 192
xraylabtool.kk.result, 172
xraylabtool.kk.spectrum, 181
xraylabtool.kk.transform, 175
xraylabtool.logging_utils, 122
xraylabtool.typing_extensions, 123
MultiF1F2Plot (*class in xraylabtool.gui.widgets.sweep_plots*), 158
MultiTabMixin (*class in xraylabtool.gui.main_window.multi_tab*), 149

N

n (*ScatteringTables attribute*), 71, 75
n_added (*RefineOutcome attribute*), 193
n_inside (*MergeStatus attribute*), 188
n_knots (*DeviceTables attribute*), 196
n_negative (*MergeStatus attribute*), 188
n_pole_skipped (*RefineOutcome attribute*), 193
n_refined_points (*KKResult attribute*), 173
n_segments (*MergeStatus attribute*), 188
n_segments (*Spectrum attribute*), 180
name (*ColorPalette attribute*), 160
nearedge_scaled_kev_f2 (*KKResult attribute*), 173
NearEdgeSpectrum (*class in xraylabtool.kk.nearedge*), 184
nonfinite (*MergeStatus attribute*), 188
normalize_intensity() (*in module xraylabtool.utils*), 138

O

optical_properties() (*in module xraylabtool.calculators.optics*), 77
OpticalProps (*class in xraylabtool.calculators.optics*), 72
optimize_array_memory_layout() (*in module xraylabtool.typing_extensions*), 124
OverlayScrollbarMarginHelper (*class in xraylabtool.gui.widgets.scrollbar_helper*), 159

P

pack_materials() (*in module xraylabtool.calculators.optics*), 78
panel_bg (*ColorPalette attribute*), 160
parse_chemical_formula() (*in module xraylabtool.data_handling.compound_analysis*), 82
parse_energy_string() (*in module xraylabtool.interfaces.cli*), 91
parse_energy_string() (*in module xraylabtool.interfaces.cli.utils*), 93
parse_formula() (*in module xraylabtool.utils*), 131
PerformanceMetrics (*class in xraylabtool.typing_extensions*), 123
PI (*in module xraylabtool.constants*), 141
PIPENV (*EnvironmentType attribute*), 101
PLANCK (*in module xraylabtool.constants*), 141
plot_bg (*ColorPalette attribute*), 161
plot_cycle (*ColorPalette attribute*), 161
plot_multi() (*PlotCanvas method*), 157
plot_single() (*PlotCanvas method*), 157
PlotCanvas (*class in xraylabtool.gui.widgets.plot_canvas*), 156
POETRY (*EnvironmentType attribute*), 101
points (*EnergyConfig attribute*), 150
PowerShellCompletionGenerator (*class in xraylabtool.interfaces.completion_v2.shells*), 100
preload_elements() (*FastAtomicDataProvider method*), 82
process_batch_chunk() (*in module xraylabtool.data_handling.batch_processing*), 86
process_single_calculation() (*in module xraylabtool.data_handling.batch_processing*), 86
progress (*WorkerSignals attribute*), 154
progress_bar() (*in module xraylabtool.utils*), 139
properties (*ComparisonResult attribute*), 146

Q

q_from_angle() (*in module xraylabtool.utils*), 136

R

RADIANS_TO_DEGREES (*in module xraylabtool.constants*), 141
raise_or_log() (*in module xraylabtool.kk.nearedge*), 190
re_sld (*OpticalProps attribute*), 74
recommendations (*ComparisonResult attribute*), 146
refine_exhausted (*KKResult attribute*), 173
refine_spectrum() (*in module xraylabtool.kk.refine*), 194
RefineOutcome (*class in xraylabtool.kk.refine*), 192
relativistic_correction (*KKResult attribute*), 173
relativistic_correction() (*in module xraylabtool.kk.transform*), 175
remove_selected() (*MaterialTable method*), 156
render_multi() (*MultiF1F2Plot method*), 158
render_result() (*F1F2Plot method*), 157
reset_logging() (*in module xraylabtool.logging_utils*), 123
reset_transfer_counts() (*in module xraylabtool.backend.runtime*), 164
resizeEvent() (*SharedHelpersMixin method*), 149

result (*KKComputation attribute*), 151
run() (*CalculationWorker method*), 154

S

save_batch_results() (*in module xraylabtool.data_handling.batch_processing*), 87
save_calculation_results() (*in module xraylabtool.io.file_operations*), 118
save_processed_data() (*in module xraylabtool.utils*), 139
scaled_f2 (*MergeResult attribute*), 189
SCATTERING_FACTOR (*in module xraylabtool.constants*), 141
ScatteringData (*class in xraylabtool.calculators.scattering_data*), 68
ScatteringTables (*class in xraylabtool.calculators.optics*), 74
ScatteringTables (*class in xraylabtool.calculators.tables*), 70
seg_ev (*Spectrum attribute*), 180
set_backend() (*in module xraylabtool.backend*), 163
set_data() (*KKPlot method*), 158
set_scales() (*PlotCanvas method*), 156
set_theme() (*ThemeManager method*), 153
SharedHelpersMixin (*class in xraylabtool.gui.main_window.shared*), 149
shell_name (*BashCompletionGenerator property*), 99
shell_name (*CompletionGenerator property*), 99
shell_name (*FishCompletionGenerator property*), 100
shell_name (*PowerShellCompletionGenerator property*), 100
shell_name (*ZshCompletionGenerator property*), 99
show_completion_info() (*in module xraylabtool.interfaces.completion_v2.cli*), 104
show_toast() (*Toast method*), 148, 150
SingleTabMixin (*class in xraylabtool.gui.main_window.single_tab*), 149
smooth_data() (*in module xraylabtool.utils*), 137
Spectrum (*class in xraylabtool.kk*), 179
spectrum (*MergeResult attribute*), 189
SPEED_OF_LIGHT (*in module xraylabtool.constants*), 141
SQRT_2 (*in module xraylabtool.constants*), 142
start_core_warm_up() (*SingleTabMixin method*), 149
start_kev (*EnergyConfig attribute*), 150
start_kk_warm_up() (*KKTabMixin method*), 149
staticMetaObject (*F1F2Plot attribute*), 158
staticMetaObject (*KKPlot attribute*), 159
staticMetaObject (*MainWindow attribute*), 148
staticMetaObject (*MaterialInputForm attribute*), 155
staticMetaObject (*MaterialTable attribute*), 156
staticMetaObject (*MultiF1F2Plot attribute*), 158
staticMetaObject (*OverlayScrollbarMarginHelper attribute*), 159
staticMetaObject (*PlotCanvas attribute*), 157
staticMetaObject (*ThemeManager attribute*), 153
staticMetaObject (*Toast attribute*), 148, 150
staticMetaObject (*WorkerSignals attribute*), 154
status (*MergeResult attribute*), 189
status() (*CompletionInstaller method*), 97
success (*ColorPalette attribute*), 161
summary_stats (*ComparisonResult attribute*), 146
suppress_qt_noise() (*in module xraylabtool.gui.logging_filters*), 154
symbols (*AsfDatabase attribute*), 197
symbols (*ScatteringTables attribute*), 71, 75
SYSTEM (*EnvironmentType attribute*), 101

T

TableFormatter (*class in xraylabtool.gui.table_formatter*), 162
TableInterpolator (*class in xraylabtool.calculators.cache*), 66
TABULATED_MAX_KEV (*in module xraylabtool.constants*), 142
TABULATED_MIN_KEV (*in module xraylabtool.constants*), 142
tabulated_step (*MergeStatus attribute*), 189
test_completion() (*CompletionInstaller method*), 98
test_completion_installation() (*in module xraylabtool.interfaces.completion_v2.integration*), 103
text_primary (*ColorPalette attribute*), 160
text_secondary (*ColorPalette attribute*), 160
theme_changed (*ThemeManager attribute*), 153
ThemeManager (*class in xraylabtool.gui.theme_manager*), 153
THOMPSON (*in module xraylabtool.constants*), 142
thread_transfer_counts() (*in module xraylabtool.backend.runtime*), 165
threadpool (*KKTabMixin attribute*), 149
threadpool (*MultiTabMixin attribute*), 149
threadpool (*SingleTabMixin attribute*), 149
to_array() (*EnergyConfig method*), 150
to_asf_scale() (*in module xraylabtool.kk.nearedge*), 186
to_asf_scale_device() (*in module xraylabtool.kk.nearedge*), 186
to_dict() (*EnvironmentInfo method*), 102
to_numpy() (*Spectrum method*), 180
to_qpalette() (*ColorPalette method*), 161
to_xray_result() (*KKResult method*), 174
Toast (*class in xraylabtool.gui.main_window*), 148
Toast (*class in xraylabtool.gui.main_window.toast*), 150
toggle_theme() (*ThemeManager method*), 153
transfer_counts() (*in module xraylabtool.backend.runtime*), 165
transmission() (*in module xraylabtool.calculators.optics*), 78
TWO_PI (*in module xraylabtool.constants*), 142

U

uninstall() (*CompletionInstaller method*), 97

uninstall_bash_completion() (*CompletionInstaller method*), 98
uninstall_completion() (*CompletionInstaller method*), 98
uninstall_completion_main() (*in module xraylabtool.interfaces.completion_v2.cli*), 104
uninstall_completion_main() (*in module xraylabtool.interfaces.completion_v2.integration*), 104
update_theme() (*F1F2Plot method*), 157
update_theme() (*KKPlot method*), 159
update_theme() (*MultiF1F2Plot method*), 158
update_theme() (*PlotCanvas method*), 157
upload() (*in module xraylabtool.backend.runtime*), 165
used_points_ev (*MergeResult attribute*), 189

V

v_high (*MergeStatus attribute*), 189
v_low (*MergeStatus attribute*), 189
validate_complex_array() (*in module xraylabtool.typing_extensions*), 124
validate_constants() (*in module xraylabtool.constants*), 143
validate_energy_array() (*in module xraylabtool.typing_extensions*), 123
value_exponent() (*in module xraylabtool.kk.nearedge*), 184
values (*NearEdgeSpectrum attribute*), 185
values() (*MaterialInputForm method*), 155
VENV (*EnvironmentType attribute*), 101
VIRTUALENV (*EnvironmentType attribute*), 101

W

warm_cache_for_compounds() (*in module xraylabtool.data_handling.atomic_cache*), 80
warm_up() (*in module xraylabtool.kk.api*), 171
warm_up_cache() (*in module xraylabtool.data_handling.atomic_cache*), 80
warm_up_core() (*in module xraylabtool.gui.services*), 151
warm_up_kk() (*in module xraylabtool.gui.services*), 152
wavelength (*OpticalProps attribute*), 73
wavelength_angstrom_to_energy() (*in module xraylabtool.constants*), 143
wavelength_to_energy() (*in module xraylabtool.utils*), 134
window_bg (*ColorPalette attribute*), 160
worker_guard() (*in module xraylabtool.backend.runtime*), 165
WorkerSignals (*class in xraylabtool.gui.workers*), 154

X

x (*ScatteringTables attribute*), 71, 75
xraylabtool.analysis
    module, 145
xraylabtool.analysis.comparator
    module, 146
xraylabtool.backend
    module, 163
xraylabtool.backend.interpolation
    module, 167
xraylabtool.backend.runtime
    module, 163
xraylabtool.backend.shapes
    module, 165
xraylabtool.calculators.cache
    module, 66
xraylabtool.calculators.derived_quantities
    module, 57
xraylabtool.calculators.optics
    module, 72
xraylabtool.calculators.scattering_data
    module, 68
xraylabtool.calculators.tables
    module, 70
xraylabtool.constants
    module, 140
xraylabtool.data_handling.atomic_cache
    module, 79
xraylabtool.data_handling.batch_processing
    module, 85
xraylabtool.data_handling.compound_analysis
    module, 82
xraylabtool.device
    module, 121
xraylabtool.gui
    module, 148
xraylabtool.gui.logging_filters
    module, 154
xraylabtool.gui.main_window
    module, 148
xraylabtool.gui.main_window.kk_tab
    module, 149
xraylabtool.gui.main_window.multi_tab
    module, 149
xraylabtool.gui.main_window.shared
    module, 149
xraylabtool.gui.main_window.single_tab
    module, 149
xraylabtool.gui.main_window.toast
    module, 150
xraylabtool.gui.services
    module, 150
xraylabtool.gui.style
    module, 159
xraylabtool.gui.table_formatter
    module, 162
xraylabtool.gui.theme_manager
    module, 153
xraylabtool.gui.widgets
    module, 155
xraylabtool.gui.widgets.kk_plot
    module, 158
xraylabtool.gui.widgets.material_form
    module, 155
xraylabtool.gui.widgets.material_table
    module, 156

xraylabtool.gui.widgets.plot_canvas
    module, 156
xraylabtool.gui.widgets.scrollbar_helper
    module, 159
xraylabtool.gui.widgets.sweep_plots
    module, 157
xraylabtool.gui.workers
    module, 154
xraylabtool.interfaces.cli
    module, 89
xraylabtool.interfaces.cli.commands.atomic
    module, 94
xraylabtool.interfaces.cli.commands.batch
    module, 94
xraylabtool.interfaces.cli.commands.bragg
    module, 95
xraylabtool.interfaces.cli.commands.calc
    module, 93
xraylabtool.interfaces.cli.commands.compare
    module, 94
xraylabtool.interfaces.cli.commands.completion
    module, 95
xraylabtool.interfaces.cli.commands.convert
    module, 94
xraylabtool.interfaces.cli.commands.formula
    module, 94
xraylabtool.interfaces.cli.commands.kk
    module, 94
xraylabtool.interfaces.cli.commands.list_cmd
    module, 95
xraylabtool.interfaces.cli.formatting
    module, 93
xraylabtool.interfaces.cli.parser
    module, 91
xraylabtool.interfaces.cli.utils
    module, 93
xraylabtool.interfaces.completion_v2.cli
    module, 104
xraylabtool.interfaces.completion_v2.environment
    module, 101
xraylabtool.interfaces.completion_v2.installer
    module, 97
xraylabtool.interfaces.completion_v2.integration
    module, 103
xraylabtool.interfaces.completion_v2.shells
    module, 98
xraylabtool.io
    module, 116
xraylabtool.kk
    module, 169
xraylabtool.kk.api
    module, 169
xraylabtool.kk.database
    module, 195
xraylabtool.kk.nearedge
    module, 184
xraylabtool.kk.refine
    module, 192
xraylabtool.kk.result
    module, 172
xraylabtool.kk.spectrum
    module, 181
xraylabtool.kk.transform
    module, 175
xraylabtool.logging_utils
    module, 122
xraylabtool.typing_extensions
    module, 123

Z

z (*TableInterpolator attribute*), 66
ZshCompletionGenerator (*class in xraylabtool.interfaces.completion_v2.shells*), 99